SQL Handbook — Vol 1 (Ch 1–11)

Queries · joins · window functions · data modeling · normalization · indexes · transactions · scaling · 40+ interview problems — PostgreSQL first

11 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is | problem | how it works | trade-offs | real world | interview | self-check | TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — Databases and the Relational Model**
   - 1.1 Before databases: the file processing era
   - 1.2 What a database is
   - 1.3 What a DBMS does
   - 1.4 Inside a DBMS: the core components
   - 1.5 Database architectures
   - 1.6 The relational model
   - 1.7 RDBMS — relational database management systems
   - 1.8 Relational vs non-relational databases
   - 1.9 What SQL is
   - 1.10 Your private PostgreSQL server and psql
   - 1.11 The sample database
   - Key terms (chapter 1)
   - Chapter 1 TL;DR
2. **Chapter 2 — SELECT Basics: Reading, Filtering, Sorting and Paging**
   - 2.1 SELECT: choosing what comes back
   - 2.2 WHERE: filtering rows
   - 2.3 Operators
   - 2.4 Ranges, lists and patterns: BETWEEN, IN, LIKE/ILIKE, regex
   - 2.5 NULL and three-valued logic
   - 2.6 DISTINCT: removing duplicate rows
   - 2.7 ORDER BY: sorting results
   - 2.8 LIMIT, OFFSET and FETCH: returning a page
   - 2.9 How a SELECT is evaluated (logical processing order)
   - Putting it together
   - Chapter 2 TL;DR
3. **Chapter 3 — Functions and Expressions**
   - 3.1 Data types and conversions
   - 3.2 Number functions
   - 3.3 String functions
   - 3.4 Date and time functions
   - 3.5 Conditional expressions: CASE, COALESCE, NULLIF, GREATEST, LEAST
   - 3.6 Worked exercises
   - Chapter 3 TL;DR
4. **Chapter 4 — Aggregation: Summarising Many Rows**
   - 4.1 Aggregate functions
   - 4.2 GROUP BY
   - 4.3 HAVING
   - 4.4 Conditional aggregation: FILTER and CASE
   - 4.5 The logical order of query processing
   - 4.6 Subtotals: GROUPING SETS, ROLLUP and CUBE
   - Chapter 4 TL;DR
5. **Chapter 5 — Joins: Combining Tables**
   - 5.1 Join theory: the Cartesian product and CROSS JOIN
   - 5.2 The four main join types at a glance
   - 5.3 INNER JOIN
   - 5.4 LEFT OUTER JOIN
   - 5.5 RIGHT OUTER JOIN
   - 5.6 FULL OUTER JOIN
   - 5.7 SELF JOIN
   - 5.8 Semi-joins and anti-joins
   - 5.9 Join pitfalls
   - 5.10 USING, NATURAL JOIN, multi-table joins, and how joins are executed
   - Chapter 5 TL;DR
6. **Chapter 6 — Subqueries and Set Operations**
   - 6.1 Subqueries: queries inside queries
   - 6.2 Multi-row subqueries: IN and NOT IN
   - 6.3 Correlated subqueries
   - 6.4 EXISTS and NOT EXISTS
   - 6.5 ANY, SOME and ALL
   - 6.6 Table subqueries: derived tables and row subqueries
   - 6.7 LATERAL subqueries
   - 6.8 Set operations and UNION
   - 6.9 INTERSECT
   - 6.10 EXCEPT
   - 6.11 Precedence, sorting, and choosing the right tool
   - Chapter 6 TL;DR
7. **Chapter 7 — CTEs, Views and Recursion: Naming Queries and Walking Trees**
   - 7.1 Common table expressions (WITH)
   - 7.2 Data-modifying CTEs (PostgreSQL)
   - 7.3 Recursive CTEs: the mechanics
   - 7.4 Hierarchies: trees stored as parent pointers
   - 7.5 Graphs and cycles
   - 7.6 generate_series: making rows (PostgreSQL)
   - 7.7 Views
   - 7.8 Materialized views
   - 7.9 Choosing the right tool
   - Chapter 7 TL;DR
8. **Chapter 8 — Window Functions: Rankings, Running Totals and Row-to-Row Comparisons**
   - 8.1 What a window function is
   - 8.2 PARTITION BY: per-group values next to each row
   - 8.3 ORDER BY inside OVER: running calculations and the default frame
   - 8.4 Frames: ROWS, RANGE, GROUPS and EXCLUDE
   - 8.5 Ranking functions
   - 8.6 Value (offset) functions: LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE

- 8.7 Running totals, moving averages and growth
- 8.8 Top-N per group and de-duplication
- 8.9 Gaps and islands
- 8.10 Named windows, FILTER, evaluation order and performance
- Chapter 8 TL;DR

9. **Chapter 9 — DDL, DML and Constraints: Building Tables and Changing Data Safely**
   - 9.1 Data types
   - 9.2 CREATE: databases, schemas and tables
   - 9.3 Keys and constraints
   - 9.4 Generated keys: identity, sequences, serial, UUID, generated columns
   - 9.5 ALTER: changing existing tables
   - 9.6 DROP and TRUNCATE (vs DELETE)
   - 9.7 Indexes (in brief)
   - 9.8 INSERT
   - 9.9 UPDATE
   - 9.10 DELETE
   - 9.11 Upsert: INSERT … ON CONFLICT, and MERGE
   - 9.12 Backup and restore
   - Chapter 9 TL;DR
10. **Chapter 10 — Data Modeling and ER Diagrams**
   - 10.1 What data modeling is
   - 10.2 The three levels of a data model
   - 10.3 The data modeling process
   - 10.4 Best practices in data modeling
   - 10.5 The Entity-Relationship (ER) model and its notations
   - 10.6 Entities and the five kinds of attribute
   - 10.7 Relationships: degree, cardinality and participation
   - 10.8 Attributes of relationships
   - 10.9 Weak entities
   - 10.10 Worked example — an employee-management ER diagram
   - 10.11 The relational model
   - 10.12 Relations, tuples, attributes — the precise vocabulary
   - 10.13 Keys in relational databases
   - 10.14 Relational integrity constraints
   - 10.15 Converting an ER diagram to tables
   - 10.16 Best practices for ER → relational mapping
   - Key terms
   - Chapter 10 TL;DR
11. **Chapter 11 — Normalization**
   - 11.1 Functional dependencies
   - 11.2 Types of functional dependencies
   - 11.3 Armstrong's inference rules
   - 11.4 Attribute closure and finding candidate keys
   - 11.5 Minimal (canonical) cover
   - 11.6 Anomalies — why normalization exists
   - 11.7 First normal form (1NF)
   - 11.8 Second normal form (2NF)
   - 11.9 Third normal form (3NF)
   - 11.10 Decomposition done right: lossless joins and dependency preservation
   - 11.11 Boyce–Codd normal form (BCNF)
   - 11.12 Fourth normal form (4NF) — independent multivalued facts
   - 11.13 Fifth normal form (5NF) — facts that need three tables
   - 11.14 Putting it together — a normalization procedure
   - 11.15 Exercises (with solutions)
   - 11.16 Denormalization — when and how to break the rules
   - Key terms
   - Chapter 11 TL;DR

Chapter 1 — Databases and the Relational Model

Almost every program you use keeps its facts somewhere: the orders in a shop, the balance in a bank, the messages in a chat app. This chapter explains where those facts live and why the answer is usually "in a relational database". You will see the problems that existed before databases, what a database management system (DBMS) does for you, how a DBMS is built inside, how relational databases compare with NoSQL stores (databases that keep data in shapes other than tables, covered in 1.8), and what SQL is. The chapter ends with a working PostgreSQL server (PostgreSQL is a free, open-source relational database product; a *server* is a program that runs in the background and answers requests from other programs) on your machine and the **sample database** that the next chapters query.

What you'll learn

#	Topic	You will be able to…
1.1	Life before databases	name the seven problems of file-based systems
1.2	What a database is	define database, DBMS and database system precisely
1.3	What a DBMS does	list its six jobs and explain data independence
1.4	Inside a DBMS	follow one query through parser, optimizer, executor, storage and log
1.5	Database architectures	compare two-tier, three-tier, n-tier, cloud and distributed set-ups
1.6	The relational model	use the vocabulary (relation, tuple, key…) and the four table rules
1.7	RDBMS products	describe what an RDBMS adds and how the main products differ
1.8	Relational vs non-relational	choose a store by data shape and access pattern
1.9	What SQL is	sort any statement into DDL, DML, DQL, DCL or TCL
1.10	Your private PostgreSQL and psql	start the server, connect, and use the psql commands you need daily
1.11	The sample database	know the `company` tables used in chapters 2 onward

All code is in `code/`. Every output below was pasted from a real run on the book's private PostgreSQL 14 server (section 1.10 shows how to start it).

> **Dialect note.** This book writes PostgreSQL first. Where MySQL, SQLite or SQL Server (three other widely used database products, compared in 1.7) behave differently, a small table says so. (Each product speaks its own *dialect* — its version of SQL; see 1.9.) Most of what you learn transfers directly; the differences are mainly in functions (chapter 3), limits/paging (chapter 2) and data types (chapter 9).

1.1 Before databases: the file processing era

What it is

Until the late 1960s most business software stored data in **files owned by one program each**. A payroll program had its payroll files, the billing program had its billing files. The layout of each file (which bytes meant "name", which meant "salary") was written into the program's code, not stored with the file. This arrangement is called a **file processing system**.

Think of a school where the admissions office, the library and the exam office each keep their own paper register of students. Each register works fine on its own. Trouble starts when a student moves house.

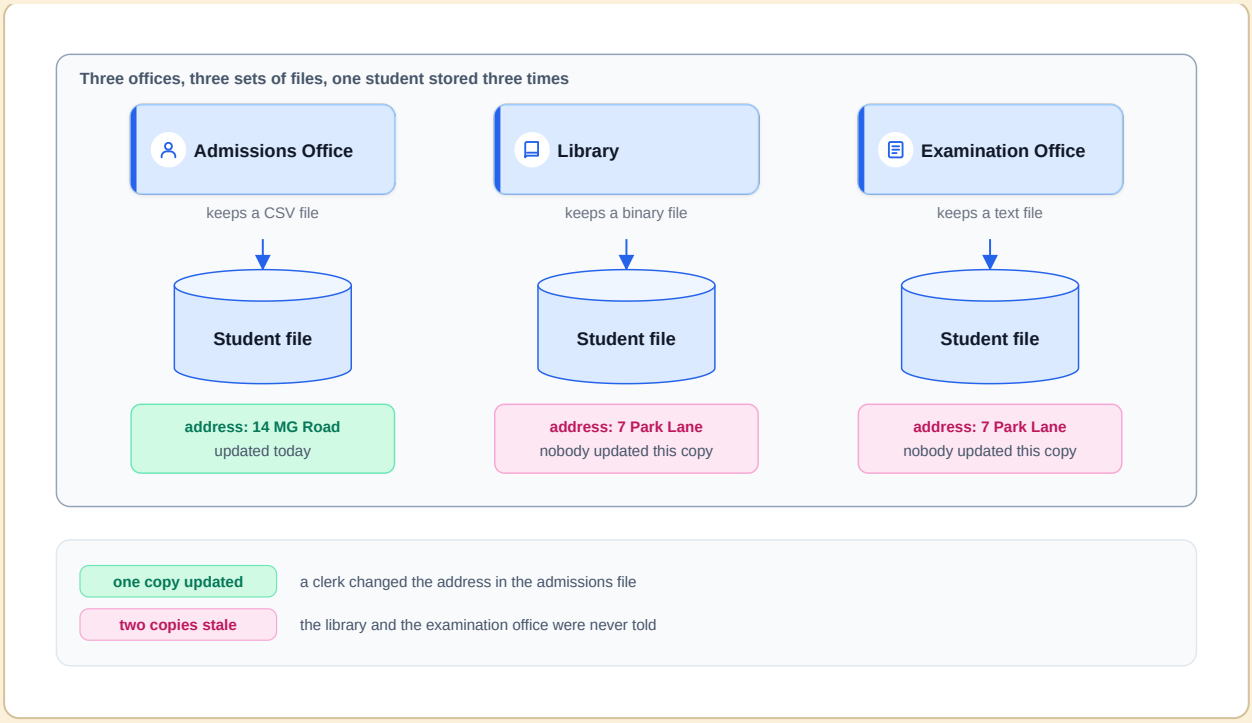


Figure — One fact stored in three separate files: an update reaches one of them and the other two go stale (course diagram).

Records in such files were found in one of two ways. **Sequential access** reads the file from the beginning until the wanted record appears. **Indexed access** keeps a second, hand-maintained file that maps a key (say, a student number) to a byte position, which is faster but is one more thing to keep in sync.

The problem it solves — seven recurring failures

As companies added programs and data grew, the same seven problems appeared everywhere. Each one later became a feature of the DBMS.

#	Problem	What went wrong	What a DBMS does instead
1	**Redundancy → inconsistency**	A customer's address is copied into several files; an update reaches one copy, the copies disagree, and nothing notices.	Store each fact once; every program reads the same copy.
2	**Data isolation**	Files sit on different machines in different formats (CSV = plain text with commas between values; fixed-width binary = raw bytes at set positions); a cross-department question needs custom parsing code.	One store with one self-describing format.
3	**No ad-hoc questions**	No query language: "which customers owe more than ₹500?" means days of writing a new program, so many questions are never asked.	A declarative query language (SQL — you state *what* you want, not *how* to get it): a new question is a new query.
4	**Integrity rules buried in code**	"Marks must be 0–100" lives as an `if` in one program. Any other program writing that file can skip it.	Constraints (rules declared on the table, such as "0 to 100") declared once and enforced for every writer.
5	**Unsafe concurrent access**	Two clerks both read "1 copy left" and both save 0: two copies go out, the count dropped once — a **lost update**. Locking the whole file blocks everybody.	Row-level locks (a marker that makes others wait while one program changes a row) and transactions (all-or-nothing groups of changes, defined in 1.4).
6	**No crash recovery**	A transfer is two writes; power fails after the first and the money has left one account and reached none. Only an old backup can repair it.	A log (an append-only diary of every change) that lets the system undo or redo half-finished work.
7	**All-or-nothing security**	File permissions cover the whole file. A clerk who needs addresses can also read salaries.	Permissions per table, column and even row.

How the DBMS changed things

A DBMS is a layer of software that **every application must go through**; no program touches the stored files directly.

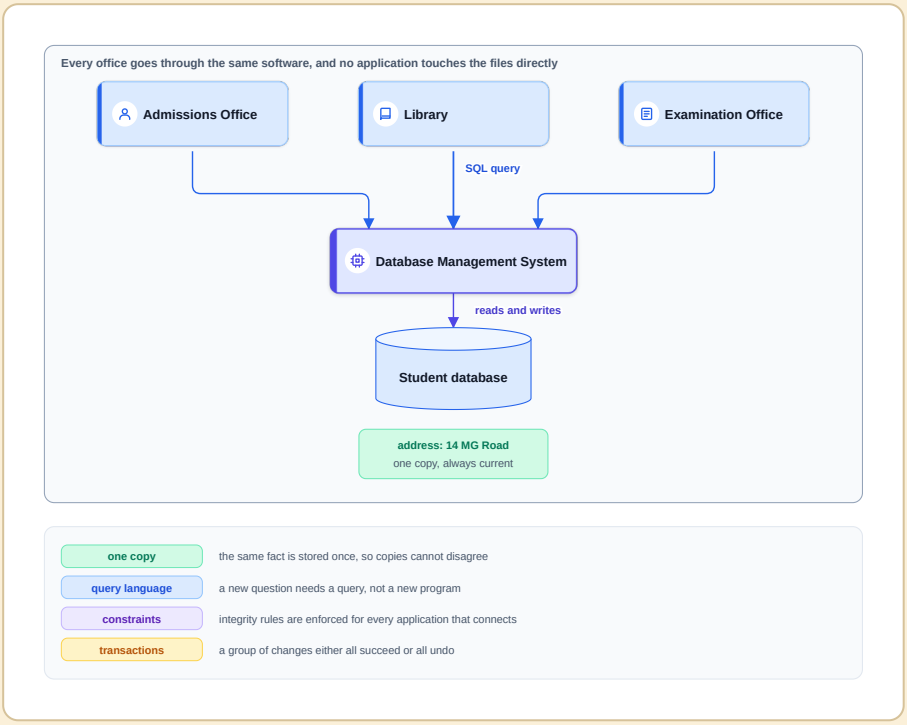


Figure — With a DBMS in the middle, a single shared copy of the data lives alongside its protecting rules (course diagram).

Because every read and write passes through one place, that place can apply all seven fixes in the table above.

A short timeline helps place the ideas:

Year	Milestone
1960s	Navigational databases: IBM's IMS (hierarchical, a tree of records) and Charles Bachman's IDS / the CODASYL network model (records linked in a web, not just a tree). Programs followed stored pointers (saved disk addresses of other records) from record to record.
1970	Edgar F. Codd (IBM) publishes *A Relational Model of Data for Large Shared Data Banks*.
1974–79	IBM's System R and the query language SEQUEL (later renamed SQL); UC Berkeley's Ingres; Oracle ships the first commercial SQL database (1979).
1986–87	SQL becomes an ANSI (1986), then ISO (1987) standard (ANSI and ISO are the US and international standards bodies). In 1986 UC Berkeley also starts POSTGRES, the ancestor of PostgreSQL.
2000s	Open-source (free, with public source code) RDBMSs (relational DBMSs, see 1.7) dominate the web (MySQL, PostgreSQL); SQLite runs inside phones and browsers.
late 2000s	"NoSQL" systems (key-value, document, wide-column, graph) for web-scale workloads.
2010s–now	Cloud-managed databases (run for you by a provider such as AWS) and distributed SQL ("NewSQL": relational databases spread over many machines).

Self-check

1. *Why does storing the same fact in three files lead to wrong answers, not just wasted space?* → Updates reach some copies and not others, so the copies drift apart (inconsistency), and nothing checks that they match.
2. *What is a lost update?* → Two programs read the same value, each computes a new value from it, and the second write overwrites the first, so one change disappears.
3. *Which file-system problem does a transaction log solve?* → Recovery from a crash in the middle of a multi-step change.

1.2 What a database is

What it is

A **database** is an organised, persistent collection of related data that you can query and update, stored *along with a description of its own structure*. Your phone's contact list is a (tiny) database; so is a bank's decade of transactions. Size changes the hardware and the cost, not the definition.

Four properties make a pile of data a database:

Property	Meaning	Example
Related	Everything describes one chosen slice of reality, often called the *mini-world* (a school, a shop, a hospital).	A shop database holds customers, products and orders — not your holiday photos.
Self-describing	The database stores **metadata** (data about data: table names, column names, types, rules) in its **catalog**, also called the **data dictionary**.	Any program can ask "which columns does `employees` have?" instead of hard-coding the layout.
Purposeful	Built for specific people and the questions they ask; requirements are gathered before any table exists.	A hospital's schema is shaped by what doctors, nurses and billing staff need to look up.
Persistent	Data survives after the program ends and the machine restarts.	An in-memory list that disappears on exit is not a database.

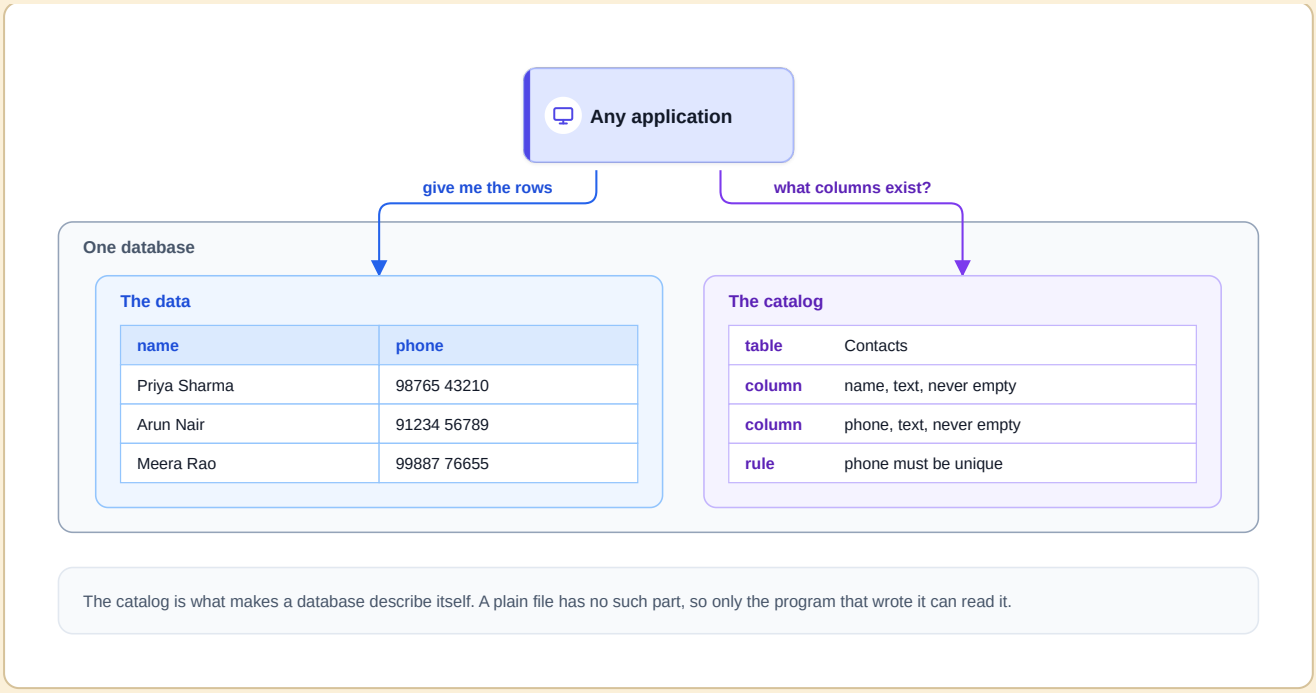


Figure — The catalog sits inside the database next to the data it describes (course diagram).

You can see self-description in action: PostgreSQL answers questions about its own tables through the standard `information_schema` views (a **view** is a saved query you can read like a table — chapter 7) and its own `pg_catalog` tables.

```
-- code/02_catalog_metadata.sql (part)
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'departments'
ORDER BY ordinal_position;
```

Output:

```
 column_name | data_type | is_nullable
-------------+-----------+-------------
 dept_id     | integer   | NO
 name        | text      | NO
 location    | text      | YES
 budget      | numeric   | NO
(4 rows)
```

```
SELECT conname AS constraint_name, contype AS kind, conrelid::regclass AS on_table
FROM pg_constraint
WHERE conrelid = 'employees'::regclass
ORDER BY conname;
```

Output:

```
      constraint_name      | kind | on_table
---------------------------+------+-----------
 employees_dept_id_fkey    | f    | employees
 employees_email_key       | u    | employees
 employees_manager_id_fkey | f    | employees
 employees_pkey            | p    | employees
 employees_salary_check    | c    | employees
(5 rows)
```

The `kind` letters are `p` primary key, `f` foreign key, `u` unique, `c` check. A **constraint** is a rule attached to a table that the DBMS checks on every write: a *primary key* names the column that identifies each row, a *foreign key* says a value must exist in another table (both defined properly in 1.6), *unique* forbids repeats, and a *check* is a yes/no test every row must pass (here: salary must be positive). The rules you declared live in the catalog, which is how the DBMS can enforce them for every program.

Database vs DBMS vs database system

These three are often mixed up — including in interviews.

Term	What it is	Example
Database	The stored data plus its description	the `company` tables in this book
DBMS (database management system)	The software that creates, stores, queries and protects databases	PostgreSQL, MySQL, Oracle
Database system	DBMS + databases + the applications that use them	PostgreSQL + `company` + the web shop that talks to it

Why it matters

Databases make large amounts of data fast to find, keep it accurate (integrity rules) and private (access control — deciding who may see what; encryption — scrambling data so only key holders can read it), let many users work at once, and feed the reports behind business decisions. A good one is well structured, scalable (able to handle more data and users by adding hardware), reliable through failures, richly queryable and usable from many tools and languages.

Real-world examples

A shop must never lose a paid order; a hospital must never show records to the wrong person; a bank must never lose or double-apply a transfer; a social app must not fall over when millions read the same post. Each requirement maps to a DBMS job in the next section.

Self-check

1. *What is metadata and where is it kept?* → Data describing the data (tables, columns, types, constraints, permissions); in the catalog / data dictionary inside the database.
2. *Is MySQL a database?* → Strictly, no — it is a DBMS. The tables you create inside it form a database.

1.3 What a DBMS does

What it is

A **DBMS** is the program that stands between applications and stored data. Applications send requests ("give me these rows", "store this order"); the DBMS decides where bytes go on disk, how to find them again, and how to keep them safe.

How it works — the six jobs

Job	Meaning	In SQL / PostgreSQL
Define	Describe tables, columns, types and rules; saved in the catalog	`CREATE TABLE`, `ALTER TABLE`
Construct	Physically write the data to storage; you never choose file names or byte offsets	PostgreSQL chooses files under its data directory (the folder where the server keeps its files)
Manipulate	Read, add, change, remove data	`SELECT`, `INSERT`, `UPDATE`, `DELETE`
Share	Let many users and programs work on the same data at once without corrupting it	transactions, locks, MVCC (keeping several versions of a row so readers and writers don't block each other; chapter 13)
Protect	*Security* (who may read/write what) and *failure protection* (stay correct after a crash)	`GRANT` / `REVOKE`, the write-ahead log (a change diary saved to disk *before* the data itself, used to repair after a crash)
Maintain	Let the schema and storage evolve for years without rewriting applications	add a column, add an index (a lookup structure, explained below), move to faster disks

File processing systems managed the first three poorly and the last three not at all.

Who talks to a DBMS

Role	What they do
Application programs	Issue SQL for the people using them (a shopper clicks buttons, never writes SQL).
Database administrators (DBAs)	Create users and permissions, plan backups, tune and monitor.
Database designers	Decide which tables exist and how they connect (chapter 10).
Analysts	Query the data themselves to produce reports and insights.

Data independence

Data independence means a change at one level of the system's description does not force changes at the level above. It is the reason a 20-year-old report query can still run after the database moved to new hardware.

Kind	You change…	…and this keeps working	Example
Physical data independence	how data is stored (indexes, files, disks, page layout)	the tables and every query	add an index; move to SSDs (fast flash-memory disks)
Logical data independence	the tables (add a column or a table)	applications that don't use the changed part	add `phone` to `employees`; a report reading names is unaffected

What does "logical is separate from physical" mean? The **logical level** is what you query — tables, columns, views. The **physical level** is how the bytes are laid out — data files, 8 kB pages (fixed-size chunks the database reads and writes as one unit), index trees.

> **Key term — index:** An index is an extra, sorted structure the database keeps beside a table so it can find rows by a column's value without reading every row — like the index at the back of a book. It speeds up reads but costs disk space and a little extra work on every insert or update. PostgreSQL's default index is a *B-tree*, a balanced tree that stays sorted as rows change. Chapter

> 12 covers indexes in depth. SQL names only logical objects, so the DBMS may reorganise the physical side freely and your queries keep their meaning.

The demo below changes both levels and runs the same "application" query each time.

```sql
-- code/06_data_independence.sql
SELECT first_name, last_name, job_title FROM employees WHERE last_name = 'Verma';
EXPLAIN (COSTS OFF) SELECT first_name, last_name, job_title FROM employees WHERE last_name = 'Verma';

-- Physical change: add an index. The query text is unchanged.
CREATE INDEX employees_last_name_idx ON employees (last_name);
SET enable_seqscan = off;   -- the table is tiny; nudge the planner so we can SEE it use the index
EXPLAIN (COSTS OFF) SELECT first_name, last_name, job_title FROM employees WHERE last_name = 'Verma';
SELECT first_name, last_name, job_title FROM employees WHERE last_name = 'Verma';
RESET enable_seqscan;

-- Logical change: add a column the query does not use. Still unchanged.
ALTER TABLE employees ADD COLUMN phone text;
SELECT first_name, last_name, job_title FROM employees WHERE last_name = 'Verma';
```

Output (abridged to the interesting parts):

```
 first_name | last_name | job_title
------------+-----------+-----------
 Rahul      | Verma     | Engineer
(1 row)

              QUERY PLAN
---------------------------------------
 Seq Scan on employees
   Filter: (last_name = 'Verma'::text)
(2 rows)

                      QUERY PLAN
------------------------------------------------------
 Index Scan using employees_last_name_idx on employees
   Index Cond: (last_name = 'Verma'::text)
(2 rows)

...
ALTER TABLE
 first_name | last_name | job_title
------------+-----------+-----------
 Rahul      | Verma     | Engineer
(1 row)
```

`EXPLAIN` shows the **plan** — the method the DBMS chose. The **planner** is PostgreSQL's name for the optimizer, the part that picks the plan. Before the index it scans every row (`Seq Scan`); after, it jumps through the index (`Index Scan`). The result and the query text never changed. (`enable_seqscan = off` is a teaching switch only; on a 15-row table the planner would rightly prefer the full scan. Chapter 12 covers plans properly.)

Self-check

1. *Name the six jobs of a DBMS.* → Define, construct, manipulate, share, protect, maintain.
2. *Adding an index without touching any query is an example of…?* → Physical data independence.

1.4 Inside a DBMS: the core components

What it is

A DBMS is several cooperating parts, each with one job. Knowing them makes database behaviour predictable: why the first run of a query is slower than the second, why a crash does not lose

committed data, why two users can update different rows at once.

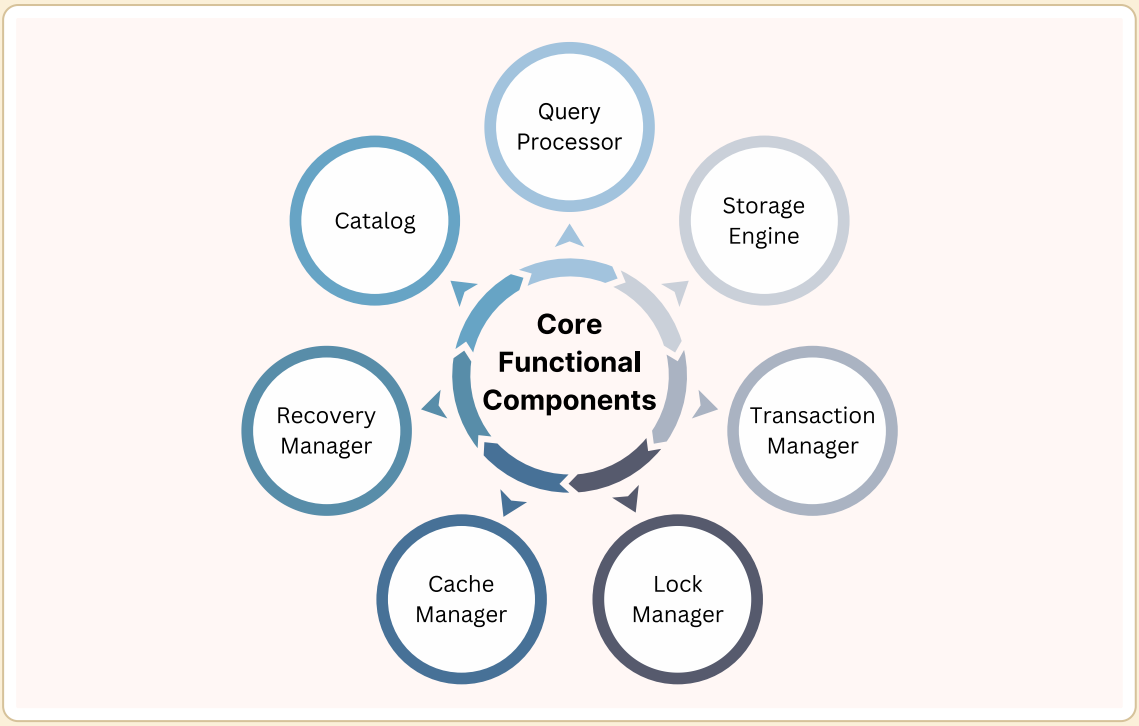
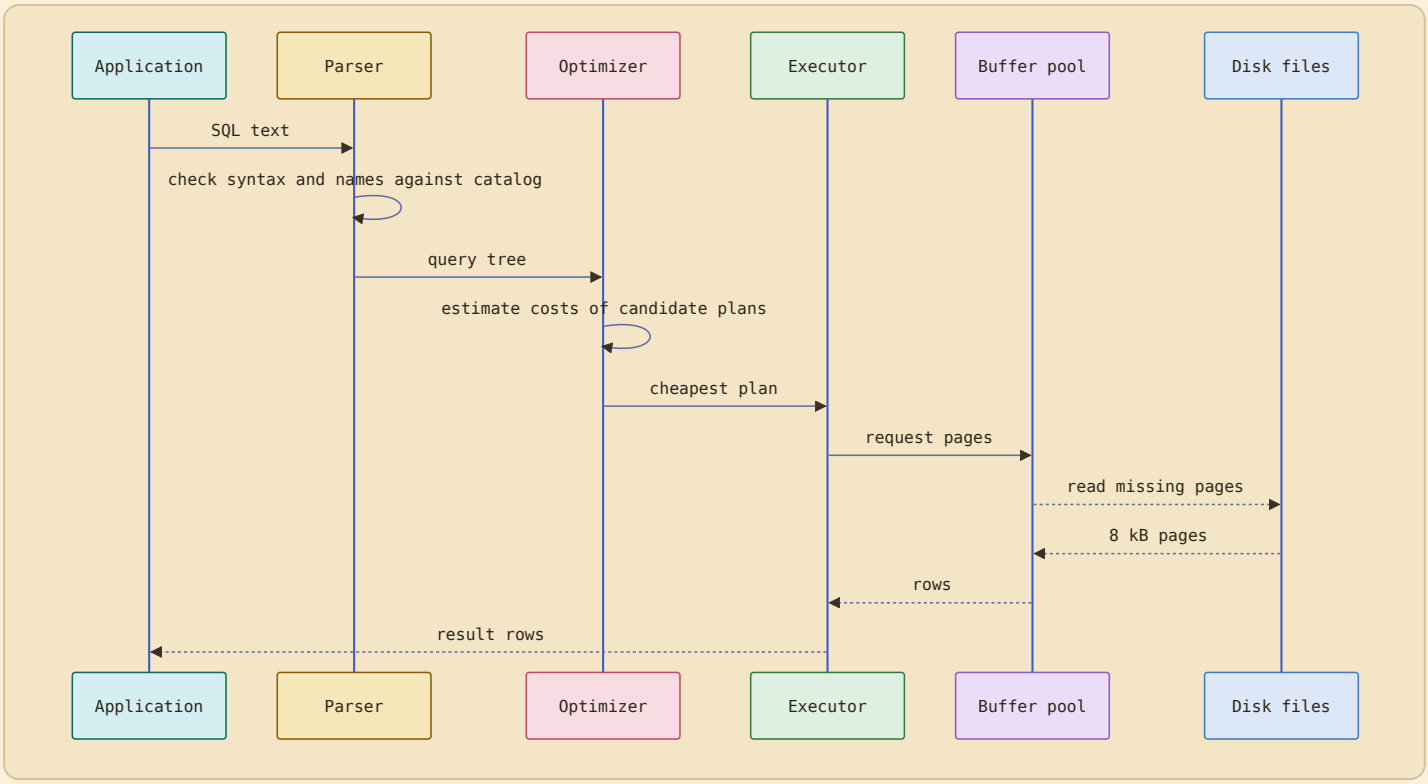


Figure — Query processor, storage engine, transaction manager, lock manager, cache manager, recovery manager and catalog (course diagram).

Component	Job	Sub-parts / tools
Query processor	Turn SQL text into results	**Parser** checks syntax and meaning (do these tables/columns exist? may you use them?) and builds an internal tree. **Optimizer** considers alternative plans and picks the cheapest by estimated cost. **Execution engine** runs the plan, asking the storage engine for rows.
Storage engine (storage manager)	Store and fetch bytes	Organises data into fixed-size **pages** (blocks) inside files; maintains indexes; may store rows (row store) or columns (column store). Examples: InnoDB in MySQL, WiredTiger in MongoDB, PostgreSQL's heap + index access methods.
Cache / buffer manager	Keep hot pages in memory	The **buffer pool** holds recently used pages; a query whose pages are already cached avoids disk reads.
Transaction manager	Make groups of operations all-or-nothing and isolated	Begins, commits (makes permanent) and rolls back (cancels) transactions; guarantees ACID (all-or-nothing, rule-preserving, isolated, durable — defined in 1.8; chapter 13).
Lock manager	Coordinate concurrent access	**Shared (S) locks**: many readers may hold one at once. **Exclusive (X) locks**: one writer, nobody else. Prevents lost updates and dirty reads (seeing another transaction's uncommitted change); must also detect **deadlocks** (two transactions each waiting for the other's lock).
Recovery manager	Restore a consistent state after a crash	Writes a **log** of every change *before* applying it; after a crash, **redoes** committed work (roll forward) and **undoes** uncommitted work (roll back).
Catalog (data dictionary)	Describe everything	Tables, columns, types, constraints, indexes, views, users and permissions, storage sizes. Every other component reads it.

A **transaction** bundles several operations into one all-or-nothing unit: if any step fails, none of the steps remain. The classic example is a bank transfer — the debit and the credit succeed together or not at all.

How it works — the journey of one query



The optimizer's candidate plans differ in scan method (full scan or index), join order and join algorithm; it estimates each one's cost from **statistics** about the data. That division of labour is what **declarative** means: your SQL names the result, the DBMS picks the algorithm. For writes, the change is first recorded in the log, the lock manager holds off conflicting writers, and the change becomes durable at `COMMIT`.

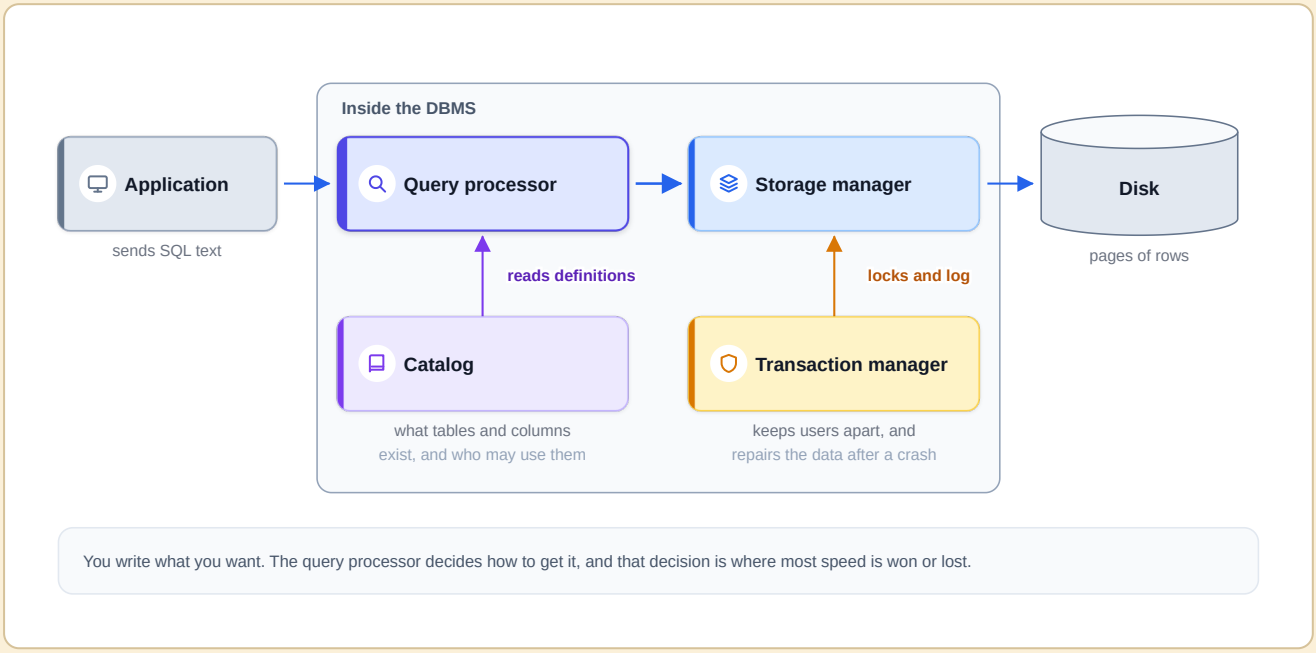


Figure — A query flows from the application through the query processor to storage; the catalog and the transaction manager support every step (course diagram).

Real-world example — the same parts in PostgreSQL

Generic component	PostgreSQL name
Parser / optimizer / executor	parser → rewriter → **planner** → executor (each connection gets its own server process, i.e. its own running copy of the server program)
Pages	8 kB blocks (`SHOW block_size` → `8192`) in files under the data directory
Buffer pool	`shared_buffers` (default `128MB`) plus the operating-system cache (file data the OS itself keeps in spare memory)
Recovery log	**WAL** (write-ahead log) in `pg_wal/`
Lock manager + concurrency	table/row locks **plus MVCC** (multi-version concurrency control: readers see a snapshot and don't block writers)
Catalog	`pg_catalog` tables and the standard `information_schema` views

(Both numbers were read on the book's server by `code/02_catalog_metadata.sql`.)

Common mistakes

- Assuming SQL runs "top to bottom as written" — the optimizer may reorder joins and filters as long as the result is the same.
- Judging a query by its first (cold-cache) run — the second may be served from the buffer pool.
- Confusing the cache (a fast in-memory copy, lost in a crash) with durability (the guarantee that committed data survives — provided by the log on disk).

In the interview

> **30-second answer:** "A DBMS has a query processor — parser, optimizer, executor — that turns SQL into a plan and runs it; a storage engine that keeps data in fixed-size pages and indexes; a buffer pool that caches pages in memory; a transaction manager and lock manager that give ACID and safe concurrency; a recovery manager that uses a write-ahead log to redo or undo work after a crash; and a catalog that describes everything."

- *Why does the optimizer need statistics?* → To estimate how many rows each step returns, which decides whether an index or a full scan, and which join order, is cheaper.
- *What is write-ahead logging?* → The log record describing a change is flushed to disk before the changed data page, so after a crash the system can replay or undo from the log.
- *Shared vs exclusive lock?* → Shared allows concurrent readers; exclusive allows one writer and blocks others.

Self-check

1. *Which component decides between an index scan and a full scan?* → The optimizer (planner).
2. *What does the recovery manager do for a committed transaction whose pages weren't yet written?* → Redoes it from the log (roll forward).

1.5 Database architectures

What it is

An **architecture** describes where the database runs relative to its users and how requests travel. The same PostgreSQL can sit in any of these set-ups.

Two-tier: client–server

The **client** application (the program asking for data) talks **directly** to the database server: it opens a **connection** (a logged-in, two-way network channel to the server), sends SQL, gets rows back.

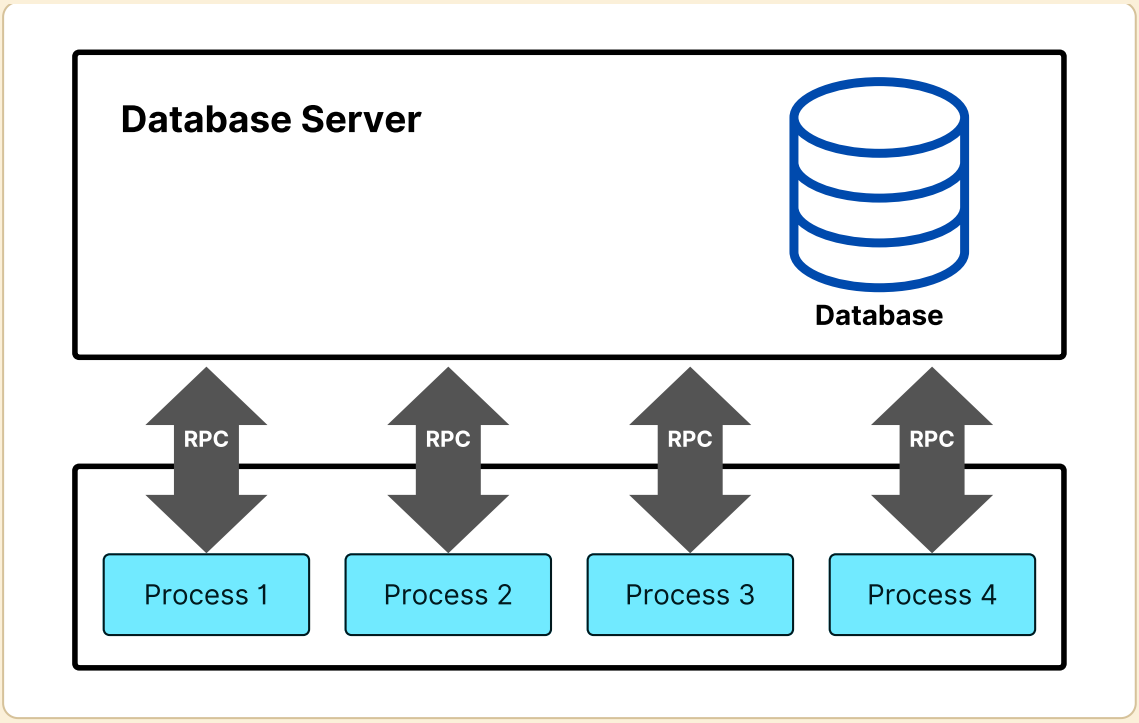


Figure — Two-tier client–server: many clients, one database server (course diagram).

- **Good:** simple; fine for small teams and internal tools (a desktop accounting app on an office server, `psql` itself).
- **Bad:** one server struggles with many client connections; business rules live in every client, so a change means redeploying (reinstalling an updated version of) them all; clients hold database credentials (user name and password).

Three-tier

A middle **application tier** sits between the user interface and the database.

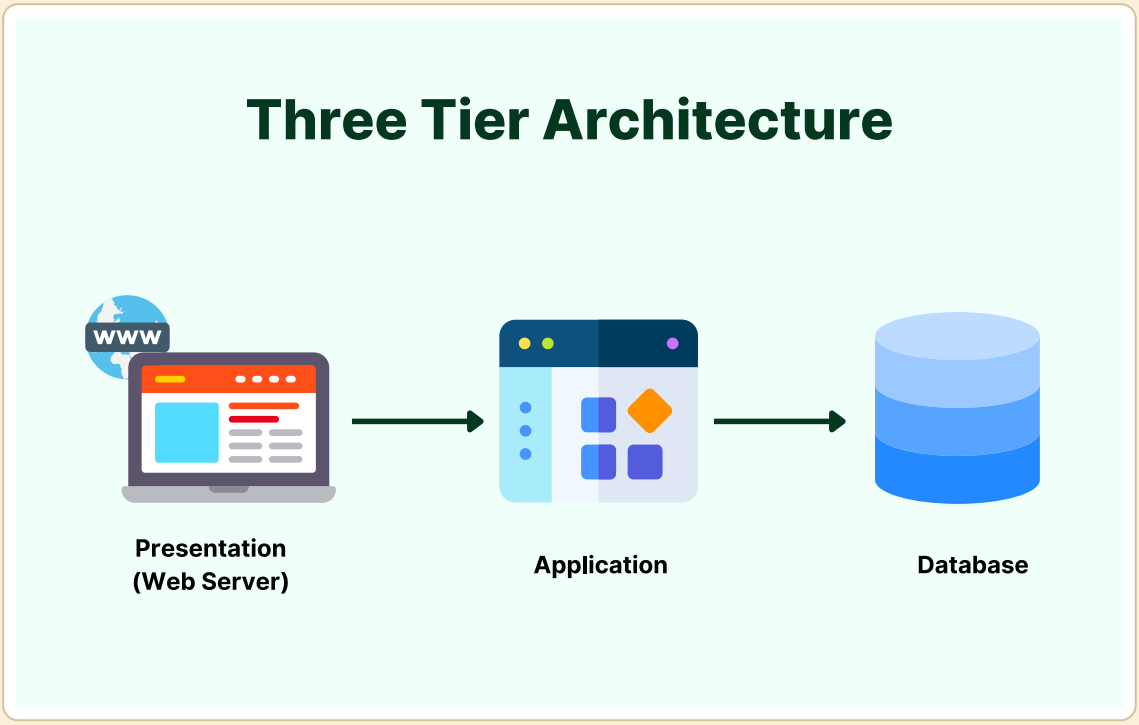


Figure — Three-tier: presentation, application (business logic) and data layers (course diagram).

Tier	Job	Example
Presentation	User interface	browser page or mobile app
Application (middle tier)	Business rules, validation, coordination, connection pooling (reusing a few open connections for many users)	a Java/Python/Node web service
Data	Storage and queries	PostgreSQL

- **Good:** tiers scale separately (more app servers behind a **load balancer** — a component that spreads incoming requests across several servers); rules change in one place; the database stays off the internet. **Bad:** more moving parts, a little **latency** (delay, the time a request takes) per hop (each network trip between tiers). Both terms are explained further in System Design Fundamentals.
- **Typical use:** almost every web application — shops, ERP (enterprise resource planning) and CRM (customer relationship management) systems.

N-tier

Split further into specialised layers: an authentication service (checks who a user is), a payment service, a cache tier (e.g. Redis — a fast in-memory store that keeps copies of hot data), a search tier, message queues (buffers where one service drops work for another to pick up later).

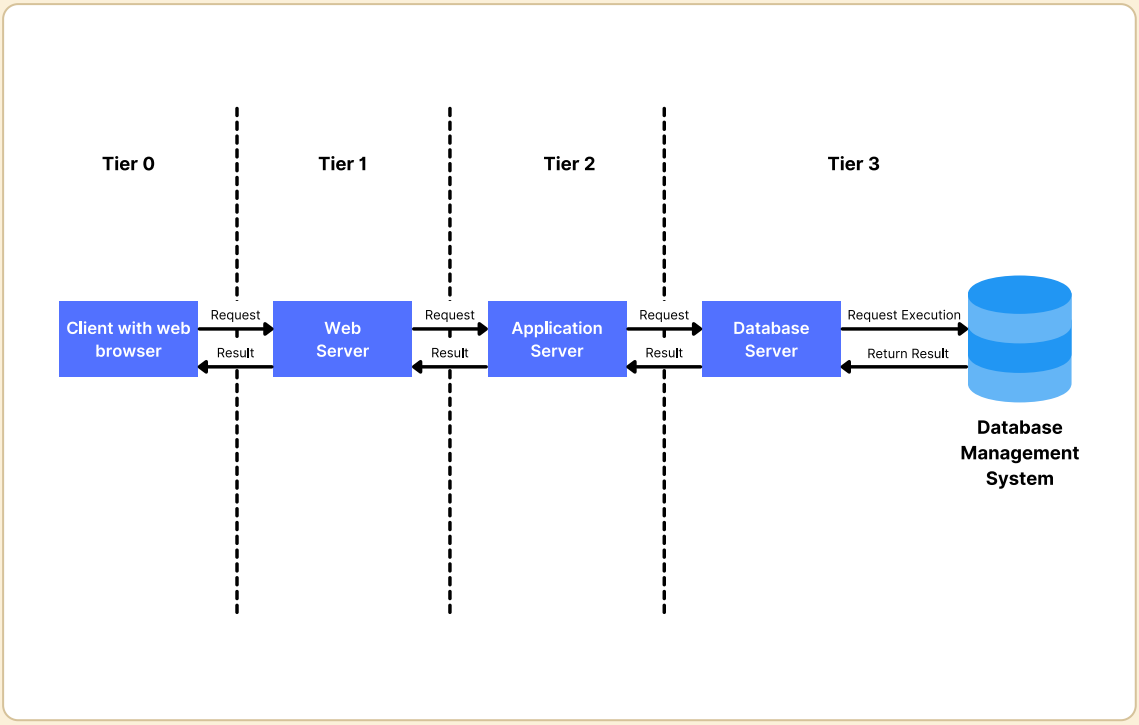


Figure — N-tier: extra service, integration and caching layers (course diagram).

- **Good:** layers scale, deploy and fail independently. **Bad:** harder to operate and debug; more network hops. Microservices (an app split into many small, independently deployed services) are an extreme form.

Cloud-hosted databases

The database runs on a cloud provider's infrastructure. Three service models:

Model	You manage	Provider manages	Example
IaaS (Infrastructure as a Service)	OS, DBMS install, backups, upgrades	hardware, virtual machines (software-simulated computers)	PostgreSQL you install on an AWS EC2 or Azure VM
PaaS / DBaaS (Platform / Database as a Service)	schema, queries, some settings	install, patching, backups, failover (switching to a standby copy when the main one dies)	Amazon RDS, Google Cloud SQL, Azure Database for PostgreSQL
SaaS (Software as a Service)	only your data inside the app	everything, including the database	Salesforce, Google Workspace

- **Good:** elastic capacity (grow or shrink on demand), less operational work, global reach. **Bad:** depends on the network; cost can grow quietly; data location must satisfy compliance (data-residency) rules.

Distributed databases

Data is spread over several machines, often in several regions. Each machine in such a group is called a **node**.

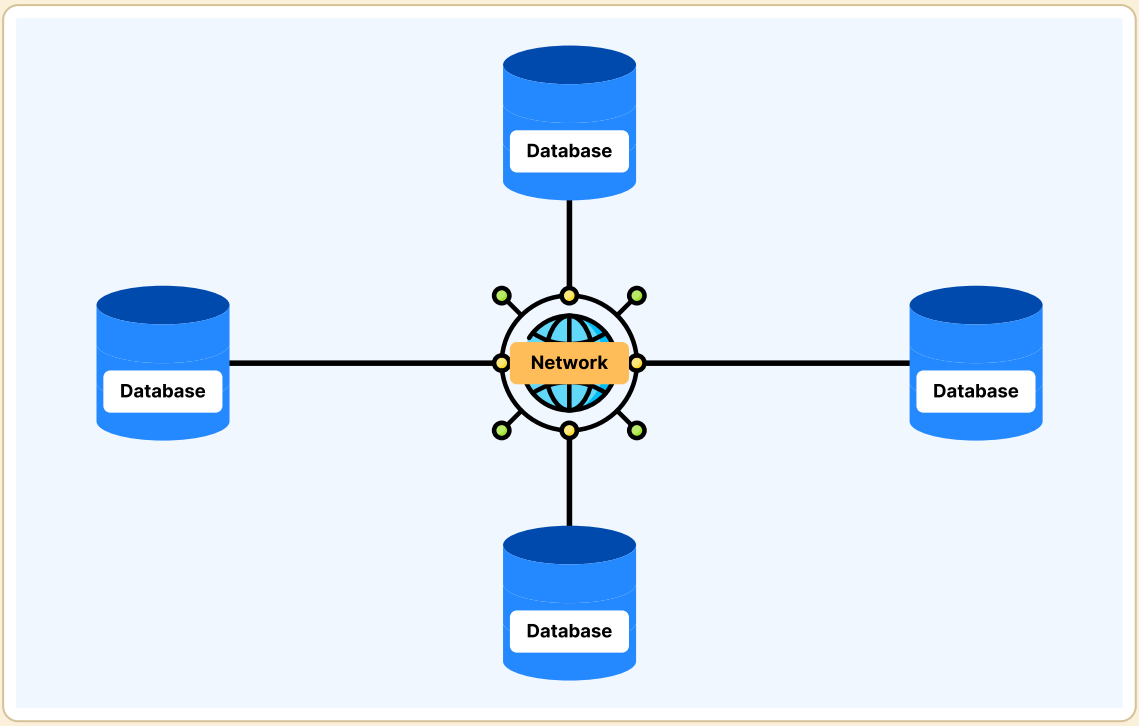


Figure — A distributed database: nodes in different locations, data partitioned and replicated (course diagram).

- **Partitioning / sharding** — split data into pieces, each on a different node (e.g. customers A–M on node 1, N–Z on node 2).
- **Replication** — keep copies (**replicas**) of the same data on several nodes for availability (staying up when a machine fails) and read scaling (spreading reads over the copies). See System Design Fundamentals for more on sharding and replication.
- **Consistency models** — how quickly all copies agree, from **strong** (every read sees the latest write) to **eventual** (copies converge after a while).
- **Good:** scales past one machine, survives node failures, puts data near users. **Bad:** much harder to reason about; cross-node queries and transactions pay network costs.

All of this is chapter 15 (Scaling databases); see also the DDIA notes (DDIA = the book *Designing Data-Intensive Applications*; `../../../ddia/`).

Self-check

1. *Why is exposing the database directly to thousands of browser clients a bad idea?* → Too many connections, credentials on clients, and business rules duplicated in every client; a middle tier pools connections and centralises rules.
2. *Amazon RDS is which cloud model?* → PaaS / managed database service.
3. *Partitioning vs replication?* → Partitioning splits different data across nodes; replication copies the same data to several nodes.

1.6 The relational model

What it is

The **relational model** stores *all* data in tables and connects tables using **values they share**, not pointers or file addresses. Codd proposed it in 1970 because the navigational databases of the day forced programs to follow stored pointers, so reorganising data meant rewriting programs. His proposal: represent everything as plain values in tables and leave the retrieval strategy to the system — the query states the goal, the DBMS finds the route.

A common misconception: "relational" does **not** come from relationships between tables. It comes from **relation**, the mathematical word for a table (a set of tuples).

The vocabulary

Formal term	Everyday term	Meaning
Relation	table	a named set of rows that all have the same columns
Tuple	row, record	one item — one employee, one order
Attribute	column, field	one property — a name, a price
Domain	data type (+ rules)	the allowed values for a column (a *data type* is the kind of value: integer, text, date…)
Degree	number of columns	`departments` has degree 4
Cardinality	number of rows	`departments` has cardinality 6
Schema	table/database definition	the list of tables, columns, types and constraints

(Careful: "cardinality" also has a second meaning in ER modelling — one-to-many, many-to-many — covered in chapter 10.)

The four rules every table obeys

1. **Same columns in every row.** No row has extra private fields. A missing value is stored as `NULL`, meaning "unknown" or "not applicable" (chapter 2 treats `NULL` in depth).
2. **One value per cell (atomic values).** Not a list, not a nested record. This is the heart of *First Normal Form* (chapter 11).
3. **Row order means nothing.** The table is a set; if you need an order, store it in a column and ask for it with `ORDER BY`.
4. **No duplicate rows — in theory.** A mathematical set cannot contain the same element twice. SQL is looser: a table without a primary key or unique constraint will happily store identical rows, which is why good designs always declare one.

Keys and referential integrity

- A **primary key (PK)** is a column (or set of columns) whose value identifies exactly one row; it may not repeat and may not be `NULL`. Example: `customers.customer_id`.

- A **foreign key (FK)** is a column whose values must match a primary (or unique) key in another table. Example: `orders.customer_id` → `customers.customer_id`.
- **Referential integrity** is the DBMS rule that every foreign-key value points at a real row. Inserting an order for a customer who doesn't exist is rejected. Deleting a customer who still has orders is either refused or **cascades** (automatically repeats the delete on the dependent rows) to delete the orders too, depending on the `ON DELETE` rule chosen when the table was created (chapter 9).
- **Entity integrity** is the rule that a primary key is never `NULL`.

Here the sample database enforces all of these (errors are deliberate):

```
-- code/04_integrity.sql
INSERT INTO orders (order_id, customer_id, placed_at, status)
VALUES (200, 99, '2025-07-01 10:00', 'pending');          -- no customer 99
DELETE FROM customers WHERE customer_id = 1;               -- customer 1 has orders
INSERT INTO orders (order_id, customer_id, placed_at, status)
VALUES (201, 1, '2025-07-01 10:00', 'lost');              -- not an allowed status
INSERT INTO departments VALUES (1, 'Legal', 'Pune', 1000);  -- duplicate key
INSERT INTO departments VALUES (NULL, 'Legal', 'Pune', 1000);
SELECT count(*) AS orders_still FROM orders;
```

Output:

```
ERROR:  insert or update on table "orders" violates foreign key constraint "orders_customer_id_fkey"
DETAIL:  Key (customer_id)=(99) is not present in table "customers".
ERROR:  update or delete on table "customers" violates foreign key constraint "orders_customer_id_fkey" on table
"orders"
DETAIL:  Key (customer_id)=(1) is still referenced from table "orders".
ERROR:  new row for relation "orders" violates check constraint "orders_status_check"
DETAIL:  Failing row contains (201, 1, null, 2025-07-01 10:00:00, lost, null).
ERROR:  duplicate key value violates unique constraint "departments_pkey"
DETAIL:  Key (dept_id)=(1) already exists.
ERROR:  null value in column "dept_id" of relation "departments" violates not-null constraint
DETAIL:  Failing row contains (null, Legal, Pune, 1000.00).
 orders_still
--------------
           14
(1 row)
```

No application code checked anything — the rules live in the schema, so they hold for every program that connects.

Connecting tables with a join

A **join** combines rows from two tables whose shared values match. Joins are the everyday workhorse of relational querying (chapter 5 covers every kind). Which products did customer 1 buy?

```
-- code/01_first_queries.sql (part)
SELECT c.first_name, c.last_name, o.order_id, p.name AS product
FROM customers   c
JOIN orders      o  ON o.customer_id = c.customer_id
JOIN order_items oi ON oi.order_id   = o.order_id
JOIN products    p  ON p.product_id  = oi.product_id
WHERE c.customer_id = 1
ORDER BY o.order_id, p.name;
```

Output:

```
 first_name | last_name | order_id |       product
------------+-----------+----------+---------------------
 Anita      | Shah      |      101 | Laptop Pro 14
 Anita      | Shah      |      101 | Wireless Mouse
 Anita      | Shah      |      104 | Ergonomic Chair
 Anita      | Shah      |      104 | Standing Desk
 Anita      | Shah      |      113 | Laptop Pro 14
 Anita      | Shah      |      113 | Mechanical Keyboard
(6 rows)
```

Nothing in the tables "points" anywhere: the value `1` in `orders.customer_id` and the value `1` in `customers.customer_id` are the whole connection.

Because queries name only tables and columns, the model gives data independence and lets the optimizer choose (and later improve) the method; because it is mathematical, it also gives exact tests for design flaws — the normal forms of chapter 11.

Common mistakes

- Relying on "the order rows came back in" without `ORDER BY` — it can change at any time.
- Storing lists in one cell (`'red,green,blue'`), which breaks rule 2 and makes querying painful.
- Tables without a primary key, which lets exact duplicates creep in.

In the interview

> **30-second answer:** "The relational model, from Codd in 1970, stores data as relations — tables of tuples with typed attributes — and links them only by values: a foreign-key column stores the key of a row in another table, and the DBMS guarantees that row exists. Rows are unordered and cells hold single values. Because queries are declarative and never mention storage, the DBMS can optimise and reorganise freely."

- *Where does the word 'relational' come from?* → From *relation*, the mathematical name for a table — not from relationships.
- *What happens if you delete a parent row that has children?* → Depends on the FK's `ON DELETE` action: `NO ACTION`/`RESTRICT` (error, the default), `CASCADE`, `SET NULL` or `SET DEFAULT`.

Self-check

1. *Degree and cardinality of `products`?* → Degree 5 (columns), cardinality 9 (rows).
2. *Why is a primary key not allowed to be NULL?* → It must identify a row; an unknown identifier identifies nothing (entity integrity).
3. *Which rule does storing `'555-1234, 555-9876'` in one phone column break?* → One value per cell (First Normal Form).

1.7 RDBMS — relational database management systems

What it is

An **RDBMS** is a DBMS that implements the relational model and speaks SQL. Its defining features:

Feature	What it gives you
Tables of rows and columns	one uniform structure for all data
Primary and foreign keys	identity and enforced relationships
SQL	one standard language to define, change, query and secure data
Transactions with **ACID** guarantees	Atomicity, Consistency, Isolation, Durability (defined in 1.8, in depth in chapter 13)
Integrity constraints	types, `NOT NULL`, `UNIQUE`, `CHECK`, keys — bad data is refused at the door
Separation of logical and physical	data independence (1.3)

Advantages: the same tables answer countless questions without being reorganised, SQL is far shorter than file-access code, ACID and constraints keep data correct, and modern products handle large volumes and many concurrent users.

The main products

Product	Licence	Known for	Notes for this book
PostgreSQL	open source (PostgreSQL licence)	standards compliance, correctness, extensibility (JSONB = stored JSON documents, PostGIS = maps and geography, custom types)	this book's main dialect
MySQL (and fork MariaDB — a copy that became its own project)	open source (GPL, a licence that keeps derived code open) + commercial	ubiquity on the web, simple operations	the SQL course's original dialect; differences noted per chapter
Oracle Database	commercial	enterprise features, PL/SQL (Oracle's procedural programming language)	mentioned where syntax differs notably
Microsoft SQL Server	commercial	Microsoft ecosystem integration, T-SQL (Transact-SQL, its SQL dialect)	differences noted in small tables
SQLite	public domain	serverless (no separate server program): a whole database in one file, embedded in apps, phones and browsers	`*.sqlite.sql` examples run in `sqlite3`

Self-check

1. *What makes SQLite different from the others?* → No separate server process; a library that stores the whole database in a single file.

1.8 Relational vs non-relational databases

What it is

Non-relational databases store data in shapes other than tables. The umbrella name is **NoSQL**; people now tend to expand it as "**not only SQL**", since many such products later gained SQL-flavoured query languages. Neither family is "better": the shape of the data decides which questions are cheap.

Picture a food-delivery app. Orders and payments fit tables perfectly. Restaurant menus do not: one restaurant has thirty dishes with sizes and toppings, another four fixed meals, a third rewrites its menu weekly. That is the moment teams consider a second kind of store.

The four NoSQL families

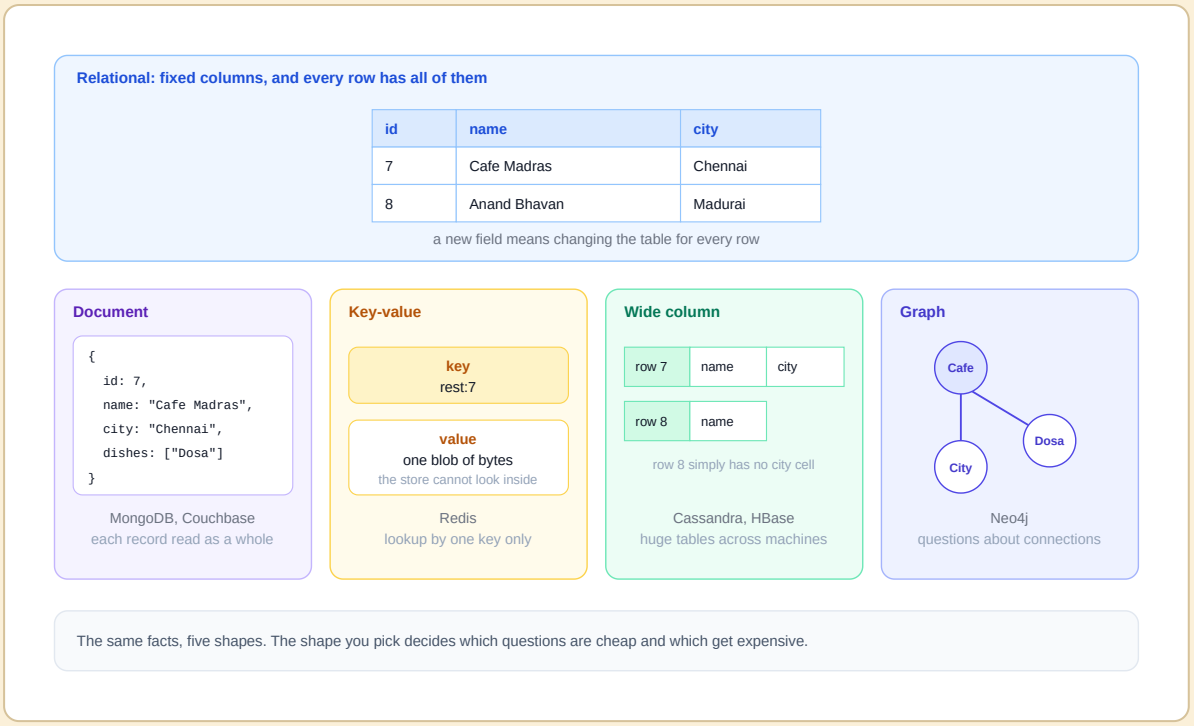


Figure — One dataset, four non-relational shapes, next to the relational version (course diagram).

Family	Data shape	Strength	Weakness	Examples	Typical use
Document	self-contained JSON/BSON (or XML) documents (JSON = text of nested `{"key": value}` pairs; BSON = binary JSON; XML = tag-based text); documents in one collection may differ in fields	reading/writing one whole object; varied shapes	cross-document joins are limited	MongoDB, Couchbase, CouchDB	catalogs, content, user profiles, menus
Key-value	a value stored under a key; mainly `put` and `get`	extremely fast lookups by key	cannot search inside values	Redis, Amazon DynamoDB, Riak	caches, sessions (a logged-in user's temporary state), feature flags (on/off switches for app features)
Wide-column (column-family)	rows grouped into column families; rows may have different columns; partitioned by key	very high write throughput (writes per second) across many machines	queries must follow the partition key (the column that decides which machine holds a row)	Apache Cassandra, HBase	time series (values recorded over time), event logs, IoT (Internet of Things — sensors and devices)
Graph	nodes, edges and properties	multi-hop relationship questions ("friends of friends who…")	not built for bulk aggregation	Neo4j, Amazon Neptune	social networks, recommendations, fraud detection

Chapter 16 (Beyond relational) builds each of these in practice.

ACID vs BASE

ACID — the guarantees of a relational transaction:

Letter	Guarantee
Atomicity	all operations in the transaction happen, or none do
Consistency	after a transaction finishes, every declared rule (keys, checks, constraints) is still satisfied
Isolation	transactions running side by side behave as if they ran alone; half-done changes stay invisible to others
Durability	once committed, changes survive crashes and power loss

BASE — how many distributed NoSQL systems describe themselves:

Letter(s)	Meaning
Basically Available	requests still get answers while some nodes are failing, though the data returned may be stale (out of date)
Soft state	stored state may keep changing without new writes as replicas catch up
Eventual consistency	if writes stop, all replicas converge; until then two readers may see different values

BASE gives up immediate agreement to stay available — acceptable for a product catalogue, unacceptable for an account balance.

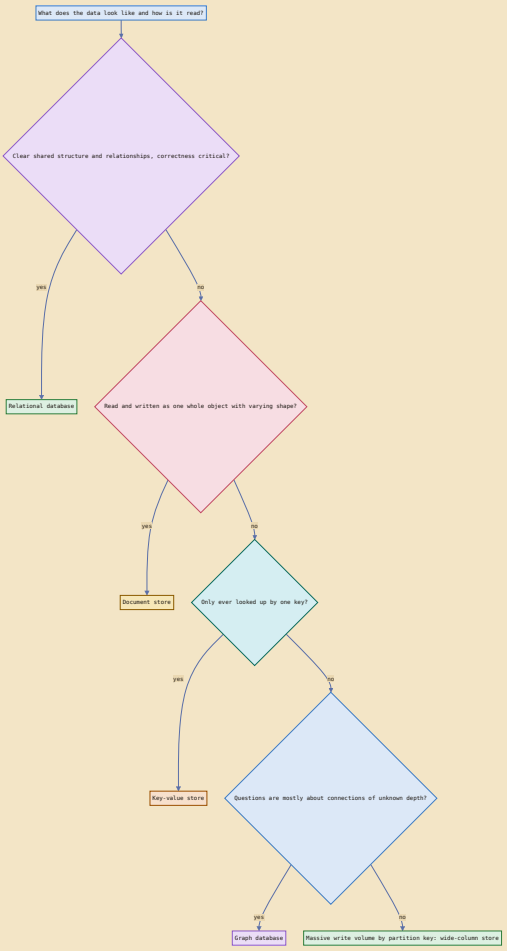
Three claims that need care

1. **"Relational databases only scale vertically."** *False.* **Vertical scaling** = a bigger machine; **horizontal scaling** = more machines. Relational systems scale out with read replicas (read-only copies that share the query load), partitioning/sharding and distributed SQL (CockroachDB, Google Spanner, YugabyteDB, Citus for PostgreSQL). The real issue is *cost*: cross-machine joins and transactions are expensive, and NoSQL models avoid them by design.
2. **"NoSQL databases don't support transactions."** *Out of date.* MongoDB has had multi-document ACID transactions since version 4.0 (2018); DynamoDB added transactions in 2018; Cassandra offers lightweight (compare-and-set: "write only if the value is still what I last saw") transactions per partition. Limits and costs vary, so check the specific product.
3. **"NoSQL databases have no schema."** *Misleading.* The database doesn't *enforce* one, but your code still expects `price` to be a number — the schema has moved into the application. The names: **schema-on-write** (the database checks data as it's written — relational) vs **schema-on-read** (the reader interprets whatever is stored). Schema-on-read makes change cheap and bad data easy.

Comparison

Aspect	Relational	Non-relational
Schema	checked by the DBMS as rows are written (schema-on-write)	mostly checked by application code when reading (schema-on-read)
Query language	SQL, standardised	product-specific (MongoDB's query API, Cassandra's CQL, Neo4j's Cypher…), some SQL-like
Joins	built in, optimised	few or none; applications join in code or embed related data instead
Transactions	full ACID across tables	available in many products, often scoped (one document / one partition) or slower
Best fit	structured, related data where correctness matters (orders, payments, bookings, inventory, ERP, CRM)	huge volumes, varied/changing record shapes, one dominant access pattern (catalogues, caches, feeds, telemetry, real-time analytics)

How to choose



Real systems often combine several: orders and payments in PostgreSQL, menus in a document store, sessions in Redis. Using different stores for different jobs is **polyglot persistence**. It is not free — every extra store must be run, backed up, monitored and kept in step with the others — so start with one database (a relational one is the safe default) and add another only when a concrete requirement demands it.

In the interview

> **30-second answer:** "I pick from the access pattern, not the fashion. Orders need multi-table transactions and constraints, so relational. Menus differ per restaurant and are fetched as one unit, which suits a document store. Sessions are key lookups, so Redis. Relational databases do scale horizontally — replicas, sharding, distributed SQL — the real cost is cross-node joins and transactions."

- *Why not one database for everything?* → Often you should! Each extra store adds operational cost and consistency problems between stores; add one only when a requirement justifies it.

Self-check

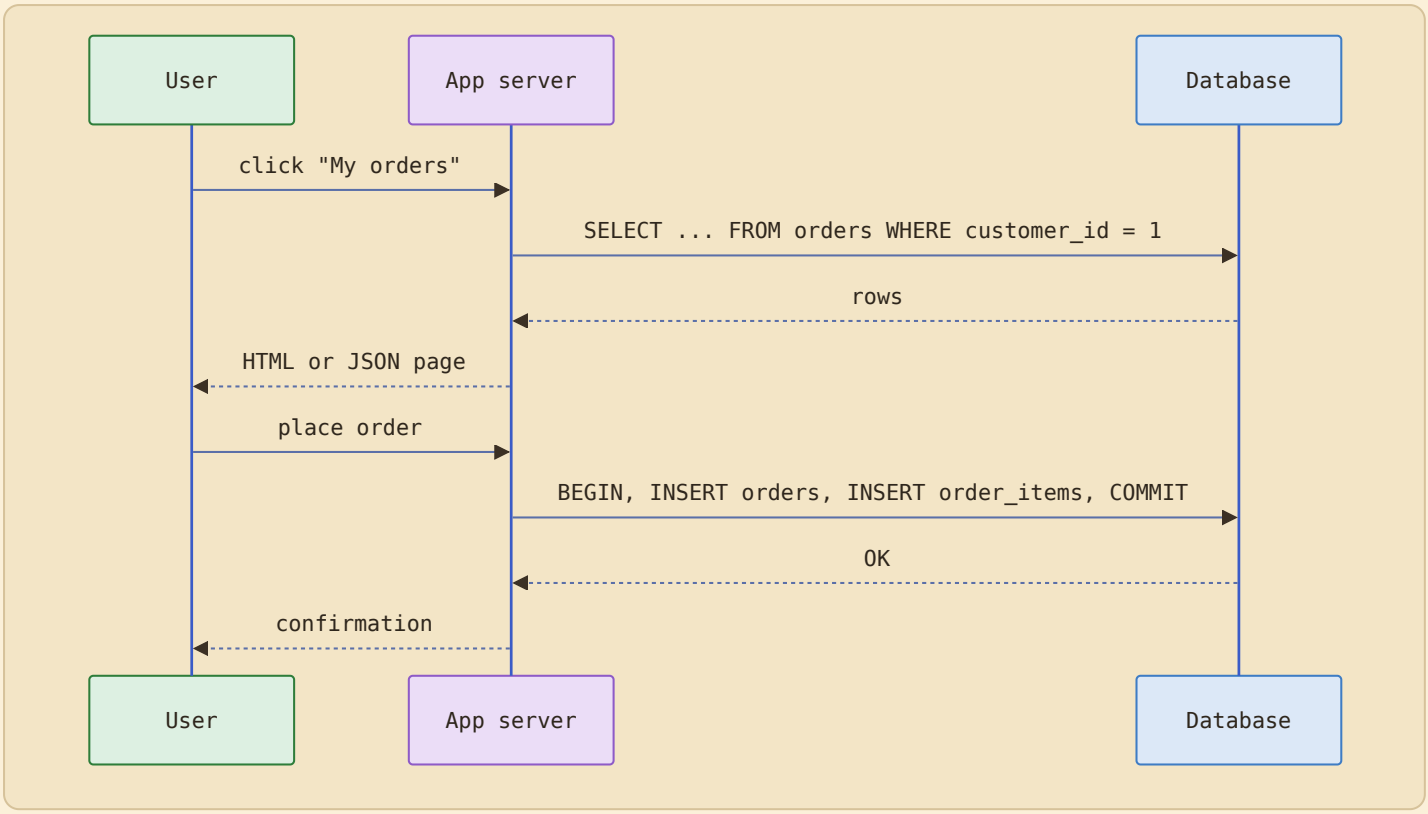
1. *A feature-flag service reads flags by name millions of times per second. Which family?* → Key-value.
2. *"Who are the friends of my friends who also like jazz?" Which family?* → Graph.
3. *Horizontal vs vertical scaling?* → More machines vs a bigger machine.

1.9 What SQL is

What it is

SQL (Structured Query Language) is the standard language for defining, changing, querying and securing data in relational databases. It is **declarative**: you describe the result you want, not the steps to compute it. Developed at IBM in the 1970s as SEQUEL (renamed SQL for trademark reasons), it became an ANSI standard in 1986 and an ISO standard in 1987; the current edition is **SQL:2023**. Every RDBMS implements the standard's core plus its own extensions — a **dialect** (PostgreSQL's, MySQL's, T-SQL for SQL Server, PL/SQL-flavoured Oracle SQL).

SQL is the traffic between applications and databases:



The five sub-languages

SQL statements are traditionally grouped into five families. (Some texts fold DQL into DML; interviewers accept either, but know both.)

Sub-language	Purpose	Main statements
DDL — Data Definition Language	define and change *structure*	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `COMMENT`, `RENAME` (via `ALTER`)
DML — Data Manipulation Language	change *data*	`INSERT`, `UPDATE`, `DELETE`, `MERGE` (PostgreSQL 15+), `COPY` (PostgreSQL)
DQL — Data Query Language	read data	`SELECT`
DCL — Data Control Language	control *access*	`GRANT`, `REVOKE` (and role management such as `CREATE ROLE`)
TCL — Transaction Control Language	group changes into transactions	`BEGIN` / `START TRANSACTION`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`, `ROLLBACK TO SAVEPOINT`, `RELEASE SAVEPOINT`

A frequent mix-up: **granting and revoking permissions is DCL, not TCL.** TCL is only about transaction boundaries (commit, rollback, savepoints).

How it works — one statement of each kind

```sql
-- code/03_sql_sublanguages.sql
-- DDL: define structure
CREATE TABLE suppliers (
    supplier_id integer PRIMARY KEY,
    name        text NOT NULL,
    country     text
);

-- DML: change data
INSERT INTO suppliers VALUES (1, 'Deskworks', 'India'), (2, 'Clicky Ltd', 'UK');
UPDATE suppliers SET country = 'Ireland' WHERE supplier_id = 2;

-- DQL: read data
SELECT * FROM suppliers ORDER BY supplier_id;

-- DCL: control who may do what
CREATE ROLE sql01_report_reader;
GRANT SELECT ON suppliers TO sql01_report_reader;
SELECT grantee, privilege_type
FROM information_schema.role_table_grants
WHERE table_name = 'suppliers' AND grantee = 'sql01_report_reader';
REVOKE SELECT ON suppliers FROM sql01_report_reader;

-- TCL: group changes into a transaction, with a savepoint
BEGIN;
INSERT INTO suppliers VALUES (3, 'Chairmakers', 'India');
SAVEPOINT before_mistake;
DELETE FROM suppliers;                -- oops: no WHERE clause
ROLLBACK TO SAVEPOINT before_mistake;  -- undo only the delete
COMMIT;
SELECT * FROM suppliers ORDER BY supplier_id;
```

Output (psql echoes a *command tag* such as `INSERT 0 2` after each change — `0` is a legacy OID (object identifier) field, `2` is the number of rows):

```
CREATE TABLE
INSERT 0 2
UPDATE 1
 supplier_id |    name    | country
-------------+------------+---------
           1 | Deskworks  | India
           2 | Clicky Ltd | Ireland
(2 rows)

CREATE ROLE
GRANT
       grantee       | privilege_type
---------------------+----------------
 sql01_report_reader | SELECT
(1 row)

REVOKE
BEGIN
INSERT 0 1
SAVEPOINT
DELETE 3
ROLLBACK
COMMIT
 supplier_id |    name     | country
-------------+-------------+---------
           1 | Deskworks   | India
           2 | Clicky Ltd  | Ireland
           3 | Chairmakers | India
(3 rows)
```

The `DELETE 3` wiped every row, but `ROLLBACK TO SAVEPOINT` undid just that step; the insert before the savepoint survived the `COMMIT`.

SQL syntax basics you'll use from chapter 2

Rule	Detail
Statements end with `;`	psql waits for the semicolon before running anything
Keywords are case-insensitive	`select` = `SELECT`; this book capitalises keywords for readability
Unquoted identifiers fold to lower case in PostgreSQL	`Employees` means `employees`; `"Employees"` (double quotes) keeps the case exactly
Strings use single quotes	`'Mumbai'`; a quote inside a string is doubled: `'O''Brien'`
Comments	`-- to end of line` and `/* block */`
Whitespace is free	break long queries over lines

Dialect differences at a glance

Feature	PostgreSQL	MySQL	SQLite	SQL Server
Limit rows	`LIMIT n` / `FETCH FIRST n ROWS ONLY`	`LIMIT n`	`LIMIT n`	`TOP n` / `OFFSET … FETCH`
Identifier quotes	`"name"`	`` `name` `` (or `"` in ANSI mode)	`"name"`	`[name]` or `"name"`
String concat	`||`, `concat()`	`concat()` (`||` is OR by default)	`||`	`+`, `concat()`
Auto-increment	`GENERATED … AS IDENTITY` / `serial`	`AUTO_INCREMENT`	`INTEGER PRIMARY KEY`	`IDENTITY(1,1)`
DDL inside transactions	yes, can be rolled back	no (implicit commit)	yes	mostly yes

Self-check

1. *Classify:* *`TRUNCATE`*, *`GRANT`*, *`SAVEPOINT`*, *`MERGE`*, *`SELECT`*. → DDL, DCL, TCL, DML, DQL.
2. *What does "declarative" mean for SQL?* → You state the result; the DBMS decides the algorithm.
3. *Why did `ROLLBACK TO SAVEPOINT` not remove supplier 3?* → The insert happened before the savepoint; only work after the savepoint is undone.

TL;DR

SQL is the declarative, standardised language of relational databases, split into DDL (structure), DML (changes), DQL (queries), DCL (permissions) and TCL (transactions). Each product speaks a dialect; this book uses PostgreSQL's.

1.10 Your private PostgreSQL server and psql

What it is

The book ships a small script, `tools_pg.sh`, in the project root. It runs a **private PostgreSQL 14 server** that belongs only to this book: its data lives in `~/.pg-books-system-design/` (outside the iCloud-synced Documents folder), it listens on port **5544** (a *port* is a numbered network door a server waits on) through a Unix socket in `/tmp` (a special file programs on the same machine use to talk to each other), and it never touches any other PostgreSQL you may have installed (which normally uses port 5432). **psql** is PostgreSQL's command-line client (a text-only program run in a terminal): you type SQL, it prints results.

How it works — step by step

Run these from the project root (`system_design_notes/`).

1. **Start the server** (the first run also creates the data directory):

   ```
   bash tools_pg.sh start
   ```

   ```
   postgres ready on port 5544 (socket /tmp)
   ```

2. **Create a database** for the sample data:

   ```
   bash tools_pg.sh psql -d postgres -c "CREATE DATABASE company ENCODING 'UTF8' TEMPLATE template0"
   ```

   ```
   CREATE DATABASE
   ```

   An **encoding** is the rule for turning characters into bytes; **UTF-8** is the universal one that covers every language. Asking for `UTF8` explicitly guarantees that accented and non-Latin text (`café`, `मुंबई`) is stored and measured correctly, whatever encoding the server was initialised with; `TEMPLATE template0` is required when the encoding differs from the default template's.

3. **Load the sample schema** (section 1.11):

   ```
   bash tools_pg.sh psql -d company -q -f sql/chapters/01-databases-and-the-relational-model/code/00_schema.sql
   ```

4. **Check it**:

   ```
   bash tools_pg.sh psql -d company -c '\conninfo' -c 'SELECT count(*) FROM employees'
   ```

   ```
   You are connected to database "company" as user "postgres" via socket in "/tmp" at port "5544".
    count
   -------
       15
   (1 row)
   ```

5. **Open an interactive session**: `bash tools_pg.sh psql -d company`. The prompt `company=#` means "connected to `company` as a superuser" (an account allowed to do everything) (`=>` for ordinary users). If you press Enter before the `;`, the prompt changes to `company-#` — psql is waiting for the rest of the statement. Type `\q` to leave.

6. **Stop the server** when you're done: `bash tools_pg.sh stop`.

`tools_pg.sh psql` is a thin wrapper around `psql -h /tmp -p 5544 -U postgres -X`: `-h /tmp` uses the socket directory, `-p` the port, `-U` the user, and `-X` skips any personal `~/.psqlrc` so output looks the same for every reader.

psql meta-commands

Lines starting with a backslash are **meta-commands**: psql handles them itself instead of sending them to the server. They do not need a semicolon.

Command	Does
`\?` / `\h SELECT`	help on meta-commands / SQL syntax help for a statement
`\l`	list databases
`\c company`	connect to another database
`\conninfo`	show current connection
`\dt` / `\dt+`	list tables (with sizes)
`\d employees`	describe a table: columns, types, indexes, constraints, references
`\dn`, `\df`, `\dv`, `\di`, `\du`	schemas, functions, views, indexes, roles
`\x` (or `\x auto`)	expanded display: one field per line, good for wide rows
`\pset null '(null)'`	show NULLs visibly instead of as blanks (used throughout this book)
`\timing`	print how long each statement took
`\i file.sql`	run a file of SQL
`\e`	edit the last query in your `$EDITOR` (the environment variable naming your preferred text editor)
`\copy`	import/export CSV from your machine (client side)
`\q`	quit

Meta-commands also work inside script files:

```sql
-- code/05_psql_meta_commands.sql
\dt
\d departments
\x on
SELECT * FROM employees WHERE emp_id = 7;
\x off
```

Output (abridged):

```
               List of relations
 Schema |       Name        | Type  |  Owner
--------+-------------------+-------+----------
 public | customers         | table | postgres
 public | departments       | table | postgres
 ...
 public | projects          | table | postgres
(8 rows)

                Table "public.departments"
  Column  |     Type      | Collation | Nullable | Default
----------+---------------+-----------+----------+---------
 dept_id  | integer       |           | not null |
 name     | text          |           | not null |
 location | text          |           |          |
 budget   | numeric(12,2) |           | not null |
Indexes:
    "departments_pkey" PRIMARY KEY, btree (dept_id)
    "departments_name_key" UNIQUE CONSTRAINT, btree (name)
Check constraints:
    "departments_budget_check" CHECK (budget >= 0::numeric)
Referenced by:
    TABLE "employees" CONSTRAINT "employees_dept_id_fkey" FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
    TABLE "projects" CONSTRAINT "projects_dept_id_fkey" FOREIGN KEY (dept_id) REFERENCES departments(dept_id)

Expanded display is on.
-[ RECORD 1 ]--+--------------------------
emp_id         | 7
first_name     | Priya
last_name      | Nair
email          | priya.nair@company.example
...
hire_date      | 2020-02-10

Expanded display is off.
```

Running the book's code

- Run **one file** against your `company` database: `bash tools_pg.sh psql -d company -f sql/chapters/02-select-basics/code/02_where.sql`. (Files that change data, like 1.3's demo, are best run on a fresh copy: re-run step 3 to reset.)
- Run **every chapter's code** the way the book was checked: `bash tools_run_sql.sh sql`. For each file it creates a fresh database, loads that chapter's `00_schema.sql`, runs the file, and prints `OK` or `FAIL`.

Common mistakes

- Forgetting the `;` — psql shows `company-#` and waits. Type `;` and Enter.
- Typing SQL at the shell prompt (`$`) instead of inside psql.
- Connecting to the wrong server: always go through `tools_pg.sh psql` (port 5544), not a bare `psql` (default port 5432).
- A typo inside a transaction puts it in an aborted state (`current transaction is aborted…`); type `ROLLBACK;` to start again.

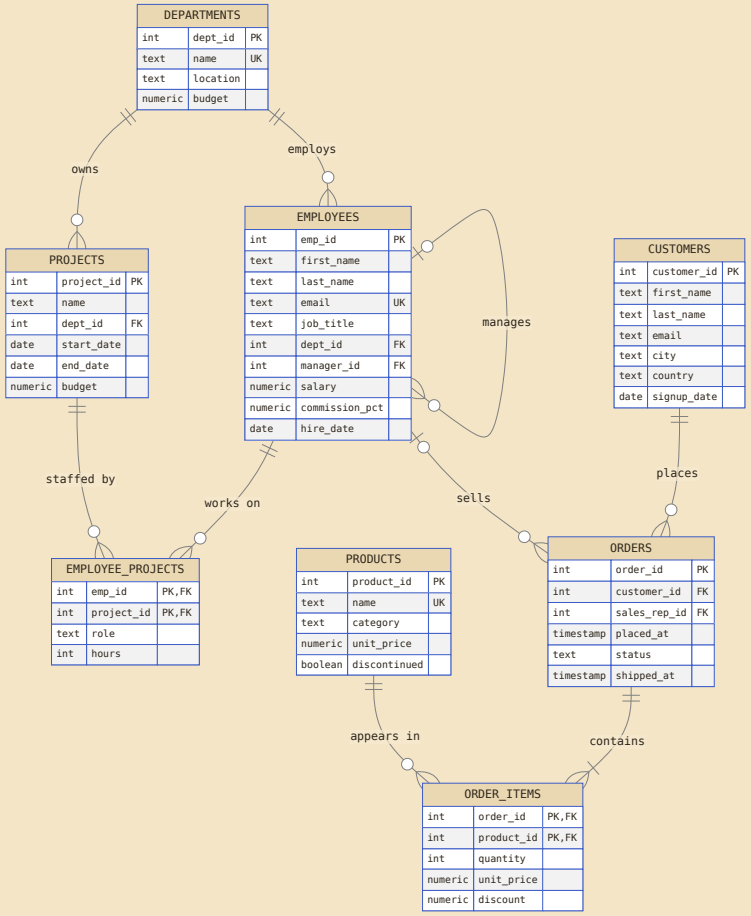
TL;DR

`bash tools_pg.sh start` → create the UTF-8 `company` database → load `00_schema.sql` → `bash tools_pg.sh psql -d company`. Use `\dt`, `\d table`, `\x`, `\pset null`, `\timing`, `\q`. `tools_run_sql.sh sql` runs all examples on fresh databases.

1.11 The sample database

What it is

Chapters 2 onward query one small, realistic dataset: `company`, a fictional firm that sells office equipment. It has staff organised in departments and projects, and a small shop side with customers, products and orders. Every chapter's `code/00_schema.sql` is a copy of `code/00_schema.sql`, so you can load it once and follow along.



(Columns that may be NULL are listed in the table below. How to read the crow's-foot symbols — `||` exactly one, `o{` zero or more, `|{` one or more, `|o` zero or one — is explained in chapter 10.)

The tables

Table	Rows	What a row is	Notable columns
`departments`	6	a department	`location` may be NULL, `budget`
`employees`	15	a member of staff	`manager_id` → another employee (a self-reference: a foreign key pointing at its own table), `commission_pct` only for some sales staff
`projects`	5	an internal project owned by a department	`end_date` NULL = still running
`employee_projects`	13	"employee X works on project Y" (a many-to-many link)	composite key (a key made of two or more columns together) `(emp_id, project_id)`, `role`, `hours`
`customers`	10	a shop customer	`city`, `email` may be NULL
`products`	9	a product for sale	`category` may be NULL, `discontinued`
`orders`	14	an order header	`placed_at`, `status` ∈ pending / shipped / delivered / cancelled, `shipped_at`, `sales_rep_id` NULL for online orders
`order_items`	22	one product line in an order	`quantity`, `unit_price` at time of sale, `discount` (0.10 = 10 %)

Deliberate quirks (so later chapters have something to find)

Quirk	Where	Useful for
CEO (Asha) and a new intern (Rohan) have no department	`employees.dept_id` NULL	NULL handling, outer joins
Research department has no employees; Support has no location	`departments`	outer joins (keep unmatched rows), anti-joins (find rows with *no* match), NULL sorting
Salary ties: Rahul & Meera 98,000; Arjun & Sara 68,000	`employees.salary`	`DISTINCT`, `WITH TIES`, ranking functions
Missing emails (Meera, Ananya) and commissions	`employees`	`IS NULL`, `COALESCE`, `count(col)`
One email stored in mixed case (`Neha.Gupta@Company.example`)	`employees.email`	case-insensitive matching
Anita Shah entered twice, second time with spaces and capitals in the email and city `mumbai`	`customers` 1 and 8	de-duplication, `trim`, `lower`
Customers 9 and 10 never ordered; product "Desk Lamp" never sold; "Gift Card" has no category	`customers`, `products`	anti-joins, NULL groups
Orders across Jan–Jun 2025, one at 23:55 on 31 January	`orders.placed_at`	date ranges, month boundaries
Pending/cancelled orders have no `shipped_at`	`orders`	NULLs in date arithmetic

A first look:

```sql
SELECT * FROM departments;
```

Output:

```
 dept_id |    name     | location  |  budget
---------+-------------+-----------+-----------
       1 | Engineering | Bengaluru | 900000.00
       2 | Sales       | Mumbai    | 400000.00
       3 | Marketing   | Mumbai    | 250000.00
       4 | Finance     | Pune      | 200000.00
       5 | Support     | (null)    | 150000.00
       6 | Research    | Pune      | 300000.00
(6 rows)
```

Key terms (chapter 1)

Every term is defined where it is first used (in **bold**). For flash-card revision: file processing system, redundancy/inconsistency, lost update, metadata, catalog/data dictionary, DBMS, database system, data independence (physical/logical), buffer pool, page, WAL, transaction, ACID, BASE, relation/tuple/attribute/domain, degree, cardinality, primary key, foreign key, entity and referential integrity, schema-on-write/-on-read, horizontal/vertical scaling, polyglot persistence, DDL/DML/DQL/DCL/TCL, declarative. The book-wide glossary is in chapter 20.

Chapter 1 TL;DR

- File-based systems suffered seven problems (redundancy, isolation, no queries, rules in code, lost updates, no recovery, coarse security); the **DBMS** fixes them by mediating every access.
- A **database** is related, self-describing (catalog), purposeful, persistent data; the **DBMS** is the software that **defines, constructs, manipulates, shares, protects and maintains** it, with **physical and logical data independence**.
- Inside: **parser → optimizer → executor**, a **storage engine** with pages, a **buffer pool**, **transaction/lock managers**, a **recovery manager** with a write-ahead log, and a **catalog**.
- Architectures: two-tier, three-tier, n-tier, cloud (IaaS/PaaS/SaaS), distributed (partitioning + replication).
- The **relational model**: relations of tuples and attributes; same columns per row, atomic cells, no row order, no duplicates; **primary keys** identify, **foreign keys** link, the DBMS enforces **referential integrity**.
- NoSQL families (document, key-value, wide-column, graph) trade enforced structure and joins for flexible shapes and easier scale-out; choose by access pattern.
- **SQL** is declarative and standard: **DDL, DML, DQL, DCL, TCL**.
- Your lab: `bash tools_pg.sh start`, `bash tools_pg.sh psql -d company`; the `company` sample database powers the next chapters.

Next: Chapter 2 — SELECT basics.

Chapter 2 — SELECT Basics: Reading, Filtering, Sorting and Paging

Reading data is most of what anyone does with SQL. This chapter teaches the single-table `SELECT` statement completely: choosing columns, computing new ones, filtering rows with `WHERE`, the operators you can use there, the peculiar but essential rules of `NULL`, removing duplicates with `DISTINCT`, sorting with `ORDER BY`, and returning one page of results with `LIMIT`/`OFFSET` or the standard `FETCH`. Joins (combining tables), grouping and subqueries (queries nested inside another query) build on exactly these pieces in chapters 4–6.

What you'll learn

#	Topic	You will be able to…
2.1	`SELECT` and aliases	pick columns, compute expressions, rename outputs
2.2	`WHERE` and logical operators	filter rows and combine conditions without precedence bugs
2.3	Operators	use arithmetic, comparison, logical and string operators correctly (incl. integer division)
2.4	`BETWEEN`, `IN`, `LIKE`/`ILIKE`, regex	express ranges, lists and text patterns
2.5	`NULL` and three-valued logic	test for missing values and avoid the classic `NOT IN` trap
2.6	`DISTINCT` / `DISTINCT ON`	remove duplicates and keep one row per group
2.7	`ORDER BY`	sort by several keys, control where NULLs go, custom orders
2.8	`LIMIT`/`OFFSET`/`FETCH`	page through results, handle ties, prefer keyset paging
2.9	Logical processing order	know why an alias works in `ORDER BY` but not in `WHERE`

Setup. All examples use the `company` sample database from chapter 1, section 1.11. Files are in `code/`; each starts with `\pset null '(null)'` so missing values print as `(null)` instead of blanks. To follow along: `bash tools_pg.sh psql -d company`, then type `\pset null '(null)'`.

2.1 SELECT: choosing what comes back

What it is

`SELECT` reads rows from a table and returns a new, temporary table called the **result set**. Think of the table as a spreadsheet: `SELECT` chooses which columns you see, and (with `WHERE`, next section) which rows.

```
SELECT column1, column2, ...     -- what to show
FROM   table_name;               -- where to read it
```

How it works

1. `FROM` names the table to read.
2. The `SELECT` list says which columns (or computed values) to return, in which order.
3. `*` is a wildcard meaning "every column, in table order".

All columns:

```
-- code/01_select_columns.sql
SELECT * FROM products;
```

Output:

```
 product_id |            name             |  category   | unit_price | discontinued
------------+-----------------------------+-------------+------------+--------------
          1 | Laptop Pro 14               | Electronics |    1299.00 | f
          2 | Wireless Mouse              | Electronics |      24.99 | f
          3 | USB-C Hub                   | Accessories |      49.50 | f
          4 | Mechanical Keyboard         | Accessories |      89.00 | f
          5 | Standing Desk               | Furniture   |     499.00 | f
          6 | Ergonomic Chair             | Furniture   |     349.99 | f
          7 | Noise-Cancelling Headphones | Electronics |     199.95 | f
          8 | Desk Lamp                   | Furniture   |      35.00 | t
          9 | Gift Card                   | (null)      |      50.00 | f
(9 rows)
```

(`f`/`t` are how psql prints boolean false/true; a **boolean** is a true/false value.)

Chosen columns — list them after `SELECT`, in the order you want:

```
SELECT first_name, last_name, job_title FROM employees;
```

Output (first rows):

```
 first_name | last_name |      job_title
------------+-----------+---------------------
 Asha       | Rao       | CEO
 Vikram     | Menon     | Engineering Manager
 Neha       | Gupta     | Senior Engineer
 Rahul      | Verma     | Engineer
 ...
(15 rows)
```

Expressions and column aliases

The `SELECT` list can hold any **expression** — arithmetic, function calls (a *function* is a named built-in operation, such as `round(x, 2)`, that takes inputs and returns a value), constants — not just column names. An **alias** (`AS new_name`) names the output column. Use double quotes if the alias has spaces or capitals you want to keep.

```
SELECT name,
       unit_price,
       unit_price * 1.18        AS price_with_gst,
       'per unit'               AS note,
       unit_price * 1.18 AS "Price incl. GST"
FROM products
WHERE product_id <= 3;
```

Output:

```
      name      | unit_price | price_with_gst |   note   | Price incl. GST
----------------+------------+----------------+----------+-----------------
 Laptop Pro 14  |    1299.00 |      1532.8200 | per unit |       1532.8200
 Wireless Mouse |      24.99 |        29.4882 | per unit |         29.4882
 USB-C Hub      |      49.50 |        58.4100 | per unit |         58.4100
(3 rows)
```

(GST is India's 18 % goods-and-services sales tax, hence `* 1.18`.) The table is not changed — computed columns exist only in the result. (Rounding to 2 decimals is `round(x, 2)`, chapter 3.)

Table aliases give a table a short name, which you'll need constantly once queries involve several tables. The `AS` keyword is optional for both kinds of alias, but writing it for column aliases avoids confusing bugs (see *Common mistakes*).

```
SELECT e.first_name, e.salary / 12 monthly
FROM employees AS e
WHERE e.emp_id IN (1, 2);
```

Output:

```
 first_name |      monthly
------------+--------------------
 Asha       | 20833.333333333333
 Vikram     | 15416.666666666667
(2 rows)
```

SELECT without FROM — PostgreSQL (and MySQL, SQLite, SQL Server) let you evaluate expressions directly, which makes psql a handy calculator. Oracle required `FROM dual` (a built-in one-row dummy table) until version 23ai made `FROM` optional. Below, `now()` returns the current date and time, and `::date` **casts** it — converts it to another type, here a plain date.

```
SELECT 2 + 3 AS five, now()::date >= DATE '2025-01-01' AS after_2025;
```

Output:

```
 five | after_2025
------+------------
    5 | t
(1 row)
```

Common mistakes

- **`SELECT *` in application code.** It returns columns you don't need (more I/O — disk input/output — and network traffic), and code breaks or silently changes when someone adds a column. Name the columns. `SELECT *` is fine for exploring in psql.
- **A missing comma turns a column into an alias.** `SELECT first_name last_name FROM employees` returns *one* column named `last_name` containing first names. Writing `AS` for every alias makes this mistake visible.
- **Quoting aliases with single quotes.** `'total'` is a string literal (a fixed text value typed into the query); aliases use double quotes: `"Total"`.

Self-check

1. *What does `SELECT *` return for a table with 5 columns?* → All 5 columns in table-definition order.
2. *Does `SELECT unit_price * 2 FROM products` change prices?* → No; it only computes a value in the result set.
3. *How do you name an output column `Net Price` exactly?* → `AS "Net Price"`.

TL;DR

`SELECT cols FROM table`. Use `*` only to explore. Any expression can be a column; `AS alias` names it; table aliases shorten references.

2.2 WHERE: filtering rows

What it is

`WHERE condition` keeps only the rows for which the condition is **true**. It works the same way in `SELECT`, `UPDATE` and `DELETE` statements (chapter 9), so a filter you test with `SELECT` can be reused safely in a `DELETE`.

```
SELECT column1, column2
FROM   table_name
WHERE  condition;
```

How it works

For each row, the database evaluates the condition. The result is `TRUE`, `FALSE` or — when `NULL` is involved — `UNKNOWN` (section 2.5). Only `TRUE` rows pass.

```sql
-- code/02_where.sql
SELECT name, category, unit_price
FROM products
WHERE category = 'Electronics';
```

Output:

```
            name             |  category   | unit_price
-----------------------------+-------------+------------
 Laptop Pro 14               | Electronics |    1299.00
 Wireless Mouse              | Electronics |      24.99
 Noise-Cancelling Headphones | Electronics |     199.95
(3 rows)
```

Comparisons work on numbers, text and dates alike. Date literals are written as strings in ISO format `'YYYY-MM-DD'` (the international standard date layout: year, month, day), which PostgreSQL converts to the column's type:

```sql
SELECT first_name, hire_date FROM employees WHERE hire_date >= '2023-01-01' ORDER BY hire_date;
```

Output:

```
 first_name | hire_date
------------+------------
 Li         | 2023-02-14
 Arjun      | 2023-11-20
 Sara       | 2024-04-01
 Rohan      | 2025-06-02
(4 rows)
```

Combining conditions: AND, OR, NOT

Operator	True when	Example
`AND`	both sides are true	`dept_id = 2 AND salary > 70000`
`OR`	at least one side is true	`city = 'Delhi' OR city = 'Mumbai'`
`NOT`	the condition is false	`NOT discontinued`

Precedence: `NOT` binds tightest, then `AND`, then `OR` — just as `*` binds before `+`. So `a OR b AND c` means `a OR (b AND c)`. This silently changes results:

```sql
-- Intended: Engineering or Sales staff earning over 100k
SELECT first_name, dept_id, salary
FROM employees
WHERE dept_id = 1 OR dept_id = 2 AND salary > 100000
ORDER BY emp_id;
```

Output:

```
 first_name | dept_id |  salary
------------+---------+-----------
 Vikram     |       1 | 185000.00
 Neha       |       1 | 142000.00
 Rahul      |       1 |  98000.00
 Meera      |       1 |  98000.00
 Daniel     |       2 | 150000.00
(5 rows)
```

Rahul and Meera earn less than 100k but appear, because the query really says "*everyone* in dept 1, or dept-2 staff over 100k". Parentheses fix it:

```
SELECT first_name, dept_id, salary
FROM employees
WHERE (dept_id = 1 OR dept_id = 2) AND salary > 100000
ORDER BY emp_id;
```

Output:

```
 first_name | dept_id |  salary
------------+---------+-----------
 Vikram     |       1 | 185000.00
 Neha       |       1 | 142000.00
 Daniel     |       2 | 150000.00
(3 rows)
```

`NOT` negates the condition after it:

```
SELECT name FROM products WHERE NOT discontinued AND NOT unit_price < 100 ORDER BY name;
```

Output:

```
            name
----------------------------
 Ergonomic Chair
 Laptop Pro 14
 Noise-Cancelling Headphones
 Standing Desk
(4 rows)
```

Case sensitivity

In PostgreSQL, text comparison with `=` is **case-sensitive**: `'Mumbai' = 'mumbai'` is false. Our customers table has one `Mumbai` and one `mumbai`, and `WHERE city = 'Mumbai'` finds only one:

```
 exact_mumbai
--------------
            1
```

MySQL's default collations (the rules a database uses to compare and sort text, see 2.7) are case-*insensitive*, so the same query there returns 2 — a real portability trap (code that behaves differently when moved to another database). In PostgreSQL, compare `lower(city) = 'mumbai'` or use `ILIKE` (2.4).

Common mistakes

- Mixing `AND` and `OR` without parentheses. Rule of thumb: whenever both appear, add parentheses.
- Comparing with `= NULL` — never true; use `IS NULL` (2.5).
- Writing `"Mumbai"` (double quotes) for a string: PostgreSQL reads it as a column name and errors with *`column "Mumbai" does not exist`*.

Self-check

1. *How is `a AND b OR c` evaluated?* → `(a AND b) OR c`.
2. *Does `WHERE` also apply to `DELETE`?* → Yes; `DELETE FROM t WHERE …` removes only matching rows (and without `WHERE`, all rows).
3. *Why does `WHERE city = 'Mumbai'` miss `'mumbai'` in PostgreSQL?* → String equality is case-sensitive.

TL;DR

`WHERE` keeps rows whose condition is TRUE. `NOT` > `AND` > `OR` in precedence — parenthesise mixed conditions. Text equality is case-sensitive in PostgreSQL.

2.3 Operators

What it is

An **operator** is a symbol or keyword that combines one or more values (its **operands**) into a new value: `+`, `=`, `AND`, `||`.

Arithmetic operators

Operator	Meaning	Example	Result
`+`	add	`5 + 3`	8
`-`	subtract	`10 - 4`	6
`*`	multiply	`3 * 7`	21
`/`	divide	`20 / 5`	4
`%`	remainder (modulo)	`17 % 3`	2
`^`	power (PostgreSQL)	`2 ^ 10`	1024
`|/`, `@`	square root, absolute value (PostgreSQL)	`|/ 25`, `@ -5`	5, 5

```sql
-- code/08_operators.sql
SELECT 5 + 3 AS add, 10 - 4 AS sub, 3 * 7 AS mul, 20 / 5 AS div, 17 % 3 AS modulo;
SELECT 7 / 2 AS int_div, 7 / 2.0 AS num_div, 7::numeric / 2 AS cast_div, -7 / 2 AS neg_int_div;
SELECT 2 ^ 10 AS power, |/ 25 AS sqrt, @ -5 AS abs;
SELECT 2 + 3 * 4 AS no_parens, (2 + 3) * 4 AS parens;
```

Output:

```
 add | sub | mul | div | modulo
-----+-----+-----+-----+--------
   8 |   6 |  21 |   4 |      2
(1 row)

 int_div |      num_div       |      cast_div      | neg_int_div
---------+--------------------+--------------------+-------------
       3 | 3.5000000000000000 | 3.5000000000000000 |          -3
(1 row)

 power | sqrt | abs
-------+------+-----
  1024 |    5 |   5
(1 row)

 no_parens | parens
-----------+--------
        14 |     20
(1 row)
```

Integer division truncates toward zero (simply drops the fractional part) in PostgreSQL, SQL Server and SQLite: `7 / 2` is `3`, and `-7 / 2` is `-3`. If either side is a decimal (`2.0`) or cast to `numeric` (PostgreSQL's exact-decimal number type), you get the fraction. MySQL is the odd one out: `/` always returns a decimal (`7 / 2` = `3.5000`), and `DIV` does integer division. This is one of the most common "my average is wrong" bugs when moving between databases.

Operators work on columns row by row — for example the value of each order line:

```sql
SELECT order_id, product_id, quantity, unit_price, discount,
       quantity * unit_price * (1 - discount) AS line_total
FROM order_items WHERE order_id IN (102, 104) ORDER BY order_id, product_id;
```

Output:

```
 order_id | product_id | quantity | unit_price | discount | line_total
----------+------------+----------+------------+----------+------------
      102 |          7 |        2 |     199.95 |     0.10 |   359.9100
      104 |          5 |        1 |     479.00 |     0.00 |   479.0000
      104 |          6 |        1 |     349.99 |     0.05 |   332.4905
(3 rows)
```

Comparison operators

Operator	Meaning	Example
`=`	equal	`unit_price = 49.50`
`<>` or `!=`	not equal (`<>` is standard)	`status <> 'cancelled'`
`<`, `>`, `<=`, `>=`	ordering	`salary >= 100000`
`BETWEEN a AND b`	inclusive range	2.4
`IN (…)`	membership in a list	2.4
`LIKE`, `ILIKE`, `~`	pattern match	2.4
`IS [NOT] NULL`, `IS [NOT] DISTINCT FROM`	NULL-aware tests	2.5

Logical operators

`AND`, `OR`, `NOT` — see 2.2 for precedence and 2.5 for how they behave with `NULL`.

String concatenation

`||` joins strings in PostgreSQL, SQLite and Oracle:

```
SELECT first_name || ' ' || last_name AS full_name FROM employees WHERE emp_id <= 3;
```

Output:

```
  full_name
--------------
 Asha Rao
 Vikram Menon
 Neha Gupta
(3 rows)
```

In MySQL `||` means logical OR by default; use `CONCAT()` there. SQL Server uses `+` or `CONCAT()`. Chapter 3 covers `concat()` and how each treats `NULL`.

Compound assignment operators

Some SQL dialects have shorthand assignment operators, e.g. SQL Server's T-SQL: `SET @x += 3` (add and assign), also `-=`, `*=`, `/=`, `%=`. They apply to **procedural variables** (named values inside step-by-step code such as a stored procedure, rather than columns — plus T-SQL's `UPDATE … SET col += 1`). They are **not** part of standard SQL and PostgreSQL does not support them in SQL statements — write `SET col = col + 1` in an `UPDATE`, or `x := x + 3` inside a PL/pgSQL function (PL/pgSQL is PostgreSQL's built-in programming language for functions; chapter 14).

Dialect	Add-and-assign
PostgreSQL	`UPDATE t SET n = n + 3`; PL/pgSQL `x := x + 3`
SQL Server	`SET @x += 3`; `UPDATE t SET n += 3`
MySQL	`SET @x = @x + 3` (no `+=`)

Operator precedence (highest first, simplified)

Level	Operators
1	`::` (cast)
2	unary `-` / `+`
3	`^`
4	`*`, `/`, `%`
5	binary `+`, `-`
6	other operators such as `||`
7	`BETWEEN`, `IN`, `LIKE`, `ILIKE`
8	`<`, `>`, `=`, `<=`, `>=`, `<>`
9	`IS NULL`, `IS DISTINCT FROM` …
10	`NOT`
11	`AND`
12	`OR`

(*Unary* operators take one operand, like the `-` in `-5`; *binary* ones take two, like `a - b`.) Because unary minus binds tighter than `^`, `SELECT -2 ^ 2` returns `4` in PostgreSQL, not `-4`. When in doubt, add parentheses — they cost nothing and document intent.

Self-check

1. *`SELECT 9 / 4` in PostgreSQL?* → `2` (integer division). `9 / 4.0` → `2.25`.
2. *How do you write "not equal" in standard SQL?* → `<>` (PostgreSQL also accepts `!=`).
3. *Why might `first_name || ' ' || last_name` return nothing useful in MySQL?* → `||` is logical OR there by default.

TL;DR

`+ - * / %` (and PostgreSQL's `^`), with integer division truncating. `<>` for not-equal. `||` concatenates in PostgreSQL. Compound `+=` is a T-SQL/procedural feature, not standard SQL.

2.4 Ranges, lists and patterns: BETWEEN, IN, LIKE/ILIKE, regex

BETWEEN — inclusive ranges

`x BETWEEN a AND b` means `x >= a AND x <= b`. **Both ends are included.**

```sql
-- code/03_between_in_like.sql
SELECT name, unit_price FROM products WHERE unit_price BETWEEN 49.50 AND 199.95 ORDER BY unit_price;
```

Output:

```
            name             | unit_price
-----------------------------+------------
 USB-C Hub                   |      49.50
 Gift Card                   |      50.00
 Mechanical Keyboard         |      89.00
 Noise-Cancelling Headphones |     199.95
(4 rows)
```

`NOT BETWEEN a AND b` is the complement. `a` must be the smaller value; `BETWEEN 200 AND 50` matches nothing (PostgreSQL's `BETWEEN SYMMETRIC` accepts either order).

The timestamp trap. `placed_at` is a timestamp (date *and* time). The string `'2025-01-31'` becomes midnight at the *start* of 31 January, so an order at 23:55 that day is excluded:

```
SELECT order_id, placed_at FROM orders
WHERE placed_at BETWEEN '2025-01-01' AND '2025-01-31'
ORDER BY order_id;
```

Output:

```
 order_id |      placed_at
----------+---------------------
      101 | 2025-01-03 10:15:00
      102 | 2025-01-17 22:40:00
(2 rows)
```

Order 103 (31 Jan, 23:55) is missing. The robust pattern for any time range is **half-open**: `>=` the start and `<` the next period's start.

```
SELECT order_id, placed_at FROM orders
WHERE placed_at >= '2025-01-01' AND placed_at < '2025-02-01'
ORDER BY order_id;
```

Output:

```
 order_id |      placed_at
----------+---------------------
      101 | 2025-01-03 10:15:00
      102 | 2025-01-17 22:40:00
      103 | 2025-01-31 23:55:00
(3 rows)
```

IN — membership in a list

`x IN (a, b, c)` is shorthand for `x = a OR x = b OR x = c`; `NOT IN` is its negation.

```
SELECT order_id, status FROM orders WHERE status IN ('pending', 'cancelled') ORDER BY order_id;
SELECT first_name, job_title FROM employees WHERE job_title NOT IN ('Engineer', 'Sales Rep', 'Intern') ORDER BY
emp_id;
```

Output:

```
 order_id |  status
----------+-----------
      105 | cancelled
      111 | cancelled
      113 | pending
      114 | pending
(4 rows)

 first_name |      job_title
------------+---------------------
 Asha       | CEO
 Vikram     | Engineering Manager
 Neha       | Senior Engineer
 Daniel     | Sales Manager
 Lucas      | Marketing Lead
 Ananya     | Content Writer
 Kiran      | Finance Manager
 Omar       | Accountant
 Li         | Support Engineer
(9 rows)
```

`IN` can also take a subquery (`WHERE customer_id IN (SELECT …)`) — chapter 6. **Warning:** `NOT IN` with a `NULL` in the list returns no rows at all; see 2.5.

LIKE and ILIKE — simple patterns

`LIKE` matches text against a pattern with two wildcards:

Wildcard	Matches	Example	Matches…
`%`	any sequence of zero or more characters	`'%Desk%'`	"Standing Desk", "Desk Lamp"
`_`	exactly one character	`'_a%'`	"Daniel", "Rahul", "Sara" (second letter *a*)

```
SELECT name FROM products WHERE name LIKE '%Desk%' ORDER BY name;
SELECT first_name FROM employees WHERE first_name LIKE '_a%' ORDER BY first_name;
```

Output:

```
     name
--------------
 Desk Lamp
 Standing Desk
(2 rows)

 first_name
------------
 Daniel
 Rahul
 Sara
(3 rows)
```

`LIKE` is **case-sensitive** in PostgreSQL. `ILIKE` (PostgreSQL-only) ignores case:

```
SELECT email FROM employees WHERE email LIKE 'neha%';
SELECT email FROM employees WHERE email ILIKE 'neha%';
```

Output:

```
 email
-------
(0 rows)

           email
---------------------------
 Neha.Gupta@Company.example
(1 row)
```

Portable alternative: `lower(email) LIKE 'neha%'`.

Matching a literal `%` or `_` — escape it. An **escape character** is a marker that means "treat the next character as plain text, not as a wildcard". PostgreSQL's default escape character is backslash; `ESCAPE` chooses another:

```
SELECT label FROM (VALUES ('10% off'), ('100 off'), ('10_off')) AS t(label)
WHERE label LIKE '10\%%' ORDER BY label;
SELECT label FROM (VALUES ('10% off'), ('100 off'), ('10_off')) AS t(label)
WHERE label LIKE '10!_%' ESCAPE '!' ORDER BY label;
```

Output:

```
  label
---------
 10% off
(1 row)

 label
--------
 10_off
(1 row)
```

(`VALUES (…), (…) AS t(label)` builds a tiny inline table — handy for experiments.)

Regular expressions (PostgreSQL)

For richer patterns PostgreSQL offers POSIX regular expressions. A **regular expression** (regex) is a mini-language for describing text patterns — `^` means "start of text", `[A-Z]` "one capital letter", `+` "one or more of the previous"; POSIX is the Unix standard that fixes this syntax. The operators are `~` (case-sensitive match), `~*` (case-insensitive), `!~` and `!~*` (no match). Names made of a capitalised word, a space, then another capital:

```
SELECT name FROM products WHERE name ~ '^[A-Z][a-z]+ [A-Z]' ORDER BY name;
```

Output:

```
        name
--------------------
 Desk Lamp
 Ergonomic Chair
 Gift Card
 Laptop Pro 14
 Mechanical Keyboard
 Standing Desk
 Wireless Mouse
(7 rows)
```

`USB-C Hub` fails (`S` isn't lower case) and `Noise-Cancelling…` fails (hyphen, not space). The standard's `SIMILAR TO` also exists but is rarely used.

Performance note

`LIKE 'abc%'` (a fixed prefix) can use a B-tree index — an **index** is a sorted lookup structure kept beside the table so the database can jump to matching rows instead of reading them all, and a **B-tree** is the default, ordered kind (in PostgreSQL the index needs the `text_pattern_ops` operator class — a setting that tells the index how to compare values — unless the database uses the `C` collation, i.e. plain byte-by-byte comparison); `LIKE '%abc'` with a leading wildcard cannot, and scans every row. Chapter 12 shows trigram (`pg_trgm`) indexes for "contains" searches (a *trigram* is every 3-character slice of the text).

Dialect notes

Feature	PostgreSQL	MySQL	SQLite	SQL Server
`LIKE` case-sensitivity	sensitive	depends on collation (default insensitive)	insensitive for ASCII (plain English letters) by default	depends on collation (default insensitive)
Case-insensitive LIKE	`ILIKE`	`LIKE` (default)	`LIKE`	`LIKE` (default)
Regex	`~`, `~*`	`REGEXP` / `RLIKE`	`REGEXP` only when a `regexp()` function is loaded (the `sqlite3` shell includes one)	none built in (SQL Server 2025 adds `REGEXP_LIKE`)

Common mistakes

- `BETWEEN` on timestamps with date-only upper bounds (use half-open ranges).
- Forgetting that `_` is a wildcard: `LIKE 'user_%'` also matches `userA…`.
- Expecting `LIKE` to be case-insensitive in PostgreSQL.

Self-check

1. *Is `BETWEEN 10 AND 20` inclusive of 20?* → Yes, both ends.
2. *Pattern for "exactly 4 characters ending in `n`"?* → `LIKE '___n'`.

3. *How do you match names containing* `%`*?* → `LIKE '%\%%'` (or `ESCAPE`).

TL;DR

`BETWEEN` is inclusive (use `>=`/`<` for timestamps), `IN` is shorthand for ORs, `LIKE` uses `%` and `_` and is case-sensitive in PostgreSQL (`ILIKE` isn't), regex via `~`.

2.5 NULL and three-valued logic

What it is

`NULL` is a **marker for a missing value** — unknown ("we don't know Meera's email") or not applicable ("interns earn no commission"). It is not zero, not an empty string, and not a value at all, so it cannot be *equal* to anything — not even another `NULL`.

Why it matters

Real data is full of gaps, and `NULL` changes how every comparison, filter and calculation behaves. Most "missing rows" bugs in SQL come from it.

How it works — three-valued logic

Any comparison involving `NULL` yields the third truth value **UNKNOWN** (displayed as `NULL`). Arithmetic and concatenation with `NULL` yield `NULL`.

```
-- code/04_nulls.sql
SELECT NULL = NULL AS null_eq_null, NULL <> 1 AS null_ne_1, NULL IS NULL AS is_null,
       1 + NULL AS plus, 'a' || NULL AS concat;
```

Output:

```
 null_eq_null | null_ne_1 | is_null |  plus  | concat
--------------+-----------+---------+--------+--------
 (null)       | (null)    | t       | (null) | (null)
(1 row)
```

`AND`, `OR` and `NOT` follow these truth tables (generated by the database itself):

```
SELECT a, b, a AND b AS "a AND b", a OR b AS "a OR b", NOT a AS "NOT a"
FROM (VALUES (true), (false), (NULL::boolean)) AS x(a)
CROSS JOIN (VALUES (true), (false), (NULL::boolean)) AS y(b);
```

Output:

```
   a    |   b    | a AND b | a OR b | NOT a
--------+--------+---------+--------+--------
 t      | t      | t       | t      | f
 t      | f      | f       | t      | f
 t      | (null) | (null)  | t      | f
 f      | t      | f       | t      | t
 f      | f      | f       | f      | t
 f      | (null) | f       | (null) | t
 (null) | t      | (null)  | t      | (null)
 (null) | f      | f       | (null) | (null)
 (null) | (null) | (null)  | (null) | (null)
(9 rows)
```

Read it as "UNKNOWN might be either true or false": `FALSE AND UNKNOWN` is `FALSE` whatever the unknown is; `TRUE OR UNKNOWN` is `TRUE`; anything else involving UNKNOWN stays UNKNOWN. And `NOT UNKNOWN` is still UNKNOWN.

`WHERE` **keeps only TRUE** — both FALSE and UNKNOWN rows are dropped.

Testing for NULL: IS NULL / IS NOT NULL

```
SELECT count(*) AS eq_null  FROM employees WHERE email = NULL;
SELECT count(*) AS is_null  FROM employees WHERE email IS NULL;
SELECT first_name, last_name FROM employees WHERE email IS NULL ORDER BY emp_id;
```

Output:

```
 eq_null
---------
       0
(1 row)

 is_null
---------
       2
(1 row)

 first_name | last_name
------------+-----------
 Meera      | Iyer
 Ananya     | Das
(2 rows)
```

`= NULL` is always UNKNOWN, so it never matches. (SQL Server has a legacy `ANSI_NULLS OFF` mode where `= NULL` works; don't rely on it.)

The "missing rows" trap

Sales reps have commission `0.10`, `0.10` and `NULL`. Asking for `= 0.10` and then `<> 0.10` should cover everyone — but Sara (NULL) appears in neither:

```
SELECT first_name, commission_pct FROM employees WHERE job_title = 'Sales Rep' AND commission_pct =  0.10 ORDER BY
emp_id;
SELECT first_name, commission_pct FROM employees WHERE job_title = 'Sales Rep' AND commission_pct <> 0.10 ORDER BY
emp_id;
```

Output:

```
 first_name | commission_pct
------------+----------------
 Priya      |           0.10
 Arjun      |           0.10
(2 rows)

 first_name | commission_pct
------------+----------------
(0 rows)
```

`IS DISTINCT FROM` is the NULL-safe "not equal": it treats two NULLs as equal and NULL vs a value as different, and always returns TRUE or FALSE.

```
SELECT first_name, commission_pct FROM employees
WHERE job_title = 'Sales Rep' AND commission_pct IS DISTINCT FROM 0.10 ORDER BY emp_id;
```

Output:

```
 first_name | commission_pct
------------+----------------
 Sara       |         (null)
(1 row)
```

(MySQL spells NULL-safe equality `<=>`; SQL Server supports `IS [NOT] DISTINCT FROM` from version 2022.)

The NOT IN trap

`x NOT IN (1, 2, NULL)` expands to `x <> 1 AND x <> 2 AND x <> NULL`. The last part is always UNKNOWN, so the whole condition can never be TRUE:

```
SELECT count(*) AS not_in_with_null FROM departments WHERE dept_id NOT IN (1, 2, NULL);
SELECT count(*) AS not_in_no_null   FROM departments WHERE dept_id NOT IN (1, 2);
```

Output:

```
 not_in_with_null
------------------
                0
(1 row)

 not_in_no_null
----------------
              4
(1 row)
```

This bites hardest with subqueries — `WHERE id NOT IN (SELECT manager_id FROM employees)` returns nothing because the CEO's `manager_id` is NULL. Use `NOT EXISTS` instead (chapter 6).

NULL is not zero or ''

```
SELECT count(*) AS rows, count(commission_pct) AS non_null_commissions FROM employees;
SELECT '' IS NULL AS empty_string_is_null;
```

Output:

```
 rows | non_null_commissions
------+----------------------
   15 |                    3
(1 row)

 empty_string_is_null
----------------------
 f
(1 row)
```

`count(*)` counts rows; `count(column)` counts non-NULL values (chapter 4). And in PostgreSQL an empty string is a real value, not NULL. (**Oracle** is the exception: it stores `''` as NULL.)

Allowing or forbidding NULL in a column

Columns accept NULL **unless** declared `NOT NULL`. Writing `NULL` in a column definition just states the default explicitly.

```
CREATE TABLE students (
    id    integer PRIMARY KEY,
    email text NULL,          -- explicit "NULL allowed" (the default anyway)
    name  text NOT NULL       -- a value is required
);
INSERT INTO students VALUES (1, NULL, 'Ravi');
INSERT INTO students VALUES (2, 'kim@school.example', NULL);
SELECT * FROM students WHERE email IS NULL;
```

Output:

```
CREATE TABLE
INSERT 0 1
ERROR:  null value in column "name" of relation "students" violates not-null constraint
DETAIL:  Failing row contains (2, kim@school.example, null).
 id | email  | name
----+--------+------
  1 | (null) | Ravi
(1 row)
```

Replacing NULLs in output (`COALESCE`, `NULLIF`) is in chapter 3; NULLs in sorting are in 2.7, in `DISTINCT` in 2.6, in aggregates in chapter 4, in joins in chapter 5.

In the interview

30-second answer: "NULL means unknown, so comparisons with it give UNKNOWN, and WHERE only keeps TRUE rows. Use IS NULL / IS NOT NULL, or IS DISTINCT FROM for NULL-safe comparison. Watch NOT IN with a NULL in the list or subquery — it returns nothing — so prefer NOT EXISTS. Aggregates like COUNT(col) and SUM skip NULLs."

- *What does* `NULL = NULL` *return?* → NULL (UNKNOWN), not TRUE.
- *Why can* `WHERE x <> 5` *miss rows?* → Rows where `x` is NULL give UNKNOWN and are filtered out.
- *Is* `''` *NULL?* → Not in PostgreSQL, MySQL, SQL Server or SQLite; yes in Oracle.

Self-check

1. `TRUE OR NULL`*?* → TRUE.
2. `FALSE AND NULL`*?* → FALSE.
3. `NOT NULL` *(as a boolean expression)?* → NULL.
4. *How many rows does* `WHERE email = NULL` *return?* → Zero, always.

TL;DR

NULL = unknown. Comparisons with NULL are UNKNOWN; `WHERE` keeps only TRUE. Test with `IS [NOT] NULL`; compare NULL-safely with `IS [NOT] DISTINCT FROM`; never put a nullable list inside `NOT IN`.

2.6 DISTINCT: removing duplicate rows

What it is

`SELECT DISTINCT` removes duplicate rows from the **result** (the table itself is untouched). It answers questions like "which categories exist?" or "which cities do our customers live in?".

```
SELECT DISTINCT column1, column2
FROM table_name
WHERE condition;
```

How it works

```
-- code/05_distinct.sql
SELECT category FROM products ORDER BY category;
SELECT DISTINCT category FROM products ORDER BY category;
```

Output:

```
 category
-------------
 Accessories
 Accessories
 Electronics
 Electronics
 Electronics
 Furniture
 Furniture
 Furniture
 (null)
(9 rows)

 category
-------------
 Accessories
 Electronics
 Furniture
 (null)
(4 rows)
```

Two rules to notice:

1. **NULLs count as duplicates of each other** for `DISTINCT` (unlike `=`) — all NULL categories collapse into one row.
2. `DISTINCT` **applies to the whole selected row**, not to the first column. With two columns, you get distinct *pairs*:

```
SELECT DISTINCT country, city FROM customers ORDER BY country, city;
```

Output:

```
 country |   city
---------+-----------
 India   | Bengaluru
 India   | Delhi
 India   | Mumbai
 India   | mumbai
 India   | (null)
 UK      | London
 USA     | New York
(7 rows)
```

`Mumbai` and `mumbai` are different strings, so both survive — `DISTINCT` removes only **exact** duplicates. Dirty data needs cleaning first (`trim` removes spaces at the start and end of a string; `lower` turns it into lower case). Our customer Anita Shah was entered twice with differently formatted emails:

```
SELECT DISTINCT first_name, last_name, email FROM customers WHERE last_name = 'Shah';
SELECT DISTINCT first_name, last_name, lower(trim(email)) AS email FROM customers WHERE last_name = 'Shah';
```

Output:

```
 first_name | last_name |           email
------------+-----------+---------------------------
 Anita      | Shah      |   Anita.Shah@Mail.example
 Anita      | Shah      | anita.shah@mail.example
(2 rows)

 first_name | last_name |          email
------------+-----------+------------------------
 Anita      | Shah      | anita.shah@mail.example
(1 row)
```

Counting distinct values uses `count(DISTINCT expr)`, which ignores NULLs (chapter 4 covers aggregates — functions such as `count` and `sum` that combine many rows into one value):

```
SELECT count(*) AS rows, count(city) AS with_city, count(DISTINCT city) AS distinct_cities,
       count(DISTINCT lower(city)) AS distinct_cities_ci
FROM customers;
```

Output:

```
 rows | with_city | distinct_cities | distinct_cities_ci
------+-----------+-----------------+--------------------
   10 |         9 |               6 |                  5
(1 row)
```

DISTINCT ON (PostgreSQL)

`DISTINCT ON (expr)` keeps the **first row of each group** of equal `expr` values, where "first" is decided by `ORDER BY` (which must start with the same expressions). A classic use is "the top earner in each department":

```
SELECT DISTINCT ON (dept_id) dept_id, first_name, salary
FROM employees
WHERE dept_id IS NOT NULL
ORDER BY dept_id, salary DESC, emp_id;
```

Output:

```
 dept_id | first_name |  salary
---------+------------+-----------
       1 | Vikram     | 185000.00
       2 | Daniel     | 150000.00
       3 | Lucas      | 120000.00
       4 | Kiran      | 135000.00
       5 | Li         |  64000.00
(5 rows)
```

Other databases solve "first row per group" with window functions (functions that look at related rows without merging them into one, such as `ROW_NUMBER()` — chapter 8).

Trade-offs and when not to use it

- `DISTINCT` needs a sort or hash of the whole result (hashing turns each row into a number used to bucket equal rows together) — cheap here, expensive on millions of rows.
- **Don't use `DISTINCT` to hide a bad join.** If a join produces duplicates you didn't expect, the join condition is probably wrong (chapter 5 calls this *fan-out*); `DISTINCT` masks the symptom and can hide real duplicates too.
- If you need counts per value, you want `GROUP BY` (which gathers rows with equal values into groups, chapter 4), not `DISTINCT`.

Self-check

1. *Does `SELECT DISTINCT a, b` make `a` unique?* → No; it makes each `(a, b)` pair unique.
2. *Do two NULLs count as duplicates under DISTINCT?* → Yes.
3. *Why did `count(DISTINCT city)` give 6 but the case-insensitive version 5?* → `Mumbai` and `mumbai` differ as strings but match after `lower()`.

TL;DR

`DISTINCT` removes exact duplicate result rows (NULLs grouped together); it applies to the whole select list. Clean data (`lower`, `trim`) before de-duplicating; use PostgreSQL's `DISTINCT ON` for "first row per group".

2.7 ORDER BY: sorting results

What it is

Without `ORDER BY`, SQL returns rows in **no guaranteed order** — whatever is fastest for the database at that moment. `ORDER BY` is the only way to get a defined order.

```
SELECT column1, column2
FROM table_name
WHERE condition
ORDER BY column1 [ASC | DESC], column2 [ASC | DESC];
```

`ASC` (ascending, smallest first) is the default; `DESC` is descending.

How it works

```
-- code/06_order_by.sql
SELECT first_name, salary FROM employees ORDER BY salary DESC;
```

Output:

```
 first_name |  salary
------------+-----------
 Asha       | 250000.00
 Vikram     | 185000.00
 Daniel     | 150000.00
 Neha       | 142000.00
 Kiran      | 135000.00
 Lucas      | 120000.00
 Meera      |  98000.00
 Rahul      |  98000.00
 Omar       |  76000.00
 Priya      |  72000.00
 Arjun      |  68000.00
 Sara       |  68000.00
 Li         |  64000.00
 Ananya     |  61000.00
 Rohan      |  30000.00
(15 rows)
```

Meera and Rahul tie; their relative order is **not guaranteed** and may differ on another run. Add a **tie-breaker** column whenever order matters (especially for paging), so the order is **deterministic** — the same every run.

Several sort keys — the second key only matters within ties of the first, and each key has its own direction:

```
SELECT dept_id, first_name, salary FROM employees
ORDER BY dept_id, salary DESC, first_name;
```

Output:

```
 dept_id | first_name |  salary
---------+------------+-----------
       1 | Vikram     | 185000.00
       1 | Neha       | 142000.00
       1 | Meera      |  98000.00
       1 | Rahul      |  98000.00
       2 | Daniel     | 150000.00
       2 | Priya      |  72000.00
       2 | Arjun      |  68000.00
       2 | Sara       |  68000.00
       3 | Lucas      | 120000.00
       3 | Ananya     |  61000.00
       4 | Kiran      | 135000.00
       4 | Omar       |  76000.00
       5 | Li         |  64000.00
  (null) | Asha       | 250000.00
  (null) | Rohan      |  30000.00
(15 rows)
```

Where do NULLs go?

PostgreSQL treats NULL as **larger than any value**: NULLs come **last in ASC** and **first in DESC**. Override with `NULLS FIRST` / `NULLS LAST`:

```
SELECT name, location FROM departments ORDER BY location;
SELECT name, location FROM departments ORDER BY location NULLS FIRST;
SELECT name, location FROM departments ORDER BY location DESC NULLS LAST;
```

Output:

```
    name     | location
-------------+-----------
 Engineering | Bengaluru
 Sales       | Mumbai
 Marketing   | Mumbai
 Finance     | Pune
 Research    | Pune
 Support     | (null)
(6 rows)

    name     | location
-------------+-----------
 Support     | (null)
 Engineering | Bengaluru
 Sales       | Mumbai
 Marketing   | Mumbai
 Finance     | Pune
 Research    | Pune
(6 rows)

    name     | location
-------------+-----------
 Finance     | Pune
 Research    | Pune
 Sales       | Mumbai
 Marketing   | Mumbai
 Engineering | Bengaluru
 Support     | (null)
(6 rows)
```

Database	NULLs in ASC	`NULLS FIRST/LAST` supported?
PostgreSQL, Oracle	last	yes
MySQL, SQL Server, SQLite	first	SQLite yes (3.30+); MySQL/SQL Server no — use `ORDER BY col IS NULL, col` (MySQL) or a `CASE` expression

Sorting by alias, expression or position

```
SELECT name, unit_price * 1.18 AS gross FROM products ORDER BY gross DESC LIMIT 3;
SELECT first_name, last_name FROM employees ORDER BY length(last_name) DESC, last_name LIMIT 3;
SELECT first_name, hire_date FROM employees ORDER BY 2 LIMIT 3;
```

Output:

```
      name       |   gross
-----------------+-----------
 Laptop Pro 14   | 1532.8200
 Standing Desk   |  588.8200
 Ergonomic Chair |  412.9882
(3 rows)

 first_name | last_name
------------+-----------
 Daniel     | Brooks
 Omar       | Farouk
 Lucas      | Martin
(3 rows)

 first_name | hire_date
------------+------------
 Asha       | 2015-01-05
 Kiran      | 2016-12-01
 Daniel     | 2017-09-25
(3 rows)
```

`ORDER BY 2` means "the second column of the select list". It's quick in psql but fragile in code — reorder the select list and the sort silently changes.

Custom order with CASE

Status values should sort by workflow, not alphabetically:

```
SELECT order_id, status FROM orders
ORDER BY CASE status WHEN 'pending' THEN 1 WHEN 'shipped' THEN 2
                    WHEN 'delivered' THEN 3 ELSE 4 END, order_id;
```

Output (abridged):

```
 order_id |  status
----------+-----------
      113 | pending
      114 | pending
      107 | shipped
      110 | shipped
      112 | shipped
      101 | delivered
      ...
      109 | delivered
      105 | cancelled
      111 | cancelled
(14 rows)
```

(`CASE` is explained in chapter 3.)

Text order depends on collation

A **collation** is the rule set for comparing and sorting text in a language. The book's server uses the simple `C` collation, which sorts by byte value — so all capital letters come before all lower-case ones:

```
SELECT city FROM customers WHERE city IS NOT NULL ORDER BY city;
SELECT city FROM customers WHERE city IS NOT NULL ORDER BY lower(city), city;
```

Output:

```
   city
-----------
 Bengaluru
 Delhi
 Delhi
 London
 London
 Mumbai
 New York
 New York
 mumbai
(9 rows)

   city
-----------
 Bengaluru
 Delhi
 Delhi
 London
 London
 Mumbai
 mumbai
 New York
 New York
(9 rows)
```

A database created with a language collation such as `en_US.UTF-8` would sort `mumbai` next to `Mumbai` without help. You can also choose per query: `ORDER BY city COLLATE "en-x-icu"` (needs ICU support — ICU, International Components for Unicode, is a library of language-aware text rules).

Common mistakes

- Relying on insertion order or "it always came back sorted" — without `ORDER BY` there is no order.
- Forgetting a unique tie-breaker, then paging (2.8) shows some rows twice and others never.
- Sorting numbers stored as text: `'10' < '9'` alphabetically. Store numbers in numeric columns.

Self-check

1. *Default direction?* → ASC.
2. *Where does PostgreSQL put NULLs with* `ORDER BY x DESC`*?* → First.
3. *Why add* `emp_id` *to* `ORDER BY salary DESC`*?* → As a unique tie-breaker so the order is deterministic.

TL;DR

Only `ORDER BY` guarantees order. Multiple keys, each `ASC`/`DESC`; control NULLs with `NULLS FIRST/LAST`; always add a unique tie-breaker when order matters; text sorting follows the collation.

2.8 LIMIT, OFFSET and FETCH: returning a page

What it is

`LIMIT n` returns at most `n` rows. `OFFSET m` skips the first `m` rows. Together they return "page *k*" of a result. The SQL-standard spelling is `OFFSET m ROWS FETCH FIRST n ROWS ONLY`. **Always pair them with `ORDER BY`** — "the first 3 rows" means nothing without an order.

How it works

```
-- code/07_limit_offset.sql
-- Top 3 earners
SELECT first_name, salary FROM employees ORDER BY salary DESC LIMIT 3;

-- Page 2 with 5 rows per page (skip 5)
SELECT emp_id, first_name FROM employees ORDER BY emp_id LIMIT 5 OFFSET 5;

-- Standard SQL spelling of the same thing
SELECT emp_id, first_name FROM employees ORDER BY emp_id
OFFSET 5 ROWS FETCH FIRST 5 ROWS ONLY;
```

Output:

```
 first_name |  salary
------------+-----------
 Asha       | 250000.00
 Vikram     | 185000.00
 Daniel     | 150000.00
(3 rows)

 emp_id | first_name
--------+------------
      6 | Daniel
      7 | Priya
      8 | Arjun
      9 | Sara
     10 | Lucas
(5 rows)

 emp_id | first_name
--------+------------
      6 | Daniel
      7 | Priya
      8 | Arjun
      9 | Sara
     10 | Lucas
(5 rows)
```

For page *p* (starting at 1) with page size *s*: `LIMIT s OFFSET (p - 1) * s`.

Ties: FETCH … WITH TIES

`LIMIT 1` on a tie picks one row arbitrarily. `FETCH FIRST n ROWS WITH TIES` (PostgreSQL 13+, requires `ORDER BY`) also returns every row that ties with the last one:

```
SELECT first_name, salary FROM employees WHERE dept_id IN (1, 2)
ORDER BY salary LIMIT 1;
SELECT first_name, salary FROM employees WHERE dept_id IN (1, 2)
ORDER BY salary FETCH FIRST 1 ROWS WITH TIES;
```

Output:

```
 first_name |  salary
------------+----------
 Arjun      | 68000.00
(1 row)

 first_name |  salary
------------+----------
 Arjun      | 68000.00
 Sara       | 68000.00
(2 rows)
```

"Lowest-paid person" has two correct answers; `WITH TIES` returns both. (Ranking functions in chapter 8 generalise this.)

Why OFFSET gets slow — keyset pagination

`OFFSET 100000` still makes the database produce and throw away 100,000 rows, so deep pages get slower and slower. Rows inserted or deleted between page loads also shift the pages, so users see duplicates or miss rows. **Keyset** (or "seek") pagination remembers the last key shown and asks for rows after it:

```
-- page after the one that ended at emp_id 10
SELECT emp_id, first_name FROM employees WHERE emp_id > 10 ORDER BY emp_id LIMIT 5;
```

Output:

```
 emp_id | first_name
--------+------------
     11 | Ananya
     12 | Kiran
     13 | Omar
     14 | Li
     15 | Rohan
(5 rows)
```

With an index on the key, every page costs the same no matter how deep. The trade-off: you can go "next/previous" but not jump to page 500. Details in chapter 12.

Dialect differences

Database	First 10 rows	Skip 20, take 10
PostgreSQL	`LIMIT 10` or `FETCH FIRST 10 ROWS ONLY`	`LIMIT 10 OFFSET 20` or `OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`
MySQL	`LIMIT 10`	`LIMIT 10 OFFSET 20` or `LIMIT 20, 10` (offset first!)
SQLite	`LIMIT 10`	`LIMIT 10 OFFSET 20`
SQL Server	`SELECT TOP (10) …`	`ORDER BY … OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`
Oracle 12c+	`FETCH FIRST 10 ROWS ONLY`	`OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`

Common mistakes

- `LIMIT` without `ORDER BY` — "random" rows that can change between runs.
- MySQL's `LIMIT 20, 10` read as "20 rows starting at 10" — it's the reverse.
- Paging on a non-unique sort key — rows at page boundaries repeat or vanish. Add a unique tie-breaker.

In the interview

30-second answer: "`ORDER BY` plus `LIMIT n OFFSET m`, or the standard `OFFSET … FETCH`. For deep pages OFFSET is O(offset) ("Big-O" notation: its work grows in proportion to the offset) and unstable under inserts, so I'd use keyset pagination — `WHERE (sort_key, id) > (:last_key, :last_id) ORDER BY sort_key, id LIMIT n` — backed by an index." (`:last_key` and `:last_id` are placeholders the application fills with the last row it showed.)

- *How do you get the top 3 including ties?* → `FETCH FIRST 3 ROWS WITH TIES`, or `DENSE_RANK()` (chapter 8).
- *SQL Server equivalent of LIMIT?* → `TOP (n)` or `OFFSET … FETCH`.

Self-check

1. *Rows returned by `LIMIT 5 OFFSET 12` over 15 rows?* → 3 (rows 13–15).
2. *Page 4, size 20?* → `LIMIT 20 OFFSET 60`.
3. *What does keyset pagination need?* → A stable, unique ordering key (and ideally an index on it).

TL;DR

`ORDER BY … LIMIT n OFFSET m` (or `OFFSET … FETCH`). `WITH TIES` keeps ties. Deep `OFFSET` is slow and unstable; prefer keyset pagination.

2.9 How a SELECT is evaluated (logical processing order)

What it is

You *write* a query in the order `SELECT … FROM … WHERE … ORDER BY … LIMIT`, but the database *evaluates it logically* in a different order. Knowing that order explains many error messages.



(Chapter 4 inserts `GROUP BY` and `HAVING` between `WHERE` and `SELECT`, and window functions after them.)

Consequences:

- `WHERE` **cannot use a column alias** defined in the `SELECT` list, because `SELECT` hasn't happened yet.
- `ORDER BY` **can use aliases**, because it runs after `SELECT` — but in PostgreSQL only a bare alias: `ORDER BY gross` works, `ORDER BY gross + 0` fails with *column "gross" does not exist* (inside an expression, names refer to input columns).
- `LIMIT` is applied last, to the already sorted result.

```sql
-- code/01_select_columns.sql (end)
SELECT name, unit_price * 1.18 AS gross FROM products WHERE gross > 500;
SELECT name, unit_price * 1.18 AS gross FROM products WHERE unit_price * 1.18 > 500 ORDER BY gross DESC;
```

Output:

```
ERROR:  column "gross" does not exist
LINE 1: ...e, unit_price * 1.18 AS gross FROM products WHERE gross > 50...
                                                             ^
     name      |   gross
---------------+-----------
 Laptop Pro 14 | 1532.8200
 Standing Desk |  588.8200
(2 rows)
```

This is the *logical* order that defines the meaning; the optimizer (the database component that chooses how to run a query) may physically do things differently (e.g. use an index to read rows already sorted) as long as the result is the same.

Self-check

1. *Why does* `WHERE gross > 500` *fail?* → `WHERE` is evaluated before the `SELECT` list defines `gross`.
2. *Is* `ORDER BY gross` *allowed?* → Yes; `ORDER BY` comes after `SELECT`.

TL;DR

Logical order: FROM → WHERE → SELECT → DISTINCT → ORDER BY → LIMIT. Aliases exist from SELECT onward.

Putting it together

A query using every clause from this chapter — "the three most recent non-cancelled 2025 orders placed through a sales rep":

```sql
-- code/09_putting_it_together.sql
SELECT order_id, customer_id, sales_rep_id, placed_at, status
FROM orders
WHERE status <> 'cancelled'
  AND sales_rep_id IS NOT NULL
  AND placed_at >= '2025-01-01' AND placed_at < '2026-01-01'
ORDER BY placed_at DESC, order_id DESC
LIMIT 3;
```

Output:

```
 order_id | customer_id | sales_rep_id |      placed_at      | status
----------+-------------+--------------+---------------------+---------
      113 |           1 |            7 | 2025-06-02 10:30:00 | pending
      112 |           5 |            9 | 2025-05-20 21:05:00 | shipped
      110 |           7 |            8 | 2025-04-25 11:10:00 | shipped
(3 rows)
```

Chapter 2 TL;DR

- `SELECT cols FROM t` — name your columns; expressions and `AS` aliases are free; table aliases shorten references.
- `WHERE` keeps TRUE rows only. Precedence `NOT` > `AND` > `OR`: parenthesise.
- Integer division truncates in PostgreSQL (`7/2 = 3`); `||` concatenates; compound `+=` is not standard SQL.
- `BETWEEN` is inclusive (use half-open ranges for timestamps); `IN` = ORs; `LIKE` (`%`, `_`) is case-sensitive, `ILIKE` isn't; `~` for regex.
- NULL means unknown → three-valued logic. Use `IS [NOT] NULL`, `IS DISTINCT FROM`; beware `NOT IN` with NULLs.
- `DISTINCT` removes exact duplicate rows (NULLs grouped); `DISTINCT ON` keeps first row per group.
- Only `ORDER BY` gives order; add tie-breakers; `NULLS FIRST/LAST`; collation affects text.
- `LIMIT/OFFSET` or `OFFSET … FETCH`; `WITH TIES`; keyset pagination for deep pages.
- Logical order FROM → WHERE → SELECT → DISTINCT → ORDER BY → LIMIT.

Next: Chapter 3 — Functions and expressions.

Chapter 3 — Functions and Expressions

Chapter 2 selected and filtered stored values. Real reports need to *transform* them: round a price, tidy a messy email, work out how many days an order took to ship, label a product "premium", or show "no email on file" instead of a blank. This chapter covers the building blocks for all of that: SQL's **type system** and conversions, **number**, **string** and **date/time** functions, and the **conditional expressions** `CASE`, `COALESCE`, `NULLIF`, `GREATEST` and `LEAST`.

The SQL course these notes draw on was written for MySQL. Function names differ more between databases than anything else in SQL, so every section ends with a **dialect map** from MySQL (and SQL Server, SQLite) to PostgreSQL.

What you'll learn

#	Topic	You will be able to…
3.1	Types and casts	name PostgreSQL's core types, convert with `CAST` / `::`, avoid float surprises
3.2	Number functions	round, truncate, take powers/logs/roots, generate random numbers, format numbers
3.3	String functions	join, cut, measure, clean, search, pad, split and pattern-replace text
3.4	Date and time functions	get "now", do date arithmetic, extract/truncate parts, format/parse, handle time zones
3.5	Conditional expressions	use `CASE`, `COALESCE`, `NULLIF`, `GREATEST` / `LEAST`
3.6	Worked exercises	combine functions into realistic report columns

Setup. Examples use the `company` sample database (chapter 1, 1.11); code is in `code/`. Outputs come from PostgreSQL 14.

Functions in one paragraph

A **function** takes zero or more **arguments** and returns one value: `round(9.876, 1)` → `9.9`. Functions that work on one row at a time (everything in this chapter) are **scalar functions**; functions that squash many rows into one (`sum`, `count`, `avg`) are **aggregate functions**, covered in chapter 4. Functions nest — `lower(trim(email))` — and can appear anywhere an expression can: the `SELECT` list, `WHERE`, `ORDER BY`, `GROUP BY`, `UPDATE … SET`.

3.1 Data types and conversions

What it is

Every value in SQL has a **data type** that decides what it can hold and which operators and functions apply. `'42'` (text) and `42` (integer) look alike but behave differently: `'42' || '1'` is `'421'`, `42 + 1` is `43`. PostgreSQL is **strict**: it rarely converts types silently, which catches bugs early.

The core PostgreSQL types

Family	Type	Holds	Notes
Integer	`smallint`, `integer` (`int`), `bigint`	2-, 4-, 8-byte whole numbers	`integer` max is 2,147,483,647
Exact decimal	`numeric(p, s)` / `decimal(p, s)`	exact numbers, *p* total digits, *s* after the point	use for **money**; slower than floats
Floating point	`real`, `double precision`	approximate binary fractions	science/measurements, never money
Text	`text`, `varchar(n)`, `char(n)`	strings	in PostgreSQL `text` and `varchar` perform the same; `char(n)` pads with spaces
Boolean	`boolean`	`true`, `false`, `NULL`	
Date/time	`date`, `time`, `timestamp`, `timestamptz`, `interval`	calendar dates, clock times, instants, durations	3.4
Other	`uuid`, `json`/`jsonb`, arrays (`int[]`), `bytea`, enums	uuid = random 128-bit ID; jsonb = stored JSON; array = list in one cell; bytea = raw bytes; enum = fixed set of labels	JSON and arrays in chapter 14

Concept	PostgreSQL	MySQL	SQL Server	SQLite
Exact decimal	`numeric(10,2)`	`DECIMAL(10,2)`	`DECIMAL(10,2)`	stored as REAL/INTEGER (type *affinity*)
Variable text	`text` / `varchar(n)`	`VARCHAR(n)`, `TEXT`	`VARCHAR(n)` / `NVARCHAR(n)`	`TEXT`
Boolean	`boolean`	`TINYINT(1)` alias `BOOLEAN`	`BIT`	integer 0/1
Timestamp with zone	`timestamptz`	`TIMESTAMP` (converted to UTC, the world reference time)	`DATETIMEOFFSET`	text or number, by convention

(SQLite is the outlier: a column's type is only a *preference* — it will store text in an integer column. The full DDL (Data Definition Language — the `CREATE`/`ALTER` statements that define tables) story is in chapter 9.)

How it works — asking for and changing types

`pg_typeof(x)` reports a value's type:

```
-- code/01_types_and_casts.sql
SELECT pg_typeof(42) AS int_lit, pg_typeof(42.0) AS dec_lit, pg_typeof(4.2e1) AS sci_lit,
       pg_typeof('hi') AS str_lit, pg_typeof(true) AS bool_lit, pg_typeof(now()) AS now_type;
SELECT pg_typeof(salary) AS salary, pg_typeof(hire_date) AS hire_date, pg_typeof(email) AS email
FROM employees LIMIT 1;
```

Output:

```
 int_lit | dec_lit | sci_lit | str_lit | bool_lit |         now_type
---------+---------+---------+---------+----------+--------------------------
 integer | numeric | numeric | unknown | boolean  | timestamp with time zone
(1 row)

 salary  | hire_date | email
---------+-----------+-------
 numeric | date      | text
(1 row)
```

Notice: `42.0` is `numeric` (exact), not a float, and a quoted literal starts out as type `unknown` — PostgreSQL decides its type from context. That's why `WHERE hire_date >= '2023-01-01'` works: the string is read as a `date` because it's compared with one.

Casting (explicit conversion) has three spellings:

Form	Standard?	Example
`CAST(expr AS type)`	yes — works everywhere	`CAST('2025-03-18' AS date)`
`expr::type`	PostgreSQL shorthand	`'2025-03-18'::date`
`type 'literal'`	standard, literals only	`date '2025-03-18'`

```
SELECT CAST('2025-03-18' AS date) AS std_cast, '2025-03-18'::date AS pg_cast, date '2025-03-18' AS typed_literal;
SELECT CAST(100 AS numeric(5,2)) AS to_numeric, 3.99::integer AS rounds_not_truncates,
       '42'::integer + 1 AS text_to_int, 123::text || 'abc' AS int_to_text;
```

Output:

```
  std_cast  |  pg_cast   | typed_literal
------------+------------+--------------
 2025-03-18 | 2025-03-18 | 2025-03-18
(1 row)

 to_numeric | rounds_not_truncates | text_to_int | int_to_text
------------+----------------------+-------------+-------------
     100.00 |                    4 |          43 | 123abc
(1 row)
```

Casting a decimal to `integer` **rounds** (3.99 → 4); use `trunc()` or `floor()` if you want to cut off the fraction.

Exact vs approximate numbers

`double precision` stores binary fractions, and most decimal fractions (like 0.1) have no exact binary form:

```
SELECT 0.1 + 0.2 = 0.3 AS numeric_equal,
       0.1::double precision + 0.2::double precision = 0.3 AS float_equal,
       0.1::double precision + 0.2::double precision AS float_sum;
SELECT 12.345::numeric(5,2) AS rounded_to_scale;
```

Output:

```
 numeric_equal | float_equal |      float_sum
---------------+-------------+---------------------
 t             | f           | 0.30000000000000004
(1 row)

 rounded_to_scale
------------------
            12.35
(1 row)
```

Rule: **money and anything that must add up exactly → `numeric`**. A `numeric(p,s)` value is rounded to *s* decimal places when stored.

Errors a strict type system gives you

```
SELECT 2147483647::integer + 1;
SELECT 'abc'::integer;
SELECT 1000::numeric(5,2);
SELECT first_name + 1 FROM employees LIMIT 1;
```

Output:

```
ERROR:  integer out of range
ERROR:  invalid input syntax for type integer: "abc"
LINE 1: SELECT 'abc'::integer;
               ^
ERROR:  numeric field overflow
DETAIL:  A field with precision 5, scale 2 must round to an absolute value less than 10^3.
ERROR:  operator does not exist: text + integer
LINE 1: SELECT first_name + 1 FROM employees LIMIT 1;
                          ^
HINT:  No operator matches the given name and argument types. You might need to add explicit type casts.
```

MySQL would quietly turn `'abc' + 1` into `1` (with a warning) and wrap or clip some overflows (values too big for their type) depending on its SQL mode (a server setting that controls how strict MySQL is). PostgreSQL refuses — annoying for a minute, valuable for years.

Booleans accept several spellings on input:

```sql
SELECT 'yes'::boolean AS yes, 'f'::boolean AS f, 'on'::boolean AS on_;
```

```
 yes | f | on_
-----+---+-----
 t   | f | t
(1 row)
```

Common mistakes

- Storing money in `real` / `double precision`.
- Storing numbers or dates in text columns — sorting and comparisons become alphabetical (`'10' < '9'`).
- Expecting `::integer` to truncate; it rounds.

Self-check

1. *Type of the literal `42.0` in PostgreSQL?* → `numeric`.
2. *Portable way to convert text to a date?* → `CAST('2025-03-18' AS date)`.
3. *Why is `0.1::float8 + 0.2::float8 = 0.3` false?* → Binary floating point can't represent 0.1 or 0.2 exactly.

TL;DR

Everything has a type. Use `numeric` for money, `text` for strings, `timestamptz` for instants. Convert with `CAST(x AS t)` or `x::t`. PostgreSQL won't silently coerce (automatically convert) mismatched types — that's a feature.

3.2 Number functions

What it is

Numeric functions do arithmetic you'd otherwise hand-code: rounding for display, absolute differences, powers and logarithms for growth (a **logarithm** answers "to what power must I raise the base to get this number?" — log base 10 of 1000 is 3), modulo for "every n-th", random numbers for sampling.

The functions

Function (PostgreSQL)	Does	Example → result
`abs(x)`	absolute value	`abs(-10)` → 10
`round(x)` / `round(x, d)`	round to *d* decimals (negative *d* rounds left of the point)	`round(9.876, 1)` → 9.9; `round(1234.567, -2)` → 1200
`trunc(x, d)`	cut to *d* decimals, no rounding	`trunc(9.876, 1)` → 9.8
`ceil(x)` / `ceiling(x)`	smallest integer ≥ x (round **up**)	`ceil(4.3)` → 5; `ceil(-4.3)` → -4
`floor(x)`	largest integer ≤ x (round **down**)	`floor(4.8)` → 4; `floor(-4.3)` → -5
`sqrt(x)`, `cbrt(x)`	square / cube root	`sqrt(25)` → 5
`power(a, b)`, `a ^ b`	a to the power b	`power(2, 3)` → 8
`mod(a, b)`, `a % b`	remainder	`mod(17, 5)` → 2
`div(a, b)`	integer quotient	`div(17, 5)` → 3
`exp(x)`	e^x (e ≈ 2.718, the natural growth constant)	`exp(2)` → 7.389…
`ln(x)`	natural log (base e)	`ln(10)` → 2.302585…
`log(x)`, `log10(x)`	**base-10** log in PostgreSQL	`log(100)` → 2
`log(b, x)`	log of x in base b	`log(2, 8)` → 3
`sign(x)`	-1, 0 or 1	`sign(-15)` → -1
`pi()`, `degrees(r)`, `radians(d)`	π and angle conversion	`radians(60)` → 1.047…
`greatest(…)`, `least(…)`	largest / smallest argument	`greatest(3, 9, 4)` → 9
`gcd(a, b)`, `lcm(a, b)`	greatest common divisor / least common multiple (PostgreSQL 13+)	`gcd(12, 18)` → 6
`random()`, `setseed(s)`	random `double precision` in [0, 1) (0 included, 1 excluded); seed it (give a fixed starting value) for repeatable output	
`to_char(x, fmt)`	format a number as text	`to_char(1234567, 'FM9,999,999.00')` → `1,234,567.00`

How it works — run them

```sql
-- code/02_number_functions.sql
SELECT abs(-10) AS abs, round(9.876, 1) AS round_1, round(9.876) AS round_0,
       round(1234.567, -2) AS round_neg, trunc(9.876, 1) AS trunc_1,
       ceil(4.3) AS ceil, floor(4.8) AS floor, ceil(-4.3) AS ceil_neg, floor(-4.3) AS floor_neg;
```

Output:

```
 abs | round_1 | round_0 | round_neg | trunc_1 | ceil | floor | ceil_neg | floor_neg
-----+---------+---------+-----------+---------+------+-------+----------+-----------
  10 |     9.9 |      10 |      1200 |     9.8 |    5 |     4 |       -4 |        -5
(1 row)
```

With negative numbers `ceil` moves toward zero and `floor` away from it — `floor(-4.3)` is -5, not -4. `trunc` always moves toward zero.

Halfway cases depend on the type:

```
SELECT round(2.5) AS num_2_5, round(-2.5) AS num_neg_2_5,
       round(2.5::double precision) AS dbl_2_5, round(3.5::double precision) AS dbl_3_5;
```

Output:

```
 num_2_5 | num_neg_2_5 | dbl_2_5 | dbl_3_5
---------+-------------+---------+---------
       3 |          -3 |       2 |       4
(1 row)
```

`numeric` rounds halves **away from zero** (2.5 → 3); `double precision` typically rounds halves **to the nearest even** number ("banker's rounding", 2.5 → 2, 3.5 → 4) because it uses the platform's floating-point rounding. Another reason to keep money in `numeric`.

```
SELECT sqrt(25) AS sqrt, power(2, 3) AS power, 2 ^ 3 AS caret, mod(17, 5) AS mod,
       17 % 5 AS pct, mod(-17, 5) AS mod_neg, div(17, 5) AS int_div;
SELECT exp(2) AS exp_2, ln(10) AS ln_10, log(100) AS log_100, log10(100) AS log10_100,
       log(2, 8) AS log_base2_of_8;
SELECT sign(-15) AS sign, pi() AS pi, degrees(1.047) AS degrees, radians(60) AS radians;
SELECT greatest(3, 9, 4) AS greatest, least(3, 9, 4) AS least, gcd(12, 18) AS gcd, lcm(4, 6) AS lcm;
```

Output:

```
 sqrt | power | caret | mod | pct | mod_neg | int_div
------+-------+-------+-----+-----+---------+---------
    5 |     8 |     8 |   2 |   2 |      -2 |       3
(1 row)

      exp_2       |       ln_10       | log_100 | log10_100 |   log_base2_of_8
------------------+-------------------+---------+-----------+--------------------
 7.38905609893065 | 2.302585092994046 |       2 |         2 | 3.0000000000000000
(1 row)

 sign |        pi         |      degrees      |      radians
------+-------------------+-------------------+--------------------
   -1 | 3.141592653589793 | 59.98868115019719 | 1.0471975511965976
(1 row)

 greatest | least | gcd | lcm
----------+-------+-----+-----
        9 |     3 |   6 |  12
(1 row)
```

Two things to remember: the remainder keeps the sign of the **dividend** (the number being divided; `mod(-17, 5)` = -2), and `log(x)` means **base 10** in PostgreSQL but **natural log** in MySQL and SQL Server. Use `ln()` when you mean natural log — it's unambiguous everywhere that has it.

Random numbers

`random()` returns a new value each call. `setseed()` makes the sequence repeatable for the rest of the session (your current connection to the database) — useful for tests and reproducible samples.

```
SELECT setseed(0.42);
SELECT round(random()::numeric, 4) AS r1, round(random()::numeric, 4) AS r2;
SELECT floor(random() * 6)::int + 1 AS dice_roll;
```

Output:

```
   r1   |   r2
--------+--------
 0.5699 | 0.2954
(1 row)

 dice_roll
-----------
         6
(1 row)
```

`floor(random() * n)::int + 1` gives a whole number from 1 to *n*. (Sampling rows randomly — `ORDER BY random() LIMIT k` or `TABLESAMPLE` — comes up in chapter 12.)

Formatting numbers for people

`to_char(number, pattern)` turns numbers into display strings. `9` is a digit position, `0` a digit that's always shown, `,` a group separator, `.` the decimal point, and `FM` ("fill mode") strips the padding spaces `to_char` otherwise adds.

```sql
SELECT to_char(1234567, 'FM9,999,999.00') AS with_commas,
       to_char(0.0725, 'FM990.00%') AS naive_pct,
       to_char(0.0725 * 100, 'FM990.00') || '%' AS pct,
       to_char(42, '00000') AS zero_padded;
```

Output:

```
 with_commas  | naive_pct |  pct  | zero_padded
--------------+-----------+-------+-------------
 1,234,567.00 | 0.07%     | 7.25% |  00042
(1 row)
```

`%` in the pattern is printed literally — it does *not* multiply by 100, so do that yourself. Without `FM`, `to_char` reserves a leading space for the sign (see `' 00042'`). Format only at the very end: the result is text and no longer sorts or sums as a number.

Real use on the sample data

```sql
SELECT order_id, product_id,
       quantity * unit_price * (1 - discount)           AS exact,
       round(quantity * unit_price * (1 - discount), 2)  AS rounded
FROM order_items WHERE order_id IN (104, 107, 109) ORDER BY order_id, product_id;

-- Salary per month, rounded up to the next hundred
SELECT first_name, salary, ceil(salary / 12 / 100) * 100 AS monthly_rounded_up
FROM employees WHERE emp_id <= 4 ORDER BY emp_id;
```

Output:

```
 order_id | product_id |  exact   | rounded
----------+------------+----------+---------
      104 |          5 | 479.0000 |  479.00
      104 |          6 | 332.4905 |  332.49
      107 |          6 | 629.9820 |  629.98
      109 |          4 | 151.3000 |  151.30
(4 rows)

 first_name |  salary   | monthly_rounded_up
------------+-----------+--------------------
 Asha       | 250000.00 |              20900
 Vikram     | 185000.00 |              15500
 Neha       | 142000.00 |              11900
 Rahul      |  98000.00 |               8200
(4 rows)
```

The "divide, round to a unit, multiply back" trick (`ceil(x / 100) * 100`) works for any step size — to the nearest 5 minutes, 1,000 rupees, and so on.

Dialect map (numbers)

Task	MySQL	PostgreSQL	SQL Server	SQLite
Truncate decimals	`TRUNCATE(x, d)`	`trunc(x, d)`	`ROUND(x, d, 1)`	— (cast tricks)
Random 0–1	`RAND()`	`random()`	`RAND()`	`random()` returns a 64-bit integer
Natural log	`LOG(x)` or `LN(x)`	`ln(x)`	`LOG(x)`	`ln(x)` (3.35+ math functions)
Base-10 log	`LOG10(x)`	`log(x)` / `log10(x)`	`LOG10(x)`	`log(x)` / `log10(x)`
Power	`POWER(a,b)` / `POW`	`power(a,b)` / `a ^ b`	`POWER(a,b)` (`^` is XOR, a bit-flipping operator!)	`pow(a,b)`
Number with commas	`FORMAT(x, 2)`	`to_char(x, 'FM999,999,990.00')`	`FORMAT(x, 'N2')`	no built-in (`printf('%,d', x)` groups integers only)
Convert type	`CONVERT(x, DECIMAL(5,2))` / `CAST`	`CAST(x AS numeric(5,2))` / `x::numeric(5,2)`	`CONVERT(DECIMAL(5,2), x)` / `CAST`	`CAST(x AS REAL)`
Integer division of ints	`DIV`	`/` or `div()`	`/`	`/`

Common mistakes

- Using `log()` expecting natural log in PostgreSQL.
- Formatting (`to_char`) inside calculations or before sorting.
- `round()` on `double precision` expecting "half up".

Self-check

1. *`floor(-2.5)`, `ceil(-2.5)`, `trunc(-2.5)`?* → -3, -2, -2.
2. *`round(123.456, -1)`?* → 120.
3. *PostgreSQL function for MySQL's `LOG(x)`?* → `ln(x)`.

TL;DR

`round/trunc/ceil/floor` for rounding (watch negatives and halves), `power/sqrt/exp/ln/log` for maths (PostgreSQL `log` is base 10), `mod` / `%` for remainders, `random` + `setseed` for repeatable randomness, `to_char` for display only.

3.3 String functions

What it is

String (text) functions join, cut, measure, clean and search text. They are the tools for turning messy input — stray spaces, mixed case, combined fields — into consistent values.

The functions

Function (PostgreSQL)	Does	Example → result
`concat(a, b, …)` / `a || b`	join strings	`concat('Hello', ' ', 'World')` → `Hello World`
`concat_ws(sep, a, b, …)`	join with a separator, skipping NULLs	`concat_ws('-', '2024', '01', '01')` → `2024-01-01`
`substring(s FROM start FOR len)` / `substr(s, start, len)`	part of a string (positions start at 1)	`substring('SQL Functions' FROM 1 FOR 3)` → `SQL`
`left(s, n)`, `right(s, n)`	first / last *n* characters (negative *n* = all but)	`left('Database', 3)` → `Dat`
`length(s)` / `char_length(s)`	number of characters	`length('Data')` → 4
`octet_length(s)`	number of bytes	`octet_length('Data')` → 4
`upper(s)`, `lower(s)`, `initcap(s)`	change case	`initcap('desk lamp')` → `Desk Lamp`
`replace(s, from, to)`	replace every occurrence of a substring	`replace('I love SQL', 'SQL', 'Postgres')`
`translate(s, from, to)`	replace character by character	`translate('2025/01/03', '/', '-')`
`trim(s)`, `ltrim`, `rtrim`, `trim(BOTH 'x' FROM s)`	remove spaces (or given characters) from ends	`trim(' SQL ')` → `SQL`
`position(sub IN s)` / `strpos(s, sub)`	1-based position of first match, 0 if absent	`strpos('Hello World', 'World')` → 7
`lpad(s, n, fill)`, `rpad(s, n, fill)`	pad (or cut) to length *n*	`lpad('7', 3, '0')` → `007`
`repeat(s, n)`, `reverse(s)`	repeat / reverse	`repeat('La', 3)` → `LaLaLa`
`split_part(s, delim, n)`	*n*-th field of a delimited string (fields separated by a marker such as `.` or `,`) (negative *n* from the end, PostgreSQL 14+)	`split_part('a@b.com', '@', 2)` → `b.com`
`format(fmt, …)`	printf-style building (fill `%s` slots in a template); `%I` quotes identifiers (table/column names), `%L` literals	
`regexp_replace`, `regexp_match`	regular-expression replace / capture	

How it works — concatenation and NULL

```sql
-- code/03_string_functions.sql
SELECT concat('Hello', ' ', 'World') AS concat, 'Hello' || ' ' || 'World' AS pipes,
       concat_ws('-', '2024', '01', '01') AS concat_ws;
SELECT concat('a', NULL, 'b') AS concat_null, 'a' || NULL || 'b' AS pipes_null,
       concat_ws(', ', 'Pune', NULL, 'India') AS ws_null;
```

Output:

```
   concat    |    pipes    | concat_ws
-------------+-------------+------------
 Hello World | Hello World | 2024-01-01
(1 row)

 concat_null | pipes_null |   ws_null
-------------+------------+-------------
 ab          | (null)     | Pune, India
(1 row)
```

In PostgreSQL `concat()` **ignores** NULL arguments, while `||` returns NULL if any part is NULL. `concat_ws` skips NULLs, so it's ideal for addresses with optional parts. **MySQL differs:** its `CONCAT()` returns NULL if any argument is NULL (its `CONCAT_WS` skips them).

Extracting parts

```
SELECT substring('SQL Functions' FROM 1 FOR 3) AS std_form, substring('SQL Functions', 5) AS from_5,
       substr('SQL Functions', 5, 4) AS substr, left('Database', 3) AS left_3,
       right('Postgres', 3) AS right_3, left('Database', -2) AS left_neg;
```

Output:

```
 std_form |  from_5   | substr | left_3 | right_3 | left_neg
----------+-----------+--------+--------+---------+----------
 SQL      | Functions | Func   | Dat    | res     | Databa
(1 row)
```

String positions in SQL start at **1**, not 0.

Measuring

```
SELECT length('Data') AS length, char_length('Data') AS char_length, octet_length('Data') AS bytes,
       length('  padded  ') AS with_spaces;
```

Output:

```
 length | char_length | bytes | with_spaces
--------+-------------+-------+-------------
      4 |           4 |     4 |          10
(1 row)
```

For plain ASCII (the 128 basic English letters, digits and symbols, one byte each), characters and bytes are the same. In a UTF-8 database (UTF-8 is the universal character encoding — the rule for storing any language's characters as bytes, using 1 to 4 bytes each) — which is what the book's runner creates (`ENCODING 'UTF8'`, `C` collation) — `é` counts as one character yet occupies two bytes:

```
SELECT current_setting('server_encoding') AS encoding, char_length('café') AS chars,
       octet_length('café') AS bytes, upper('café') AS upper_c_collation,
       upper('café' COLLATE "en-x-icu") AS upper_icu;
```

Output:

```
 encoding | chars | bytes | upper_c_collation | upper_icu
----------+-------+-------+-------------------+-----------
 UTF8     |     4 |     5 | CAFé              | CAFÉ
(1 row)
```

Notice the last two columns: case conversion follows the **collation**, and the plain `C` collation only knows the ASCII letters, so `é` stays lower case. A language-aware collation (here the ICU collation `en-x-icu`, available when PostgreSQL is built with ICU — International Components for Unicode, a widely used library of language rules) converts it properly. (A database created with the legacy `SQL_ASCII` encoding would not understand multibyte characters (characters stored in more than one byte) at all and would report 5 characters — always create databases as UTF-8.)

Portability trap: MySQL's `LENGTH()` counts **bytes** (use `CHAR_LENGTH()` for characters); SQL Server's `LEN()` counts characters but ignores trailing spaces (`DATALENGTH()` gives bytes).

Changing case, replacing, repeating

```
SELECT upper('hello') AS upper, lower('WORLD') AS lower, initcap('noise-cancelling headphones') AS initcap;
SELECT replace('I love SQL', 'SQL', 'Postgres') AS replace, translate('2025/01/03', '/', '-') AS translate,
       reverse('stressed') AS reverse, repeat('La', 3) AS repeat;
```

Output:

```
 upper | lower |           initcap
-------+-------+----------------------------
 HELLO | world | Noise-Cancelling Headphones
(1 row)

     replace     | translate  | reverse  | repeat
-----------------+------------+----------+--------
 I love Postgres | 2025-01-03 | desserts | LaLaLa
(1 row)
```

`upper`/`lower` are the standard way to compare case-insensitively: `WHERE lower(email) = lower(:input)` (`:input` stands for a value the application supplies). `replace` swaps whole substrings; `translate` maps single characters one-to-one.

Trimming

```
SELECT '[' || trim('   SQL   ') || ']' AS trim, '[' || ltrim('   SQL   ') || ']' AS ltrim,
       '[' || rtrim('   SQL   ') || ']' AS rtrim, trim(BOTH '*' FROM '**SQL**') AS trim_chars;
```

Output:

```
 trim  |  ltrim   |  rtrim   | trim_chars
-------+----------+----------+------------
 [SQL] | [SQL   ] | [   SQL] | SQL
(1 row)
```

(The brackets make the invisible spaces visible.) `trim` removes only leading and trailing characters, never those in the middle.

Searching and padding

```
SELECT position('abc' IN 'xxabcdef') AS position, strpos('Hello World', 'World') AS strpos,
       position('zzz' IN 'Hello') AS not_found;
SELECT lpad('SQL', 5, '*') AS lpad, rpad('SQL', 5, '*') AS rpad, lpad('7', 3, '0') AS zero_pad,
       lpad('Postgres', 3) AS lpad_truncates;
```

Output:

```
 position | strpos | not_found
----------+--------+-----------
        3 |      7 |         0
(1 row)

 lpad  | rpad  | zero_pad | lpad_truncates
-------+-------+----------+----------------
 **SQL | SQL** | 007      | Pos
(1 row)
```

`lpad` both pads and **truncates** to the requested length — careful with long values.

Splitting delimited text

```
SELECT split_part('www.example.com', '.', 1) AS first, split_part('www.example.com', '.', -1) AS last,
       array_to_string((string_to_array('www.example.com', '.'))[1:2], '.') AS first_two;
```

Output:

```
 first | last |  first_two
-------+------+-------------
 www   | com  | www.example
(1 row)
```

`split_part` replaces MySQL's `SUBSTRING_INDEX`: `SUBSTRING_INDEX(s, '.', -1)` (text after the last dot) is `split_part(s, '.', -1)`; "everything before the second dot" (`SUBSTRING_INDEX(s, '.', 2)`) needs the array form shown (an **array** is an ordered list of values held in one cell; `[1:2]` takes its first two elements). (Splitting into *rows* uses `string_to_table` / `unnest`, chapter 14.)

format() and regular expressions

```
SELECT format('%s earns %s', first_name, salary) AS sentence,
       format('SELECT * FROM %I WHERE name = %L', 'my table', 'O''Brien') AS safe_sql
FROM employees WHERE emp_id = 7;
SELECT regexp_replace('(022) 555-0199', '[^0-9]', '', 'g') AS digits_only,
       (regexp_match('order #A-1042 shipped', '#([A-Z])-(\d+)'))[2] AS order_number;
```

Output:

```
       sentence       |                    safe_sql
----------------------+-------------------------------------------------
 Priya earns 72000.00 | SELECT * FROM "my table" WHERE name = 'O''Brien'
(1 row)

 digits_only | order_number
-------------+--------------
 0225550199  | 1042
(1 row)
```

`%I` and `%L` quote an identifier and a literal safely — the basis of safe dynamic SQL (SQL text built as a string while the program runs, then executed; quoting stops *SQL injection*, where crafted input changes the meaning of the query — chapter 14). `regexp_replace(…, 'g')` replaces all matches (`g` = global); `regexp_match` returns an array of captured groups (the pieces of text matched by each bracketed part of the pattern), and `[2]` picks the second.

Cleaning real data

Our two Anita Shah rows differ only in formatting. Normalising them shows they're the same person:

```
SELECT customer_id, email AS raw, lower(trim(email)) AS clean, initcap(city) AS city_fixed
FROM customers WHERE last_name = 'Shah' ORDER BY customer_id;
```

Output:

```
 customer_id |           raw             |          clean          | city_fixed
-------------+---------------------------+-------------------------+------------
           1 | anita.shah@mail.example   | anita.shah@mail.example | Mumbai
           8 |   Anita.Shah@Mail.example | anita.shah@mail.example | Mumbai
(2 rows)
```

Better still, clean on the way *in* (application validation, a `CHECK` constraint — a rule the database tests on every insert — or a unique index on `lower(email)`, which refuses a second row with the same value — chapter 12).

Full names and email domains:

```sql
SELECT first_name || ' ' || last_name AS full_name,
       split_part(email, '@', 2) AS domain
FROM employees WHERE emp_id IN (1, 3, 5) ORDER BY emp_id;
```

Output:

```
 full_name  |     domain
------------+-----------------
 Asha Rao   | company.example
 Neha Gupta | Company.example
 Meera Iyer | (null)
(3 rows)
```

Meera has no email, so the domain is NULL — functions given NULL generally return NULL.

Dialect map (strings)

Task	MySQL	PostgreSQL	SQL Server	SQLite
Concatenate	`CONCAT(a,b)` (NULL if any NULL)	`a || b`, `concat()` (skips NULL)	`a + b`, `CONCAT()` (NULL → '')	`a || b`, `concat()` (3.44+)
Characters	`CHAR_LENGTH(s)`	`length(s)` / `char_length(s)`	`LEN(s)` (no trailing spaces)	`length(s)`
Bytes	`LENGTH(s)`	`octet_length(s)`	`DATALENGTH(s)`	`length(CAST(s AS BLOB))` (BLOB = raw binary data)
Find position	`LOCATE(sub, s)`, `INSTR(s, sub)`, `POSITION(sub IN s)`	`position(sub IN s)`, `strpos(s, sub)`	`CHARINDEX(sub, s)`	`instr(s, sub)`
Split field	`SUBSTRING_INDEX(s, d, n)`	`split_part(s, d, n)`	`STRING_SPLIT` (rows) / `PARSENAME`	— (use `instr` + `substr`)
Left/right	`LEFT`, `RIGHT`	`left`, `right`	`LEFT`, `RIGHT`	`substr(s, 1, n)`, `substr(s, -n)`
Pad	`LPAD`, `RPAD`	`lpad`, `rpad`	`RIGHT(REPLICATE('0', n) + s, n)`	`printf('%05d', n)`
Repeat	`REPEAT`	`repeat`	`REPLICATE`	—
Regex replace	`REGEXP_REPLACE` (8.0+)	`regexp_replace`	`REGEXP_REPLACE` (SQL Server 2025)	extension

Common mistakes

- Assuming positions start at 0.
- Using `||` when a part may be NULL and getting a NULL result; use `concat`/`concat_ws` or `COALESCE`.
- Counting bytes instead of characters (MySQL `LENGTH`).
- Comparing case-insensitively without `lower()`/`ILIKE` in PostgreSQL.

Self-check

1. *Get the text after the `@` in an email in PostgreSQL?* → `split_part(email, '@', 2)`.
2. *`'a' || NULL` vs `concat('a', NULL)`?* → NULL vs `'a'`.
3. *Pad an order number 42 to `00042`?* → `lpad(42::text, 5, '0')` (or `to_char(42, 'FM00000')`).

TL;DR

`||`/`concat`/`concat_ws` join (mind NULLs), `substring`/`left`/`right`/`split_part` cut, `length`/`octet_length` measure, `lower`/`upper`/`initcap`/`trim`/`replace` clean, `position`/`strpos` search, `lpad`/`rpad` pad, `format` and `regexp_*` for power tasks.

3.4 Date and time functions

What it is

Time values cause more subtle SQL bugs than any other kind of data: months have different lengths, time zones shift, and "now" depends on when you ask. PostgreSQL has a rich, precise set of types and functions for them.

The types

Type	Holds	Example
`date`	a calendar date	`2025-03-18`
`time`	a time of day	`19:30:00`
`timestamp` (without time zone)	date + time, *no* zone — a wall-clock reading	`2025-03-18 19:30:00`
`timestamptz` (with time zone)	an absolute **instant** (one exact moment, the same everywhere on Earth); stored in UTC, shown in the session's time zone	`2025-03-18 14:00:00+00`
`interval`	a duration	`3 days`, `1 mon`, `02:30:00`

Guideline: use `timestamptz` for "when did it happen" (events, orders, logins) and `date` for calendar facts (birthdays, hire dates). Our sample `orders.placed_at` is a plain `timestamp` to keep outputs identical on every machine.

"Now"

A **transaction** is a group of statements that succeed or fail together, started with `BEGIN` and ended with `COMMIT` (a single statement on its own is also a transaction). Several "now" functions freeze at the transaction's start:

Function	Returns	Changes within a transaction?
`current_date`	today's `date`	no
`current_time`	time of day with zone (`timetz`)	no
`localtime`	time of day without zone	no
`now()` = `current_timestamp` = `transaction_timestamp()`	`timestamptz` at transaction start	**no**
`localtimestamp`	`timestamp` at transaction start	no
`statement_timestamp()`	start of the current statement	per statement
`clock_timestamp()`	the actual current time	**yes, every call**

```sql
-- code/04_date_functions.sql (the file sets SET TIME ZONE 'UTC' first)
SELECT current_date AS today, current_time AS time_with_tz, localtime AS local_time,
       now() AS now_tstz, localtimestamp AS local_ts;
```

Output (from the run that produced this book; yours will show your current time):

```
   today    |    time_with_tz    |   local_time    |           now_tstz            |          local_ts
------------+--------------------+-----------------+-------------------------------+---------------------------
 2026-09-28 | 19:15:38.702642+00 | 19:15:38.702642 | 2026-09-28 19:15:38.702642+00 | 2026-09-28 19:15:38.702642
(1 row)
```

`now()` is frozen for the whole transaction, so every row inserted in one transaction gets the same timestamp — usually what you want. `clock_timestamp()` keeps moving:

```
BEGIN;
SELECT now() AS tx_start, clock_timestamp() AS wall_clock;
SELECT pg_sleep(0.2);
SELECT now() AS tx_start_again, clock_timestamp() AS wall_clock_later;
COMMIT;
```

Output:

```
           tx_start            |          wall_clock
-------------------------------+-------------------------------
 2026-09-28 19:15:38.702968+00 | 2026-09-28 19:15:38.703095+00
(1 row)

        tx_start_again         |       wall_clock_later
-------------------------------+-------------------------------
 2026-09-28 19:15:38.702968+00 | 2026-09-28 19:15:38.905532+00
(1 row)
```

Taking the date part of a timestamp

MySQL's `DATE(ts)` is a cast in PostgreSQL:

```
SELECT placed_at, placed_at::date AS just_date, date_trunc('day', placed_at) AS truncated
FROM orders WHERE order_id = 103;
```

Output:

```
      placed_at      | just_date  |      truncated
---------------------+------------+---------------------
 2025-01-31 23:55:00 | 2025-01-31 | 2025-01-31 00:00:00
(1 row)
```

`::date` returns a `date`; `date_trunc('day', …)` returns a timestamp at midnight.

Date arithmetic

Expression	Result type	Meaning
`date + integer`	date	add days
`date - date`	integer	days between
`date/timestamp ± interval`	timestamp	add/subtract a duration
`timestamp - timestamp`	interval	elapsed time
`age(later, earlier)`	interval	"years mons days" difference

```
SELECT date '2024-01-01' + 3 AS plus_3_days, date '2024-01-01' + interval '3 days' AS plus_interval,
       date '2024-01-01' - interval '1 month' AS minus_month, date '2024-01-05' - date '2024-01-01' AS
days_between;
```

Output:

```
 plus_3_days |    plus_interval    |     minus_month     | days_between
-------------+---------------------+---------------------+--------------
 2024-01-04  | 2024-01-04 00:00:00 | 2023-12-01 00:00:00 |            4
(1 row)
```

Month arithmetic clamps to the last valid day, which makes it non-reversible:

```
SELECT date '2025-01-31' + interval '1 month' AS jan31_plus_month,
       date '2024-02-29' + interval '1 year' AS leap_plus_year,
       date '2025-01-31' + interval '1 month' - interval '1 month' AS round_trip;
```

Output:

```
  jan31_plus_month   |   leap_plus_year    |     round_trip
---------------------+---------------------+---------------------
 2025-02-28 00:00:00 | 2025-02-28 00:00:00 | 2025-01-28 00:00:00
(1 row)
```

31 January + 1 month = 28 February, and going back a month lands on 28 January, not the 31st. Billing systems must decide explicitly what "one month later" means.

Extracting fields: EXTRACT and date_part

`extract(field FROM value)` is standard SQL; `date_part('field', value)` is PostgreSQL's function form. Since PostgreSQL 14, `extract` returns `numeric`.

```
SELECT extract(YEAR FROM date '2024-01-01') AS year, extract(MONTH FROM date '2024-01-01') AS month,
       extract(DAY FROM date '2024-01-01') AS day, extract(DOW FROM date '2024-01-01') AS dow_0_sun,
       extract(ISODOW FROM date '2024-01-01') AS isodow_1_mon, extract(QUARTER FROM date '2024-08-15') AS quarter,
       date_part('week', date '2024-01-01') AS iso_week;
```

Output:

```
 year | month | day | dow_0_sun | isodow_1_mon | quarter | iso_week
------+-------+-----+-----------+--------------+---------+----------
 2024 |     1 |   1 |         1 |            1 |       3 |        1
(1 row)
```

1 January 2024 was a **Monday**: `DOW` numbers Sunday 0 … Saturday 6, `ISODOW` numbers Monday 1 … Sunday 7. Other useful fields: `HOUR`, `MINUTE`, `SECOND`, `DOY` (day of year), `EPOCH` (seconds since 1970-01-01 UTC).

Truncating to a period: date_trunc

`date_trunc(unit, ts)` rounds *down* to the start of the unit — the workhorse of time-series reports (reports of values over time, such as "sales per month"):

```
SELECT placed_at, date_trunc('month', placed_at) AS month_start, date_trunc('week', placed_at) AS week_start,
       date_trunc('hour', placed_at) AS hour_start
FROM orders WHERE order_id = 107;
```

Output:

```
      placed_at      |     month_start     |     week_start      |     hour_start
---------------------+---------------------+---------------------+---------------------
 2025-03-18 19:30:00 | 2025-03-01 00:00:00 | 2025-03-17 00:00:00 | 2025-03-18 19:00:00
(1 row)
```

Weeks start on **Monday** (ISO 8601, the international standard for writing dates and times). Grouping by `date_trunc('month', …)` gives one sortable key per month — see exercise E5 in 3.6.

Formatting and parsing

`to_char(value, pattern)` formats; `to_date` / `to_timestamp` parse; `make_date` / `make_timestamp` build from numbers.

Pattern	Meaning	Example
`YYYY`, `MM`, `DD`	year, month number, day	`2025`, `03`, `18`
`Month`, `Mon`	month name, abbreviation	`March`, `Mar`
`Day`, `Dy`	weekday name, abbreviation	`Tuesday`, `Tue`
`HH24`, `HH12`, `MI`, `SS`, `AM`	hour, minute, second, meridiem	`19`, `07`, `30`, `00`, `PM`
`Q`	quarter	`1`
`FM` prefix	drop padding spaces/zeros	`FMDay` → `Monday`
`"text"`	literal text	`"Q"`

```
SELECT to_char(date '2024-01-01', 'FMDay, FMMonth FMDD, YYYY') AS long_form,
       to_char(timestamp '2025-03-18 19:30', 'DD Mon YYYY HH12:MI AM') AS short_form,
       to_char(timestamp '2025-03-18 19:30', 'YYYY-"Q"Q') AS quarter_label;
SELECT to_date('18/03/2025', 'DD/MM/YYYY') AS parsed_date,
       to_timestamp('2025-03-18 19:30', 'YYYY-MM-DD HH24:MI') AS parsed_ts,
       make_date(2025, 3, 18) AS made_date;
```

Output:

```
        long_form        |      short_form      | quarter_label
-------------------------+----------------------+---------------
 Monday, January 1, 2024 | 18 Mar 2025 07:30 PM | 2025-Q1
(1 row)

 parsed_date |       parsed_ts        | made_date
-------------+------------------------+------------
 2025-03-18  | 2025-03-18 19:30:00+00 | 2025-03-18
(1 row)
```

`to_timestamp` returns a `timestamptz` interpreted in the session time zone (UTC here). For ISO strings (`'2025-03-18'`) a plain cast is simpler; use `to_date` for other layouts like `18/03/2025`, which is ambiguous otherwise.

Differences between moments

```
SELECT order_id, placed_at, shipped_at,
       shipped_at - placed_at AS time_to_ship,
       round(extract(EPOCH FROM shipped_at - placed_at) / 3600, 1) AS hours_to_ship,
       shipped_at::date - placed_at::date AS calendar_days
FROM orders WHERE shipped_at IS NOT NULL AND order_id <= 104 ORDER BY order_id;
```

Output:

```
 order_id |      placed_at      |     shipped_at      |  time_to_ship  | hours_to_ship | calendar_days
----------+---------------------+---------------------+----------------+---------------+---------------
      101 | 2025-01-03 10:15:00 | 2025-01-04 09:00:00 | 22:45:00       |          22.8 |             1
      102 | 2025-01-17 22:40:00 | 2025-01-19 11:30:00 | 1 day 12:50:00 |          36.8 |             2
      103 | 2025-01-31 23:55:00 | 2025-02-02 08:10:00 | 1 day 08:15:00 |          32.3 |             2
      104 | 2025-02-11 09:05:00 | 2025-02-12 16:45:00 | 1 day 07:40:00 |          31.7 |             1
(4 rows)
```

Three different answers to "how long did shipping take?": an `interval`, a number of hours (via `EPOCH` seconds ÷ 3600), and the number of calendar-date boundaries crossed. Order 101 took under a day but crossed one midnight. Decide which one the business means.

`age()` gives a human-style difference:

```
SELECT first_name, hire_date, age(date '2025-07-01', hire_date) AS tenure_on_2025_07_01
FROM employees WHERE emp_id IN (1, 9, 15) ORDER BY emp_id;
```

Output:

```
 first_name | hire_date  |  tenure_on_2025_07_01
------------+------------+------------------------
 Asha       | 2015-01-05 | 10 years 5 mons 27 days
 Sara       | 2024-04-01 | 1 year 3 mons
 Rohan      | 2025-06-02 | 29 days
(3 rows)
```

`age(x)` with one argument measures from `current_date` at midnight. Whole years of service: `extract(YEAR FROM age(current_date, hire_date))`.

Expected delivery ten days after ordering:

```
SELECT order_id, placed_at::date AS order_date, (placed_at + interval '10 days')::date AS expected_by
FROM orders WHERE order_id IN (101, 105, 113) ORDER BY order_id;
```

Output:

```
 order_id | order_date | expected_by
----------+------------+-------------
      101 | 2025-01-03 | 2025-01-13
      105 | 2025-02-28 | 2025-03-10
      113 | 2025-06-02 | 2025-06-12
(3 rows)
```

Time zones

A `timestamptz` is one instant; how it *prints* depends on the session's `TimeZone` setting. `AT TIME ZONE` converts between instants and wall-clock readings:

Expression	Gives
`timestamptz AT TIME ZONE 'Zone'`	the wall-clock `timestamp` in that zone at that instant
`timestamp AT TIME ZONE 'Zone'`	the `timestamptz` instant when that zone's clock showed that reading

```
SET TIME ZONE 'UTC';
SELECT timestamptz '2025-03-18 19:30:00+05:30' AS shown_in_utc;
SET TIME ZONE 'Asia/Kolkata';
SELECT timestamptz '2025-03-18 19:30:00+05:30' AS shown_in_ist,
       timestamptz '2025-03-18 19:30:00+05:30' AT TIME ZONE 'America/New_York' AS wall_clock_ny,
       timestamp '2025-03-18 19:30' AT TIME ZONE 'Europe/London' AS london_wall_clock_as_instant;
```

Output:

```
      shown_in_utc
------------------------
 2025-03-18 14:00:00+00
(1 row)

       shown_in_ist        |    wall_clock_ny    | london_wall_clock_as_instant
---------------------------+---------------------+------------------------------
 2025-03-18 19:30:00+05:30 | 2025-03-18 10:00:00 | 2025-03-19 01:00:00+05:30
(1 row)
```

7:30 pm in India is 14:00 UTC and 10:00 in New York (already on daylight-saving time on 18 March). 7:30 pm in London (still on GMT, Greenwich Mean Time — the UK's winter clock, equal to UTC) is 01:00 the next day in India. Use named zones (`'Asia/Kolkata'`), not fixed offsets, so daylight-saving rules are applied.

A calendar of dates

`generate_series` produces rows — here, one per day — which is how you build calendars or fill gaps in reports (chapter 7):

```
SELECT d::date AS day, to_char(d, 'Dy') AS weekday
FROM generate_series(date '2025-03-01', date '2025-03-05', interval '1 day') AS d;
```

Output:

```
    day     | weekday
------------+---------
 2025-03-01 | Sat
 2025-03-02 | Sun
 2025-03-03 | Mon
 2025-03-04 | Tue
 2025-03-05 | Wed
(5 rows)
```

Dialect map (dates)

Task	MySQL	PostgreSQL	SQL Server	SQLite
Now	`NOW()`	`now()`	`SYSDATETIME()` / `GETDATE()`	`datetime('now')`
Today	`CURRENT_DATE` / `CURDATE()`	`current_date`	`CAST(GETDATE() AS date)`	`date('now')`
Date part of datetime	`DATE(ts)`	`ts::date`	`CAST(ts AS date)`	`date(ts)`
Add 3 days	`DATE_ADD(d, INTERVAL 3 DAY)`	`d + interval '3 days'` (or `d + 3` for a date)	`DATEADD(day, 3, d)`	`date(d, '+3 days')`
Subtract a month	`DATE_SUB(d, INTERVAL 1 MONTH)`	`d - interval '1 month'`	`DATEADD(month, -1, d)`	`date(d, '-1 month')`
Year / month / day	`YEAR(d)`, `MONTH(d)`, `DAY(d)`, `EXTRACT`	`extract(YEAR FROM d)` …	`YEAR(d)`, `DATEPART(month, d)`	`strftime('%Y', d)`
Format	`DATE_FORMAT(d, '%W, %M %e, %Y')`	`to_char(d, 'FMDay, FMMonth FMDD, YYYY')`	`FORMAT(d, 'dddd, MMMM d, yyyy')`	`strftime(…)`
Days between	`DATEDIFF(end, start)`	`end::date - start::date`	`DATEDIFF(day, start, end)`	`julianday(end) - julianday(start)`
Difference in a unit	`TIMESTAMPDIFF(HOUR, start, end)`	`extract(EPOCH FROM end - start) / 3600`	`DATEDIFF(hour, start, end)`	`(julianday(end) - julianday(start)) * 24`
Truncate to month	`DATE_FORMAT(d, '%Y-%m-01')`	`date_trunc('month', d)`	`DATETRUNC(month, d)` (2022+)	`date(d, 'start of month')`

Note the argument orders: MySQL `DATEDIFF(end, start)` vs SQL Server `DATEDIFF(unit, start, end)`. SQL Server's `DATEDIFF` counts *boundaries crossed* (like `calendar_days` above), not elapsed time.

Common mistakes

- Comparing a timestamp column to a date-only upper bound (see chapter 2, 2.4) — use half-open ranges.
- Applying a function to an indexed column (one with an **index** — a sorted lookup structure that lets the database find rows without reading them all) in `WHERE` (`WHERE date_trunc('month', placed_at) = …`) — it can stop the index being used; rewrite as a range (chapter 12).
- Storing local times in `timestamp` for a multi-time-zone app — use `timestamptz`.
- Assuming "+1 month" can be undone with "−1 month".

In the interview

> **30-second answer:** "I store instants as timestamptz, truncate with date_trunc for per-day or per-month reports, filter with half-open ranges so indexes still work, and compute durations with timestamp subtraction or EXTRACT(EPOCH …). I'm careful that month arithmetic clamps and that now() is fixed per transaction."

- *Orders per month including months with zero orders?* → Generate the months with `generate_series` and left-join the counts (a *left join* keeps every row of the first table even when the second has no match; chapters 5 and 7).
- *Difference between `timestamp` and `timestamptz`?* → `timestamptz` is an absolute instant converted to/from the session zone; `timestamp` is a zone-less wall-clock reading.

Self-check

1. *`date '2025-03-31' - interval '1 month'`?* → `2025-02-28 00:00:00`.
2. *First day of the month of `placed_at`?* → `date_trunc('month', placed_at)`.
3. *Why might two `now()` calls in one transaction be equal?* → `now()` returns the transaction start time.
4. *Hours between two timestamps as a number?* → `extract(EPOCH FROM b - a) / 3600`.

TL;DR

`date`, `timestamp`, `timestamptz`, `interval`. `now()` (transaction start) vs `clock_timestamp()`. Add intervals; subtract dates for days; `extract` fields; `date_trunc` to periods; `to_char`/`to_date` to format/parse; `AT TIME ZONE` converts; month maths clamps.

3.5 Conditional expressions: CASE, COALESCE, NULLIF, GREATEST, LEAST

What it is

Conditional expressions put *if-then-else* logic inside a query. They are expressions, so they return a value and can be used wherever a column can.

CASE

Searched CASE tests conditions in order and returns the result of the first true one:

```sql
-- code/05_case_coalesce_nullif.sql
SELECT name, unit_price,
       CASE WHEN unit_price >= 300 THEN 'premium'
            WHEN unit_price >= 50  THEN 'standard'
            ELSE 'budget' END AS price_band
FROM products ORDER BY unit_price DESC;
```

Output:

```
            name             | unit_price | price_band
-----------------------------+------------+------------
 Laptop Pro 14               |    1299.00 | premium
 Standing Desk               |     499.00 | premium
 Ergonomic Chair             |     349.99 | premium
 Noise-Cancelling Headphones |     199.95 | standard
 Mechanical Keyboard         |      89.00 | standard
 Gift Card                   |      50.00 | standard
 USB-C Hub                   |      49.50 | budget
 Desk Lamp                   |      35.00 | budget
 Wireless Mouse              |      24.99 | budget
(9 rows)
```

Order matters: a 1,299 laptop is also `>= 50`, but the first matching branch wins. Write branches from most to least specific.

Simple CASE compares one expression with a list of values:

```
SELECT order_id, status,
       CASE status WHEN 'pending' THEN 'Awaiting dispatch'
                   WHEN 'shipped' THEN 'On the way'
                   WHEN 'delivered' THEN 'Done'
       END AS customer_message
FROM orders WHERE order_id IN (101, 105, 107, 113) ORDER BY order_id;
```

Output:

```
 order_id |  status   | customer_message
----------+-----------+------------------
      101 | delivered | Done
      105 | cancelled | (null)
      107 | shipped   | On the way
      113 | pending   | Awaiting dispatch
(4 rows)
```

No `ELSE` means `ELSE NULL` — the cancelled order gets NULL. Always write an `ELSE` unless NULL is really what you want. Also, simple `CASE x WHEN NULL` never matches (it uses `=`); test NULLs with a searched `CASE WHEN x IS NULL`.

`CASE` appears everywhere: in `ORDER BY` for custom sort orders (chapter 2, 2.7), inside aggregates for conditional counts (`count(*) FILTER (…)` / `sum(CASE …)`, chapter 4), in `UPDATE … SET` for conditional changes (chapter 9), and for pivots (turning row values into columns, e.g. one column per month; chapter 14).

COALESCE — first non-NULL

`coalesce(a, b, c, …)` returns the first argument that isn't NULL (or NULL if all are). It's the standard way to supply defaults.

```
SELECT first_name, email, coalesce(email, 'no email on file') AS contact,
       commission_pct, coalesce(commission_pct, 0) AS commission_or_zero
FROM employees WHERE emp_id IN (5, 6, 9, 11) ORDER BY emp_id;
```

Output:

```
 first_name |             email             |            contact            | commission_pct | commission_or_zero
------------+-------------------------------+-------------------------------+----------------+--------------------
 Meera      | (null)                        | no email on file              |         (null) |                  0
 Daniel     | daniel.brooks@company.example | daniel.brooks@company.example |           0.05 |               0.05
 Sara       | sara.khan@company.example     | sara.khan@company.example     |         (null) |                  0
 Ananya     | (null)                        | no email on file              |         (null) |                  0
(4 rows)
```

Why it matters — NULL "poisons" arithmetic:

```
SELECT first_name, salary, commission_pct,
       salary * (1 + commission_pct)              AS naive_total,
       salary * (1 + coalesce(commission_pct, 0)) AS total
FROM employees WHERE dept_id = 2 ORDER BY emp_id;
```

Output:

```
 first_name |  salary   | commission_pct | naive_total |    total
------------+-----------+----------------+-------------+-------------
 Daniel     | 150000.00 |           0.05 | 157500.0000 | 157500.0000
 Priya      |  72000.00 |           0.10 |  79200.0000 |  79200.0000
 Arjun      |  68000.00 |           0.10 |  74800.0000 |  74800.0000
 Sara       |  68000.00 |         (null) |      (null) |    68000.00
(4 rows)
```

Without `coalesce`, Sara's total pay is unknown. All arguments must share a compatible type — `coalesce(salary, 'n/a')` fails; convert first (`coalesce(salary::text, 'n/a')`).

NULLIF — turn a value into NULL

`nullif(a, b)` returns NULL if `a = b`, otherwise `a`. Two classic uses:

```
-- 1. Avoid division by zero
SELECT a, b, a / NULLIF(b, 0) AS safe_ratio
FROM (VALUES (10, 2), (10, 0)) AS t(a, b);

-- 2. Treat blank strings as missing
SELECT email, NULLIF(trim(email), '') AS email_or_null
FROM (VALUES ('x@y.example'), ('   '), (NULL)) AS t(email);
```

Output:

```
 a  | b | safe_ratio
----+---+------------
 10 | 2 |          5
 10 | 0 |     (null)
(2 rows)

    email    | email_or_null
-------------+---------------
 x@y.example | x@y.example
             | (null)
 (null)      | (null)
(3 rows)
```

`x / 0` raises *division by zero* and aborts the whole query; `x / NULLIF(0, 0)` is `x / NULL` = NULL. Combine them: `coalesce(a / nullif(b, 0), 0)` if a zero is more useful than NULL.

CASE with dates and NULLs together

```
SELECT name, start_date, end_date,
       coalesce(end_date, date '2025-12-31') AS planned_or_open_end,
       CASE WHEN end_date IS NULL THEN 'running' ELSE 'finished' END AS state
FROM projects ORDER BY project_id;
```

Output:

```
       name       | start_date |  end_date  | planned_or_open_end |  state
------------------+------------+------------+---------------------+----------
 Checkout Revamp  | 2024-09-01 | 2025-03-31 | 2025-03-31          | finished
 Mobile App       | 2025-01-15 | (null)     | 2025-12-31          | running
 Spring Campaign  | 2025-02-01 | 2025-04-30 | 2025-04-30          | finished
 Year-End Audit   | 2024-11-01 | 2025-01-31 | 2025-01-31          | finished
 Helpdesk Chatbot | 2025-04-01 | (null)     | 2025-12-31          | running
(5 rows)
```

GREATEST and LEAST

Return the largest / smallest of their arguments (not across rows — that's `max`/`min`, chapter 4). In PostgreSQL they **ignore NULLs**:

```
SELECT greatest(1, NULL, 3) AS greatest, least(1, NULL, 3) AS least;
```

```
 greatest | least
----------+-------
        3 |     1
(1 row)
```

MySQL and Oracle return NULL if any argument is NULL — wrap arguments in `coalesce` for portable code. Typical use: `greatest(price - discount, 0)` to floor a value at zero.

Dialect map (conditionals)

Task	Standard / PostgreSQL	MySQL	SQL Server	Oracle
If-then-else	`CASE WHEN … THEN … ELSE … END`	also `IF(cond, a, b)`	also `IIF(cond, a, b)`	also `DECODE(…)`
First non-NULL	`COALESCE(a, b, …)`	also `IFNULL(a, b)`	also `ISNULL(a, b)`	also `NVL(a, b)`
Value → NULL	`NULLIF(a, b)`	same	same	same
Greatest/least	`GREATEST`, `LEAST` (ignore NULLs)	same names (NULL if any NULL)	`GREATEST` / `LEAST` (2022+, ignore NULLs)	same names (NULL if any NULL)

Prefer the standard forms (`CASE`, `COALESCE`, `NULLIF`) — they work everywhere.

Common mistakes

- Forgetting `ELSE` and getting unexpected NULLs.
- Overlapping `WHEN` conditions in the wrong order.
- `CASE x WHEN NULL …` — never true.
- Mixing result types in branches (`THEN 1 ELSE 'none'`) — PostgreSQL errors; make them the same type.

In the interview

> **30-second answer:** "CASE is SQL's if-else and works anywhere an expression does — SELECT, ORDER BY, inside SUM for conditional aggregation. COALESCE returns the first non-null, so I use it for defaults and to stop NULL wiping out arithmetic. NULLIF turns a value into NULL, mainly to avoid division by zero."

- *Count delivered and cancelled orders in one row?* → `sum(CASE WHEN status='delivered' THEN 1 ELSE 0 END)`, or `count(*) FILTER (WHERE status='delivered')` in PostgreSQL (chapter 4).
- *`ISNULL` vs `COALESCE` in SQL Server?* → `ISNULL` takes two arguments and uses the first argument's type; `COALESCE` is standard, takes many, and uses type precedence (the rule for which type wins when arguments differ).

Self-check

1. *What does a CASE return when no branch matches and there's no ELSE?* → NULL.
2. *Safe average price per item when quantity may be 0?* → `total / NULLIF(quantity, 0)`.
3. *`coalesce(NULL, NULL, 'x', 'y')`?* → `'x'`.

TL;DR

`CASE` = if/else (first true branch wins; no `ELSE` → NULL). `COALESCE` = first non-NULL (defaults). `NULLIF` = value → NULL (safe division, blank → NULL). `GREATEST`/`LEAST` compare arguments (PostgreSQL skips NULLs).

3.6 Worked exercises

Try each before reading the solution. They combine this chapter with a preview of aggregates (`sum`, `count`, `GROUP BY` — chapter 4) and a simple join (`JOIN … USING` — a **join** pairs rows of two tables whose shared column, here `order_id`, matches; chapter 5).

E1. Total value of all delivered orders, rounded to 2 decimals and shown with thousands separators.

```sql
-- code/06_function_exercises.sql
SELECT to_char(round(sum(quantity * unit_price * (1 - discount)), 2), 'FM999,999,990.00') AS total_sales
FROM order_items oi
JOIN orders o USING (order_id)
WHERE o.status = 'delivered';
```

```
 total_sales
-------------
 4,508.11
(1 row)
```

Compute with numbers, round, and only then format — `to_char` is the last step.

E2. The square root of the number of order lines, and the natural log of total revenue.

```sql
SELECT round(sqrt(count(*))::numeric, 3) AS sqrt_lines,
       round(ln(sum(quantity * unit_price * (1 - discount))), 4) AS ln_revenue
FROM order_items;
```

```
 sqrt_lines | ln_revenue
------------+------------
      4.690 |     9.2130
(1 row)
```

`sqrt` of an integer returns `double precision`, and `round(x, d)` with a decimal-places argument needs `numeric` — hence the cast. Note `ln`, not `log`, for the natural logarithm.

E3. Customer display names like "Shah, A." and lower-case email domains.

```sql
SELECT customer_id,
       last_name || ', ' || left(first_name, 1) || '.' AS display_name,
       lower(split_part(trim(email), '@', 2))          AS domain
FROM customers ORDER BY customer_id LIMIT 5;
```

```
 customer_id | display_name |    domain
-------------+--------------+--------------
           1 | Shah, A.     | mail.example
           2 | Carter, B.   | mail.example
           3 | Li, C.       | mail.example
           4 | Menon, D.    | (null)
           5 | Walker, E.   | mail.example
(5 rows)
```

E4. Revenue per year and month, excluding cancelled orders.

```sql
SELECT extract(YEAR FROM o.placed_at)  AS year,
       extract(MONTH FROM o.placed_at) AS month,
       round(sum(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue
FROM orders o
JOIN order_items oi USING (order_id)
WHERE o.status <> 'cancelled'
GROUP BY 1, 2
ORDER BY 1, 2;
```

```
 year | month | revenue
------+-------+---------
 2025 |     1 | 1871.90
 2025 |     2 |  811.49
 2025 |     3 |  804.95
 2025 |     4 | 2148.75
 2025 |     5 |  249.93
 2025 |     6 | 1516.60
(6 rows)
```

E5. The same idea with one sortable month key (orders per month):

```sql
SELECT to_char(date_trunc('month', o.placed_at), 'YYYY-MM') AS month,
       count(DISTINCT o.order_id) AS orders
FROM orders o
WHERE o.status <> 'cancelled'
GROUP BY date_trunc('month', o.placed_at)
ORDER BY date_trunc('month', o.placed_at);
```

```
  month  | orders
---------+--------
 2025-01 |      3
 2025-02 |      1
 2025-03 |      2
 2025-04 |      3
 2025-05 |      1
 2025-06 |      2
(6 rows)
```

Grouping by `date_trunc` keeps a real timestamp for sorting; `to_char` only dresses it up for display. The expected-delivery exercise (order date + 10 days) is in 3.4.

Chapter 3 TL;DR

- **Types:** everything has one; `numeric` for money, `text` for strings, `date`/`timestamptz` for time; convert with `CAST(x AS t)` or `x::t`; PostgreSQL refuses silent coercions.
- **Numbers:** `round`/`trunc`/`ceil`/`floor` (numeric rounds half away from zero, floats to even), `power`, `sqrt`, `exp`, `ln`, `log` (base 10 in PostgreSQL!), `mod`/`%`, `random` + `setseed`, `to_char` for display.
- **Strings:** `||` vs `concat` (NULL handling), `substring`/`left`/`right`/`split_part`, `length` vs `octet_length`, `lower`/`upper`/`initcap`, `trim`, `replace`/`translate`, `position`/`strpos`, `lpad`/`rpad`, `format`, `regexp_replace`.
- **Dates:** `now()` vs `clock_timestamp()`, `+ interval`, `date - date`, `age`, `extract`, `date_trunc`, `to_char`/`to_date`, `AT TIME ZONE`; month arithmetic clamps.
- **Conditionals:** `CASE` (no ELSE → NULL), `COALESCE` (defaults), `NULLIF` (safe division), `GREATEST`/`LEAST`.
- MySQL names differ a lot (`DATE_ADD`, `DATE_FORMAT`, `SUBSTRING_INDEX`, `IFNULL`, `LOG` = ln, `LENGTH` = bytes) — use the dialect maps.

Next: Chapter 4 — Aggregation.

Chapter 4 — Aggregation: Summarising Many Rows

Chapters 2 and 3 worked **row by row**: `WHERE` kept or dropped each row, and functions such as `upper()` or `round()` turned one row's value into another value for the same row. Most business questions are not about single rows, though. "How many orders did we ship last month?", "What is the average salary per department?", "Which customers spent more than 1,000?" — each of these squeezes **many rows into one answer**. That squeezing is called **aggregation**, and it is the subject of this chapter.

What you'll learn

- The aggregate functions (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX` and PostgreSQL's useful extras) and exactly how each treats `NULL` and empty input.
- `GROUP BY` — splitting rows into groups and getting one result row per group — and the rule about which columns you may select.
- `HAVING` — filtering whole groups after they have been summarised — and how it differs from `WHERE`.
- Conditional aggregation with `FILTER (WHERE …)` and `CASE`, the trick behind most "count several things in one pass" and pivot-style reports (reports that turn row values into columns, §4.4).
- The **logical query processing order** — the order in which the database *conceptually* evaluates the clauses of a `SELECT` — and why it explains almost every "column does not exist" error.
- Subtotals and grand totals in a single query with `GROUPING SETS`, `ROLLUP` and `CUBE`.

How to run the examples. Every query in this chapter is in `code/` and was run on the book's private PostgreSQL 14 server; the output under each query is the real `psql` output. `(null)` in the output is how we asked `psql` to display `NULL` (`\pset null '(null)'`).

```
bash tools_run_sql.sh sql '04-*'                        # run this chapter's files (fresh DB per file)
bash tools_pg.sh psql -d <db> -f sql/chapters/04-aggregation/code/02_group_by.sql
```

The sample data. We keep using the book's `company` database from Chapter 1 (`code/00_schema.sql` is an exact copy). The tables that matter most here:

Table	Rows	What is in it (the quirks we will meet)
`employees`	15	`emp_id`, `first_name`, `dept_id`, `salary`, `commission_pct`, `hire_date`, `manager_id`. The CEO (Asha) and the intern (Rohan) have **no department** (`dept_id` is `NULL`); only 3 people have a commission; two salaries are shared by two people each.
`departments`	6	Engineering, Sales, Marketing, Finance, Support, Research (Research has **no** employees).
`orders`	14	`status` is one of pending / shipped / delivered / cancelled; `sales_rep_id` is `NULL` for online orders.
`order_items`	22	One row per product in an order: `quantity`, `unit_price` (price paid), `discount` (0.10 = 10 % off).
`products`, `customers`	9, 10	One product has no category; two customers have never ordered.

4.1 Aggregate functions

What it is

An **aggregate function** takes a *set of rows* and returns *one value* that summarises them. Think of a shop's till at closing time: it does not print every sale again — it prints the number of sales, the total taken, the largest single sale. Those are aggregates.

Function	Returns	Works on	Notes
`COUNT(*)`	number of **rows**	any rows	Counts rows, even rows that are all `NULL`.
`COUNT(expr)`	number of rows where `expr` is **not NULL**	any type	The usual way to count "how many have a value".
`COUNT(DISTINCT expr)`	number of **different** non-NULL values	any type	Duplicates and `NULL`s are dropped first.
`SUM(expr)`	total	numbers, intervals (durations)	`NULL` if there are no non-NULL inputs.
`AVG(expr)`	arithmetic mean = `SUM / COUNT(expr)`	numbers, intervals	Divides by the count of **non-NULL** values.
`MIN(expr)` / `MAX(expr)`	smallest / largest	anything that can be sorted: numbers, text, dates	Text uses the column's collation (alphabetical order).

Two words you will see a lot: a **scalar function** (Chapter 3) works on one row's values and returns one value *per row*; an **aggregate function** works on many rows and returns one value *per group*. Without `GROUP BY`, the whole table (after `WHERE`) is one group, so the query returns exactly one row.

Why it matters

Reports, dashboards, KPIs (key performance indicators — the headline numbers a business tracks), billing, fraud checks and most SQL interview questions are aggregations. Doing the summary inside the database is also far cheaper than shipping every row to your application and adding them up there: the database reads the rows where they live and sends back one small result.

How it works

1. The database builds the input rows (`FROM` and any joins — a **join** combines rows of several tables by matching values, Chapter 5), then removes rows that fail `WHERE`.
2. For each aggregate in the `SELECT` list, it walks the remaining rows, keeping a small running **state** (a counter, a running total, the smallest value so far …).
3. Before feeding a value into the state, it **skips** `NULL`**s** — every aggregate except `COUNT(*)` does this.
4. At the end it turns the state into the result (for `AVG`, total ÷ count).

```
SELECT count(*)              AS employees,
       sum(salary)           AS payroll,
       round(avg(salary), 2) AS avg_salary,
       min(salary)           AS lowest,
       max(salary)           AS highest
FROM employees;
```

Output:

```
 employees |  payroll   | avg_salary |  lowest  |  highest
-----------+------------+------------+----------+-----------
        15 | 1617000.00 |  107800.00 | 30000.00 | 250000.00
(1 row)
```

The three flavours of COUNT

```
SELECT count(*)                 AS all_rows,
       count(email)             AS with_email,
       count(dept_id)           AS with_dept,
       count(DISTINCT dept_id)  AS distinct_depts,
       count(DISTINCT job_title) AS distinct_titles
FROM employees;
```

Output:

```
 all_rows | with_email | with_dept | distinct_depts | distinct_titles
----------+------------+-----------+----------------+-----------------
       15 |         13 |        13 |              5 |              12
(1 row)
```

- `count(*)` = 15: every row.
- `count(email)` = 13: two employees have no email.
- `count(dept_id)` = 13 but `count(DISTINCT dept_id)` = 5: 13 people have a department, and they are spread over 5 different departments. The two `NULL`s are not counted as a sixth "value".

NULLs are skipped — and that changes averages

```sql
SELECT count(commission_pct)            AS non_null,
       avg(commission_pct)              AS avg_ignoring_nulls,
       avg(coalesce(commission_pct, 0)) AS avg_nulls_as_zero
FROM employees;
```

Output:

```
 non_null |   avg_ignoring_nulls   |   avg_nulls_as_zero
----------+------------------------+------------------------
        3 | 0.08333333333333333333 | 0.01666666666666666667
(1 row)
```

`AVG` divided by **3** (the people who have a commission), not by 15. Whether that is right depends on the question. "What is the average commission rate *among people who earn commission*?" → 0.083 is correct. "What is the average commission rate *across all staff*, treating no commission as zero?" → you must say so with `COALESCE(commission_pct, 0)`, giving 0.017. Neither is wrong; the mistake is not knowing which one you asked for.

Empty input

```sql
SELECT count(*) AS n, sum(salary) AS total, avg(salary) AS mean, max(salary) AS top
FROM employees WHERE salary > 1000000;
```

Output:

```
 n | total  |  mean  |  top
---+--------+--------+--------
 0 | (null) | (null) | (null)
(1 row)
```

Nobody earns over a million, so the aggregates see zero rows. `COUNT` answers `0`; `SUM`, `AVG`, `MIN` and `MAX` answer `NULL` ("there is no total of nothing"). You still get **one row** back — an aggregate query without `GROUP BY` always returns exactly one row. If your application expects `0` for an empty sum, write `COALESCE(SUM(x), 0)`.

Other useful points

`MIN`/`MAX` work on any sortable type, and an aggregate can take an **expression**, not just a column:

```sql
SELECT min(hire_date) AS first_hire, max(hire_date) AS latest_hire,
       min(last_name) AS first_alpha, max(last_name) AS last_alpha
FROM employees;

SELECT round(sum(quantity * unit_price * (1 - discount)), 2) AS gross_revenue
FROM order_items;
```

Output:

```
 first_hire | latest_hire | first_alpha | last_alpha
------------+-------------+-------------+------------
 2015-01-05 | 2025-06-02  | Brooks      | Wei
(1 row)

 gross_revenue
---------------
      10026.61
(1 row)
```

A `WHERE` clause simply shrinks the set being aggregated:

```
SELECT count(*) AS delivered, min(placed_at) AS first, max(placed_at) AS last
FROM orders WHERE status = 'delivered';
```

Output:

```
 delivered |        first        |        last
-----------+---------------------+---------------------
         7 | 2025-01-03 10:15:00 | 2025-04-09 16:45:00
(1 row)
```

Beyond the big five (PostgreSQL)

PostgreSQL ships many more aggregates. These are the ones you will actually use:

```
SELECT string_agg(first_name, ', ' ORDER BY first_name) AS engineers,
       array_agg(emp_id ORDER BY emp_id)                AS ids,
       bool_and(salary > 90000)                         AS all_above_90k,
       bool_or(email IS NULL)                           AS any_missing_email
FROM employees WHERE dept_id = 1;

SELECT round(stddev_samp(salary), 2) AS stddev,
       percentile_cont(0.5) WITHIN GROUP (ORDER BY salary) AS median,
       mode() WITHIN GROUP (ORDER BY salary)               AS most_common
FROM employees;
```

Output:

```
         engineers          |    ids    | all_above_90k | any_missing_email
----------------------------+-----------+---------------+-------------------
 Meera, Neha, Rahul, Vikram | {2,3,4,5} | t             | t
(1 row)

  stddev  | median | most_common
----------+--------+-------------
 57217.88 |  98000 |    68000.00
(1 row)
```

Aggregate	What it gives	Typical use
`string_agg(x, sep ORDER BY …)`	all values joined into one string	"list the products in each order"
`array_agg(x ORDER BY …)`	all values as an array (an ordered list stored as one value)	feed an app, or `unnest` (expand back into rows) later
`bool_and` / `bool_or`	TRUE if all / any inputs are true	"did every item ship?"
`stddev_samp`, `variance`	spread of the values (standard deviation ≈ how far values typically sit from the average)	analytics
`percentile_cont(f) WITHIN GROUP (ORDER BY x)`	interpolated percentile (0.5 = median)	median salary, p95 latency (the time 95 % of requests finish within)
`percentile_disc(f) …`	percentile that is an actual input value	"the median order", not an average of two
`mode() WITHIN GROUP (ORDER BY x)`	most frequent value	most common salary / status

> **Key term — percentile and median:** The *f*-th percentile is the value below which a fraction *f* of the sorted values fall; the **median** is the 50th percentile, the middle value, which (unlike the average) is not dragged by a few huge values. *Interpolated* means that when the cut falls between two values, the result is blended from both, so it may be a number that never occurs in the data. **Latency** is how long one request takes; "p95 latency" is its 95th percentile — see System Design Fundamentals.

`percentile_cont`, `percentile_disc` and `mode` are **ordered-set aggregates**: the `WITHIN GROUP (ORDER BY …)` tells them which order to read the values in.

Dialect differences

Feature	PostgreSQL	MySQL 8	SQLite	SQL Server
Join values into a string	`string_agg(x, ',')`	`GROUP_CONCAT(x SEPARATOR ',')`	`group_concat(x, ',')`	`STRING_AGG(x, ',')` (2017+)
`AVG` of an integer column	exact `numeric`	decimal	floating point	**integer** (truncated!) — cast first: `AVG(1.0 * x)`
Median	`percentile_cont(0.5) WITHIN GROUP (…)`	no built-in	no built-in	`PERCENTILE_CONT(0.5) WITHIN GROUP (…) OVER ()` (window form, Chapter 8)
Boolean aggregates	`bool_and`, `bool_or`	`MIN` / `MAX` of 0/1	`min` / `max` of 0/1	`MIN` / `MAX` of a `CASE`

Common mistakes

- **`COUNT(column)` when you meant `COUNT(*)`** (or the other way round). They differ exactly by the number of `NULL`s in that column.
- **Averaging with NULLs without thinking** — see the commission example above.
- **`SUM` of an empty set used as a number** in application code — it is `NULL`, not `0`.
- **Integer averages in SQL Server** (`AVG` of `int` returns an `int`).
- **`COUNT(DISTINCT a, b)`** — PostgreSQL does not accept two arguments; count distinct *rows* instead: `COUNT(DISTINCT (a, b))` or a subquery with `SELECT DISTINCT a, b`.

In the interview

30-second answer. "Aggregate functions reduce a set of rows to one value: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. All of them ignore `NULL` inputs except `COUNT(*)`, which counts rows. On empty input `COUNT` returns 0 and the others return `NULL`. Without `GROUP BY` the whole filtered table is one group, so you get one row."

- *Q: What is the difference between `COUNT(*)`, `COUNT(1)` and `COUNT(col)`?* — `COUNT(*)` and `COUNT(1)` both count rows (1 is never NULL), and PostgreSQL treats them the same way; `COUNT(col)` counts non-NULL values of `col`.
- *Q: `AVG(x)` vs `SUM(x)/COUNT(*)`?* — They differ when `x` has NULLs: `AVG` divides by `COUNT(x)`. Also `SUM(int)/COUNT(*)` is integer division in PostgreSQL.
- *Q: How do you compute a median in PostgreSQL?* — `percentile_cont(0.5) WITHIN GROUP (ORDER BY x)`.

Self-check

1. A column has values 10, NULL, 20. What do `COUNT(*)`, `COUNT(col)`, `SUM(col)`, `AVG(col)` return? → 3, 2, 30, 15.
2. What does `SELECT MAX(salary) FROM employees WHERE 1 = 0` return — zero rows or one row? → One row containing `NULL`.
3. How would you count employees who have *no* email? → `COUNT(*) - COUNT(email)`, or `COUNT(*) FILTER (WHERE email IS NULL)` (§4.4).
4. Can `MIN` be used on a text column? → Yes; it returns the alphabetically first value (by the column's collation).

TL;DR

Aggregates turn many rows into one value. `COUNT(*)` counts rows; every other aggregate skips `NULL`s. Empty input → `COUNT` = 0, the rest `NULL`. No `GROUP BY` → exactly one result row.

4.2 GROUP BY

What it is

`GROUP BY` splits the rows into **groups** — one group per distinct value (or combination of values) of the listed expressions — and then runs the aggregates **once per group**. The result has **one row per group**.

Analogy: a teacher sorting exam papers into piles by class, then writing the number of papers and the average mark on top of each pile. The piles are the groups; the numbers on top are the aggregates.

How it works

1. Build and filter the rows (`FROM`, `WHERE`).
2. Put rows with the **same values** in all `GROUP BY` expressions into the same group. For grouping, all `NULL`s count as equal, so every row with a `NULL` key lands in **one** `NULL` group.
3. For each group, compute each aggregate over just that group's rows.
4. Output one row per group. The `SELECT` list may contain only the grouping expressions, aggregates, constants, or expressions built from those.

```sql
SELECT dept_id, count(*) AS headcount, sum(salary) AS payroll
FROM employees
GROUP BY dept_id
ORDER BY dept_id;
```

Output:

```
 dept_id | headcount |  payroll
---------+-----------+-----------
       1 |         4 | 523000.00
       2 |         4 | 358000.00
       3 |         2 | 181000.00
       4 |         2 | 211000.00
       5 |         1 |  64000.00
  (null) |         2 | 280000.00
(6 rows)
```

Six groups: five departments plus a `NULL` **group** holding Asha and Rohan, who have no department. Notice that Research (department 6) does not appear: `GROUP BY` can only make groups from rows that exist in its input. To list departments with zero employees you need an outer join (a join that also keeps rows with no match; Chapter 5).

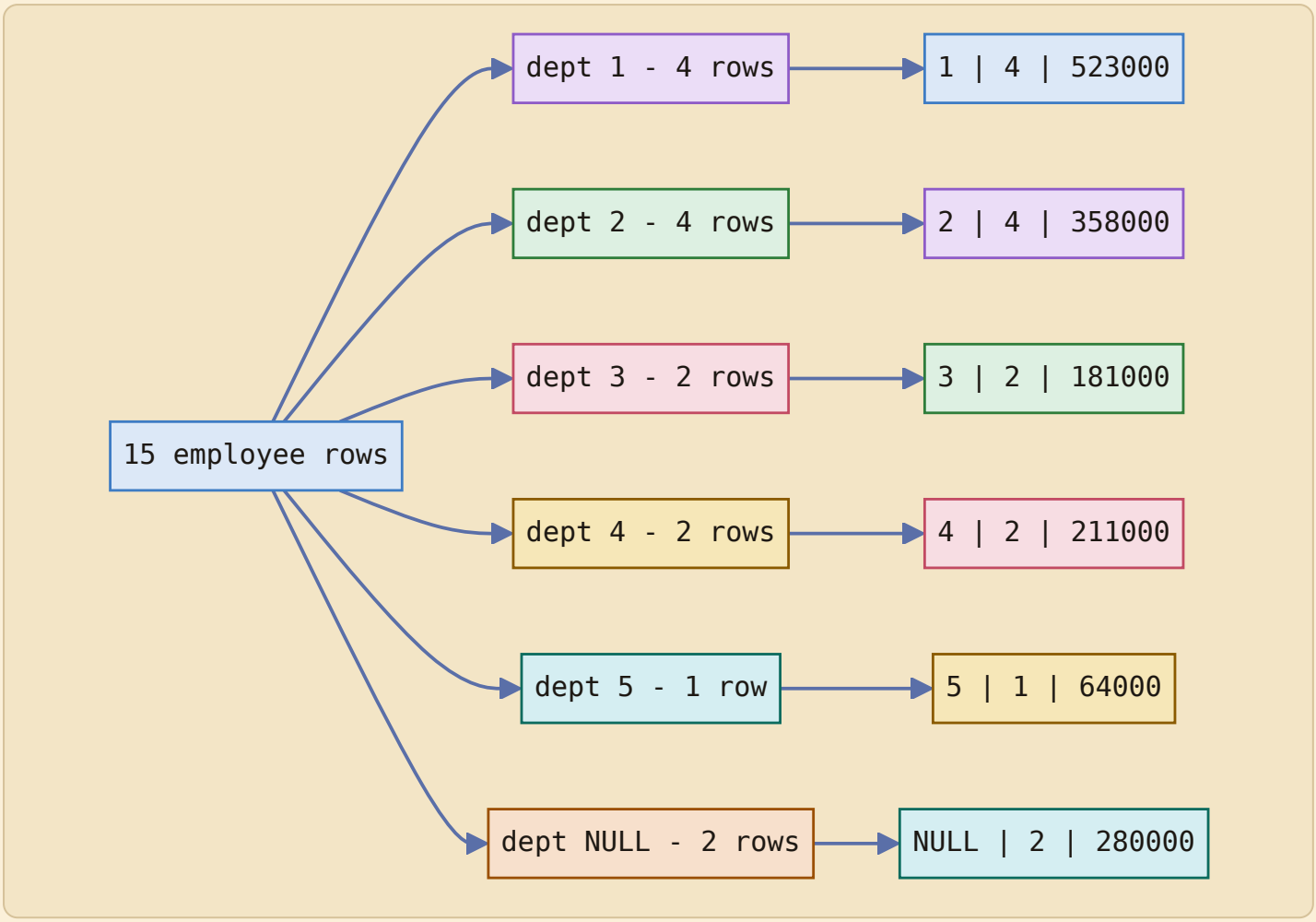


Figure 4.1 — GROUP BY dept_id: split the rows into piles, then summarise each pile into one output row.

Grouping by several columns

With two grouping columns, each **distinct pair** is a group:

```sql
SELECT dept_id, job_title, count(*) AS n, round(avg(salary)) AS avg_salary
FROM employees
WHERE dept_id IN (1, 2)
GROUP BY dept_id, job_title
ORDER BY dept_id, job_title;
```

Output:

```
 dept_id |      job_title      | n | avg_salary
---------+---------------------+---+------------
       1 | Engineer            | 2 |      98000
       1 | Engineering Manager | 1 |     185000
       1 | Senior Engineer     | 1 |     142000
       2 | Sales Manager       | 1 |     150000
       2 | Sales Rep           | 3 |      69333
(5 rows)
```

The order of columns in `GROUP BY` does not change *which* groups you get — `GROUP BY a, b` and `GROUP BY b, a` produce the same groups. Row order in the output is controlled only by `ORDER BY`.

Grouping by an expression

You can group by a computed value, such as the month of a timestamp:

```sql
SELECT date_trunc('month', placed_at)::date AS month, count(*) AS orders
FROM orders
GROUP BY date_trunc('month', placed_at)
ORDER BY month;
```

Output:

```
   month    | orders
------------+--------
 2025-01-01 |      3
 2025-02-01 |      2
 2025-03-01 |      2
 2025-04-01 |      3
 2025-05-01 |      2
 2025-06-01 |      2
(6 rows)
```

PostgreSQL also lets you refer to a `SELECT` alias or a column position in `GROUP BY` (a non-standard convenience; if an alias clashes with a real column name, the real column wins):

```sql
SELECT extract(year FROM hire_date) AS hire_year, count(*) AS hires
FROM employees
GROUP BY hire_year
ORDER BY 1;
```

Output:

```
 hire_year | hires
-----------+-------
      2015 |     1
      2016 |     1
      2017 |     1
      2018 |     1
      2019 |     2
      2020 |     1
      2021 |     2
      2022 |     2
      2023 |     2
      2024 |     1
      2025 |     1
(11 rows)
```

Grouping usually follows a join. Revenue per product category joins `order_items` to `products` first (joins are Chapter 5):

```sql
SELECT p.category, sum(oi.quantity) AS units,
       round(sum(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY revenue DESC;
```

Output:

```
  category   | units | revenue
-------------+-------+---------
 Electronics |    16 | 7314.79
 Furniture   |     5 | 1940.47
 Accessories |    11 |  671.35
 (null)      |     2 |  100.00
(4 rows)
```

A `GROUP BY` with no aggregates returns each distinct group once — the same result as `SELECT DISTINCT status FROM orders`:

```
SELECT status FROM orders GROUP BY status ORDER BY status;
```

Output:

```
  status
-----------
 cancelled
 delivered
 pending
 shipped
(4 rows)
```

The single-value rule

After grouping, each output row stands for a whole group. A column that is neither grouped nor aggregated could have **several different values** inside one group — which one should be shown? SQL refuses to guess:

```
SELECT dept_id, first_name, count(*) FROM employees GROUP BY dept_id;
```

Output:

```
ERROR:  column "employees.first_name" must appear in the GROUP BY clause or be used in an aggregate function
LINE 1: SELECT dept_id, first_name, count(*) FROM employees GROUP BY...
                        ^
```

There is one exception. If you group by a table's **primary key** (the column whose value uniquely identifies each row), every other column of that table has exactly one value per group (it is *functionally dependent* on the key — see Chapter 11), so PostgreSQL (9.1+) allows it:

```
SELECT d.dept_id, d.name, count(e.emp_id) AS headcount
FROM departments d JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_id
ORDER BY d.dept_id;
```

Output:

```
 dept_id |    name     | headcount
---------+-------------+-----------
       1 | Engineering |         4
       2 | Sales       |         4
       3 | Marketing   |         2
       4 | Finance     |         2
       5 | Support     |         1
(5 rows)
```

Database	Non-grouped, non-aggregated column in SELECT
PostgreSQL	Error, unless the column depends on a grouped primary key.
MySQL 8	Error by default (`ONLY_FULL_GROUP_BY` mode, on since 5.7.5); also detects primary-key dependency. With the mode off it returns an arbitrary value.
SQLite	Allowed ("bare column"); the value comes from an arbitrary row of the group (or from the row holding the `min`/`max` if the query has exactly one of those).
SQL Server, Oracle	Always an error.

Trade-offs and performance

The database groups rows in one of two ways (you can see which with `EXPLAIN`, the command that prints how the database plans to run a query, Chapter 12): **hash aggregation** builds an in-memory hash table (a lookup structure that turns each key into a slot number, so finding a key takes about one step) keyed by the group values (fast, no sorting, needs memory for all groups), or **sort + group aggregation** sorts the rows by the grouping key and summarises each run of equal keys (can use an index — a sorted lookup structure kept beside the table — that is already in that order, and streams rows through with little memory). Grouping by many high-cardinality columns (columns with many distinct values) creates many groups and costs more memory.

When not to use it

- If you need each **row** *and* a group total next to it (e.g. each salary and its department's average), `GROUP BY` collapses the rows you want to keep. Use a **window function** (an aggregate-like function that computes over a set of related rows but keeps every row in the output) instead: `avg(salary) OVER (PARTITION BY dept_id)` — Chapter 8.
- If you only want distinct values, `SELECT DISTINCT` states the intent more clearly.

Common mistakes

- Selecting a column that is not grouped (error above) — and "fixing" it by adding the column to `GROUP BY`, which silently creates *smaller* groups than you meant. Ask yourself: "one row per *what*?" and group by exactly that.
- Expecting groups for values that have no rows (Research). Groups come only from existing rows.
- Forgetting that `NULL` keys form their own group — often labelled with `COALESCE(dept_id::text, 'none')` in reports.
- Relying on `GROUP BY` to sort. PostgreSQL's hash aggregation returns groups in no particular order; always add `ORDER BY`.

In the interview

30-second answer. "`GROUP BY` partitions the filtered rows by the grouping columns and computes the aggregates once per partition, returning one row per group. Every selected column must be grouped or aggregated — unless it depends on a grouped primary key. `NULL`s form one group. For per-row output with group totals, use window functions instead."

- *Q: Does `GROUP BY` guarantee sorted output?* — No. Only `ORDER BY` does.
- *Q: `GROUP BY` vs `DISTINCT`?* — Without aggregates they give the same rows; `GROUP BY` is needed when you want to compute something per group.
- *Q: How do you find duplicate emails?* — `SELECT lower(email), count(*) FROM customers GROUP BY lower(email) HAVING count(*) > 1;` (the `HAVING` part is next).

Self-check

1. `GROUP BY dept_id` on our data gives 6 rows but there are 6 departments. Is the Research department one of them? → No; the sixth group is the `NULL` group. Research has no rows to group.

2. Why is `SELECT dept_id, first_name, count(*) … GROUP BY dept_id` an error? → `first_name` has several values per department; SQL will not pick one.
3. Does `GROUP BY a, b` produce more, fewer or the same number of groups as `GROUP BY a`? → The same or more — each `a` group is split by `b`.

TL;DR

`GROUP BY` = split rows into groups by key, aggregate each group, one output row per group. Select only keys and aggregates. `NULL` keys form one group. Add `ORDER BY` for order.

4.3 HAVING

What it is

`HAVING` is a filter on **groups**. It runs **after** grouping and aggregation, so its condition can use aggregates such as `count(*) > 2`. `WHERE` is a filter on **rows** and runs *before* grouping, so it cannot.

	WHERE	HAVING
Filters	individual rows	whole groups
Runs	before `GROUP BY`	after `GROUP BY` and aggregation
Can use aggregates?	**No**	Yes
Can use plain columns?	Yes	Only grouping columns (or inside aggregates)
Effect on work	fewer rows to group → cheaper	groups are already computed

How it works

```sql
SELECT dept_id, count(*) AS headcount
FROM employees
GROUP BY dept_id
HAVING count(*) > 2
ORDER BY dept_id;
```

Output:

```
 dept_id | headcount
---------+-----------
       1 |         4
       2 |         4
(2 rows)
```

1. Group the 15 employees by `dept_id` (6 groups).
2. Compute `count(*)` for each group.
3. Keep only groups where the count is greater than 2 — Engineering and Sales.

Using `WHERE` and `HAVING` together is common: `WHERE` removes unwanted *rows*, then `HAVING` removes unwanted *groups*. "Among orders that were not cancelled, which customers ordered at least twice?"

```sql
SELECT customer_id, count(*) AS orders
FROM orders
WHERE status <> 'cancelled'
GROUP BY customer_id
HAVING count(*) >= 2
ORDER BY customer_id;
```

Output:

```
 customer_id | orders
-------------+--------
           1 |      3
           2 |      2
           5 |      2
           6 |      2
(4 rows)
```

Customer 3 (Chen) has two orders in total, but one is cancelled; `WHERE` drops it first, the group then has 1 row, and `HAVING` drops the group.

`HAVING` may use an aggregate that is not in the `SELECT` list, and may combine conditions with `AND` / `OR`:

```sql
SELECT dept_id
FROM employees
GROUP BY dept_id
HAVING max(salary) > 140000
ORDER BY dept_id;

SELECT dept_id, count(*) AS n, round(avg(salary)) AS avg_salary
FROM employees
WHERE dept_id IS NOT NULL
GROUP BY dept_id
HAVING avg(salary) > 90000 AND count(*) >= 2
ORDER BY avg_salary DESC;
```

Output:

```
 dept_id
---------
       1
       2
  (null)
(3 rows)

 dept_id | n | avg_salary
---------+---+------------
       1 | 4 |     130750
       4 | 2 |     105500
       3 | 2 |      90500
(3 rows)
```

In the first result the `NULL` group qualifies because it contains the CEO's salary.

An aggregate in WHERE is an error

```sql
SELECT dept_id, count(*) FROM employees WHERE count(*) > 2 GROUP BY dept_id;
```

Output:

```
ERROR:  aggregate functions are not allowed in WHERE
LINE 1: SELECT dept_id, count(*) FROM employees WHERE count(*) > 2 G...
                                                      ^
```

When `WHERE` runs, groups do not exist yet, so there is nothing to count (§4.5 shows the full order).

Row conditions belong in WHERE

A condition on a grouping column works in either place and gives the same answer:

```sql
SELECT dept_id, count(*) FROM employees GROUP BY dept_id HAVING dept_id = 2;
SELECT dept_id, count(*) FROM employees WHERE dept_id = 2 GROUP BY dept_id;
```

Output:

```
 dept_id | count
---------+-------
       2 |     4
(1 row)

 dept_id | count
---------+-------
       2 |     4
(1 row)
```

Prefer `WHERE`: it removes rows *before* the expensive grouping, can use an index, and tells the reader "this is a row filter". (PostgreSQL's planner — the component that decides how to execute a query — often moves such a `HAVING` condition down into `WHERE` for you, but don't rely on every database doing so.)

HAVING without GROUP BY

If a query has `HAVING` but no `GROUP BY`, the whole input is treated as **one group**. The query returns that one row if the condition is true, and no rows otherwise:

```sql
SELECT count(*) AS products FROM products HAVING count(*) > 5;
SELECT count(*) AS products FROM products HAVING count(*) > 50;
```

Output:

```
 products
----------
        9
(1 row)

 products
----------
(0 rows)
```

Common mistakes

- Putting an aggregate condition in `WHERE` (error) or a row condition in `HAVING` (works but slower and confusing).
- Referring to a `SELECT` alias in `HAVING`: `HAVING headcount > 2` fails in PostgreSQL (`column "headcount" does not exist`), because `HAVING` is evaluated before `SELECT` names its outputs. Repeat the aggregate. (MySQL allows the alias as an extension.)
- Forgetting that `WHERE` changes what `HAVING` sees (the Chen example).

In the interview

30-second answer. "`WHERE` filters rows before grouping and can't use aggregates; `HAVING` filters groups after aggregation and can. Use `WHERE` for anything that doesn't need an aggregate, because it shrinks the input early."

- *Q: Find departments with more than 3 employees.* — `SELECT dept_id FROM employees GROUP BY dept_id HAVING COUNT(*) > 3;`
- *Q: Can you use `HAVING` without `GROUP BY`?* — Yes; the whole table is one group, so you get one row or none.
- *Q: Find duplicate rows.* — `GROUP BY` all the columns that define "duplicate" and `HAVING COUNT(*) > 1`.

Self-check

1. Which runs first, `WHERE` or `HAVING`? → `WHERE` (on rows), then grouping, then `HAVING` (on groups).
2. Rewrite `HAVING dept_id <> 5` in the better place. → `WHERE dept_id <> 5` (before `GROUP BY`) — careful: `WHERE dept_id <> 5` also drops `NULL` rows, just as the `HAVING` version drops the `NULL` group.

3. What does `SELECT count(*) FROM t HAVING count(*) > 0` return on an empty table? → No rows (the count is 0, so the one group is filtered out).

TL;DR

`HAVING` = `WHERE` for groups. Aggregates go in `HAVING`; row filters go in `WHERE`. No `GROUP BY` → one big group.

4.4 Conditional aggregation: FILTER and CASE

What it is

Conditional aggregation means aggregating only the rows that meet a condition, *inside* a single query, so you can compute several differently-filtered numbers side by side. PostgreSQL has a dedicated clause for it — `aggregate(...) FILTER (WHERE condition)` (SQL standard; PostgreSQL 9.4+, SQLite 3.30+). Every database supports the older trick of putting a `CASE` expression inside the aggregate.

The problem it solves

"For each customer: total orders, delivered orders, cancelled orders, online orders." With plain `WHERE` you would need four queries (or four subqueries — queries nested inside another), each scanning (reading through every row of) `orders` again. Conditional aggregation does it in **one pass** over the table.

How it works

`FILTER (WHERE cond)` feeds only the rows where `cond` is true into that one aggregate. Other aggregates in the same query still see all rows of the group.

```
SELECT customer_id,
       count(*)                                      AS total,
       count(*) FILTER (WHERE status = 'delivered')   AS delivered,
       count(*) FILTER (WHERE status = 'cancelled')   AS cancelled,
       count(*) FILTER (WHERE sales_rep_id IS NULL)   AS online
FROM orders
GROUP BY customer_id
ORDER BY customer_id;
```

Output:

```
 customer_id | total | delivered | cancelled | online
-------------+-------+-----------+-----------+--------
           1 |     3 |         2 |         0 |      1
           2 |     2 |         2 |         0 |      1
           3 |     2 |         1 |         1 |      1
           4 |     1 |         0 |         1 |      0
           5 |     2 |         1 |         0 |      1
           6 |     2 |         0 |         0 |      1
           7 |     1 |         0 |         0 |      0
           8 |     1 |         1 |         0 |      1
(8 rows)
```

The portable version uses `CASE`. Two equivalent styles:

- `SUM(CASE WHEN cond THEN 1 ELSE 0 END)` — adds 1 for each matching row.
- `COUNT(CASE WHEN cond THEN 1 END)` — the `CASE` gives `NULL` for non-matching rows, and `COUNT` skips `NULL`s.

```sql
SELECT customer_id,
       count(*)                                          AS total,
       sum(CASE WHEN status = 'delivered' THEN 1 ELSE 0 END) AS delivered,
       count(CASE WHEN status = 'cancelled' THEN 1 END)      AS cancelled
FROM orders
GROUP BY customer_id
ORDER BY customer_id;
```

Output:

```
 customer_id | total | delivered | cancelled
-------------+-------+-----------+-----------
           1 |     3 |         2 |         0
           2 |     2 |         2 |         0
           3 |     2 |         1 |         1
           4 |     1 |         0 |         1
           5 |     2 |         1 |         0
           6 |     2 |         0 |         0
           7 |     1 |         0 |         0
           8 |     1 |         1 |         0
(8 rows)
```

Ratios and percentages

Conditional aggregates make "what share of X is Y" questions easy — divide a filtered aggregate by the unfiltered one. Multiply by `100.0` (not `100`) so integer inputs don't cause integer division.

```sql
SELECT dept_id,
       sum(salary) AS payroll,
       round(100.0 * sum(salary) FILTER (WHERE hire_date < '2020-01-01') / sum(salary), 1) AS pct_veterans
FROM employees
WHERE dept_id IS NOT NULL
GROUP BY dept_id
ORDER BY dept_id;
```

Output:

```
 dept_id |  payroll  | pct_veterans
---------+-----------+--------------
       1 | 523000.00 |         62.5
       2 | 358000.00 |         41.9
       3 | 181000.00 |         66.3
       4 | 211000.00 |         64.0
       5 |  64000.00 |       (null)
(5 rows)
```

Support shows `NULL`, not 0: its only employee was hired in 2023, so the filtered `SUM` saw no rows and returned `NULL`. Wrap it in `COALESCE(…, 0)` if the report should show 0.

Pivot-style reports

One column per category of something is called a **pivot** (rows turned into columns). Conditional aggregation is the standard way to pivot in SQL when the column list is known in advance:

```sql
SELECT p.category,
       sum(oi.quantity) FILTER (WHERE o.status = 'delivered') AS delivered_units,
       sum(oi.quantity) FILTER (WHERE o.status = 'shipped')   AS shipped_units,
       sum(oi.quantity) FILTER (WHERE o.status = 'pending')   AS pending_units
FROM order_items oi
JOIN orders o   ON o.order_id = oi.order_id
JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY p.category NULLS LAST;
```

Output:

```
  category   | delivered_units | shipped_units | pending_units
-------------+-----------------+---------------+---------------
 Accessories |               6 |        (null) |             5
 Electronics |               9 |             3 |             1
 Furniture   |               2 |             3 |        (null)
 (null)      |               2 |        (null) |        (null)
(4 rows)
```

Dynamic pivots (where the columns are not known until you look at the data) and `crosstab` (a PostgreSQL extension function that builds pivot tables) are covered in Chapter 14.

Trade-offs

Approach	Portable?	Readability	Notes
`agg(x) FILTER (WHERE c)`	PostgreSQL, SQLite 3.30+ (not MySQL, SQL Server)	Best — the condition is separate from the value	Works with every aggregate, including `string_agg`, `array_agg`.
`SUM(CASE WHEN c THEN 1 ELSE 0 END)`	Everywhere	Good	Returns 0, not `NULL`, when nothing matches.
`COUNT(CASE WHEN c THEN 1 END)`	Everywhere	Good	Relies on `COUNT` skipping `NULL`.
Separate queries / subqueries	Everywhere	Verbose	Several scans of the table.

In MySQL a boolean expression is 0 or 1, so `SUM(status = 'delivered')` is a common MySQL shortcut (not valid in PostgreSQL, where `boolean` can't be summed; you would write `SUM((status = 'delivered')::int)`).

Common mistakes

- `COUNT(CASE WHEN c THEN 1 ELSE 0 END)` — counts *every* row, because 0 is not `NULL`. Use `SUM` with `ELSE 0`, or `COUNT` with no `ELSE`.
- Integer division in ratios (`100 * a / b` with integer `a`, `b`).
- Putting the condition in `WHERE` when other columns need the unfiltered rows — that filters the rows for *all* aggregates.

In the interview

30-second answer. "Conditional aggregation computes several filtered aggregates in one pass: `COUNT(*) FILTER (WHERE status='x')` in PostgreSQL, or `SUM(CASE WHEN status='x' THEN 1 ELSE 0 END)` anywhere. It's how you do per-group breakdowns, ratios and fixed pivots."

- *Q: Share of cancelled orders per month?* — `GROUP BY` month, `ROUND(100.0 * COUNT(*) FILTER (WHERE status='cancelled') / COUNT(*), 1)`.
- *Q: Why does `COUNT(CASE … ELSE 0 END)` give the wrong number?* — Because `COUNT` counts non-NULL values and 0 is non-NULL.

Self-check

1. Write "number of employees with no email per department" two ways. → `COUNT(*) FILTER (WHERE email IS NULL)` and `SUM(CASE WHEN email IS NULL THEN 1 ELSE 0 END)`.
2. What does a `FILTER`ed `SUM` return for a group where no row matches? → `NULL`.
3. Why is `100.0` used instead of `100` in percentages? → To force numeric (decimal) arithmetic instead of integer division.

TL;DR

Put the condition *inside* the aggregate (`FILTER (WHERE …)` or `CASE`) to compute many filtered numbers in one scan. It powers breakdowns, ratios and fixed pivots.

4.5 The logical order of query processing

What it is

You **write** a `SELECT` in this order: `SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY … LIMIT`. The database **conceptually evaluates** it in a different order, starting with `FROM`. This *logical query processing order* is a mental model defined by the SQL standard's semantics: each step takes the table produced by the previous step and hands a new table to the next one.

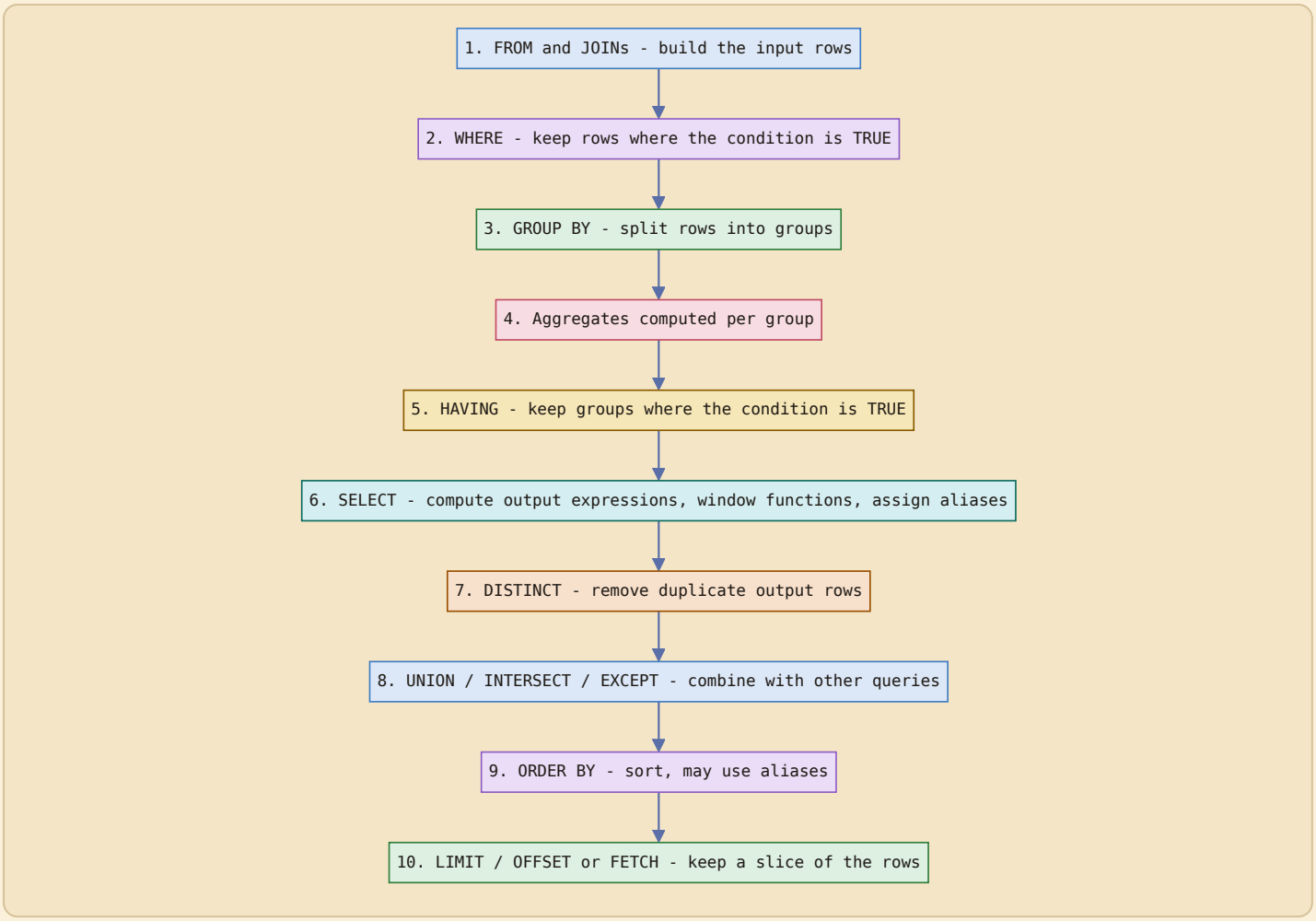


Figure 4.2 — Logical query processing order. Each step only sees what earlier steps produced.

Step	Clause	Input → output	Can refer to
1	`FROM`, `JOIN … ON`	tables → one combined row set	table columns
2	`WHERE`	rows → fewer rows	table columns (no aggregates, no `SELECT` aliases)
3–4	`GROUP BY` + aggregates	rows → one row per group	table columns / expressions
5	`HAVING`	groups → fewer groups	grouping columns, aggregates
6	`SELECT`	groups/rows → output columns	grouping columns, aggregates; **window functions run here**
7	`DISTINCT`	output rows → unique output rows	output columns
8	set operators (`UNION`, `INTERSECT`, `EXCEPT`; Chapter 6)	two results → one	output columns
9	`ORDER BY`	rows → sorted rows	output aliases, output positions, and (without `DISTINCT`/set ops) input columns
10	`LIMIT`/`OFFSET`	sorted rows → a slice	—

Why it matters

Almost every "column does not exist" or "aggregate not allowed" error is explained by this order:

- **`WHERE` can't use a `SELECT` alias** — the alias is created in step 6, `WHERE` is step 2.
- **`WHERE` can't use aggregates** — groups don't exist until step 3.
- **`ORDER BY` *can* use aliases** — it runs after `SELECT`.
- **`LIMIT` applies to the sorted, final rows** — so `ORDER BY … LIMIT 3` really means "top 3".
- **Window functions can't be used in `WHERE`** — they are computed in step 6 (Chapter 8).

It also guides efficient writing and debugging. Filtering rows as early as possible (in `WHERE`, step 2) means fewer rows to group, aggregate and sort later; and when a query returns surprising results, walking through the steps in this order — what rows does `FROM` produce, which survive `WHERE`, what groups form — usually shows where the logic went wrong.

```
SELECT emp_id, salary * 12 AS yearly FROM employees WHERE yearly > 2000000;
```

Output:

```
ERROR:  column "yearly" does not exist
LINE 1: ...mp_id, salary * 12 AS yearly FROM employees WHERE yearly > 2...
                                                             ^
```

```
SELECT emp_id, salary * 12 AS yearly FROM employees ORDER BY yearly DESC LIMIT 3;
```

Output:

```
 emp_id |   yearly
--------+------------
      1 | 3000000.00
      2 | 2220000.00
      6 | 1800000.00
(3 rows)
```

Two fixes for the `WHERE` case: repeat the expression (`WHERE salary * 12 > 2000000`), or compute it in a **derived table** (a subquery in `FROM`, Chapter 6) so that it becomes an input column of the outer query:

```sql
SELECT * FROM (SELECT emp_id, salary * 12 AS yearly FROM employees) t
WHERE yearly > 2000000;
```

Output:

```
 emp_id |   yearly
--------+------------
      1 | 3000000.00
      2 | 2220000.00
(2 rows)
```

A query that uses every clause

```sql
SELECT   dept_id, count(*) AS headcount, round(avg(salary)) AS avg_salary
FROM     employees
WHERE    hire_date >= '2018-01-01'
GROUP BY dept_id
HAVING   count(*) >= 2
ORDER BY avg_salary DESC
LIMIT    3;
```

Output:

```
 dept_id | headcount | avg_salary
---------+-----------+------------
       1 |         4 |     130750
       3 |         2 |      90500
       2 |         3 |      69333
(3 rows)
```

Tracing it step by step:

1. **FROM** `employees` → 15 rows.
2. **WHERE** hired in 2018 or later → drops Asha (2015), Kiran (2016) and Daniel (2017) → 12 rows.
3. **GROUP BY** `dept_id` → groups 1 (4 rows), 2 (3 rows), 3 (2), 4 (1: Omar), 5 (1: Li), `NULL` (1: Rohan).
4. **Aggregates** → count and average per group.
5. **HAVING** `count(*) >= 2` → keeps groups 1, 2, 3.
6. **SELECT** → computes `headcount`, `avg_salary` and names them.
7. **ORDER BY** `avg_salary DESC` (an alias — allowed here).
8. **LIMIT 3** → all three survive.

`DISTINCT` happens after `SELECT` and before `ORDER BY` and `LIMIT`, and `ORDER BY` may even sort by an aggregate that is not selected:

```sql
SELECT DISTINCT job_title FROM employees ORDER BY job_title LIMIT 4;

SELECT dept_id FROM employees WHERE dept_id IS NOT NULL
GROUP BY dept_id ORDER BY sum(salary) DESC;
```

Output:

```
   job_title
----------------
 Accountant
 CEO
 Content Writer
 Engineer
(4 rows)

 dept_id
---------
       1
       2
       4
       3
       5
(5 rows)
```

Logical order is not physical order

The logical order defines **what the answer is**, not **how the database computes it**. The query planner is free to do things in any order that gives the same answer: it may push a `WHERE` condition into a join, use an index so rows arrive already sorted and skip the sort step, stop reading once `LIMIT` rows are found, or move a `HAVING` condition on a grouping column down into the scan. `EXPLAIN` shows the physical plan (Chapter 12).

Dialect notes

Behaviour	PostgreSQL	MySQL	SQLite	SQL Server
`SELECT` alias in `GROUP BY`	allowed (extension)	allowed	allowed	**not** allowed
`SELECT` alias in `HAVING`	not allowed	allowed	allowed	not allowed
`SELECT` alias in `WHERE`	not allowed	not allowed	allowed (quirk)	not allowed
Row limit	`LIMIT n OFFSET m` / `FETCH FIRST n ROWS ONLY`	`LIMIT`	`LIMIT`	`TOP n` / `OFFSET … FETCH`

The extensions work because those databases look the name up among the output columns; they do not change the logical order.

In the interview

30-second answer. "Logically, SQL runs `FROM`/joins, then `WHERE`, `GROUP BY`, aggregates, `HAVING`, `SELECT` (including window functions), `DISTINCT`, set operators, `ORDER BY`, and finally `LIMIT` / `OFFSET`. That's why `WHERE` can't see aliases or aggregates but `ORDER BY` can. The optimizer may physically reorder work as long as the result is the same."

- *Q: Why can't I filter on `ROW_NUMBER()` in `WHERE`?* — Window functions are computed in the `SELECT` step, after `WHERE`. Wrap the query in a subquery or CTE (Common Table Expression — a named subquery written with `WITH`, Chapter 7) and filter outside.
- *Q: Is `LIMIT` applied before or after `ORDER BY`?* — After. Without `ORDER BY`, which rows you get is undefined.
- *Q: Does `WHERE` run before the join?* — Logically after `FROM` builds the joined rows; physically the planner often filters each table before joining.

Self-check

1. Put in logical order: `HAVING`, `FROM`, `ORDER BY`, `SELECT`, `WHERE`, `GROUP BY`, `LIMIT`. → `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY`, `LIMIT`.

2. Why does `ORDER BY yearly` work but `WHERE yearly > …` fail? → `ORDER BY` runs after `SELECT` creates the alias; `WHERE` runs before.
3. Where do window functions run? → In the `SELECT` step — after `HAVING`, before `DISTINCT`/`ORDER BY`.

TL;DR

From → **W**here → **G**roup by → **H**aving → **S**elect → **D**istinct → **O**rder by → **L**imit. A clause can only use what earlier steps produced. Logical order ≠ execution plan.

4.6 Subtotals: GROUPING SETS, ROLLUP and CUBE

What it is

A normal `GROUP BY` produces one level of detail. Reports often need **several levels at once**: revenue per country *and* category, a subtotal per country, and a grand total. `GROUPING SETS`, `ROLLUP` and `CUBE` (SQL:1999; PostgreSQL 9.5+) produce all of those levels in one query, as if several `GROUP BY` queries had been glued together with `UNION ALL` (which stacks the rows of several queries into one result).

Syntax	Groupings produced	Example with `(country, category)`
`GROUPING SETS ((a, b), (a), ())`	exactly the ones you list	whatever you list
`ROLLUP (a, b)`	`(a, b)`, `(a)`, `()` — a hierarchy, dropping columns from the right	detail, per-country subtotal, grand total
`CUBE (a, b)`	every subset: `(a, b)`, `(a)`, `(b)`, `()` — 2^n groupings	detail, per-country, per-category, grand total

In a subtotal row, a column that was "rolled up" is shown as `NULL`. The function `GROUPING(col)` returns 1 when `col` is `NULL` *because it was rolled up*, and 0 when it is a real value — which is how you tell a subtotal `NULL` from a genuine `NULL` in the data.

How it works

We first define a small view (a saved query, Chapter 7) with one row per non-cancelled order line:

```sql
CREATE VIEW sales AS
SELECT c.country, coalesce(p.category, 'Other') AS category,
       oi.quantity * oi.unit_price * (1 - oi.discount) AS revenue
FROM order_items oi
JOIN orders o    ON o.order_id = oi.order_id
JOIN customers c ON c.customer_id = o.customer_id
JOIN products p  ON p.product_id = oi.product_id
WHERE o.status <> 'cancelled';
```

ROLLUP — detail, then a subtotal per country, then the grand total:

```sql
SELECT country, category, round(sum(revenue), 2) AS revenue
FROM sales
GROUP BY ROLLUP (country, category)
ORDER BY country NULLS LAST, category NULLS LAST;
```

Output:

```
 country |  category   | revenue
---------+-------------+---------
 India   | Accessories |  621.85
 India   | Electronics | 2558.04
 India   | Furniture   | 1441.47
 India   | (null)      | 4621.36
 UK      | Accessories |   49.50
 UK      | Electronics | 1808.86
 UK      | Furniture   |  499.00
 UK      | (null)      | 2357.36
 USA     | Electronics |  324.90
 USA     | Other       |  100.00
 USA     | (null)      |  424.90
 (null)  | (null)      | 7403.62
(12 rows)
```

The rows with `category = (null)` are the per-country subtotals; the last row (both `NULL`) is the grand total. Replacing those `NULL`s with labels using `GROUPING()`:

```sql
SELECT CASE WHEN grouping(country) = 1 THEN 'ALL countries' ELSE country END AS country,
       CASE WHEN grouping(category) = 1 THEN 'ALL categories' ELSE category END AS category,
       round(sum(revenue), 2) AS revenue
FROM sales
GROUP BY ROLLUP (country, category)
ORDER BY grouping(country), country, grouping(category), category;
```

Output:

```
    country    |    category    | revenue
---------------+----------------+---------
 India         | Accessories    |  621.85
 India         | Electronics    | 2558.04
 India         | Furniture      | 1441.47
 India         | ALL categories | 4621.36
 UK            | Accessories    |   49.50
 UK            | Electronics    | 1808.86
 UK            | Furniture      |  499.00
 UK            | ALL categories | 2357.36
 USA           | Electronics    |  324.90
 USA           | Other          |  100.00
 USA           | ALL categories |  424.90
 ALL countries | ALL categories | 7403.62
(12 rows)
```

CUBE adds the per-category totals across all countries. `grouping(country, category)` returns a bit mask (a number whose binary digits each flag one column: here the left bit is `country`, the right bit `category`) — 0 for detail rows, 1 when `category` was rolled up, 2 when `country` was, 3 for the grand total:

```sql
SELECT country, category, round(sum(revenue), 2) AS revenue,
       grouping(country, category) AS level_bits
FROM sales
GROUP BY CUBE (country, category)
ORDER BY level_bits, country, category;
```

Output:

```
 country |  category   | revenue | level_bits
---------+-------------+---------+------------
 India   | Accessories |  621.85 |          0
 India   | Electronics | 2558.04 |          0
 India   | Furniture   | 1441.47 |          0
 UK      | Accessories |   49.50 |          0
 UK      | Electronics | 1808.86 |          0
 UK      | Furniture   |  499.00 |          0
 USA     | Electronics |  324.90 |          0
 USA     | Other       |  100.00 |          0
 India   | (null)      | 4621.36 |          1
 UK      | (null)      | 2357.36 |          1
 USA     | (null)      |  424.90 |          1
 (null)  | Accessories |  671.35 |          2
 (null)  | Electronics | 4691.80 |          2
 (null)  | Furniture   | 1940.47 |          2
 (null)  | Other       |  100.00 |          2
 (null)  | (null)      | 7403.62 |          3
(16 rows)
```

GROUPING SETS lets you pick exactly the levels you want — here one total per country, one per category, and the grand total, but no country × category detail:

```sql
SELECT country, category, count(*) AS lines
FROM sales
GROUP BY GROUPING SETS ((country), (category), ())
ORDER BY country NULLS LAST, category NULLS LAST;
```

Output:

```
 country |  category   | lines
---------+-------------+-------
 India   | (null)      |    11
 UK      | (null)      |     5
 USA     | (null)      |     4
 (null)  | Accessories |     6
 (null)  | Electronics |     9
 (null)  | Furniture   |     4
 (null)  | Other       |     1
 (null)  | (null)      |    20
(8 rows)
```

Trade-offs

Option	Pros	Cons
`ROLLUP` / `CUBE` / `GROUPING SETS`	One statement, one scan of the input, totals always consistent	Output mixes levels — readers must understand the `NULL`s; `CUBE` grows as 2^n
Several `GROUP BY` queries + `UNION ALL`	Works in every database	Repeats the query; scans the input once per level
Totals in the application / BI tool (business-intelligence reporting software)	Flexible formatting	Moves more data out of the database

Database	Support
PostgreSQL 9.5+	`GROUPING SETS`, `ROLLUP`, `CUBE`, `GROUPING()`
SQL Server, Oracle	all of the above
MySQL 8	`GROUP BY a, b WITH ROLLUP` and `GROUPING()`; no `CUBE` or `GROUPING SETS`
SQLite	none — use `UNION ALL`

Common mistakes

- Confusing a subtotal `NULL` with a real `NULL` in the data (use `GROUPING()`; or, as the view does, `COALESCE` real `NULL`s to a label first).
- Expecting subtotals in a particular order — add an `ORDER BY` (often on `GROUPING(...)`).
- Using `CUBE` on many columns: 5 columns = 32 groupings.

In the interview

> **30-second answer.** "`ROLLUP(a, b)` gives `(a, b)`, `(a)` and the grand total — hierarchical subtotals. `CUBE(a, b)` gives every combination including `(b)` alone. `GROUPING SETS` lets you list the exact groupings. Rolled-up columns show as `NULL`, and `GROUPING(col)` tells those apart from real `NULL`s."

- *Q: How many groupings does `CUBE(a, b, c)` produce?* — $2^3 = 8$.
- *Q: How would you add a grand-total row without `ROLLUP`?* — `UNION ALL` a second query with no `GROUP BY`.

Self-check

1. Which groupings does `ROLLUP (year, month, day)` produce? → `(year, month, day)`, `(year, month)`, `(year)`, `()`.
2. What does `GROUPING(country)` return on the grand-total row? → 1.
3. Rewrite `GROUP BY ROLLUP (a)` with `GROUPING SETS`. → `GROUP BY GROUPING SETS ((a), ())`.

TL;DR

`ROLLUP` = hierarchy of subtotals + grand total; `CUBE` = all combinations; `GROUPING SETS` = exactly the ones you list. Use `GROUPING()` to label subtotal rows.

Chapter 4 TL;DR

Idea	Remember
Aggregates	`COUNT(*)` counts rows; every other aggregate (and `COUNT(col)`) skips `NULL`s. Empty input → 0 for `COUNT`, `NULL` for the rest.
`GROUP BY`	One output row per distinct key; select only keys and aggregates (or columns of a grouped primary key). `NULL`s form one group. No automatic ordering.
`HAVING`	Filters groups after aggregation; `WHERE` filters rows before. Aggregates only in `HAVING`.
Conditional aggregation	`agg(...) FILTER (WHERE …)` or `SUM(CASE …)` — many filtered numbers in one scan; ratios with `100.0 *`.
Logical order	FROM → WHERE → GROUP BY → aggregates → HAVING → SELECT (windows) → DISTINCT → set ops → ORDER BY → LIMIT.
Subtotals	`ROLLUP`, `CUBE`, `GROUPING SETS`; label rolled-up rows with `GROUPING()`.

Next: Chapter 5 — Joins combines rows from several tables, which is what most real aggregations are built on.

Chapter 5 — Joins: Combining Tables

A well-designed relational database splits information into many narrow tables: employees in one, departments in another, orders in a third. That design avoids duplication (Chapter 11 explains why), but it means the answer to almost any real question — "which department does Neha work in?", "what did Anita buy?" — is spread across tables. A **join** puts the pieces back together at query time. Joins are the single most important skill in SQL and the most common topic in SQL interviews.

What you'll learn

- What a join really is: a **Cartesian product** (every row paired with every row) filtered by a **join condition**, and `CROSS JOIN`.
- `INNER JOIN` and the three **outer joins** — `LEFT`, `RIGHT` and `FULL` — with Venn-style pictures (overlapping-circle diagrams) *and* row-by-row result tables that show exactly which rows come back and where `NULL` padding appears.
- **Self joins** (a table joined to itself: managers, pairs, sequences).
- **Semi-joins** and **anti-joins** ("customers who ordered" / "customers who never ordered") and the three ways to write them.
- Join pitfalls: **fan-out** (rows multiplying), `ON` vs `WHERE` in outer joins, `NULL` keys, `COUNT(*)` after an outer join.
- `USING`, `NATURAL JOIN`, multi-table joins, and how databases physically execute joins.

How to run the examples. All queries are in `code/` and ran on the book's private PostgreSQL 14; outputs are real `psql` output, with `NULL` displayed as `(null)`.

```
bash tools_run_sql.sh sql '05-*'
bash tools_pg.sh psql -d <db> -f sql/chapters/05-joins/code/04_outer_joins.sql
```

We use the book's `company` database (`code/00_schema.sql`, same as Chapter 1). The relationships that the joins in this chapter follow:

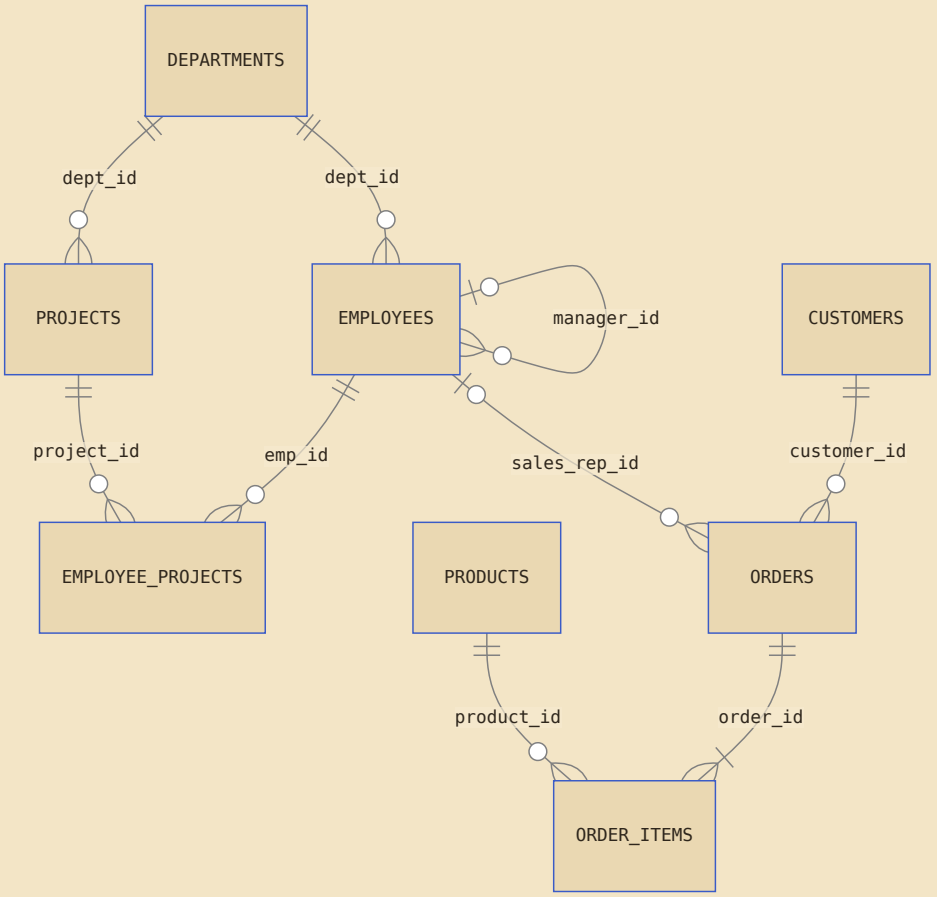


Figure 5.1 — The sample schema's foreign keys: every line is a join path.

A **foreign key** is a column whose values must match the **primary key** (the column that uniquely identifies each row, never `NULL`) of another table — `employees.dept_id` must be an existing `departments.dept_id`. Foreign keys are the natural paths along which tables are joined.

The quirks that make joins interesting: **Asha (CEO) and Rohan (intern) have no department**, the **Research** department has **no employees**, two customers (Hana, Ivan) **never ordered**, one product (Desk Lamp) was **never sold**, and online orders have **no sales rep**.

5.1 Join theory: the Cartesian product and CROSS JOIN

What it is

The **Cartesian product** of two tables (named after René Descartes' coordinate pairs) is the set of **every row of the first table paired with every row of the second**. If `A` has *m* rows and `B` has *n* rows, the product has *m* × *n* rows. SQL writes it as `CROSS JOIN` (or, in the old style, a comma between tables).

Analogy: a clothes shop with 3 sizes and 2 colours can print 3 × 2 = 6 different labels — every size with every colour.

```
CREATE TABLE sizes  (size text);
CREATE TABLE colors (color text);
INSERT INTO sizes  VALUES ('S'), ('M'), ('L');
INSERT INTO colors VALUES ('red'), ('blue');

SELECT size, color FROM sizes CROSS JOIN colors ORDER BY size, color;
SELECT count(*) AS pairs FROM sizes, colors;
```

Output:

```
 size | color
------+-------
 L    | blue
 L    | red
 M    | blue
 M    | red
 S    | blue
 S    | red
(6 rows)

 pairs
-------
     6
(1 row)
```

Every join is a filtered Cartesian product

This is the key idea of the whole chapter. Conceptually, `A JOIN B ON condition`:

1. Forms every pair (a, b) — the Cartesian product.
2. Keeps only the pairs for which the **join condition** (the `ON` clause) is `TRUE`.
3. For outer joins only: adds back the rows that found **no partner**, padding the missing side with `NULL`s.

(Real databases never build the full product when they can avoid it — §5.10 — but the *result* is defined this way.)

```sql
SELECT count(*) AS all_pairs FROM employees, departments;
SELECT count(*) AS matching_pairs FROM employees e, departments d WHERE e.dept_id = d.dept_id;
```

Output:

```
 all_pairs
-----------
        90
(1 row)

 matching_pairs
----------------
             13
(1 row)
```

15 employees × 6 departments = 90 pairs, of which only 13 make sense (the employee really belongs to that department). The 77 others are noise. Forgetting the join condition — an **accidental Cartesian product** — is a classic bug: the query "works" but returns far too many rows, and aggregates over it are wildly wrong.

When CROSS JOIN is what you want

A deliberate cross join is useful for building **every combination**: a report grid (every department × every month, so that months with no data still appear), test data, or a calendar.

```sql
SELECT d.name, m.month
FROM departments d
CROSS JOIN (VALUES ('2025-01'), ('2025-02')) AS m(month)
WHERE d.dept_id <= 2
ORDER BY d.name, m.month;
```

Output:

```
    name     |  month
-------------+---------
 Engineering | 2025-01
 Engineering | 2025-02
 Sales       | 2025-01
 Sales       | 2025-02
(4 rows)
```

Join syntax: explicit vs implicit

Style	Example	Notes
Explicit (ANSI SQL-92, the 1992 SQL standard)	`FROM a JOIN b ON a.k = b.k`	Join condition sits next to the join; required for outer joins. **Use this.**
Implicit (comma)	`FROM a, b WHERE a.k = b.k`	Older style; easy to forget a condition in a long `WHERE`; can't express outer joins in standard SQL.

A few vocabulary words used below: an **equi-join** uses only `=` in its condition; a **non-equi join** uses other operators (`<`, `BETWEEN` …); the **join key** is the column(s) compared; **left** and **right** table simply mean "written before" and "written after" the `JOIN` keyword.

5.2 The four main join types at a glance

Before looking at each join in detail, here they are side by side on two tiny tables, so you can see **every row** each one returns. There is no foreign key between them on purpose, so we can have a student in a club that doesn't exist.

```sql
CREATE TABLE clubs    (club_id int PRIMARY KEY, club text);
CREATE TABLE students (id int PRIMARY KEY, name text, club_id int);   -- no FK on purpose
INSERT INTO clubs    VALUES (10, 'Chess'), (20, 'Drama'), (30, 'Robotics');
INSERT INTO students VALUES (1, 'Ana', 10), (2, 'Ben', 20), (3, 'Cai', 20),
                            (4, 'Dev', NULL), (5, 'Eli', 40);
```

Output (the two inputs):

```
 id | name | club_id
----+------+---------
  1 | Ana  |      10
  2 | Ben  |      20
  3 | Cai  |      20
  4 | Dev  |  (null)
  5 | Eli  |      40
(5 rows)

 club_id |   club
---------+----------
      10 | Chess
      20 | Drama
      30 | Robotics
(3 rows)
```

Row by row, which pairs match on `c.club_id = s.club_id`?

Student	s.club_id	Matching club	Why
Ana	10	Chess	10 = 10
Ben	20	Drama	20 = 20
Cai	20	Drama	20 = 20 (Drama matches **two** students)
Dev	`NULL`	none	`NULL = anything` is UNKNOWN, never TRUE
Eli	40	none	no club 40 (an "orphan" row)
—	—	Robotics (30)	no student has 30

The Venn-style picture

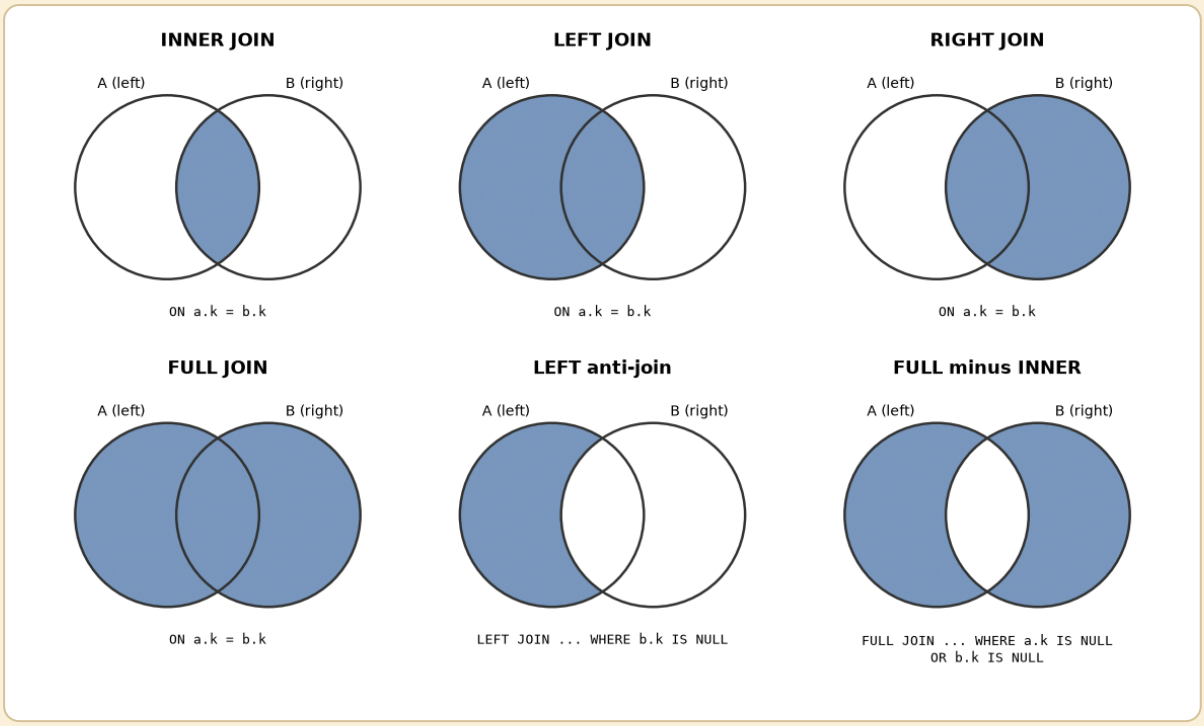


Figure 5.2 — Join "Venn" cheat sheet (drawn by `code/make_join_venn.py`). Left circle = rows of the left table, right circle = rows of the right table, overlap = pairs that satisfy the `ON` condition. Shaded = what the query returns.

A caution about this picture: a Venn diagram describes **sets of rows**, but a join produces **pairs**. One left row can pair with several right rows (Drama pairs with Ben *and* Cai), so the "overlap" can contain more rows than either table has. Use the diagram to remember *which unmatched rows are kept*, and the result tables below to see *how many rows* you get.

The result of each join, row by row

`INNER JOIN` — only the matched pairs (3 rows):

```
SELECT s.id, s.name, s.club_id AS s_club, c.club_id AS c_club, c.club
FROM students s INNER JOIN clubs c ON c.club_id = s.club_id
ORDER BY s.id;
```

```
 id | name | s_club | c_club | club
----+------+--------+--------+-------
  1 | Ana  |     10 |     10 | Chess
  2 | Ben  |     20 |     20 | Drama
  3 | Cai  |     20 |     20 | Drama
(3 rows)
```

`LEFT JOIN` — the matched pairs **plus** every unmatched *left* row, padded with `NULL`s on the right (5 rows):

```sql
SELECT s.id, s.name, s.club_id AS s_club, c.club_id AS c_club, c.club
FROM students s LEFT JOIN clubs c ON c.club_id = s.club_id
ORDER BY s.id;
```

```
 id | name | s_club | c_club |  club
----+------+--------+--------+--------
  1 | Ana  |     10 |     10 | Chess
  2 | Ben  |     20 |     20 | Drama
  3 | Cai  |     20 |     20 | Drama
  4 | Dev  | (null) | (null) | (null)
  5 | Eli  |     40 | (null) | (null)
(5 rows)
```

`RIGHT JOIN` — the matched pairs **plus** every unmatched *right* row, padded with `NULL`s on the left (4 rows):

```sql
SELECT s.id, s.name, s.club_id AS s_club, c.club_id AS c_club, c.club
FROM students s RIGHT JOIN clubs c ON c.club_id = s.club_id
ORDER BY c.club_id, s.id;
```

```
   id   |  name  | s_club | c_club |   club
--------+--------+--------+--------+----------
      1 | Ana    |     10 |     10 | Chess
      2 | Ben    |     20 |     20 | Drama
      3 | Cai    |     20 |     20 | Drama
 (null) | (null) | (null) |     30 | Robotics
(4 rows)
```

`FULL JOIN` — matched pairs, plus unmatched rows from **both** sides (6 rows):

```sql
SELECT s.id, s.name, s.club_id AS s_club, c.club_id AS c_club, c.club
FROM students s FULL JOIN clubs c ON c.club_id = s.club_id
ORDER BY s.id NULLS LAST, c.club_id;
```

```
   id   |  name  | s_club | c_club |   club
--------+--------+--------+--------+----------
      1 | Ana    |     10 |     10 | Chess
      2 | Ben    |     20 |     20 | Drama
      3 | Cai    |     20 |     20 | Drama
      4 | Dev    | (null) | (null) | (null)
      5 | Eli    |     40 | (null) | (null)
 (null) | (null) | (null) |     30 | Robotics
(6 rows)
```

Summary of the same example:

Join	Matched pairs	+ unmatched left (Dev, Eli)	+ unmatched right (Robotics)	Rows
`INNER`	✓			3
`LEFT`	✓	✓		5
`RIGHT`	✓		✓	4
`FULL`	✓	✓	✓	6
`CROSS`	every pair, no condition			5 × 3 = 15

Notice two things in the `LEFT` result. Dev's row has `s_club = (null)` because Dev's own value is `NULL`; Eli's has `s_club = 40` but `c_club = (null)` because the `NULL`s were **added by the join**. A `NULL` in a right-side column after a `LEFT JOIN` therefore means "no partner was found" — unless that column can also be `NULL` in the data, which is why tests for "no match" should use a column that is never `NULL` in the right table, typically its primary key.

The edges of the Venn diagram: "only in one side"

Adding `WHERE <right key> IS NULL` after a `LEFT JOIN` keeps just the rows that found no partner — the left-only crescent (an **anti-join**, §5.8). The same idea works for the right side and for "only in one of them":

```
SELECT s.name FROM students s LEFT JOIN clubs c ON c.club_id = s.club_id
WHERE c.club_id IS NULL ORDER BY s.id;

SELECT c.club FROM students s RIGHT JOIN clubs c ON c.club_id = s.club_id
WHERE s.id IS NULL;

SELECT s.name, c.club FROM students s FULL JOIN clubs c ON c.club_id = s.club_id
WHERE s.id IS NULL OR c.club_id IS NULL ORDER BY s.id NULLS LAST;
```

Output:

```
 name
------
 Dev
 Eli
(2 rows)

   club
----------
 Robotics
(1 row)

  name  |   club
--------+----------
 Dev    | (null)
 Eli    | (null)
 (null) | Robotics
(3 rows)
```

The rest of the chapter goes through each join type in depth on the company data.

5.3 INNER JOIN

What it is

An **inner join** returns only the pairs of rows for which the join condition is true. Rows with no partner on the other side disappear from the result. `JOIN` on its own means `INNER JOIN`.

Analogy: matching guests to reserved seats — a guest without a reservation and a seat nobody booked are both left out of the seating list.

How it works

1. For each row of the left table, find the rows of the right table where the `ON` condition is `TRUE`.
2. Output one combined row per matching pair — so a left row with 3 matches appears 3 times, and a left row with 0 matches not at all.
3. Rows where the key is `NULL` never match (the comparison is UNKNOWN).

```
SELECT e.emp_id, e.first_name, d.name AS department
FROM employees e
INNER JOIN departments d ON d.dept_id = e.dept_id
ORDER BY e.emp_id;
```

Output:

```
 emp_id | first_name | department
--------+------------+-------------
      2 | Vikram     | Engineering
      3 | Neha       | Engineering
      4 | Rahul      | Engineering
      5 | Meera      | Engineering
      6 | Daniel     | Sales
      7 | Priya      | Sales
      8 | Arjun      | Sales
      9 | Sara       | Sales
     10 | Lucas      | Marketing
     11 | Ananya     | Marketing
     12 | Kiran      | Finance
     13 | Omar       | Finance
     14 | Li         | Support
(13 rows)
```

13 rows: Asha (1) and Rohan (15) are missing because their `dept_id` is `NULL`, and Research is missing because no employee points to it.

Table aliases (`e`, `d`) are short names for the tables in this query. They are needed when a column name exists in both tables (`dept_id`), and make long queries readable. Always qualify columns with the alias in joins — `e.dept_id`, not just `dept_id` — or you will get `column reference "dept_id" is ambiguous`.

Filters with joins

For an inner join, a condition gives the same result whether you put it in `ON` or in `WHERE`. Convention: the relationship goes in `ON`, other filters go in `WHERE`. (For outer joins the placement **changes the result** — §5.9.)

```sql
SELECT e.first_name, e.salary, d.name AS department, d.location
FROM employees e
JOIN departments d ON d.dept_id = e.dept_id
WHERE d.location = 'Mumbai' AND e.salary > 70000
ORDER BY e.salary DESC;
```

Output:

```
 first_name |  salary   | department | location
------------+-----------+------------+----------
 Daniel     | 150000.00 | Sales      | Mumbai
 Lucas      | 120000.00 | Marketing  | Mumbai
 Priya      |  72000.00 | Sales      | Mumbai
(3 rows)
```

Joining three or more tables

Each additional `JOIN` joins the result so far with one more table. An order joined to its customer and its sales rep:

```sql
SELECT o.order_id, c.first_name AS customer, e.first_name AS rep, o.status
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN employees e ON e.emp_id = o.sales_rep_id
ORDER BY o.order_id;
```

Output:

```
 order_id | customer |  rep  |  status
----------+----------+-------+-----------
      101 | Anita    | Priya | delivered
      103 | Chen     | Arjun | delivered
      105 | Divya    | Priya | cancelled
      107 | Fatima   | Sara  | shipped
      108 | Ben      | Priya | delivered
      110 | George   | Arjun | shipped
      112 | Ethan    | Sara  | shipped
      113 | Anita    | Priya | pending
(8 rows)
```

Only 8 of the 14 orders appear — the 6 online orders have `sales_rep_id = NULL` and the inner join to `employees` dropped them. That may or may not be what you want; §5.10 fixes it with a `LEFT JOIN`.

A **many-to-many** relationship (employees ↔ projects: one employee works on many projects, one project has many employees) goes through a **junction table** (`employee_projects`, whose rows each link one employee to one project), so you join through it:

```sql
SELECT p.name AS project, d.name AS dept, e.first_name, ep.role, ep.hours
FROM employee_projects ep
JOIN employees   e ON e.emp_id     = ep.emp_id
JOIN projects    p ON p.project_id = ep.project_id
JOIN departments d ON d.dept_id    = p.dept_id
WHERE p.project_id IN (1, 2)
ORDER BY p.project_id, ep.hours DESC;
```

Output:

```
     project     |    dept     | first_name |   role    | hours
-----------------+-------------+------------+-----------+-------
 Checkout Revamp | Engineering | Neha       | Developer |   400
 Checkout Revamp | Engineering | Rahul      | Developer |   350
 Checkout Revamp | Engineering | Vikram     | Lead      |   120
 Mobile App      | Engineering | Neha       | Lead      |   300
 Mobile App      | Engineering | Meera      | Developer |   280
 Mobile App      | Engineering | Rohan      | Tester    |    90
(6 rows)
```

Joins and aggregation work together — join first, then group (Chapter 4):

```sql
SELECT e.first_name AS rep, count(DISTINCT o.order_id) AS orders,
       round(sum(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue
FROM orders o
JOIN employees e    ON e.emp_id = o.sales_rep_id
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status <> 'cancelled'
GROUP BY e.emp_id, e.first_name
ORDER BY revenue DESC;
```

Output:

```
  rep  | orders | revenue
-------+--------+---------
 Priya |      3 | 4141.04
 Sara  |      2 |  879.91
 Arjun |      2 |  687.00
(3 rows)
```

`count(DISTINCT o.order_id)` rather than `count(*)`: after joining to `order_items`, each order appears once *per line*, so `count(*)` would count lines, not orders (the fan-out effect, §5.9).

Non-equi joins

The join condition can be any boolean expression. Here we pair each order line with its product only when the price paid was below the current list price:

```sql
SELECT oi.order_id, p.name, p.unit_price AS list_price, oi.unit_price AS paid
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
               AND oi.unit_price < p.unit_price
ORDER BY oi.order_id;
```

Output:

```
 order_id |     name      | list_price |  paid
----------+---------------+------------+---------
      104 | Standing Desk |     499.00 |  479.00
      108 | Laptop Pro 14 |    1299.00 | 1249.00
(2 rows)
```

Other common non-equi joins: matching a value to a range (`ON s.amount BETWEEN b.low AND b.high` for tax brackets or grade bands), and time windows (`ON b.t > a.t AND b.t <= a.t + interval '1 day'`).

In the interview

30-second answer. "An inner join returns the combinations of rows from both tables that satisfy the join condition; unmatched rows on either side are dropped, and `NULL` keys never match. If a row has several matches it appears several times."

- *Q: Is `JOIN` the same as `INNER JOIN`?* — Yes.
- *Q: `ON` vs `WHERE` for an inner join?* — Same result; put the relationship in `ON` for readability. For outer joins they differ.
- *Q: Why did my row count go up after a join?* — One-to-many (one left row relates to many right rows): each left row repeats once per matching right row.

Self-check

1. `employees` has 15 rows. Can `employees JOIN departments` return more than 15 rows? → Not here, because each employee has at most one department (the join key is the department's primary key). Joining in the other direction to a many-side (e.g. `orders JOIN order_items`) can.
2. Why are the CEO and the intern missing from the inner join? → Their `dept_id` is `NULL`, and `NULL = x` is never true.
3. What error do you get if you write `SELECT dept_id FROM employees e JOIN departments d ON …`? → `column reference "dept_id" is ambiguous`.

TL;DR

`INNER JOIN` = matched pairs only. Unmatched and `NULL`-keyed rows vanish; multiple matches multiply rows. Qualify columns with aliases.

5.4 LEFT OUTER JOIN

What it is

A **left outer join** (`LEFT JOIN` or `LEFT OUTER JOIN` — `OUTER` is optional) returns **every row of the left table**. Rows with a partner are combined with it, exactly as in an inner join; rows **without** a partner are still returned once, with every column from the right table set to `NULL` (called **NULL padding** or **NULL extension**).

Analogy: a class register. Every student is listed; next to each is their exam mark — or a blank if they missed the exam. The blank doesn't remove the student from the register.

Why it matters

It answers the questions that inner joins silently get wrong: "list **all** customers with their order count — **including zero**", "all employees with their department — even if unassigned", "every product and

how much it sold".

How it works (rows of employees LEFT JOIN departments)

The two inputs around the edge cases:

```
SELECT emp_id, first_name, dept_id FROM employees WHERE emp_id IN (1, 2, 14, 15) ORDER BY emp_id;
SELECT dept_id, name FROM departments ORDER BY dept_id;
```

```
 emp_id | first_name | dept_id
--------+------------+---------
      1 | Asha       |  (null)
      2 | Vikram     |       1
     14 | Li         |       5
     15 | Rohan      |  (null)
(4 rows)

 dept_id |    name
---------+-------------
       1 | Engineering
       2 | Sales
       3 | Marketing
       4 | Finance
       5 | Support
       6 | Research
(6 rows)
```

```
SELECT e.first_name, e.dept_id AS e_dept, d.dept_id AS d_dept, d.name
FROM employees e LEFT JOIN departments d ON d.dept_id = e.dept_id
ORDER BY d.dept_id NULLS LAST, e.emp_id;
```

Output:

```
 first_name | e_dept | d_dept |    name
------------+--------+--------+-------------
 Vikram     |      1 |      1 | Engineering
 Neha       |      1 |      1 | Engineering
 Rahul      |      1 |      1 | Engineering
 Meera      |      1 |      1 | Engineering
 Daniel     |      2 |      2 | Sales
 Priya      |      2 |      2 | Sales
 Arjun      |      2 |      2 | Sales
 Sara       |      2 |      2 | Sales
 Lucas      |      3 |      3 | Marketing
 Ananya     |      3 |      3 | Marketing
 Kiran      |      4 |      4 | Finance
 Omar       |      4 |      4 | Finance
 Li         |      5 |      5 | Support
 Asha       | (null) | (null) | (null)
 Rohan      | (null) | (null) | (null)
(15 rows)
```

1. The 13 employees who have a department are matched exactly as in the inner join.
2. Asha and Rohan find no partner. Instead of being dropped, each is output **once** with `d_dept` and `name` padded with `NULL`.
3. Research is not in the result: a left join keeps unmatched rows of the **left** table only.

Rule of thumb: `LEFT JOIN` **never returns fewer rows than the left table has.** It returns more if some left rows match several right rows.

The everyday use: counts that include zero

```sql
SELECT c.customer_id, c.first_name, count(o.order_id) AS orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name
ORDER BY orders, c.customer_id;
```

Output:

```
 customer_id | first_name | orders
-------------+------------+--------
           9 | Hana       |      0
          10 | Ivan       |      0
           4 | Divya      |      1
           7 | George     |      1
           8 | Anita      |      1
           2 | Ben        |      2
           3 | Chen       |      2
           5 | Ethan      |      2
           6 | Fatima     |      2
           1 | Anita      |      3
(10 rows)
```

Hana and Ivan have 0 orders. It must be `count(o.order_id)` — a column from the right table — not `count(*)`: their padded row still counts as one row for `count(*)` (§5.9).

When not to use it

- If you only need **left rows that have a match** (and don't need right-side columns), a semi-join with `EXISTS` is clearer and never duplicates rows (§5.8).
- If you actually require a match, use `INNER JOIN` — a `LEFT JOIN` followed by a `WHERE` on a right column silently becomes an inner join anyway (§5.9).

Common mistakes

- Filtering the right table in `WHERE` (turns it back into an inner join) — put the condition in `ON`.
- Following a `LEFT JOIN` with an `INNER JOIN` to a table hanging off the right side: `a LEFT JOIN b … JOIN c ON c.x = b.x` drops the padded rows again, because `b.x` is `NULL`. Make the second join a `LEFT JOIN` too.
- Using `count(*)` for "how many matches".

In the interview

30-second answer. "A left join returns every row of the left table; where there is a matching right row the columns are filled in, otherwise they're `NULL`. It's the standard way to keep rows with no related data — like customers with zero orders — and, with `WHERE right.pk IS NULL`, to find rows with no match."

- *Q: How many rows can a left join return?* — At least as many as the left table; more if there are one-to-many matches.
- *Q: Find employees without a department.* — `LEFT JOIN departments … WHERE d.dept_id IS NULL`, or simply `WHERE dept_id IS NULL` on employees — plus rows whose `dept_id` points to a missing department, if there is no foreign key.

Self-check

1. After `employees LEFT JOIN departments`, which employees have `d.name = NULL`? → Asha and Rohan.
2. Would the result change if we swapped the tables? → Yes, it becomes "all departments" (see §5.5).
3. Why use `count(o.order_id)` rather than `count(*)` in the customer query? → `count(*)` counts the `NULL`-padded row as 1.

TL;DR

`LEFT JOIN` keeps all left rows and pads missing right-side data with `NULL`. It's how you include "zero" and find "missing".

5.5 RIGHT OUTER JOIN

What it is

A **right outer join** is the mirror image: it keeps **every row of the right table**, padding the left side with `NULL`s where there's no match. `A RIGHT JOIN B` returns the same rows as `B LEFT JOIN A` (only the column order differs).

```
SELECT e.first_name, e.dept_id AS e_dept, d.dept_id AS d_dept, d.name
FROM employees e RIGHT JOIN departments d ON d.dept_id = e.dept_id
ORDER BY d.dept_id, e.emp_id;
```

Output:

```
 first_name | e_dept | d_dept |    name
------------+--------+--------+-------------
 Vikram     |      1 |      1 | Engineering
 Neha       |      1 |      1 | Engineering
 Rahul      |      1 |      1 | Engineering
 Meera      |      1 |      1 | Engineering
 Daniel     |      2 |      2 | Sales
 Priya      |      2 |      2 | Sales
 Arjun      |      2 |      2 | Sales
 Sara       |      2 |      2 | Sales
 Lucas      |      3 |      3 | Marketing
 Ananya     |      3 |      3 | Marketing
 Kiran      |      4 |      4 | Finance
 Omar       |      4 |      4 | Finance
 Li         |      5 |      5 | Support
 (null)     | (null) |      6 | Research
(14 rows)
```

Now Research appears with the **employee** columns padded with `NULL`, and Asha and Rohan are gone (they're unmatched *left* rows). Swapping the tables and using `LEFT JOIN` gives the same 14 rows:

```
SELECT count(*) AS left_swapped FROM departments d LEFT JOIN employees e ON d.dept_id = e.dept_id;
```

```
 left_swapped
--------------
           14
(1 row)
```

When to use it

Rarely. Most teams write every outer join as a `LEFT JOIN` and put the "keep all rows of this" table first, because a query that reads "start from departments, attach employees where possible" is easier to follow. `RIGHT JOIN` is handy when you are appending one more table to a long chain of joins and want to keep all rows of *that* new table without restructuring the query. SQLite supported `RIGHT` and `FULL` joins only from version 3.39.0 (2022).

Self-check

1. Rewrite `orders o RIGHT JOIN customers c ON …` as a left join. → `customers c LEFT JOIN orders o ON …`.
2. Which department does the right join add compared with the inner join? → Research, with `NULL` employee columns.

TL;DR

`RIGHT JOIN` = `LEFT JOIN` with the tables swapped. Prefer writing it as a `LEFT JOIN`.

5.6 FULL OUTER JOIN

What it is

A **full outer join** keeps **all rows from both tables**: matched pairs are combined; unmatched left rows are padded on the right; unmatched right rows are padded on the left. It is the union of the left-join and right-join results (without repeating the matched pairs).

```
SELECT e.first_name, e.dept_id AS e_dept, d.dept_id AS d_dept, d.name
FROM employees e FULL JOIN departments d ON d.dept_id = e.dept_id
ORDER BY d.dept_id NULLS LAST, e.emp_id;
```

Output:

```
 first_name | e_dept | d_dept |    name
------------+--------+--------+-------------
 Vikram     |      1 |      1 | Engineering
 Neha       |      1 |      1 | Engineering
 Rahul      |      1 |      1 | Engineering
 Meera      |      1 |      1 | Engineering
 Daniel     |      2 |      2 | Sales
 Priya      |      2 |      2 | Sales
 Arjun      |      2 |      2 | Sales
 Sara       |      2 |      2 | Sales
 Lucas      |      3 |      3 | Marketing
 Ananya     |      3 |      3 | Marketing
 Kiran      |      4 |      4 | Finance
 Omar       |      4 |      4 | Finance
 Li         |      5 |      5 | Support
 (null)     | (null) |      6 | Research
 Asha       | (null) | (null) | (null)
 Rohan      | (null) | (null) | (null)
(16 rows)
```

The row counts of the four joins on the same data line up exactly with the edge cases — 13 matched pairs, 2 unmatched employees, 1 unmatched department:

```
SELECT (SELECT count(*) FROM employees e JOIN departments d ON d.dept_id = e.dept_id)       AS inner_rows,
       (SELECT count(*) FROM employees e LEFT JOIN departments d ON d.dept_id = e.dept_id)  AS left_rows,
       (SELECT count(*) FROM employees e RIGHT JOIN departments d ON d.dept_id = e.dept_id) AS right_rows,
       (SELECT count(*) FROM employees e FULL JOIN departments d ON d.dept_id = e.dept_id)  AS full_rows;
```

```
 inner_rows | left_rows | right_rows | full_rows
------------+-----------+------------+-----------
         13 |        15 |         14 |        16
(1 row)
```

In general: `full = inner + unmatched left + unmatched right`, `left = inner + unmatched left`, `right = inner + unmatched right`.

Real-world use: reconciliation

`FULL JOIN` shines in **reconciliation** — checking that two records of the same facts agree — i.e. **comparing two lists** where each side may have entries the other lacks: planned vs actual, yesterday's snapshot vs today's, the bank statement vs the ledger.

```sql
CREATE TABLE planned (emp_id int, hours int);
CREATE TABLE actual  (emp_id int, hours int);
INSERT INTO planned VALUES (3, 300), (4, 200), (5, 250);
INSERT INTO actual  VALUES (3, 310), (5, 250), (7, 40);
SELECT coalesce(p.emp_id, a.emp_id) AS emp_id, p.hours AS planned, a.hours AS actual,
       CASE WHEN p.emp_id IS NULL THEN 'unplanned work'
            WHEN a.emp_id IS NULL THEN 'no work logged'
            WHEN p.hours <> a.hours THEN 'different'
            ELSE 'ok' END AS verdict
FROM planned p FULL JOIN actual a ON a.emp_id = p.emp_id
ORDER BY 1;
```

Output:

```
 emp_id | planned | actual |    verdict
--------+---------+--------+----------------
      3 |     300 |    310 | different
      4 |     200 | (null) | no work logged
      5 |     250 |    250 | ok
      7 |  (null) |     40 | unplanned work
(4 rows)
```

`COALESCE(p.emp_id, a.emp_id)` picks whichever side's key is present, giving one clean key column.

Dialects

Database	FULL OUTER JOIN
PostgreSQL, SQL Server, Oracle	supported
SQLite	supported since 3.39.0
MySQL / MariaDB	**not supported** — emulate: `SELECT … FROM a LEFT JOIN b ON … UNION SELECT … FROM a RIGHT JOIN b ON …` (or `UNION ALL` with the right-join half filtered to `WHERE a.key IS NULL` to avoid removing genuine duplicates)

PostgreSQL restricts `FULL JOIN` conditions to ones it can run as a merge or hash join (two of the physical join algorithms described in §5.10) (in practice: at least one equality between the two sides); an arbitrary condition such as `ON a.x < b.y` alone gives an error.

In the interview

30-second answer. "A full outer join returns every row from both tables, matching where it can and padding the other side with `NULL`s where it can't. It's used to reconcile two datasets. MySQL lacks it, so you emulate it with a `LEFT JOIN` `UNION` a `RIGHT JOIN`."

- *Q: How do you find rows that are in exactly one of two tables?* — `FULL JOIN … WHERE a.pk IS NULL OR b.pk IS NULL` (or two anti-joins, or `EXCEPT` — "rows in the first result but not the second" — both ways; Chapter 6).

Self-check

1. Inner join = 13 rows, 2 unmatched left, 1 unmatched right. How many rows in the full join? → 16.
2. Why use `COALESCE(p.emp_id, a.emp_id)`? → On unmatched rows one of the two keys is `NULL`; `COALESCE` returns the one that exists.

TL;DR

`FULL JOIN` = everything from both sides, `NULL`-padded where unmatched. Great for reconciliation; emulate with `UNION` in MySQL.

5.7 SELF JOIN

What it is

A **self join** joins a table **to itself**. It is not a new keyword — it's an ordinary inner or outer join in which the same table appears twice under two different aliases, so SQL can tell "the employee row" from "the manager row". You need it whenever rows of a table refer to other rows of the **same** table (a hierarchy through `manager_id`), or when you want to compare rows of a table with each other (pairs, sequences).

Analogy: a class photo with two copies. On one copy you circle a student; on the other you find the student sitting next to them. Same photo, two roles.

How it works

```
SELECT e.emp_id, e.first_name AS employee, m.first_name AS manager
FROM employees e
LEFT JOIN employees m ON m.emp_id = e.manager_id
ORDER BY e.emp_id;
```

Output:

```
 emp_id | employee | manager
--------+----------+---------
      1 | Asha     | (null)
      2 | Vikram   | Asha
      3 | Neha     | Vikram
      4 | Rahul    | Vikram
      5 | Meera    | Vikram
      6 | Daniel   | Asha
      7 | Priya    | Daniel
      8 | Arjun    | Daniel
      9 | Sara     | Daniel
     10 | Lucas    | Asha
     11 | Ananya   | Lucas
     12 | Kiran    | Asha
     13 | Omar     | Kiran
     14 | Li       | Vikram
     15 | Rohan    | Vikram
(15 rows)
```

1. `e` is "the employee" copy and `m` is "the manager" copy of the same table.
2. The condition `m.emp_id = e.manager_id` pairs each employee with the row whose id equals their `manager_id`.
3. `LEFT JOIN` keeps Asha, who has no manager (`manager_id` is `NULL`); an inner join would drop her.

More self-join patterns

Compare each employee with their manager:

```
SELECT e.first_name, e.salary, m.first_name AS manager, m.salary AS mgr_salary
FROM employees e JOIN employees m ON m.emp_id = e.manager_id
WHERE e.salary >= 0.7 * m.salary
ORDER BY e.emp_id;
```

```
 first_name |  salary   | manager | mgr_salary
------------+-----------+---------+------------
 Vikram     | 185000.00 | Asha    |  250000.00
 Neha       | 142000.00 | Vikram  |  185000.00
(2 rows)
```

Count direct reports (join, then group by the manager):

```
SELECT m.first_name AS manager, count(*) AS direct_reports
FROM employees e JOIN employees m ON m.emp_id = e.manager_id
GROUP BY m.emp_id, m.first_name
ORDER BY direct_reports DESC, manager;
```

```
 manager | direct_reports
---------+----------------
 Vikram  |              5
 Asha    |              4
 Daniel  |              3
 Kiran   |              1
 Lucas   |              1
(5 rows)
```

Find **pairs** of rows that share a value. The condition `a.emp_id < b.emp_id` removes pairing a row with itself (Arjun–Arjun) and mirror duplicates (Arjun–Sara *and* Sara–Arjun):

```
SELECT a.first_name AS emp_a, b.first_name AS emp_b, a.salary
FROM employees a JOIN employees b ON a.salary = b.salary AND a.emp_id < b.emp_id
ORDER BY a.salary;
```

```
 emp_a | emp_b |  salary
-------+-------+----------
 Arjun | Sara  | 68000.00
 Rahul | Meera | 98000.00
(2 rows)
```

Go up **two** levels with a second self join (for arbitrary depth, use a recursive CTE — a CTE, Common Table Expression, is a named subquery written with `WITH`, and a *recursive* one keeps joining to its own previous output until nothing new is found; Chapter 7):

```
SELECT e.first_name AS employee, m.first_name AS manager, g.first_name AS grand_manager
FROM employees e
JOIN employees m ON m.emp_id = e.manager_id
LEFT JOIN employees g ON g.emp_id = m.manager_id
WHERE e.dept_id = 1
ORDER BY e.emp_id;
```

```
 employee | manager | grand_manager
----------+---------+---------------
 Vikram   | Asha    | (null)
 Neha     | Vikram  | Asha
 Rahul    | Vikram  | Asha
 Meera    | Vikram  | Asha
(4 rows)
```

Compare rows **in time** — here, customers who ordered again within 60 days of an earlier order:

```
SELECT a.customer_id, a.order_id AS first_order, b.order_id AS next_order,
       b.placed_at::date - a.placed_at::date AS days_between
FROM orders a JOIN orders b
  ON b.customer_id = a.customer_id AND b.placed_at > a.placed_at
 AND b.placed_at <= a.placed_at + interval '60 days'
ORDER BY a.customer_id, a.order_id;
```

```
 customer_id | first_order | next_order | days_between
-------------+-------------+------------+--------------
           1 |         101 |        104 |           39
(1 row)
```

Many "compare with the previous row" problems (consecutive days, streaks) are solved either with a self join like this or, more efficiently, with `LAG` / `LEAD` window functions (functions that read a value from the previous / next row without collapsing rows; Chapter 8).

Common mistakes

- Forgetting aliases (`employees JOIN employees` is an error: `table name "employees" specified more than once`).
- Using an inner join for hierarchies and losing the root (the CEO).
- Getting every pair twice (and each row paired with itself) — use `a.id < b.id`.

In the interview

> **30-second answer.** "A self join joins a table to itself using two aliases — for hierarchies like employee → manager, for finding pairs of rows with something in common, and for comparing a row with a related row of the same table. Use a left join to keep rows without a parent, and `a.id < b.id` to avoid duplicate pairs."

- *Q: Employees who earn more than their manager?* — `FROM employees e JOIN employees m ON m.emp_id = e.manager_id WHERE e.salary > m.salary`. (In our data: nobody.)
- *Q: How do you get the whole chain of managers?* — A recursive CTE; self joins only reach a fixed depth.

Self-check

1. In the manager query, why is Asha's manager `NULL`? → She has no `manager_id`, and the `LEFT JOIN` padded the manager columns.
2. What does `a.emp_id < b.emp_id` prevent? → Self-pairs and mirrored pairs.

TL;DR

Self join = the same table twice, two aliases, two roles. Hierarchies, pairs, and row-to-row comparisons.

5.8 Semi-joins and anti-joins

What they are

Two very common questions aren't really about *combining* columns at all:

- **Semi-join** — "rows of A that **have at least one** match in B" (customers who have ordered). Each A row is returned **at most once**, and no columns of B are needed.
- **Anti-join** — "rows of A that have **no** match in B" (customers who never ordered).

SQL has no `SEMI JOIN` keyword; you write them with `EXISTS` / `IN` (semi) and `NOT EXISTS` / `LEFT JOIN … IS NULL` / `NOT IN` (anti). The query planner (the database component that decides how to execute a query) recognises these shapes and runs them with special *semi-join* and *anti-join* algorithms.

Semi-join: EXISTS vs a plain JOIN

```sql
SELECT c.customer_id, c.first_name
FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
ORDER BY c.customer_id;
```

```
 customer_id | first_name
-------------+------------
           1 | Anita
           2 | Ben
           3 | Chen
           4 | Divya
           5 | Ethan
           6 | Fatima
           7 | George
           8 | Anita
(8 rows)
```

A plain inner join answers a different question — it returns one row **per order**:

```sql
SELECT c.customer_id, c.first_name
FROM customers c JOIN orders o ON o.customer_id = c.customer_id
ORDER BY c.customer_id;
```

```
 customer_id | first_name
-------------+------------
           1 | Anita
           1 | Anita
           1 | Anita
           2 | Ben
           2 | Ben
           3 | Chen
           3 | Chen
           4 | Divya
           5 | Ethan
           5 | Ethan
           6 | Fatima
           6 | Fatima
           7 | George
           8 | Anita
(14 rows)
```

Fixing that with `SELECT DISTINCT` works but does extra work (build all pairs, then remove duplicates) and hides the intent. `EXISTS` stops at the first match for each customer.

Anti-join: three ways

```sql
-- 1. NOT EXISTS
SELECT c.customer_id, c.first_name FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
ORDER BY 1;

-- 2. LEFT JOIN ... IS NULL
SELECT c.customer_id, c.first_name FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL
ORDER BY 1;

-- 3. NOT IN
SELECT c.customer_id, c.first_name FROM customers c
WHERE c.customer_id NOT IN (SELECT customer_id FROM orders)
ORDER BY 1;
```

Output (identical for all three here):

```
 customer_id | first_name
-------------+------------
           9 | Hana
          10 | Ivan
(2 rows)
```

Form	NULL-safe?	Notes
`NOT EXISTS (correlated subquery)` (a subquery that refers to the outer row)	**Yes**	Clearest intent; PostgreSQL plans it as an anti join. **Recommended.**
`LEFT JOIN … WHERE b.pk IS NULL`	Yes, if you test a never-NULL column (the PK)	Also planned as an anti join; handy when you want other columns from the join.
`NOT IN (subquery)`	**No**	If the subquery returns even one `NULL`, the result is **empty** — see Chapter 6. Also often planned less efficiently.

The plan (printed by `EXPLAIN`, which shows the strategy the database chose; `Seq Scan` means reading the whole table) shows PostgreSQL executing `NOT EXISTS` as a **Hash Anti Join**:

```
EXPLAIN (COSTS OFF)
SELECT c.customer_id FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

```
                  QUERY PLAN
----------------------------------------------
 Hash Anti Join
   Hash Cond: (c.customer_id = o.customer_id)
   ->  Seq Scan on customers c
   ->  Hash
         ->  Seq Scan on orders o
(5 rows)
```

More anti-joins on our data — the product never sold and the department with no staff:

```
SELECT p.name FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id);

SELECT d.name FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

```
   name
-----------
 Desk Lamp
(1 row)

   name
----------
 Research
(1 row)
```

In the interview

30-second answer. "A semi-join returns rows of A that have a match in B, once each — write it with `EXISTS` or `IN`. An anti-join returns rows of A with no match — write it with `NOT EXISTS` or `LEFT JOIN … WHERE b.pk IS NULL`. Avoid `NOT IN` with a subquery on a nullable column: one `NULL` makes it return nothing."

- *Q: Customers who never placed an order?* — Any of the three forms; say why you prefer `NOT EXISTS`.
- *Q: Why not `JOIN` + `DISTINCT` for a semi-join?* — It multiplies then de-duplicates; `EXISTS` stops at the first match and states the intent.

Self-check

1. Which is NULL-safe: `NOT IN` or `NOT EXISTS`? → `NOT EXISTS`.

2. In the `LEFT JOIN … IS NULL` pattern, why test `o.order_id` rather than, say, `o.sales_rep_id`? → `sales_rep_id` can be `NULL` on real rows (online orders); the primary key is never `NULL`, so a `NULL` there must be padding.

> **TL;DR**

Semi-join = `EXISTS` ("has a match", no duplicates). Anti-join = `NOT EXISTS` or `LEFT JOIN … pk IS NULL` ("has no match"). Beware `NOT IN` with `NULL`s.

5.9 Join pitfalls

Pitfall 1 — fan-out (row multiplication)

When you join a parent to **two different child tables** at once, every child row of one kind pairs with every child row of the other kind. Engineering has 4 employees and 2 projects, so joining both gives 4 × 2 = 8 rows, and every salary is counted twice:

```
SELECT d.name, count(*) AS joined_rows, sum(e.salary) AS wrong_payroll
FROM departments d
JOIN employees e ON e.dept_id = d.dept_id
JOIN projects  p ON p.dept_id = d.dept_id
WHERE d.dept_id = 1
GROUP BY d.name;
```

```
    name     | joined_rows | wrong_payroll
-------------+-------------+---------------
 Engineering |           8 |    1046000.00
(1 row)
```

The real payroll is 523,000. **Fix:** aggregate each child table separately (in a derived table — a subquery used in `FROM` — or a CTE), so each has one row per department, *then* join:

```
SELECT d.name, e.payroll, p.projects
FROM departments d
JOIN (SELECT dept_id, sum(salary) AS payroll FROM employees GROUP BY dept_id) e ON e.dept_id = d.dept_id
JOIN (SELECT dept_id, count(*) AS projects FROM projects GROUP BY dept_id)     p ON p.dept_id = d.dept_id
ORDER BY d.dept_id;
```

```
    name     |  payroll  | projects
-------------+-----------+----------
 Engineering | 523000.00 |        2
 Marketing   | 181000.00 |        1
 Finance     | 211000.00 |        1
 Support     |  64000.00 |        1
(4 rows)
```

Signs of fan-out: totals that are suspiciously exact multiples of the right answer, and `count(*)` larger than the number of parent rows. `count(DISTINCT …)` hides it for counts but **not** for sums.

Pitfall 2 — a WHERE on the right table undoes a LEFT JOIN

Goal: every customer with their number of **delivered** orders, 0 if none.

```
SELECT c.customer_id, count(o.order_id) AS delivered
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'delivered'
GROUP BY c.customer_id ORDER BY c.customer_id;          -- wrong: customers with 0 vanish
```

```
 customer_id | delivered
-------------+-----------
           1 |         2
           2 |         2
           3 |         1
           5 |         1
           8 |         1
(5 rows)
```

Only 5 customers. Why: the `LEFT JOIN` produced padded rows for Hana and Ivan (with `o.status = NULL`), and rows for customers whose only orders were not delivered — then `WHERE o.status = 'delivered'` ran **after** the join (logical order, Chapter 4) and threw all of those away. Put the right-table condition **in the `ON` clause**, so it limits which rows *match*, not which rows *survive*:

```sql
SELECT c.customer_id, count(o.order_id) AS delivered
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
                                   AND o.status = 'delivered'
GROUP BY c.customer_id ORDER BY c.customer_id;          -- right: condition in ON
```

```
 customer_id | delivered
-------------+-----------
           1 |         2
           2 |         2
           3 |         1
           4 |         0
           5 |         1
           6 |         0
           7 |         0
           8 |         1
           9 |         0
          10 |         0
(10 rows)
```

Condition on the right table of a LEFT JOIN placed in…	Effect
`ON`	Decides which right rows count as a match; unmatched left rows are still kept (padded).
`WHERE`	Filters the joined result; padded rows have `NULL` there and are removed → behaves like an `INNER JOIN`.

(A `WHERE right.pk IS NULL` test is the deliberate exception — that's the anti-join.) Conditions on the **left** table belong in `WHERE`; put in `ON` they don't remove left rows, they just stop them matching.

Pitfall 3 — NULL keys never match

```sql
CREATE TABLE a (k int, label text);
CREATE TABLE b (k int, label text);
INSERT INTO a VALUES (1, 'a1'), (2, 'a2'), (NULL, 'a-null');
INSERT INTO b VALUES (1, 'b1'), (3, 'b3'), (NULL, 'b-null');
SELECT a.label, b.label FROM a JOIN b ON a.k = b.k;
SELECT a.label, b.label FROM a JOIN b ON a.k IS NOT DISTINCT FROM b.k ORDER BY 1;
```

```
 label | label
-------+-------
 a1    | b1
(1 row)

 label  | label
--------+--------
 a-null | b-null
 a1     | b1
(2 rows)
```

`NULL = NULL` is UNKNOWN, so the two `NULL`-keyed rows don't pair. If you really want `NULL` to match `NULL` (for example when comparing two snapshots of a nullable column), use `IS NOT DISTINCT FROM` — but know that PostgreSQL can't use a hash or merge join for it, so it can be slow on large tables.

Pitfall 4 — COUNT(*) after an outer join

```
SELECT d.name, count(*) AS count_star, count(e.emp_id) AS count_col
FROM departments d LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE d.dept_id IN (5, 6)
GROUP BY d.name ORDER BY d.name;
```

```
   name   | count_star | count_col
----------+------------+-----------
 Research |          1 |         0
 Support  |          1 |         1
(2 rows)
```

Research has no employees, but `count(*)` says 1 — it counted the padded row. Count a right-side, never-NULL column instead.

Other pitfalls, briefly

- **Joining on the wrong columns** — e.g. `ON e.emp_id = d.dept_id` "works" (both are integers) and returns nonsense. Foreign keys document the correct paths.
- **Type mismatches** — joining `text` to `integer` fails in PostgreSQL; casting inside the condition (`ON a.code::int = b.id`) prevents index use on `a.code` (an **index** is a sorted lookup structure kept beside a table so rows can be found by value without reading everything).
- **Duplicated keys in a lookup table** — if the "one" side isn't actually unique, every match fans out. Check with `GROUP BY key HAVING count(*) > 1`.

Self-check

1. A query sums order amounts and joins both `order_items` and `payments` to `orders`. What goes wrong? → Fan-out: each item pairs with each payment, inflating the sums. Aggregate each child first.
2. `a LEFT JOIN b ON a.id = b.a_id WHERE b.active` — what kind of join is this really? → An inner join (padded rows have `b.active = NULL`). Move `b.active` to `ON`.

TL;DR

Aggregate children separately to avoid fan-out; right-table filters of a `LEFT JOIN` go in `ON`; `NULL` keys never match; count a right-side key, not `*`.

5.10 USING, NATURAL JOIN, multi-table joins, and how joins are executed

USING

When the join columns have the **same name** in both tables, `USING (col)` is a shorthand for `ON a.col = b.col`. The shared column appears **once** in the output (and in `SELECT *`), and you refer to it without a table prefix:

```
SELECT order_id, customer_id, first_name, status
FROM orders JOIN customers USING (customer_id)
WHERE order_id <= 103
ORDER BY order_id;
```

```
 order_id | customer_id | first_name |  status
----------+-------------+------------+-----------
      101 |           1 | Anita      | delivered
      102 |           2 | Ben        | delivered
      103 |           3 | Chen       | delivered
(3 rows)
```

In a `FULL JOIN … USING`, the merged column is `COALESCE(left.col, right.col)`, so it is filled in for unmatched rows on either side:

```
SELECT dept_id, first_name, name
FROM employees FULL JOIN departments USING (dept_id)
WHERE first_name IS NULL OR name IS NULL
ORDER BY dept_id NULLS LAST;
```

```
 dept_id | first_name |   name
---------+------------+----------
       6 | (null)     | Research
  (null) | Asha       | (null)
  (null) | Rohan      | (null)
(3 rows)
```

NATURAL JOIN — and why to avoid it

`NATURAL JOIN` joins on **every** pair of columns that happen to share a name, without listing them. It looks convenient until a table gains a column:

```
SELECT count(*) AS natural_rows FROM projects NATURAL JOIN departments;
SELECT count(*) AS using_rows   FROM projects JOIN departments USING (dept_id);
```

```
 natural_rows
--------------
            0
(1 row)

 using_rows
------------
          5
(1 row)
```

`projects` and `departments` share `dept_id` — but also `name` and `budget`. `NATURAL JOIN` required all three to be equal, so no row matched. Schemas (the table definitions) change over time (someone adds `created_at` to both tables), and a `NATURAL JOIN` changes meaning silently. **Name your join columns.**

	ON	USING	NATURAL
Columns compared	any expression	listed same-named columns	*all* same-named columns
Duplicated key column in `SELECT *`	yes (both copies)	no (one merged column)	no
Safe against schema changes	yes	yes	**no**
SQL Server support	yes	no	no

Multi-table joins

A query can join many tables; each `JOIN` adds one table to the result built so far. A realistic report — every order line with customer, product and sales rep, keeping online orders:

```sql
SELECT o.order_id, c.first_name AS customer, p.name AS product, oi.quantity,
       coalesce(e.first_name, 'online') AS rep
FROM orders o
JOIN customers c    ON c.customer_id = o.customer_id
JOIN order_items oi ON oi.order_id   = o.order_id
JOIN products p     ON p.product_id  = oi.product_id
LEFT JOIN employees e ON e.emp_id    = o.sales_rep_id
WHERE o.order_id BETWEEN 101 AND 104
ORDER BY o.order_id, p.name;
```

```
 order_id | customer |           product           | quantity |  rep
----------+----------+-----------------------------+----------+--------
      101 | Anita    | Laptop Pro 14               |        1 | Priya
      101 | Anita    | Wireless Mouse              |        1 | Priya
      102 | Ben      | Noise-Cancelling Headphones |        2 | online
      103 | Chen     | Mechanical Keyboard         |        1 | Arjun
      103 | Chen     | USB-C Hub                   |        2 | Arjun
      104 | Anita    | Ergonomic Chair             |        1 | online
      104 | Anita    | Standing Desk               |        1 | online
(7 rows)
```

And a department overview built entirely from outer joins, so that Sales (no projects) and Research (nothing at all) still appear:

```sql
SELECT d.name AS department,
       count(DISTINCT e.emp_id)     AS employees,
       count(DISTINCT p.project_id) AS projects,
       coalesce(sum(ep.hours), 0)   AS hours_on_own_projects
FROM departments d
LEFT JOIN employees e          ON e.dept_id = d.dept_id
LEFT JOIN projects p           ON p.dept_id = d.dept_id
LEFT JOIN employee_projects ep ON ep.project_id = p.project_id AND ep.emp_id = e.emp_id
GROUP BY d.dept_id, d.name
ORDER BY d.dept_id;
```

```
 department  | employees | projects | hours_on_own_projects
-------------+-----------+----------+-----------------------
 Engineering |         4 |        2 |                  1450
 Sales       |         4 |        0 |                     0
 Marketing   |         2 |        1 |                   350
 Finance     |         2 |        1 |                   240
 Support     |         1 |        1 |                    60
 Research    |         0 |        0 |                     0
(6 rows)
```

This query deliberately fans out (employees × projects per department) — it is only correct because the counts use `DISTINCT` and the hours come from `employee_projects` joined on **both** keys, so each (employee, project) assignment is counted once. When in doubt, pre-aggregate as in §5.9.

Rules for multi-table joins:

1. Inner joins can be written in any order — the result is the same, and the planner picks the physical order.
2. **Outer joins are not freely reorderable**: `(a LEFT JOIN b) JOIN c` can differ from `a LEFT JOIN (b JOIN c)`. Write them in the order you mean, and keep the "keep everything from here" table first.
3. Every table after the first needs its own join condition, or you get a Cartesian product with it.

How databases execute joins

The logical definition (filtered Cartesian product) is not how joins run. The planner picks one of three physical algorithms per join, based on table sizes, indexes and statistics (stored summaries of how many rows a table has and how its values are spread) — you'll see them in `EXPLAIN` (Chapter 12):

Algorithm	How it works	Good when	Cost (roughly)
Nested loop	For each outer row, look up matching inner rows (ideally through an index)	Outer side small, index on the inner join key; also the only option for non-equi joins	*outer rows × cost of one lookup*
Hash join	Build a hash table (a lookup structure that finds a key in about one step) on the smaller input's key, then probe it with each row of the other	Equi-joins of large unsorted inputs	*O(n + m)* (work grows with the total size of both inputs), needs memory for the hash table
Merge join	Sort both inputs by the key (or read them in index order), then walk them together	Inputs already sorted / indexed; very large inputs	*O(n + m)* after sorting

Practical consequence: **index your foreign-key columns** (`orders.customer_id`, `order_items.product_id` …). PostgreSQL indexes primary keys automatically but **not** foreign keys, and nested-loop joins from a parent to its children depend on those indexes.

Dialect summary for joins

Feature	PostgreSQL	MySQL 8	SQLite	SQL Server
`INNER`, `LEFT`, `CROSS`	yes	yes	yes	yes
`RIGHT JOIN`	yes	yes	3.39+	yes
`FULL JOIN`	yes	**no** (emulate with `UNION`)	3.39+	yes
`USING`, `NATURAL`	yes	yes	yes	**no**
`LATERAL` (Chapter 6)	yes	8.0.14+	no	`CROSS APPLY` / `OUTER APPLY`
`IS NOT DISTINCT FROM`	yes	`<=>`	`IS`	2022+

In the interview

30-second answer (whole chapter). "A join pairs rows from two tables that satisfy a condition — conceptually a filtered Cartesian product. Inner join keeps matches only; left/right keep all rows of one side with `NULL` padding; full keeps both; cross gives every combination. Self joins use two aliases on one table. For 'has a match / has no match' use `EXISTS` / `NOT EXISTS`. Watch for fan-out, filters on the outer side in `WHERE`, and `NULL` keys."

- *Q: What's the difference between `WHERE` and `ON` in a left join?* — `ON` decides matches; `WHERE` filters the result and removes padded rows.
- *Q: What join algorithms do you know?* — Nested loop, hash, merge; the optimizer chooses based on sizes, indexes and sort order.
- *Q: How do you do a full outer join in MySQL?* — `LEFT JOIN` `UNION` `RIGHT JOIN`.

Self-check

1. What columns does `SELECT *` return for `orders JOIN customers USING (customer_id)`? → `customer_id` once, then the other columns of `orders` and `customers`.
2. Why is `NATURAL JOIN` risky? → It joins on every same-named column, which changes when schemas change.
3. Which physical join works for `ON a.x < b.y`? → Nested loop (hash and merge need equality).

TL;DR

Prefer explicit `ON` (or `USING`), never `NATURAL`. Inner joins reorder freely; outer joins don't. The planner chooses nested loop, hash or merge — index your foreign keys.

Chapter 5 TL;DR

Join	Returns	Typical use
`CROSS JOIN`	every pair (m × n)	grids, combinations
`INNER JOIN`	matched pairs only	"orders with their customers"
`LEFT JOIN`	all left rows + matches, `NULL`-padded	"all customers, with orders if any"
`RIGHT JOIN`	all right rows + matches	same, mirrored — write as `LEFT`
`FULL JOIN`	all rows of both sides	reconciliation
Self join	table joined to itself (two aliases)	hierarchies, pairs, previous/next
Semi-join (`EXISTS`)	left rows with ≥ 1 match, once each	"customers who ordered"
Anti-join (`NOT EXISTS`, `LEFT … IS NULL`)	left rows with no match	"customers who never ordered"

Pitfalls: fan-out, right-table filters in `WHERE` after a `LEFT JOIN`, `NULL` keys, `count(*)` on padded rows, `NOT IN` with `NULL`s, `NATURAL JOIN`.

Next: Chapter 6 — Subqueries and Set Operations.

Chapter 6 — Subqueries and Set Operations

Some questions need **the answer to another question first**. "Who earns more than the average?" needs the average before it can compare. "Which customers never ordered?" needs the list of customers who *did*. SQL lets you put one query **inside** another — a **subquery** — and use its result as a value, a list, or a whole table. The second half of the chapter covers the other way of combining queries: stacking or comparing their results with the **set operators** `UNION`, `INTERSECT` and `EXCEPT`.

What you'll learn

- What a subquery (nested query) is, where it can appear, and the three shapes of result it can return: **scalar**, **row**, **table**.
- **Non-correlated** subqueries (run once) vs **correlated** subqueries (conceptually re-run for every outer row), and how PostgreSQL actually executes them.
- Multi-row subqueries with `IN`, `EXISTS`, `ANY` / `SOME` / `ALL` — and the famous `NOT IN` + `NULL` trap.
- **Derived tables** (subqueries in `FROM`), row subqueries, and `LATERAL` for "top-N per group" (the first *N* rows inside each group, e.g. the two best-paid people in every department).
- The set operations `UNION`, `UNION ALL`, `INTERSECT`, `EXCEPT` (and their `ALL` forms): rules, `NULL` handling, precedence, sorting, and when to prefer a join or `EXISTS` instead.

How to run the examples. All queries are in `code/` and ran on the book's private PostgreSQL 14; outputs are real `psql` output, `NULL` shown as `(null)`. Data: the book's `company` database (`code/00_schema.sql`; see Chapter 1).

```
bash tools_run_sql.sh sql '06-*'
bash tools_pg.sh psql -d <db> -f sql/chapters/06-subqueries-and-set-operations/code/04_exists.sql
```

6.1 Subqueries: queries inside queries

What it is

A **subquery** (also called a **nested query** or **inner query**) is a complete `SELECT` statement written in parentheses inside another SQL statement (the **outer query**). The database evaluates the subquery and uses its result in the outer query — as a single value, as a list of values to compare against, or as a temporary table.

Analogy: asking a colleague "what is the team's average salary?" before you can answer your manager's question "who earns more than average?". The colleague's answer is the subquery.

Where a subquery can appear

Position	Example	What the subquery must return
`WHERE` / `HAVING` with `=`, `<`, `>` …	`WHERE salary > (SELECT avg(salary) …)`	**one value** (scalar)
`WHERE` with `IN`, `ANY`, `ALL`	`WHERE dept_id IN (SELECT dept_id …)`	**one column, many rows**
`WHERE` with `EXISTS`	`WHERE EXISTS (SELECT 1 …)`	anything — only "any rows?" matters
`SELECT` list	`SELECT name, (SELECT count(*) …) AS n`	one value per outer row
`FROM` / `JOIN`	`FROM (SELECT … ) AS t`	a whole table (**derived table**)
Row comparison	`WHERE (a, b) = (SELECT x, y …)`	one row with several columns
`INSERT … SELECT`, `UPDATE … SET col = (…)`, `DELETE … WHERE … IN (…)`	see §6.3	depends on position

Types of subqueries

Subqueries are classified in two independent ways:

By the shape of their result

Type	Returns	Typical operators
Scalar subquery	exactly one row, one column (or no row → `NULL`)	`=`, `<>`, `<`, `>`, arithmetic, `SELECT` list
Row subquery	one row, several columns	`(a, b) = (…)`
Column / multi-row subquery	one column, many rows	`IN`, `NOT IN`, `ANY`, `ALL`
Table subquery	many rows and columns	`FROM`, `EXISTS`, `IN` with row values

By dependence on the outer query

Type	Meaning	Evaluated
Non-correlated (simple, nested)	The inner query does not mention the outer query's tables. It can run on its own.	Conceptually **once**; the result is reused.
Correlated	The inner query refers to a column of the current outer row (e.g. `WHERE x.dept_id = e.dept_id`). It can't run on its own.	Conceptually **once per outer row** (§6.3).

Textbooks and courses name the same shapes in slightly different ways. How the common names map onto the two tables above:

Name you may meet	What it means	Example in this chapter
Nested scalar subquery	returns one value, used in a comparison	`salary > (SELECT avg(salary) …)` (below)
Single-row subquery	returns at most one row (one or several columns); used with `=`, `<`, `>` or a row constructor	`(dept_id, job_title) = (SELECT … WHERE emp_id = 4)` (§6.6)
Multiple-row subquery	returns several rows; needs `IN`, `ANY`, `ALL` or `EXISTS`	`dept_id IN (SELECT dept_id … 'Mumbai')` (§6.2)
Single-column subquery	returns one column (one or many rows)	every `IN (SELECT one_column …)`
Correlated subquery	refers to the outer row	"above *their own* department's average" (§6.3)

How a non-correlated scalar subquery works

```
SELECT first_name, salary
FROM employees
WHERE salary > (SELECT avg(salary) FROM employees)
ORDER BY salary DESC;
```

Output:

```
 first_name |  salary
------------+-----------
 Asha       | 250000.00
 Vikram     | 185000.00
 Daniel     | 150000.00
 Neha       | 142000.00
 Kiran      | 135000.00
 Lucas      | 120000.00
(6 rows)
```

1. The database runs the inner query once: the average salary is 107,800.
2. It substitutes that value, so the outer query becomes `WHERE salary > 107800`.
3. It runs the outer query.

Without a subquery you would need two round trips (a request sent to the database plus its reply) from your application (fetch the average, then send a second query) — and the value could change between them.

A scalar subquery can be used anywhere a single value can, including the `SELECT` list and arithmetic:

```
SELECT first_name, salary,
       round((SELECT avg(salary) FROM employees)) AS company_avg,
       round(salary - (SELECT avg(salary) FROM employees)) AS diff
FROM employees
WHERE dept_id = 1
ORDER BY emp_id;
```

Output:

```
 first_name |  salary   | company_avg | diff
------------+-----------+-------------+-------
 Vikram     | 185000.00 |      107800 | 77200
 Neha       | 142000.00 |      107800 | 34200
 Rahul      |  98000.00 |      107800 | -9800
 Meera      |  98000.00 |      107800 | -9800
(4 rows)
```

Two classic uses — "the row with the extreme value" and "the second-highest value":

```sql
SELECT first_name, hire_date FROM employees
WHERE hire_date = (SELECT max(hire_date) FROM employees);

SELECT max(salary) AS second_highest
FROM employees
WHERE salary < (SELECT max(salary) FROM employees);
```

Output:

```
 first_name | hire_date
------------+------------
 Rohan      | 2025-06-02
(1 row)

 second_highest
----------------
      185000.00
(1 row)
```

The second query reads "the largest salary that is smaller than the largest salary". (For the *N*-th highest in general, window functions — functions that compute over related rows while keeping every row — such as `DENSE_RANK()` are cleaner — Chapter 8 — and interview versions are in Chapter 19.)

Scalar subqueries: the two edge cases

If a scalar subquery returns **no rows**, its value is `NULL`:

```sql
SELECT (SELECT salary FROM employees WHERE emp_id = 999) AS missing;
```

```
 missing
---------
  (null)
(1 row)
```

If it returns **more than one row**, the query fails at run time:

```sql
SELECT first_name FROM employees
WHERE salary = (SELECT salary FROM employees WHERE dept_id = 1);
```

```
ERROR:  more than one row returned by a subquery used as an expression
```

This error depends on the **data**, not just the query — it can work in testing and fail in production (the live system real users depend on) once a second matching row appears. If more than one row is possible, use `IN` (§6.2), or make the subquery provably single-row (aggregate it, or filter on a unique key).

Common mistakes

- Using `=` with a subquery that can return several rows (use `IN`).
- Forgetting the parentheses, or putting `ORDER BY` inside a subquery and expecting the outer result to be ordered (only the outermost `ORDER BY` orders the final result).
- Writing a subquery where a join is simpler — e.g. fetching a column from a related table for every row (see §6.3 for when each is better).

Self-check

1. What does a scalar subquery return when it finds no row? → `NULL`.
2. Is `(SELECT avg(salary) FROM employees)` correlated? → No — it doesn't refer to any outer table, so it runs once.
3. Name the four places a subquery can appear in a `SELECT`. → `SELECT` list, `FROM`/`JOIN`, `WHERE`, `HAVING`.

TL;DR

A subquery is a `SELECT` inside parentheses. By result: scalar, row, column, table. By dependence: non-correlated (runs once) or correlated (per outer row). Scalar with 0 rows → `NULL`; with 2+ rows → error.

6.2 Multi-row subqueries: IN and NOT IN

What it is

When the subquery returns **one column with many rows**, you compare against the whole list. `x IN (subquery)` is true if `x` equals **any** value in the list; `x NOT IN (subquery)` is true if `x` equals **none** of them.

```
SELECT first_name, dept_id FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Mumbai')
ORDER BY emp_id;
```

Output:

```
 first_name | dept_id
------------+---------
 Daniel     |       2
 Priya      |       2
 Arjun      |       2
 Sara       |       2
 Lucas      |       3
 Ananya     |       3
(6 rows)
```

The subquery returns `{2, 3}` (Sales and Marketing are in Mumbai), and the outer query keeps employees whose `dept_id` is in that set. `IN` never produces duplicates in the outer result, even if the list contains a value many times — it's a semi-join (each outer row is returned at most once if it has any match; Chapter 5).

The subquery can have its own joins and filters:

```
SELECT name FROM products
WHERE product_id IN (SELECT oi.product_id
                     FROM order_items oi JOIN orders o ON o.order_id = oi.order_id
                     WHERE o.status = 'delivered')
ORDER BY name;
```

Output:

```
            name
----------------------------
 Ergonomic Chair
 Gift Card
 Laptop Pro 14
 Mechanical Keyboard
 Noise-Cancelling Headphones
 Standing Desk
 USB-C Hub
 Wireless Mouse
(8 rows)
```

Nesting several levels

Subqueries can be nested inside subqueries. Read them **from the innermost outwards**: Furniture products → orders containing them → the customers of those orders.

```
SELECT first_name, last_name FROM customers
WHERE customer_id IN (
      SELECT customer_id FROM orders
      WHERE order_id IN (
            SELECT order_id FROM order_items
            WHERE product_id IN (SELECT product_id FROM products WHERE category = 'Furniture')))
ORDER BY customer_id;
```

Output:

```
 first_name | last_name
------------+-----------
 Anita      | Shah
 Fatima     | Qureshi
 George     | Hill
(3 rows)
```

Deep nesting is legal but hard to read; the same question is usually clearer as joins plus `DISTINCT` / `EXISTS`, or as a chain of CTEs (Common Table Expressions — named subqueries written with `WITH`; Chapter 7).

The NOT IN trap

Question: which employees are **not** anybody's manager?

```
SELECT count(*) AS not_in_rows FROM employees
WHERE emp_id NOT IN (SELECT manager_id FROM employees);
```

Output:

```
 not_in_rows
-------------
           0
(1 row)
```

Zero — obviously wrong. The subquery's list contains a `NULL` (the CEO has no manager). `x NOT IN (a, b, NULL)` means `x <> a AND x <> b AND x <> NULL`. The last comparison is UNKNOWN for every `x`, so the whole condition can never be TRUE — at best UNKNOWN — and `WHERE` keeps only TRUE rows (three-valued logic, Chapter 2). One `NULL` in the list silently empties the result.

Condition	`x = 3`, list `(1, 2)`	`x = 3`, list `(1, 2, NULL)`	`x = 1`, list `(1, 2, NULL)`
`x IN (list)`	FALSE	**UNKNOWN**	TRUE
`x NOT IN (list)`	TRUE	**UNKNOWN** (row dropped)	FALSE

Two fixes:

```
-- Fix 1: filter the NULLs out of the subquery
SELECT first_name FROM employees
WHERE emp_id NOT IN (SELECT manager_id FROM employees WHERE manager_id IS NOT NULL)
ORDER BY emp_id;

-- Fix 2 (preferred): NOT EXISTS is NULL-safe
SELECT first_name FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM employees r WHERE r.manager_id = e.emp_id)
ORDER BY emp_id;
```

Output (both):

```
 first_name
------------
 Neha
 Rahul
 Meera
 Priya
 Arjun
 Sara
 Ananya
 Omar
 Li
 Rohan
(10 rows)
```

Prefer `NOT EXISTS`: it is correct whatever the data, states the intent ("there is no row that…"), and PostgreSQL plans it as an efficient anti join (a join that returns only the outer rows with *no* match), whereas `NOT IN (subquery)` must stay a filter that checks the whole list (hashed — loaded into an in-memory hash table, a structure that finds a value in about one step — if it fits in memory, otherwise re-scanned per row).

In the interview

30-second answer. "`IN (subquery)` checks membership in the list the subquery returns — a semi-join, no duplicates. `NOT IN` is dangerous: if the list contains a `NULL`, `NOT IN` is never true and returns nothing. Use `NOT EXISTS` (or filter out `NULL`s) for anti-joins."

- *Q: Why does `NOT IN` return no rows?* — The list has a `NULL`; `x <> NULL` is UNKNOWN, so the `AND` chain is never TRUE.
- *Q: Does `IN` have the same problem?* — No: a `NULL` in the list can't make a match TRUE into something else; it can only turn FALSE into UNKNOWN, and both are dropped by `WHERE`.

Self-check

1. `SELECT 5 NOT IN (1, 2, NULL)` returns? → `NULL` (UNKNOWN).
2. Which three columns in our schema would make a `NOT IN (SELECT col …)` risky? → Any nullable one, e.g. `employees.manager_id`, `employees.dept_id`, `orders.sales_rep_id`.

TL;DR

`IN` = "is in the list"; `NOT IN` + a `NULL` in the list = empty result. Anti-joins: use `NOT EXISTS`.

6.3 Correlated subqueries

What it is

A **correlated subquery** refers to a column of the **current row of the outer query**. It therefore can't be run once up front: conceptually, the database runs it again for every outer row, with that row's values plugged in.

Analogy: a teacher checking each student's mark against *that student's own class average*. The average to compare with changes as the teacher moves from student to student.

How it works

"Employees who earn more than the average **of their own department**":

```sql
SELECT e.first_name, e.dept_id, e.salary
FROM employees e
WHERE e.salary > (SELECT avg(x.salary) FROM employees x WHERE x.dept_id = e.dept_id)
ORDER BY e.dept_id, e.salary DESC;
```

Output:

```
 first_name | dept_id |  salary
------------+---------+-----------
 Vikram     |       1 | 185000.00
 Neha       |       1 | 142000.00
 Daniel     |       2 | 150000.00
 Lucas      |       3 | 120000.00
 Kiran      |       4 | 135000.00
(5 rows)
```

1. Take the first outer row, e.g. Vikram with `e.dept_id = 1`.
2. Run the inner query with `e.dept_id` replaced by 1 → average of Engineering = 130,750.
3. Keep Vikram if 185,000 > 130,750. Move to the next outer row and repeat.
4. Asha and Rohan (`dept_id` `NULL`) are never kept: `x.dept_id = NULL` matches nothing, so the average is `NULL` and the comparison is UNKNOWN.

The aliases `e` (outer) and `x` (inner) are essential — both refer to `employees`, and the alias tells SQL which copy each column belongs to. An unqualified column in a subquery is resolved to the **nearest** table that has it (the inner one first), which can silently turn a correlated subquery into a non-correlated one or vice versa.

"The top earner in each department" (ties would all be returned):

```sql
SELECT e.dept_id, e.first_name, e.salary
FROM employees e
WHERE e.salary = (SELECT max(x.salary) FROM employees x WHERE x.dept_id = e.dept_id)
ORDER BY e.dept_id;
```

Output:

```
 dept_id | first_name |  salary
---------+------------+-----------
       1 | Vikram     | 185000.00
       2 | Daniel     | 150000.00
       3 | Lucas      | 120000.00
       4 | Kiran      | 135000.00
       5 | Li         |  64000.00
(5 rows)
```

Correlated subqueries in SELECT, and the join alternative

A correlated scalar subquery in the `SELECT` list works like a per-row lookup:

```sql
SELECT c.customer_id, c.first_name,
       (SELECT count(*)       FROM orders o WHERE o.customer_id = c.customer_id) AS orders,
       (SELECT max(placed_at)::date FROM orders o WHERE o.customer_id = c.customer_id) AS last_order
FROM customers c
ORDER BY c.customer_id;
```

Output:

```
 customer_id | first_name | orders | last_order
-------------+------------+--------+------------
           1 | Anita      |      3 | 2025-06-02
           2 | Ben        |      2 | 2025-04-01
           3 | Chen       |      2 | 2025-05-06
           4 | Divya      |      1 | 2025-02-28
           5 | Ethan      |      2 | 2025-05-20
           6 | Fatima     |      2 | 2025-06-15
           7 | George     |      1 | 2025-04-25
           8 | Anita      |      1 | 2025-04-09
           9 | Hana       |      0 | (null)
          10 | Ivan       |      0 | (null)
(10 rows)
```

The same result with a `LEFT JOIN` and `GROUP BY`:

```
SELECT c.customer_id, c.first_name, count(o.order_id) AS orders, max(o.placed_at)::date AS last_order
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name
ORDER BY c.customer_id;
```

(Identical output — omitted.) How PostgreSQL's optimizer (the component that decides how to run a query) executes the subquery version, as shown by `EXPLAIN` (which prints the chosen plan; `Seq Scan` = read the whole table):

```
EXPLAIN (COSTS OFF)
SELECT c.customer_id, (SELECT count(*) FROM orders o WHERE o.customer_id = c.customer_id) AS orders
FROM customers c;
```

```
                     QUERY PLAN
-----------------------------------------------------
 Seq Scan on customers c
   SubPlan 1
     ->  Aggregate
           ->  Seq Scan on orders o
                 Filter: (customer_id = c.customer_id)
(5 rows)
```

`SubPlan 1` under the customers scan means the inner aggregate really **runs once per customer**. With an index on `orders.customer_id` (an **index** is a sorted lookup structure that finds rows by a column's value without reading the whole table) each run is a quick index lookup, and for a page of 20 rows that is perfectly fine. Over a million customers it's a million lookups, where the join version reads `orders` once and aggregates everything with a single hash join (build a hash table from one input, then probe it with each row of the other).

	Correlated subquery	**Join + `GROUP BY` (or derived table)**
Readability	Very clear for "per row, look up X"	Clear once you know joins
Execution (PostgreSQL)	`EXISTS` / `IN` forms become semi/anti joins; scalar forms in `SELECT` / `WHERE` run per outer row	Set-based (whole tables at once): one pass, hash/merge join
Multiple values from the same lookup	One subquery **per value** (repeated work)	One join gives all columns
Risk	Slow on large outer tables without an index	Fan-out if you join several children (Chapter 5 §5.9)

Rule of thumb: `EXISTS` / `NOT EXISTS` correlated subqueries are excellent and idiomatic. For scalar lookups over **many** rows, prefer a join, a derived table, or `LATERAL` (§6.7).

Correlated subqueries in UPDATE and DELETE

Correlated subqueries are also the standard way to update a table from related data:

```
ALTER TABLE projects ADD COLUMN total_hours int;
UPDATE projects p
SET total_hours = (SELECT coalesce(sum(hours), 0) FROM employee_projects ep WHERE ep.project_id = p.project_id);
SELECT project_id, name, total_hours FROM projects ORDER BY project_id;
```

Output:

```
 project_id |       name       | total_hours
------------+------------------+-------------
          1 | Checkout Revamp  |         870
          2 | Mobile App       |         670
          3 | Spring Campaign  |         370
          4 | Year-End Audit   |         240
          5 | Helpdesk Chatbot |          75
(5 rows)
```

(PostgreSQL also offers `UPDATE … FROM` for join-style updates — Chapter 9.)

Common mistakes

- Forgetting to correlate (`WHERE x.dept_id = dept_id` without the outer alias resolves to `x.dept_id = x.dept_id`, which is always true for non-NULL values).
- A correlated scalar subquery that can return two rows (error at run time).
- Using per-row subqueries for large reports without an index on the correlated column.

In the interview

30-second answer. "A correlated subquery references the outer row, so logically it's evaluated once per outer row — like 'salary above *their department's* average'. The optimizer turns `EXISTS` / `IN` forms into semi-joins, but scalar ones often really run per row, so for large sets I'd rewrite with a join or window function."

- *Q: Rewrite 'above department average' without a correlated subquery.* — Join to a derived table of department averages, or use `avg(salary) OVER (PARTITION BY dept_id)` in a subquery and filter outside.
- *Q: Can a correlated subquery appear in `SELECT`?* — Yes, as a scalar subquery; it must return at most one row.

Self-check

1. Why are Asha and Rohan never returned by the "above department average" query? → Their `dept_id` is `NULL`, so the inner average is `NULL` and the comparison is UNKNOWN.
2. What does `SubPlan` in an `EXPLAIN` tell you? → A subquery evaluated (potentially) once per outer row.
3. Is `WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)` correlated? → Yes (it references `c`).

TL;DR

Correlated = refers to the outer row, evaluated per outer row. Great with `EXISTS`; for big scalar lookups prefer joins, derived tables or window functions.

6.4 EXISTS and NOT EXISTS

What it is

`EXISTS (subquery)` is TRUE if the subquery returns **at least one row**, FALSE if it returns none. It never returns UNKNOWN, and it doesn't care what the subquery's `SELECT` list contains. `NOT EXISTS` is the opposite. They are almost always correlated.

```sql
SELECT d.name FROM departments d
WHERE EXISTS (SELECT 1 FROM projects p WHERE p.dept_id = d.dept_id AND p.end_date IS NULL)
ORDER BY d.dept_id;
```

Output:

```
    name
-------------
 Engineering
 Support
(2 rows)
```

"Departments that have at least one project still running." The database can stop searching as soon as it finds one matching project for a department.

Because only the existence of a row matters, `SELECT 1`, `SELECT *` or even `SELECT NULL` inside `EXISTS` all behave identically; `SELECT 1` is a common convention that signals "I don't need the columns".

```
SELECT count(*) FROM departments d
WHERE EXISTS (SELECT NULL FROM employees e WHERE e.dept_id = d.dept_id);
```

```
 count
-------
     5
(1 row)
```

NOT EXISTS — the anti-join

"Employees who are on no project":

```
SELECT e.first_name, e.job_title FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM employee_projects ep WHERE ep.emp_id = e.emp_id)
ORDER BY e.emp_id;
```

Output:

```
 first_name |   job_title
------------+---------------
 Asha       | CEO
 Daniel     | Sales Manager
 Arjun      | Sales Rep
 Sara       | Sales Rep
(4 rows)
```

"For all" questions: double NOT EXISTS

SQL has no "for all" operator, but "X for **all** Y" is the same as "there is **no** Y for which X is missing". Which customers bought **every** Accessories product?

```
SELECT c.customer_id, c.first_name FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM products p
    WHERE p.category = 'Accessories'
      AND NOT EXISTS (SELECT 1 FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
                      WHERE o.customer_id = c.customer_id AND oi.product_id = p.product_id));
```

Output:

```
 customer_id | first_name
-------------+------------
           3 | Chen
(1 row)
```

Read it as: "customers for whom there is no Accessories product that they have not bought". This pattern is called **relational division**. An often simpler alternative is counting: group each customer's distinct Accessories purchases and keep those whose count equals the number of Accessories products (`HAVING count(DISTINCT product_id) = (SELECT count(*) FROM products WHERE category = 'Accessories')`).

IN vs EXISTS vs NOT IN vs NOT EXISTS with a NULL

A block list that contains a `NULL` shows the difference clearly:

```
CREATE TABLE blocked (customer_id int);
INSERT INTO blocked VALUES (4), (NULL);
SELECT (SELECT count(*) FROM customers WHERE customer_id IN (SELECT customer_id FROM blocked))           AS
in_rows,
       (SELECT count(*) FROM customers c WHERE EXISTS (SELECT 1 FROM blocked b WHERE b.customer_id =
c.customer_id)) AS exists_rows,
       (SELECT count(*) FROM customers WHERE customer_id NOT IN (SELECT customer_id FROM blocked))       AS
not_in_rows,
       (SELECT count(*) FROM customers c WHERE NOT EXISTS (SELECT 1 FROM blocked b WHERE b.customer_id =
c.customer_id)) AS not_exists_rows;
```

Output:

```
 in_rows | exists_rows | not_in_rows | not_exists_rows
---------+-------------+-------------+-----------------
       1 |           1 |           0 |               9
(1 row)
```

`IN` and `EXISTS` agree (1 blocked customer). `NOT IN` returns 0 because of the `NULL`; `NOT EXISTS` correctly returns the 9 other customers.

	IN (subquery)	EXISTS (subquery)
Question asked	"is this value in the list?"	"is there any matching row?"
Correlated?	usually not	usually yes
Multiple columns	`(a, b) IN (SELECT x, y …)`	any condition in the subquery's `WHERE`
Negation with `NULL`s	`NOT IN` breaks	`NOT EXISTS` is safe
Performance in PostgreSQL	both become semi joins	both become semi joins

Old advice that "`EXISTS` is always faster than `IN`" came from older optimizers; modern optimizers (PostgreSQL included) usually turn both `IN (subquery)` and `EXISTS` into the same semi-join. Choose by clarity — but always prefer `NOT EXISTS` over `NOT IN`.

Self-check

1. What does `EXISTS` return if the subquery's single row is all `NULL`s? → TRUE — a row exists.
2. Rewrite `WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Pune')` with `EXISTS`. → `WHERE EXISTS (SELECT 1 FROM departments d WHERE d.dept_id = e.dept_id AND d.location = 'Pune')`.

TL;DR

`EXISTS` = "at least one row". Never UNKNOWN, ignores the select list, stops at the first match. `NOT EXISTS` is the safe anti-join; double `NOT EXISTS` expresses "for all".

6.5 ANY, SOME and ALL

What it is

`ANY` and `ALL` let you use any comparison operator (`=`, `<>`, `<`, `>`, `<=`, `>=`) against a list:

- `x > ANY (subquery)` — true if `x` is greater than **at least one** value (i.e. greater than the minimum). `SOME` is an exact synonym for `ANY`.
- `x > ALL (subquery)` — true if `x` is greater than **every** value (i.e. greater than the maximum).

```
SELECT first_name, salary FROM employees
WHERE salary > ANY (SELECT salary FROM employees WHERE dept_id = 2)
  AND dept_id <> 2
ORDER BY salary;

SELECT first_name, salary FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE dept_id = 2)
ORDER BY salary;
```

Output:

```
 first_name |  salary
------------+-----------
 Omar       |  76000.00
 Rahul      |  98000.00
 Meera      |  98000.00
 Lucas      | 120000.00
 Kiran      | 135000.00
 Neha       | 142000.00
 Vikram     | 185000.00
(7 rows)

 first_name |  salary
------------+-----------
 Vikram     | 185000.00
 Asha       | 250000.00
(2 rows)
```

Sales salaries range from 68,000 to 150,000. `> ANY` means "above 68,000": Ananya (61,000) and Li (64,000) fail it, while Asha and Rohan drop out for another reason — `dept_id <> 2` is UNKNOWN for their `NULL` department. `> ALL` means "above 150,000" (Vikram and Asha).

Expression	Same as
`x = ANY (list)`	`x IN (list)`
`x <> ALL (list)`	`x NOT IN (list)` — with the same `NULL` trap
`x > ANY (list)`	`x > (SELECT min(…))` (if the list is non-empty and has no `NULL`s)
`x > ALL (list)`	`x > (SELECT max(…))` (same caveats)
`x <> ANY (list)`	"differs from at least one value" — rarely what people mean

```
SELECT count(*) AS eq_any FROM products WHERE category = ANY (SELECT DISTINCT category FROM products WHERE
unit_price > 300);
SELECT count(*) AS ne_all_with_null FROM products
WHERE unit_price <> ALL (SELECT unit_price FROM order_items UNION ALL SELECT NULL);
```

Output:

```
 eq_any
--------
      6
(1 row)

 ne_all_with_null
------------------
                0
(1 row)
```

Empty lists

Over an **empty** subquery, `ALL` is TRUE (vacuously: there is no value that fails) and `ANY` is FALSE (there is no value that passes). This differs from the `min`/`max` rewrite, where `max` of nothing is `NULL`:

```sql
SELECT (5 > ALL (SELECT salary FROM employees WHERE dept_id = 6)) AS all_empty,
       (5 > ANY (SELECT salary FROM employees WHERE dept_id = 6)) AS any_empty;
```

```
 all_empty | any_empty
-----------+-----------
 t         | f
(1 row)
```

PostgreSQL also accepts an **array** (an ordered list of values held in one value) on the right-hand side, which is how application code usually passes a list as one parameter (a value sent separately from the SQL text, filled into a placeholder):

```sql
SELECT name FROM products WHERE product_id = ANY (ARRAY[1, 3, 5]) ORDER BY product_id;
```

```
     name
---------------
 Laptop Pro 14
 USB-C Hub
 Standing Desk
(3 rows)
```

Database	`ANY` / `SOME` / `ALL` with subquery	`= ANY(array)`
PostgreSQL	yes	yes
MySQL, SQL Server, Oracle	yes	no arrays
SQLite	**no** — use `IN`, `EXISTS`, `min`/`max`	no

Self-check

1. `salary > ALL (SELECT salary FROM employees WHERE dept_id = 99)` — with no department 99, which rows pass? → All rows (ALL over an empty set is TRUE).
2. What is `= ANY` equivalent to? → `IN`.

TL;DR

`ANY`/`SOME` = compare with at least one; `ALL` = compare with every one. `= ANY` ≡ `IN`; `<> ALL` ≡ `NOT IN` (NULL trap). `ALL` of empty = TRUE.

6.6 Table subqueries: derived tables and row subqueries

What it is

A subquery in the `FROM` clause is a **derived table** (PostgreSQL's docs call it a *subquery in FROM*; some people say *inline view*). Its result is treated as a temporary, unnamed table that exists only for this query. You must give it an alias (optional from PostgreSQL 16; our server is 14):

```sql
SELECT * FROM (SELECT 1 AS x);
```

```
ERROR:  subquery in FROM must have an alias
LINE 1: SELECT * FROM (SELECT 1 AS x);
                      ^
HINT:  For example, FROM (SELECT ...) [AS] foo.
```

Why it matters

Derived tables let you **build a query in stages**: compute something, then filter, join or aggregate the result. They solve the "can't use an alias in `WHERE`" and "can't aggregate an aggregate" problems from Chapter 4.

```sql
SELECT dept_id, headcount
FROM (SELECT dept_id, count(*) AS headcount FROM employees GROUP BY dept_id) AS t
WHERE headcount >= 2
ORDER BY dept_id;
```

Output:

```
 dept_id | headcount
---------+-----------
       1 |         4
       2 |         4
       3 |         2
       4 |         2
  (null) |         2
(5 rows)
```

(The same filter could be written with `HAVING`; the derived-table form becomes necessary when the next step is something `HAVING` can't do.) An **aggregate of an aggregate** — "the average department payroll" — needs two levels, because `avg(sum(salary))` is not allowed:

```sql
SELECT round(avg(payroll)) AS avg_department_payroll
FROM (SELECT dept_id, sum(salary) AS payroll
      FROM employees WHERE dept_id IS NOT NULL GROUP BY dept_id) AS per_dept;
```

Output:

```
 avg_department_payroll
------------------------
                 267400
(1 row)
```

Derived tables can also be **joined** like any table — the standard cure for join fan-out (Chapter 5 §5.9). Each order's total next to that customer's average order total:

```sql
SELECT o.order_id, o.customer_id, round(t.order_total, 2) AS order_total, round(ca.avg_total, 2) AS customer_avg
FROM orders o
JOIN (SELECT order_id, sum(quantity * unit_price * (1 - discount)) AS order_total
      FROM order_items GROUP BY order_id) t ON t.order_id = o.order_id
JOIN (SELECT o2.customer_id, avg(t2.total) AS avg_total
      FROM orders o2
      JOIN (SELECT order_id, sum(quantity * unit_price * (1 - discount)) AS total
            FROM order_items GROUP BY order_id) t2 ON t2.order_id = o2.order_id
      GROUP BY o2.customer_id) ca ON ca.customer_id = o.customer_id
WHERE o.customer_id IN (1, 5)
ORDER BY o.order_id;
```

Output:

```
 order_id | customer_id | order_total | customer_avg
----------+-------------+-------------+--------------
      101 |           1 |     1323.99 |      1151.36
      104 |           1 |      811.49 |      1151.36
      106 |           5 |      174.97 |       212.45
      112 |           5 |      249.93 |       212.45
      113 |           1 |     1318.60 |      1151.36
(5 rows)
```

Notice the order-total subquery written **twice**. That repetition is exactly what a CTE (`WITH order_totals AS (…)`) removes — Chapter 7. Since PostgreSQL 12 a simple CTE that is referenced once is inlined (pasted into the main query), i.e. executed just like the equivalent derived table; CTEs mainly win on readability and reuse.

Row subqueries

A **row subquery** returns a single row with several columns and is compared with a **row constructor** `(a, b)`. "Employees with the same department *and* job title as employee 4":

```
SELECT first_name, job_title, dept_id FROM employees
WHERE (dept_id, job_title) = (SELECT dept_id, job_title FROM employees WHERE emp_id = 4)
ORDER BY emp_id;
```

Output:

```
 first_name | job_title | dept_id
------------+-----------+---------
 Rahul      | Engineer  |       1
 Meera      | Engineer  |       1
(2 rows)
```

With `IN`, a multi-column subquery can return many rows. "Each customer's latest order" as "(customer, time) pairs that are some customer's maximum":

```
SELECT customer_id, order_id, placed_at::date
FROM orders
WHERE (customer_id, placed_at) IN (SELECT customer_id, max(placed_at) FROM orders GROUP BY customer_id)
ORDER BY customer_id;
```

Output:

```
 customer_id | order_id | placed_at
-------------+----------+------------
           1 |      113 | 2025-06-02
           2 |      108 | 2025-04-01
           3 |      111 | 2025-05-06
           4 |      105 | 2025-02-28
           5 |      112 | 2025-05-20
           6 |      114 | 2025-06-15
           7 |      110 | 2025-04-25
           8 |      109 | 2025-04-09
(8 rows)
```

(Row constructors with `IN` work in PostgreSQL, MySQL and Oracle, but **not** in SQL Server — use `EXISTS` there.)

Self-check

1. Why can't you write `SELECT avg(sum(salary)) FROM employees GROUP BY dept_id`? → Aggregates can't be nested; compute the sums in a derived table and average them outside.
2. What must every derived table have in PostgreSQL 14? → An alias.

TL;DR

Subquery in `FROM` = derived table: build queries in stages, aggregate aggregates, pre-aggregate before joins. Row subqueries compare several columns at once.

6.7 LATERAL subqueries

What it is

A normal derived table can't refer to other tables in the same `FROM` clause — it's evaluated independently. A `LATERAL` subquery **can** refer to columns of tables listed *before* it, so it is re-evaluated for each row of those tables. Think of it as a correlated subquery that may return **many rows and many columns**, placed in `FROM`.

```
SELECT d.name, t.n FROM departments d,
     (SELECT count(*) AS n FROM employees e WHERE e.dept_id = d.dept_id) t;
```

```
ERROR:  invalid reference to FROM-clause entry for table "d"
LINE 2: ... count(*) AS n FROM employees e WHERE e.dept_id = d.dept_id)...
                                                             ^
HINT:  There is an entry for table "d", but it cannot be referenced from this part of the query.
```

The problem it solves: top-N per group

"The two highest-paid employees in each department" is hard with plain subqueries (you can't `LIMIT` per group). With `LATERAL`, for each department run a small query that sorts that department's employees and takes 2:

```
SELECT d.name, t.first_name, t.salary
FROM departments d
CROSS JOIN LATERAL (
    SELECT e.first_name, e.salary FROM employees e
    WHERE e.dept_id = d.dept_id
    ORDER BY e.salary DESC, e.emp_id
    LIMIT 2) AS t
ORDER BY d.dept_id, t.salary DESC;
```

Output:

```
    name     | first_name |  salary
-------------+------------+----------
 Engineering | Vikram     | 185000.00
 Engineering | Neha       | 142000.00
 Sales       | Daniel     | 150000.00
 Sales       | Priya      |  72000.00
 Marketing   | Lucas      | 120000.00
 Marketing   | Ananya     |  61000.00
 Finance     | Kiran      | 135000.00
 Finance     | Omar       |  76000.00
 Support     | Li         |  64000.00
(9 rows)
```

`CROSS JOIN LATERAL` drops outer rows for which the subquery returns nothing (Research is missing). To keep them, use `LEFT JOIN LATERAL … ON true` — `ON true` because the correlation is already inside the subquery:

```
SELECT d.name, t.first_name, t.salary
FROM departments d
LEFT JOIN LATERAL (
    SELECT e.first_name, e.salary FROM employees e
    WHERE e.dept_id = d.dept_id
    ORDER BY e.salary DESC LIMIT 1) AS t ON true
ORDER BY d.dept_id;
```

Output:

```
    name     | first_name |  salary
-------------+------------+----------
 Engineering | Vikram     | 185000.00
 Sales       | Daniel     | 150000.00
 Marketing   | Lucas      | 120000.00
 Finance     | Kiran      | 135000.00
 Support     | Li         |  64000.00
 Research    | (null)     |    (null)
(6 rows)
```

"Each customer's most recent order, with several of its columns" — something a scalar subquery can only do one column at a time:

```sql
SELECT c.first_name, last.order_id, last.status, last.placed_at::date
FROM customers c
LEFT JOIN LATERAL (
    SELECT o.order_id, o.status, o.placed_at FROM orders o
    WHERE o.customer_id = c.customer_id
    ORDER BY o.placed_at DESC LIMIT 1) AS last ON true
ORDER BY c.customer_id;
```

Output:

```
 first_name | order_id |  status   | placed_at
------------+----------+-----------+------------
 Anita      |      113 | pending   | 2025-06-02
 Ben        |      108 | delivered | 2025-04-01
 Chen       |      111 | cancelled | 2025-05-06
 Divya      |      105 | cancelled | 2025-02-28
 Ethan      |      112 | shipped   | 2025-05-20
 Fatima     |      114 | pending   | 2025-06-15
 George     |      110 | shipped   | 2025-04-25
 Anita      |      109 | delivered | 2025-04-09
 Hana       |   (null) | (null)    | (null)
 Ivan       |   (null) | (null)    | (null)
(10 rows)
```

With an index on `orders (customer_id, placed_at)`, each lateral lookup reads just one index entry — often the fastest way to get "latest row per group" when there are few outer rows. Window functions (`ROW_NUMBER() OVER (PARTITION BY …)`, Chapter 8) are the other standard approach and are usually better when you need the answer for *every* group of a large table.

Database	Syntax
PostgreSQL 9.3+	`CROSS JOIN LATERAL (…)`, `LEFT JOIN LATERAL (…) ON true`
MySQL 8.0.14+	`LATERAL`
SQL Server, Oracle 12c+	`CROSS APPLY` / `OUTER APPLY` (Oracle also has `LATERAL`)
SQLite	not supported

TL;DR

`LATERAL` = a derived table that can see earlier `FROM` items, evaluated per outer row; can return many rows/columns. Top-N per group, latest row per group. `LEFT JOIN LATERAL … ON true` keeps outer rows with no result.

6.8 Set operations and UNION

What it is

Set operations combine the **results of two queries** into one result, by rows, like the operations on sets in mathematics (a *set* is a collection of distinct items; a *Venn picture* draws two sets as overlapping circles):

Operator	Result	Venn picture
`UNION`	rows in the first **or** the second result (duplicates removed)	both circles
`UNION ALL`	all rows of both results, duplicates kept	both circles, overlap counted twice
`INTERSECT`	rows in **both** results	only the overlap
`EXCEPT`	rows in the first result that are **not** in the second	left circle minus overlap

Joins combine tables **side by side** (more columns); set operations stack results **on top of each other** (more or fewer rows, same columns).

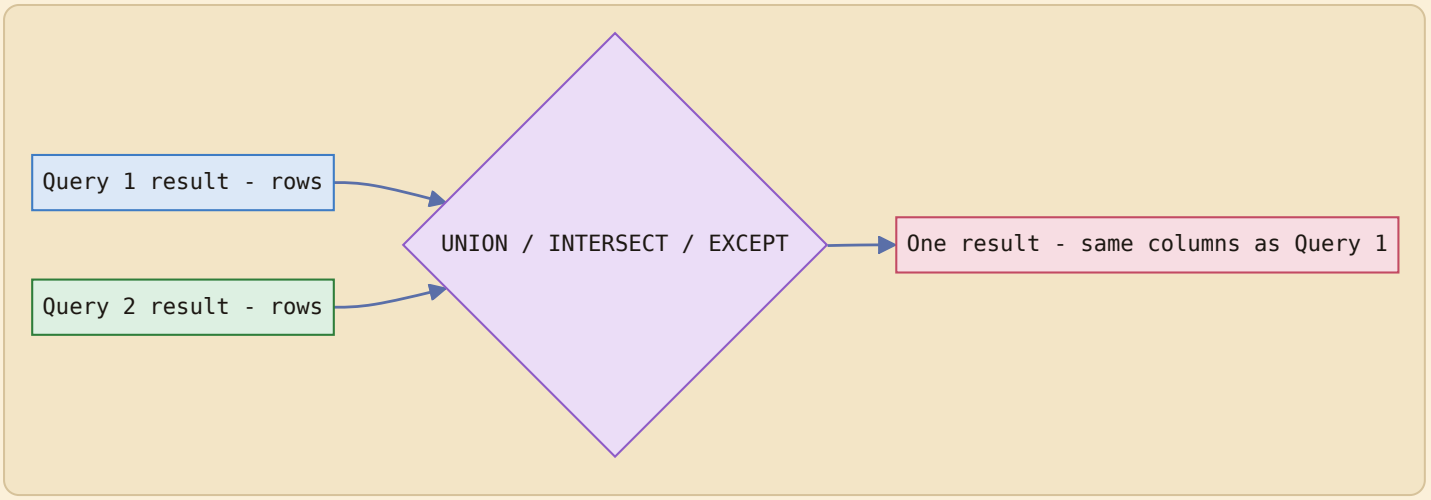


Figure 6.1 — A set operation takes two results with the same columns and produces one.

The rules

1. Both queries must return the **same number of columns**.
2. Corresponding columns must have **compatible types** (PostgreSQL resolves them to a common type, like `CASE` does).
3. Column **names** of the result come from the **first** query.
4. Rows are compared **as whole rows**, and — unlike `=` — two `NULL`s are treated as **equal** (not distinct).
5. `ORDER BY` / `LIMIT` at the end apply to the **combined** result (§6.11).

Breaking rule 1 or 2:

```
SELECT first_name, last_name FROM employees UNION SELECT first_name FROM customers;
SELECT emp_id FROM employees UNION SELECT first_name FROM customers;
```

```
ERROR:  each UNION query must have the same number of columns
LINE 1: ...first_name, last_name FROM employees UNION SELECT first_name...
                                                             ^

ERROR:  UNION types integer and text cannot be matched
LINE 1: SELECT emp_id FROM employees UNION SELECT first_name FROM cu...
                                           ^
```

UNION

One contact list from two tables, with a literal column (a fixed value typed into the query, here `'employee'`) to show where each row came from:

```
SELECT first_name, last_name, 'employee' AS kind FROM employees WHERE dept_id = 3
UNION
SELECT first_name, last_name, 'customer' FROM customers WHERE country = 'UK'
ORDER BY kind, last_name;
```

Output:

```
 first_name | last_name |   kind
------------+-----------+----------
 Ben        | Carter    | customer
 George     | Hill      | customer
 Ananya     | Das       | employee
 Lucas      | Martin    | employee
(4 rows)
```

Every city where we have customers or offices — `UNION` removes the duplicates (Mumbai, Bengaluru, Delhi …):

```
SELECT city FROM customers WHERE city IS NOT NULL
UNION
SELECT location FROM departments WHERE location IS NOT NULL
ORDER BY city;
```

Output:

```
   city
-----------
 Bengaluru
 Delhi
 London
 Mumbai
 New York
 Pune
 mumbai
(7 rows)
```

Note that `mumbai` and `Mumbai` are different strings, so both survive — clean data first (`initcap(city)`, Chapter 3) if case shouldn't matter.

UNION vs UNION ALL

```
SELECT count(*) AS union_rows     FROM (SELECT country FROM customers UNION     SELECT 'India') t;
SELECT count(*) AS union_all_rows FROM (SELECT country FROM customers UNION ALL SELECT 'India') t;
```

Output:

```
 union_rows
------------
          3
(1 row)

 union_all_rows
----------------
             11
(1 row)
```

`UNION` returned 3 (India, UK, USA — duplicates inside **each** input are removed too, not just between them). `UNION ALL` returned all 10 + 1 = 11 rows.

	UNION	UNION ALL
Duplicates	removed (whole-row comparison)	kept
Extra work	a sort or hash over the whole result	none — rows are just appended
Use when	you need distinct rows	the inputs can't overlap, or duplicates are meaningful (counting, event logs)

Default to `UNION ALL` unless you actually need de-duplication; `UNION` on large results costs a sort or hash and hides data problems.

A typical `UNION ALL`: stacking two kinds of events into one timeline:

```
SELECT placed_at AS at, 'order placed'  AS event, order_id FROM orders   WHERE customer_id = 1
UNION ALL
SELECT shipped_at,      'order shipped',          order_id FROM orders   WHERE customer_id = 1 AND shipped_at IS
NOT NULL
ORDER BY at;
```

Output:

```
         at          |     event     | order_id
---------------------+---------------+----------
 2025-01-03 10:15:00 | order placed  |      101
 2025-01-04 09:00:00 | order shipped |      101
 2025-02-11 09:05:00 | order placed  |      104
 2025-02-12 16:45:00 | order shipped |      104
 2025-06-02 10:30:00 | order placed  |      113
(5 rows)
```

And rule 4 — set operations treat `NULL`s as duplicates of each other:

```
SELECT NULL::text AS v UNION SELECT NULL::text;
```

```
   v
--------
 (null)
(1 row)
```

Self-check

1. `SELECT 1 UNION SELECT 1 UNION ALL SELECT 1` returns how many rows? → 2 (operators apply left to right: `(1 UNION 1)` = 1 row, then `UNION ALL 1` = 2 rows).
2. Where do the result's column names come from? → The first query.
3. Why prefer `UNION ALL` by default? → No de-duplication cost and no silently lost rows.

TL;DR

Set operations stack results with the same column count and compatible types; names come from the first query. `UNION` de-duplicates (cost!), `UNION ALL` doesn't. `NULL`s count as equal.

6.9 INTERSECT

What it is

`INTERSECT` returns the rows that appear in **both** results (the overlap of the Venn diagram), without duplicates.

"Cities where we have both customers and an office":

```
SELECT city FROM customers
INTERSECT
SELECT location FROM departments
ORDER BY city;
```

Output:

```
   city
-----------
 Bengaluru
 Mumbai
 (null)
(3 rows)
```

The `(null)` row is a lesson in rule 4: one customer has no city and one department has no location, and set operations treat the two `NULL`s as equal. An inner join on `c.city = d.location` would not match them. Add `WHERE city IS NOT NULL` if that row is unwanted.

"Employees who are on a project **and** manage someone":

```sql
SELECT emp_id FROM employee_projects
INTERSECT
SELECT manager_id FROM employees
ORDER BY emp_id;
```

Output:

```
 emp_id
--------
      2
     10
     12
(3 rows)
```

`INTERSECT` is equivalent to a semi-join with `EXISTS` on all columns (using `IS NOT DISTINCT FROM` for `NULL`s) plus `DISTINCT`. Use `INTERSECT` when you compare whole rows of two similar results; use `EXISTS`/`JOIN` when you need other columns from either side.

6.10 EXCEPT

What it is

`EXCEPT` returns the rows of the first result that do **not** appear in the second (the left crescent), without duplicates. Oracle calls it `MINUS` (Oracle 21c also accepts `EXCEPT`).

"Products that were never ordered":

```sql
SELECT product_id FROM products
EXCEPT
SELECT product_id FROM order_items;
```

Output:

```
 product_id
------------
          8
(1 row)
```

Unlike `UNION` and `INTERSECT`, **order matters**: `A EXCEPT B` ≠ `B EXCEPT A`.

```sql
SELECT city FROM customers EXCEPT SELECT location FROM departments ORDER BY 1;
SELECT location FROM departments EXCEPT SELECT city FROM customers ORDER BY 1;
```

Output:

```
   city
----------
 Delhi
 London
 New York
 mumbai
(4 rows)

 location
----------
 Pune
(1 row)
```

Customer cities with no office; office cities with no customers (only Pune).

Because `NULL`s compare equal in set operations, `EXCEPT` is also a handy way to **diff two queries that contain `NULL`s** — for example, check that a refactored query (rewritten for clarity or speed, meant to

behave identically) returns exactly the same rows as the old one: `(old EXCEPT new)` and `(new EXCEPT old)` should both be empty. Here Meera's row is removed even though her email is `NULL`:

```
SELECT first_name, email FROM employees WHERE dept_id = 1
EXCEPT
SELECT first_name, email FROM employees WHERE emp_id IN (2, 5)
ORDER BY 1;
```

Output:

```
 first_name |            email
------------+----------------------------
 Neha       | Neha.Gupta@Company.example
 Rahul      | rahul.verma@company.example
(2 rows)
```

Without INTERSECT and EXCEPT

Some databases lacked these operators for a long time — MySQL only gained `INTERSECT` and `EXCEPT` in 8.0.31 (2022) — so you will often see them written with semi- and anti-joins instead (Chapter 5 §5.8):

```
-- INTERSECT-like: customer cities that also have an office
SELECT DISTINCT c.city FROM customers c
WHERE EXISTS (SELECT 1 FROM departments d WHERE d.location IS NOT DISTINCT FROM c.city)
ORDER BY c.city;

-- EXCEPT-like: customers whose city has no office (whole rows)
SELECT c.customer_id, c.first_name, c.city FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM departments d WHERE d.location = c.city)
ORDER BY c.customer_id;
```

Output:

```
   city
-----------
 Bengaluru
 Mumbai
 (null)
(3 rows)

 customer_id | first_name |   city
-------------+------------+----------
           2 | Ben        | London
           4 | Divya      | Delhi
           5 | Ethan      | New York
           6 | Fatima     | (null)
           7 | George     | London
           8 | Anita      | mumbai
           9 | Hana       | New York
          10 | Ivan       | Delhi
(8 rows)
```

(A `LEFT JOIN (SELECT DISTINCT location FROM departments) d ON d.location = c.city WHERE d.location IS NULL` returns the same 8 rows — see the code file.) Two differences from the real operators matter: you must add `DISTINCT` yourself to get set semantics (no duplicate rows), and `=` does **not** treat `NULL`s as equal. That is why the first query uses `IS NOT DISTINCT FROM` to reproduce `INTERSECT`'s `(null)` row, and why Fatima (no city) appears in the `NOT EXISTS` result although `EXCEPT` would have removed her `NULL` city. In return, the `EXISTS` forms can return any columns of the outer table, not just the compared ones.

The ALL variants: bag semantics

SQL tables are really **bags** (multisets): the same row can appear several times. The `ALL` forms keep track of **how many times** each row appears:

Operator	Row appearing *m* times in A and *n* times in B appears …
`UNION ALL`	*m* + *n* times
`UNION`	once
`INTERSECT ALL`	*min(m, n)* times
`INTERSECT`	once, if *m* > 0 and *n* > 0
`EXCEPT ALL`	*max(m − n, 0)* times
`EXCEPT`	once, if *m* > 0 and *n* = 0

```
CREATE TABLE a (x int);
CREATE TABLE b (x int);
INSERT INTO a VALUES (1), (1), (2), (3);
INSERT INTO b VALUES (1), (1), (2), (2), (4);
SELECT x FROM a INTERSECT     SELECT x FROM b ORDER BY x;
SELECT x FROM a INTERSECT ALL SELECT x FROM b ORDER BY x;
SELECT x FROM a EXCEPT        SELECT x FROM b ORDER BY x;
SELECT x FROM a EXCEPT ALL    SELECT x FROM b ORDER BY x;
```

Output (in order):

```
 x
---
 1
 2
(2 rows)

 x
---
 1
 1
 2
(3 rows)

 x
---
 3
(1 row)

 x
---
 1
 3
(2 rows)
```

Check against the table: value 1 is 3 times in `a` and 2 times in `b` → `INTERSECT ALL` gives min(3, 2) = 2 copies, `EXCEPT ALL` gives 3 − 2 = 1 copy. Value 2 occurs once in `a` and twice in `b` → 1 copy in `INTERSECT ALL`, 0 in `EXCEPT ALL`. Value 3 is only in `a` → only in the `EXCEPT` results. For completeness, `UNION` gives `1, 2, 3, 4` and `UNION ALL` gives all 10 values.

Dialect support

Operator	PostgreSQL	MySQL	SQLite	SQL Server	Oracle
`UNION`, `UNION ALL`	yes	yes	yes	yes	yes
`INTERSECT`, `EXCEPT`	yes	8.0.31+	yes	yes	`INTERSECT`, `MINUS` (+ `EXCEPT` in 21c)
`INTERSECT ALL`, `EXCEPT ALL`	yes	8.0.31+	no	no	21c+

In the interview

> **30-second answer.** "`UNION` combines two results and removes duplicates; `UNION ALL` keeps them and is cheaper. `INTERSECT` keeps rows in both; `EXCEPT` keeps rows in the first but not the second. Both inputs need the same number of columns with compatible types, and set operators treat `NULL`s as equal."

- *Q: `UNION` vs `UNION ALL`?* — De-duplication vs none; use `UNION ALL` unless you need distinct rows.
- *Q: Find IDs present in table A but not in B.* — `SELECT id FROM a EXCEPT SELECT id FROM b`, or `NOT EXISTS`. `EXCEPT` compares only the selected columns and returns distinct rows; `NOT EXISTS` returns full rows of `a` (with duplicates if `a` has them).
- *Q: How do you check two queries return identical results?* — `(q1 EXCEPT ALL q2) UNION ALL (q2 EXCEPT ALL q1)` must be empty.

Self-check

1. `A = {1,1,2}`, `B = {1}`. What are `A EXCEPT B` and `A EXCEPT ALL B`? → `{2}` and `{1, 2}`.
2. Which set operator is not commutative (gives a different answer when its two sides are swapped)? → `EXCEPT`.
3. Why did `INTERSECT` return a `(null)` city? → Set operations treat two `NULL`s as the same value.

TL;DR

`INTERSECT` = in both; `EXCEPT` = in the first only (order matters). `ALL` variants count duplicates (min, difference). Great for diffing results.

6.11 Precedence, sorting, and choosing the right tool

Precedence

`INTERSECT` binds **tighter** than `UNION` and `EXCEPT` (like multiplication before addition); `UNION` and `EXCEPT` are evaluated left to right. Parentheses override:

```
SELECT 1 AS v UNION SELECT 2 INTERSECT SELECT 3;          -- = 1 UNION (2 INTERSECT 3)
(SELECT 1 AS v UNION SELECT 2) INTERSECT SELECT 3;        -- parentheses change it
```

Output:

```
 v
---
 1
(1 row)

 v
---
(0 rows)
```

When mixing operators, **always use parentheses** — readers shouldn't need to remember precedence.

ORDER BY and LIMIT

A single `ORDER BY`/`LIMIT` at the end applies to the **whole combined result**:

```
SELECT first_name, salary FROM employees WHERE dept_id = 1
UNION ALL
SELECT first_name, salary FROM employees WHERE dept_id = 2
ORDER BY salary DESC
LIMIT 3;
```

Output:

```
 first_name |  salary
------------+-----------
 Vikram     | 185000.00
 Daniel     | 150000.00
 Neha       | 142000.00
(3 rows)
```

To sort or limit **each branch**, wrap the branches in parentheses:

```
(SELECT first_name, salary FROM employees WHERE dept_id = 1 ORDER BY salary DESC LIMIT 1)
UNION ALL
(SELECT first_name, salary FROM employees WHERE dept_id = 2 ORDER BY salary DESC LIMIT 1);
```

Output:

```
 first_name |  salary
------------+-----------
 Vikram     | 185000.00
 Daniel     | 150000.00
(2 rows)
```

The final `ORDER BY` may only use **output column names or positions** — not expressions. Move the set operation into a derived table to sort by an expression:

```
SELECT first_name FROM employees UNION SELECT first_name FROM customers ORDER BY upper(first_name);
```

```
ERROR:  invalid UNION/INTERSECT/EXCEPT ORDER BY clause
LINE 1: ...s UNION SELECT first_name FROM customers ORDER BY upper(firs...
                                                             ^
DETAIL:  Only result column names can be used, not expressions or functions.
HINT:  Add the expression/function to every SELECT, or move the UNION into a FROM clause.
```

```
SELECT * FROM (SELECT first_name FROM employees UNION SELECT first_name FROM customers) t
ORDER BY upper(first_name) LIMIT 3;
```

```
 first_name
------------
 Ananya
 Anita
 Arjun
(3 rows)
```

(Without any `ORDER BY`, the row order of a set operation — like any query — is not guaranteed.)

EXCEPT vs NOT EXISTS

```
SELECT customer_id FROM customers EXCEPT SELECT customer_id FROM orders ORDER BY 1;

SELECT c.customer_id, c.first_name, c.city FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
ORDER BY 1;
```

Output:

```
 customer_id
-------------
           9
          10
(2 rows)

 customer_id | first_name |   city
-------------+------------+----------
           9 | Hana       | New York
          10 | Ivan       | Delhi
(2 rows)
```

Both find the customers who never ordered, but `EXCEPT` can only return the compared columns, while `NOT EXISTS` returns any columns of the outer table.

Choosing between joins, subqueries and set operations

You want…	Best tool
Columns from several related tables in one row	`JOIN` (Chapter 5)
Rows of A that have / don't have a match in B	`EXISTS` / `NOT EXISTS` (semi / anti-join)
Compare with one computed value (average, max)	scalar subquery
Compare with a list	`IN` (never `NOT IN` on nullable columns)
Build in stages; aggregate an aggregate; pre-aggregate before a join	derived table or CTE (Chapter 7)
Top-N / latest row per group	`LATERAL` or a window function (Chapter 8)
Stack similar results from different sources	`UNION ALL` (`UNION` if de-duplication is needed)
Rows common to / missing from two similar results, whole-row comparison, `NULL`-safe	`INTERSECT` / `EXCEPT`

In the interview

30-second answer (whole chapter). "A subquery is a query nested in another; it can return a scalar, a row, a column or a table. Non-correlated subqueries run once; correlated ones depend on the outer row. I use `EXISTS` / `NOT EXISTS` for semi- and anti-joins and avoid `NOT IN` because of `NULL`s. Derived tables and `LATERAL` let me build queries in stages. Set operators combine whole results: `UNION ALL` by default, `UNION` to de-duplicate, `INTERSECT` for common rows, `EXCEPT` for differences — same column count, compatible types, `NULL`s treated as equal."

- *Q: When is a subquery better than a join?* — When you only test existence (`EXISTS`), compare with an aggregate, or need to aggregate before joining. A join is better when you need columns from both sides.
- *Q: Can you `ORDER BY` inside a `UNION` branch?* — Only inside parentheses and normally with `LIMIT`; otherwise the final `ORDER BY` sorts the whole result.

Self-check

1. What does `SELECT 1 UNION SELECT 2 INTERSECT SELECT 2` return? → `{1, 2}` — `INTERSECT` first gives `{2}`, then `UNION` with `{1}`.
2. How do you sort a `UNION` result by `lower(name)`? → Wrap it in a derived table, or add `lower(name)` as an output column of every branch.

TL;DR

`INTERSECT` before `UNION`/`EXCEPT`; use parentheses. Final `ORDER BY`/`LIMIT` apply to the whole result and accept only output columns. Pick joins for columns, `EXISTS` for existence, set operators for whole-result comparisons.

Chapter 6 TL;DR

Tool	Returns / does	Watch out for
Scalar subquery	one value	0 rows → `NULL`; 2+ rows → run-time error
`IN (subquery)`	membership (semi-join)	—
`NOT IN (subquery)`	non-membership	any `NULL` in the list → no rows
Correlated subquery	per-outer-row evaluation	slow for big scalar lookups; qualify columns
`EXISTS` / `NOT EXISTS`	"is there a row?"	the safe semi/anti-join
`ANY` / `ALL`	compare with at least one / every value	`ALL` over empty = TRUE; `<> ALL` = `NOT IN`
Derived table	subquery in `FROM`	needs an alias (before PostgreSQL 16)
`LATERAL`	per-row subquery in `FROM`	`LEFT JOIN LATERAL … ON true` to keep all rows
`UNION` / `UNION ALL`	stack results	`UNION` costs a de-duplication
`INTERSECT` / `EXCEPT`	common / missing rows	`EXCEPT` isn't commutative; `NULL`s compare equal

Next: Chapter 7 — CTEs, Views and Recursion gives names to subqueries so that long queries read top to bottom.

Chapter 7 — CTEs, Views and Recursion: Naming Queries and Walking Trees

Real queries grow. A report that needs "order totals, then customer totals, then only the customers above average" quickly turns into three levels of nested subqueries that nobody can read. This chapter gives you the tools SQL has for **naming** a piece of a query so you can build big queries out of small, readable steps:

- a **common table expression (CTE)** names a query for the length of *one* statement;
- a **recursive CTE** calls itself, which lets plain SQL walk hierarchies (org charts, category trees, bills of materials — parts lists where each part can itself be made of parts) and graphs (networks of connected items: flight routes, friend networks) and produce sequences;
- a **view** names a query *permanently* in the database, so everyone can reuse it like a table;
- a **materialized view** also stores the query's *result*, trading freshness for speed.

It also covers `generate_series`, PostgreSQL's quick way to produce rows of numbers or dates, which you need for "fill in the missing days" reports.

What you'll learn

#	Topic	You will be able to…
7.1	CTE basics	write `WITH` queries, chain several CTEs, know when PostgreSQL inlines them
7.2	Data-modifying CTEs	archive/move rows in one atomic (all-or-nothing) statement with `RETURNING`
7.3	Recursive CTE mechanics	explain anchor, recursive term, working table and termination
7.4	Hierarchies	list a tree top-down with depth and path, walk up to the root, total a subtree
7.5	Graphs and cycles	traverse a graph without looping forever; use `CYCLE` and `SEARCH` (PostgreSQL 14)
7.6	`generate_series`	make number/date rows, fill gaps in time series, create test data
7.7	Views	create, replace, update through, and protect views (`WITH CHECK OPTION`)
7.8	Materialized views	store results, refresh them (also `CONCURRENTLY`), index them
7.9	Choosing	pick between subquery, CTE, view, materialized view and temporary table

Setup. Examples use the `company` sample database from chapter 1, section 1.11 plus one extra table, `routes` (a small flight network, see 7.5). The files are in `code/`; `code/00_schema.sql` builds everything. Subqueries and derived tables, which CTEs replace, are explained in chapter 6.

7.1 Common table expressions (WITH)

What it is

A **common table expression** is a named, temporary result set (the rows a query returns) that you define at the start of a statement with `WITH name AS (query)` and then use in the rest of that statement as if it were a table. It disappears when the statement ends.

Analogy: when you solve a maths problem you write intermediate results on scrap paper ("let x = total per customer") and then use x in the final answer. A CTE is that scrap paper, with a name.

```
WITH cte_name [(col1, col2, ...)] AS (
    SELECT ...                -- any query
)
[, another_cte AS (SELECT ... FROM cte_name ...)]   -- later CTEs may use earlier ones
SELECT ... FROM cte_name ...;  -- the main statement
```

The problem it solves

Without CTEs you nest derived tables (subqueries in `FROM`). Nesting reads *inside-out*: the first thing that runs is buried deepest. CTEs let you write the same logic *top-down*, in the order you think about it, and give every step a meaningful name. They also let you use one intermediate result **several times** without copying it.

How it works

1. Each `name AS (query)` defines one CTE. Separate several with commas; write `WITH` only once.
2. A CTE can reference any CTE defined **before** it (and, with `RECURSIVE`, itself — see 7.3).
3. The final statement (a `SELECT`, `INSERT`, `UPDATE` or `DELETE`) uses the CTEs by name.
4. The CTE exists only while this one statement runs. Nothing is stored in the catalog (the database's internal list of its tables, views and other objects).

One CTE — employees paid above their department average:

```sql
-- code/01_cte_basics.sql
WITH dept_avg AS (
    SELECT dept_id, round(avg(salary), 0) AS avg_salary
    FROM employees
    WHERE dept_id IS NOT NULL
    GROUP BY dept_id
)
SELECT e.first_name, e.dept_id, e.salary, d.avg_salary
FROM employees e
JOIN dept_avg d ON d.dept_id = e.dept_id
WHERE e.salary > d.avg_salary
ORDER BY e.dept_id, e.salary DESC;
```

Output:

```
 first_name | dept_id |  salary   | avg_salary
------------+---------+-----------+------------
 Vikram     |       1 | 185000.00 |     130750
 Neha       |       1 | 142000.00 |     130750
 Daniel     |       2 | 150000.00 |      89500
 Lucas      |       3 | 120000.00 |      90500
 Kiran      |       4 | 135000.00 |     105500
(5 rows)
```

`dept_avg` behaves like a four-row table (one row per department that has staff). The main query joins to it like any other table.

Several CTEs as a pipeline. Each step builds on the previous one — order value → customer lifetime value → the overall average — and the final `SELECT` reads almost like English:

```sql
WITH order_totals AS (          -- step 1: value of each non-cancelled order
    SELECT o.order_id, o.customer_id,
           sum(oi.quantity * oi.unit_price * (1 - oi.discount)) AS order_value
    FROM orders o
    JOIN order_items oi USING (order_id)
    WHERE o.status <> 'cancelled'
    GROUP BY o.order_id, o.customer_id
),
customer_totals AS (            -- step 2: lifetime value per customer
    SELECT customer_id, count(*) AS orders, round(sum(order_value), 2) AS lifetime_value
    FROM order_totals
    GROUP BY customer_id
),
overall AS (                    -- step 3: one number to compare against
    SELECT round(avg(lifetime_value), 2) AS avg_ltv FROM customer_totals
)
SELECT c.first_name, c.last_name, ct.orders, ct.lifetime_value, o.avg_ltv
FROM customer_totals ct
JOIN customers c USING (customer_id)
CROSS JOIN overall o
WHERE ct.lifetime_value > o.avg_ltv
ORDER BY ct.lifetime_value DESC;
```

Output:

```
 first_name | last_name | orders | lifetime_value | avg_ltv
------------+-----------+--------+----------------+---------
 Anita      | Shah      |      3 |        3454.08 | 1057.66
 Ben        | Carter    |      2 |        1858.36 | 1057.66
(2 rows)
```

`overall` is a one-row CTE, so `CROSS JOIN` simply attaches the average to every row. Written with nested subqueries this would be three levels deep.

Using a CTE twice. Here `monthly` is read by the main query *and* by a scalar subquery:

```sql
WITH monthly AS (
    SELECT date_trunc('month', placed_at)::date AS month, count(*) AS orders
    FROM orders
    GROUP BY 1
)
SELECT month, orders,
       orders = (SELECT max(orders) FROM monthly) AS is_best_month
FROM monthly
ORDER BY month;
```

Output:

```
   month    | orders | is_best_month
------------+--------+---------------
 2025-01-01 |      3 | t
 2025-02-01 |      2 | f
 ...
 2025-04-01 |      3 | t
 ...
(6 rows)
```

Naming the columns in the header. `WITH dept_size(dept, headcount) AS (...)` renames the CTE's output columns, handy when the inner query has unnamed expressions such as `count(e.emp_id)`:

```sql
WITH dept_size(dept, headcount) AS (
    SELECT d.name, count(e.emp_id)
    FROM departments d LEFT JOIN employees e USING (dept_id)
    GROUP BY d.name
)
SELECT * FROM dept_size WHERE headcount >= 2 ORDER BY headcount DESC, dept;
```

Output:

```
    dept     | headcount
-------------+-----------
 Engineering |         4
 Sales       |         4
 Finance     |         2
 Marketing   |         2
(4 rows)
```

How PostgreSQL executes a CTE: inlined or materialized

A CTE can be run in two ways:

- **Inlined** — the optimizer (the database component that chooses how to run a query) pastes the CTE's query into the main query, exactly like a subquery, and optimizes the whole thing together (for example pushing a `WHERE` filter down into it so an index — a sorted lookup structure that finds rows without reading the whole table — can be used).
- **Materialized** — the CTE runs once, its rows are stored in a temporary work area, and the main query scans that stored copy. Filters from outside cannot reach inside.

Since **PostgreSQL 12**, a non-recursive, side-effect-free (it only reads; it changes no data) CTE that is referenced **once** is inlined automatically; one referenced two or more times is materialized. You can override the choice with `AS MATERIALIZED` or `AS NOT MATERIALIZED`. (Before version 12 every CTE was materialized — an "optimization fence" that old blog posts still warn about.)

```
EXPLAIN (COSTS OFF)
WITH big AS (SELECT * FROM employees)
SELECT first_name FROM big WHERE emp_id = 3;

EXPLAIN (COSTS OFF)
WITH big AS MATERIALIZED (SELECT * FROM employees)
SELECT first_name FROM big WHERE emp_id = 3;
```

Output:

```
                  QUERY PLAN
-----------------------------------------------
 Index Scan using employees_pkey on employees
   Index Cond: (emp_id = 3)
(2 rows)

          QUERY PLAN
------------------------------
 CTE Scan on big
   Filter: (emp_id = 3)
   CTE big
     ->  Seq Scan on employees
(4 rows)
```

The first plan never mentions the CTE: the filter reached the primary-key index. The second copies the whole table into the CTE, then filters. (`EXPLAIN` prints the plan — the steps the database chose; `Seq Scan` means reading the whole table. It is covered properly in chapter 12.)

`MATERIALIZED` is useful when the CTE is expensive and referenced several times, or when you deliberately want to compute something once (for example a call to a slow function). `NOT MATERIALIZED` is useful when a multiply-referenced CTE is cheap and each use filters it differently.

Dialect notes

Feature	PostgreSQL	MySQL	SQLite	SQL Server
`WITH` (non-recursive)	yes	8.0+	3.8.3+	2005+
`WITH RECURSIVE` keyword	required for recursion	required	required	*not used* — plain `WITH` can recurse
`MATERIALIZED` hint	12+	no	3.35+	no
CTE with `INSERT/UPDATE/DELETE` inside	yes (7.2)	no	no	no (but a CTE can be the *target* of an `UPDATE` / `DELETE`)

In SQL Server the statement *before* a `WITH` must end with a semicolon, which is why you often see `;WITH` in T-SQL (SQL Server's SQL dialect) code (SQL Server — not run).

Trade-offs

	Derived table (subquery in `FROM`)	CTE
Readability	inside-out	top-down, named steps
Reuse in the same statement	copy-paste	reference by name
Recursion	no	yes
Performance (PostgreSQL 12+)	same when inlined	same when inlined; can force materialization
Scope	the one place it's written	the whole statement

When not to use it

- When the query is simple enough to read without it — one level of subquery is fine.
- When many different statements need the same logic: that is a **view** (7.7).
- When you need the intermediate result across several statements (for example to index it or look at it twice in a session — one connection's lifetime): use a **temporary table** (a real table that only your session can see and that is dropped automatically when it ends; 7.9).

Common mistakes

- **Writing `WITH` twice** (`WITH a AS (...) WITH b AS (...)`). Use one `WITH` and commas.
- **Referencing a CTE defined later** in the list. Order matters: a CTE sees only those above it (unless it is recursive and refers to itself).
- **Assuming a CTE "caches" work.** In PostgreSQL 12+ a CTE used once is inlined, so it is re-optimized, not cached. Use `MATERIALIZED` if you truly want one evaluation.
- **A missing comma between CTEs**, or an extra one after the last CTE.

Real-world examples

Analytics teams write reports as a chain of CTEs (`raw → cleaned → joined → aggregated → final`); dbt (a popular tool for building data-transformation pipelines out of SQL files) encourages exactly this style.

In the interview

30-second answer: "A CTE is a named subquery declared with `WITH` that exists for one statement. It makes complex queries readable as a sequence of steps, lets you reference the same intermediate result several times, and — with `WITH RECURSIVE` — lets a query refer to itself to walk hierarchies. In PostgreSQL 12 and later a CTE referenced once is inlined, so it costs the same as a subquery; you can force materialization with `AS MATERIALIZED`."

- *Is a CTE faster than a subquery?* → Usually identical, because modern optimizers inline it. It can be slower if it is materialized and a selective outer filter can't be pushed into it; it can be faster if an expensive CTE is materialized once and reused.
- *CTE vs temporary table?* → A CTE lasts one statement and isn't indexed; a temp table lasts the session (or transaction), can be indexed and analyzed (have statistics gathered for the optimizer), and can be reused by many statements.
- *Can you update through a CTE?* → In PostgreSQL the CTE itself can contain `UPDATE/DELETE/INSERT ... RETURNING` (next section).

Self-check

1. *How long does a CTE exist?* → Only for the single statement it belongs to.
2. *Can CTE `b` use CTE `a` if `a` is written after `b`?* → No; a CTE can only refer to CTEs defined before it (or itself when recursive).
3. *In PostgreSQL 14, is a CTE referenced once materialized?* → No, it is inlined unless you write `AS MATERIALIZED` (or it is recursive or has side effects).

TL;DR

`WITH name AS (query)` names a step for one statement. Chain steps with commas, reference them as tables, reuse them freely. PostgreSQL 12+ inlines single-use CTEs; `MATERIALIZED` / `NOT MATERIALIZED` override.

7.2 Data-modifying CTEs (PostgreSQL)

What it is

In PostgreSQL a CTE may contain an `INSERT`, `UPDATE` or `DELETE` with a `RETURNING` clause. The rows it returns can feed the rest of the statement. This lets you do several changes as **one statement** — for example "delete these rows and copy them into an archive" — which is automatically atomic: either all of it happens or none of it.

How it works

1. `DELETE ... RETURNING *` (or `UPDATE`/`INSERT ... RETURNING`) inside a CTE produces the affected rows.
2. Later CTEs or the main statement read those rows by the CTE's name.
3. Every data-modifying CTE runs **to completion exactly once**, even if nothing reads its output.
4. All parts of the statement see the **same snapshot** (a frozen picture of the data at one moment) — the data as it was *before* the statement started. They do not see each other's changes, except through `RETURNING`.

Example — archive cancelled orders and their items:

```sql
-- code/02_data_modifying_cte.sql
CREATE TABLE orders_archive (LIKE orders INCLUDING ALL);
CREATE TABLE order_items_archive (LIKE order_items INCLUDING ALL);

WITH gone_items AS (
    DELETE FROM order_items
    WHERE order_id IN (SELECT order_id FROM orders WHERE status = 'cancelled')
    RETURNING *
),
saved_items AS (
    INSERT INTO order_items_archive SELECT * FROM gone_items RETURNING order_id
),
gone_orders AS (
    DELETE FROM orders
    WHERE status = 'cancelled'
    RETURNING *
)
INSERT INTO orders_archive
SELECT * FROM gone_orders
RETURNING order_id, status;
```

Output:

```
 order_id |  status
----------+-----------
      105 | cancelled
      111 | cancelled
(2 rows)
```

```sql
SELECT (SELECT count(*) FROM orders)              AS live_orders,
       (SELECT count(*) FROM orders_archive)      AS archived_orders,
       (SELECT count(*) FROM order_items_archive) AS archived_items;
```

Output:

```
 live_orders | archived_orders | archived_items
-------------+-----------------+----------------
          12 |               2 |              2
(1 row)
```

`CREATE TABLE ... (LIKE t INCLUDING ALL)` copies the column definitions, defaults and constraints of `t` (see chapter 9).

The snapshot rule, shown:

```sql
WITH bump AS (
    UPDATE products SET unit_price = unit_price + 1 WHERE product_id = 2
    RETURNING unit_price
)
SELECT (SELECT unit_price FROM products WHERE product_id = 2) AS seen_by_select,
       (SELECT unit_price FROM bump)                          AS new_value;
```

Output:

```
 seen_by_select | new_value
----------------+-----------
          24.99 |     25.99
(1 row)
```

The outer `SELECT` still reads the old price; only `RETURNING` shows the new one. After the statement finishes, the table holds 25.99.

Common mistakes

- **Modifying the same row twice in one statement** (two CTEs updating the same row). The result is unpredictable; don't do it.

- **Expecting a later CTE to "see" an earlier change by querying the table.** It won't — read from the earlier CTE's `RETURNING` instead.
- **Porting to MySQL/SQL Server.** They don't support this; there you use a transaction (a group of statements that are committed or undone together) with separate statements (chapter 13).

TL;DR

PostgreSQL CTEs can `INSERT/UPDATE/DELETE ... RETURNING`; the statement is atomic and every part sees the pre-statement snapshot. Great for "move rows" and "update and log" in one go.

7.3 Recursive CTEs: the mechanics

What it is

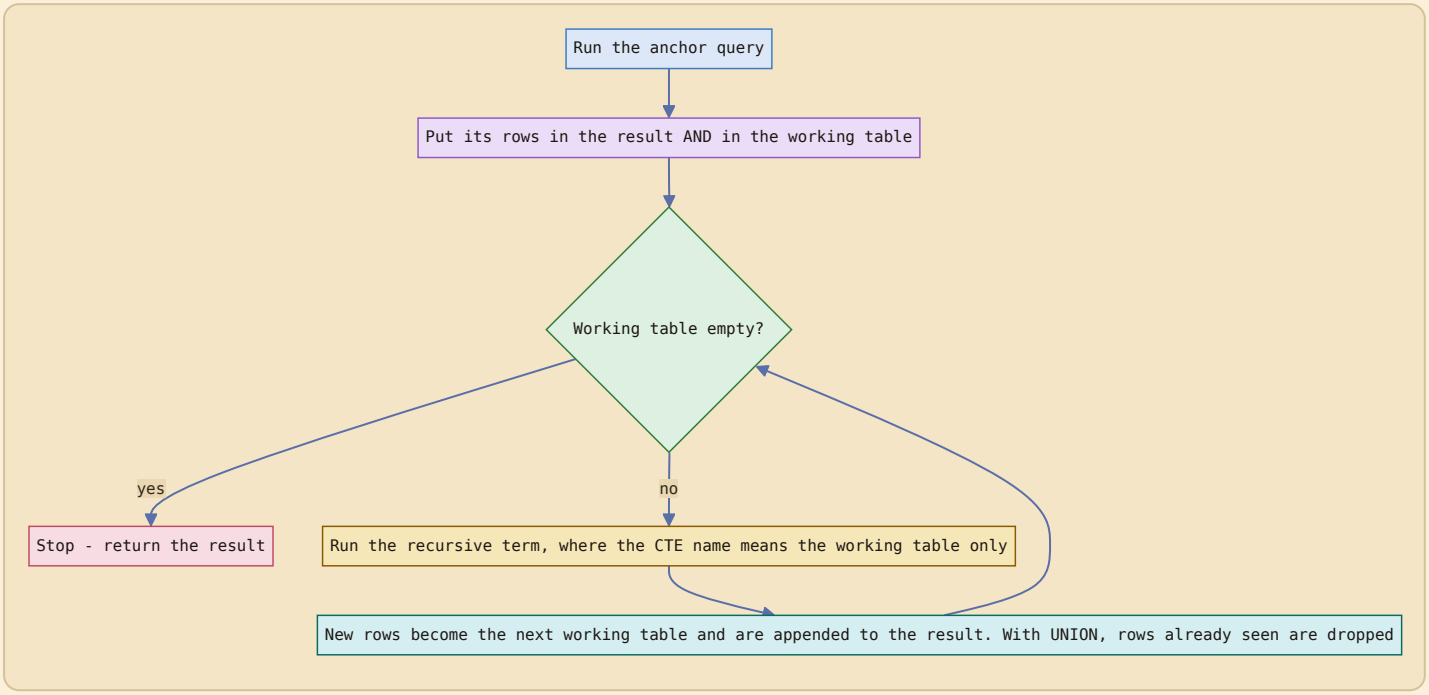
A **recursive CTE** is a CTE that refers to itself. It is written `WITH RECURSIVE name AS (anchor UNION ALL recursive_term)`. It is how standard SQL expresses *repetition*: "start with these rows, then keep deriving new rows from the last batch until no new rows appear."

Analogy: finding everyone in a family tree descended from one grandparent. You start with the grandparent (the anchor), then find their children, then the children's children, and stop when a generation has no children.

The problem it solves

Some questions have no fixed depth: "all reports under this manager, at every level", "every category inside Electronics", "every city reachable from Mumbai". A join handles exactly one level; you would need an unknown number of self-joins (chapter 5 shows the one-level self-join). A recursive CTE handles any depth.

How it works



1. **Anchor** (the non-recursive part, before `UNION [ALL]`): runs once and produces the starting rows.
2. **Recursive term** (after `UNION [ALL]`): references the CTE name. On each pass, that name means *only the rows produced by the previous pass* (the "working table"), not the whole result so far.
3. Rows produced by each pass are added to the result and become the input of the next pass.
4. **Termination:** the process stops when a pass produces no rows. Your `WHERE` or join condition must eventually make that happen.

5. `UNION ALL` keeps every row; `UNION` discards rows that are duplicates of rows already produced — which also stops loops whose state repeats.

Rules for the recursive term in PostgreSQL: it may reference the CTE only **once**, and not inside an aggregate function, a subquery, or on the nullable side of an outer join. The anchor and recursive term must produce the same number of columns with compatible types — cast in the anchor (`0::bigint`, `name::text`) if the recursive term would produce a wider type.

Counting 1 to 5:

```
-- code/03_recursive_series.sql
WITH RECURSIVE counter(n) AS (
    SELECT 1                -- anchor: the starting row(s)
    UNION ALL
    SELECT n + 1            -- recursive term: runs on the rows produced last time
    FROM counter
    WHERE n < 5             -- stop condition
)
SELECT n FROM counter;
```

Output:

```
 n
---
 1
 2
 3
 4
 5
(5 rows)
```

Trace: anchor → {1}. Pass 1 reads {1} → {2}. Pass 2 reads {2} → {3} … pass 4 reads {4} → {5}. Pass 5 reads {5}; `5 < 5` is false → no rows → stop.

Carrying state in several columns. Each row can hold more than a counter. Here each pass computes the next Fibonacci pair (each Fibonacci number is the sum of the two before it) and a running factorial ($n! = 1 \times 2 \times \ldots \times n$):

```
WITH RECURSIVE fib(i, a, b, fact) AS (
    SELECT 1, 0::bigint, 1::bigint, 1::numeric
    UNION ALL
    SELECT i + 1, b, a + b, fact * (i + 1)
    FROM fib
    WHERE i < 10
)
SELECT i, a AS fibonacci, fact AS factorial FROM fib;
```

Output:

```
 i  | fibonacci | factorial
----+-----------+-----------
  1 |         0 |         1
  2 |         1 |         2
  3 |         1 |         6
  4 |         2 |        24
 ...
 10 |        34 |   3628800
(10 rows)
```

Walking dates. The same pattern steps through time (here one week at a time):

```
WITH RECURSIVE weeks(week_start) AS (
    SELECT DATE '2025-01-06'
    UNION ALL
    SELECT (week_start + 7)::date FROM weeks WHERE week_start < DATE '2025-02-03'
)
SELECT week_start, to_char(week_start, 'Dy') AS weekday FROM weeks;
```

Output:

```
 week_start | weekday
------------+---------
 2025-01-06 | Mon
 2025-01-13 | Mon
 2025-01-20 | Mon
 2025-01-27 | Mon
 2025-02-03 | Mon
(5 rows)
```

In PostgreSQL `generate_series` (7.6) is simpler for this; the recursive form is what you use in MySQL, which has no `generate_series`.

A runaway query and two safety nets. A recursive CTE without a stop condition never ends. While experimenting, a `LIMIT` on the outer query stops it (PostgreSQL evaluates the recursion lazily, only as far as the outer query asks):

```
WITH RECURSIVE forever(n) AS (
    SELECT 1 UNION ALL SELECT n + 1 FROM forever      -- no stop condition!
)
SELECT n FROM forever LIMIT 3;
```

Output:

```
 n
---
 1
 2
 3
(3 rows)
```

And `UNION` instead of `UNION ALL` stops a computation whose state starts repeating. Doubling modulo 7 (keeping only the remainder after dividing by 7) goes 1 → 2 → 4 → 1 …; the second 1 is a duplicate, so it is discarded and the working table becomes empty:

```
WITH RECURSIVE mod_cycle(n) AS (
    SELECT 1
    UNION
    SELECT (n * 2) % 7 FROM mod_cycle
)
SELECT n FROM mod_cycle;
```

Output:

```
 n
---
 1
 2
 4
(3 rows)
```

`UNION` only helps when the *whole row* repeats. If a row carries a growing column (a depth, a path, a running total) it never repeats exactly — you need explicit cycle detection (7.5).

Dialect notes

	PostgreSQL	MySQL 8	SQLite	SQL Server
Syntax	`WITH RECURSIVE`	`WITH RECURSIVE`	`WITH RECURSIVE`	`WITH` (no keyword)
Built-in depth limit	none	`cte_max_recursion_depth` = 1000 by default	none	100 by default; `OPTION (MAXRECURSION n)`, 0 = unlimited
`UNION` (distinct) in recursion	yes	yes	yes	no, `UNION ALL` only
`SEARCH` / `CYCLE` clauses	14+	no	no	no

Oracle also supports recursive `WITH` (since 11g R2) and its older proprietary `CONNECT BY` syntax.

Common mistakes

- **No stop condition** → the query runs until it runs out of memory or disk (or, in MySQL/SQL Server, hits the depth limit with an error).
- **Type mismatch between anchor and recursive term.** The anchor fixes each column's type. A `varchar(40)` column in the anchor followed by `path || ' > ' || name` (which yields `text`) is rejected with *"column has type character varying(40) in non-recursive term but type text overall"*; and `SELECT 1` makes a column `integer`, which a growing factorial soon overflows (exceeds the largest value the type can hold). Cast in the anchor (`first_name::text`, `1::numeric`).
- **Thinking the recursive term sees the whole result**: it sees only the previous pass's rows.
- **Relying on output order.** Without `ORDER BY` the order of a recursive result is not guaranteed. Build a sort key (depth, path, or `SEARCH`, 7.5) and order by it.

Self-check

1. *What are the two parts of a recursive CTE?* → The anchor (runs once) and the recursive term (runs repeatedly on the previous pass's rows), joined by `UNION ALL` or `UNION`.
2. *When does recursion stop?* → When a pass produces no new rows.
3. *Why cast `0::bigint` in the anchor?* → The anchor fixes the column types; without the cast the column is `integer` and larger values in later passes would overflow.
4. *What does SQL Server need instead of `WITH RECURSIVE`?* → Just `WITH`; and it limits recursion to 100 levels unless `OPTION (MAXRECURSION n)`.

TL;DR

Anchor rows → repeatedly apply the recursive term to the last batch → stop when a batch is empty. Always have a stop condition, cast types in the anchor, and `ORDER BY` explicitly.

7.4 Hierarchies: trees stored as parent pointers

What it is

The most common way to store a tree in a relational table is the **adjacency list**: each row stores the key of its parent (`employees.manager_id` points to another `employees.emp_id`; `NULL` marks the root). Recursive CTEs are the standard way to query such trees.

How it works — three classic queries

Question	Anchor	Recursive step
Top-down: the whole tree (or a subtree)	the root(s): `manager_id IS NULL` (or `emp_id = X`)	children: `e.manager_id = parent.emp_id`
Bottom-up: the chain of ancestors	the starting node	parent: `m.emp_id = child.manager_id`
Subtree aggregates	every (boss, direct report) pair	extend each pair one level down, keeping the boss

Top-down with depth and path. The path column doubles as a sort key that keeps each manager's team directly under them:

```sql
-- code/04_hierarchy.sql
WITH RECURSIVE org AS (
    SELECT emp_id, first_name, manager_id, 1 AS depth,
           first_name::text AS path
    FROM employees
    WHERE manager_id IS NULL                      -- anchor: the root
    UNION ALL
    SELECT e.emp_id, e.first_name, e.manager_id, o.depth + 1,
           o.path || ' > ' || e.first_name
    FROM employees e
    JOIN org o ON e.manager_id = o.emp_id         -- children of rows found so far
)
SELECT repeat('    ', depth - 1) || first_name AS org_chart, depth, path
FROM org
ORDER BY path;
```

Output:

```
   org_chart    | depth |         path
----------------+-------+-----------------------
 Asha           |     1 | Asha
     Daniel     |     2 | Asha > Daniel
         Arjun  |     3 | Asha > Daniel > Arjun
         Priya  |     3 | Asha > Daniel > Priya
         Sara   |     3 | Asha > Daniel > Sara
     Kiran      |     2 | Asha > Kiran
         Omar   |     3 | Asha > Kiran > Omar
 ...
     Vikram     |     2 | Asha > Vikram
         Li     |     3 | Asha > Vikram > Li
 ...
         Rohan  |     3 | Asha > Vikram > Rohan
(15 rows)
```

Pass 1 found Asha's four direct reports (depth 2), pass 2 found their reports (depth 3), pass 3 found nobody, so it stopped. Sorting by a text path works here because names don't contain `>`; for a robust sort key use an array of ids (`ARRAY[emp_id]`, then `path || e.emp_id`) — arrays compare element by element. To list only one subtree, change the anchor to `WHERE emp_id = 2`.

Bottom-up: who is above the intern?

```sql
WITH RECURSIVE chain AS (
    SELECT emp_id, first_name, job_title, manager_id, 0 AS steps_up
    FROM employees WHERE emp_id = 15
    UNION ALL
    SELECT m.emp_id, m.first_name, m.job_title, m.manager_id, c.steps_up + 1
    FROM employees m
    JOIN chain c ON m.emp_id = c.manager_id
)
SELECT steps_up, first_name, job_title FROM chain ORDER BY steps_up;
```

Output:

```
 steps_up | first_name |      job_title
----------+------------+--------------------
        0 | Rohan      | Intern
        1 | Vikram     | Engineering Manager
        2 | Asha       | CEO
(3 rows)
```

The only difference from top-down is the direction of the join: `m.emp_id = c.manager_id` ("find my parent") instead of `e.manager_id = o.emp_id` ("find my children"). Real uses: breadcrumbs (`Home > Electronics > Laptops`), approval chains, permission inheritance.

Subtree totals: team size and payroll under each manager. The trick is to carry the *boss* unchanged while walking down, producing one row per (boss, anyone below them):

```
WITH RECURSIVE reports AS (
    SELECT manager_id AS boss, emp_id, salary        -- direct reports
    FROM employees WHERE manager_id IS NOT NULL
    UNION ALL
    SELECT r.boss, e.emp_id, e.salary                -- their reports, and so on
    FROM reports r
    JOIN employees e ON e.manager_id = r.emp_id
)
SELECT b.first_name AS manager, count(*) AS team_size, sum(r.salary) AS team_salary
FROM reports r JOIN employees b ON b.emp_id = r.boss
GROUP BY b.first_name
ORDER BY team_size DESC, manager;
```

Output:

```
 manager | team_size | team_salary
---------+-----------+-------------
 Asha    |        14 |  1367000.00
 Vikram  |         5 |   432000.00
 Daniel  |         3 |   208000.00
 Kiran   |         1 |    76000.00
 Lucas   |         1 |    61000.00
(5 rows)
```

Asha has all 14 other employees below her; Vikram has Neha, Rahul, Meera, Li and Rohan. The aggregate runs in the *outer* query — aggregates aren't allowed inside the recursive term.

Other ways to store trees (for comparison)

Model	How	Reads	Writes	Notes
Adjacency list (this chapter)	`parent_id` column	recursive CTE	trivial (change one pointer)	the default choice
Materialized path	store `'1/2/15'` (or PostgreSQL `ltree`, an extension type for dotted label paths)	`LIKE '1/2/%'`, indexable	moving a subtree rewrites all descendants' paths	great for breadcrumbs
Nested sets	store `lft`/`rgt` numbers	subtree = range query	inserts renumber many rows	read-heavy, rarely-changing trees
Closure table	separate table of every (ancestor, descendant, depth) pair	simple joins	many rows per insert	*SQL Antipatterns* recommends it for heavy tree querying

Common mistakes

- **Anchoring on the wrong side**: using `manager_id = X` in the anchor lists X's reports but *not X*; use `emp_id = X` if X should be included.

- **Bad data creating cycles** (A manages B, B manages A). A top-down walk from a real root never reaches such a loop, but a walk that starts inside it runs forever. Protect with a path check (7.5) or a `CHECK`/trigger (a function the database runs automatically when rows change) that prevents cycles.
- **Sorting by depth only**, which lists all level-2 rows, then all level-3 rows, losing the "team under manager" layout. Sort by the path.

In the interview

> **30-second answer:** "Store the tree as an adjacency list — each row points to its parent. To read a subtree, anchor on the root and recursively join children whose parent is in the previous level, carrying depth and a path for ordering. For ancestors, reverse the join. For totals per node, carry the top node through the recursion and aggregate outside."

- *How would you stop infinite loops if the data could contain a cycle?* → Carry an array of visited ids and add `WHERE NOT e.emp_id = ANY(path)`, or use PostgreSQL 14's `CYCLE` clause.
- *What if the tree is huge and queried constantly?* → Index `manager_id`; consider a closure table or materialized path (`ltree` with a GiST index — a PostgreSQL index type for data that isn't simply sortable, such as paths or shapes) for fast subtree reads.

Self-check

1. *Which join condition walks down, which walks up?* → Down: `child.manager_id = found.emp_id`; up: `parent.emp_id = found.manager_id`.
2. *Why carry `path`?* → To show the route from the root and to sort each subtree under its parent.
3. *Why is the `count(*)` outside the recursive CTE in the team-size query?* → Aggregates are not allowed in the recursive term; you generate all (boss, report) pairs first, then group.

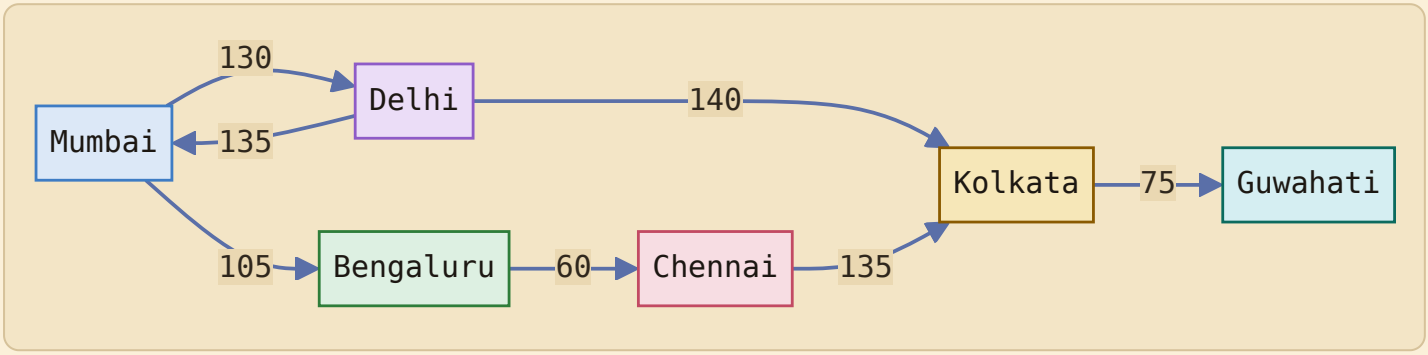
TL;DR

Adjacency list + recursive CTE = any-depth tree queries. Down: join children; up: join parent; totals: carry the ancestor, aggregate outside. Carry depth/path for display and ordering.

7.5 Graphs and cycles

What it is

A **graph** is like a tree but nodes can have several parents and paths can loop back (a **cycle**). Flight routes, road networks, "follows" relationships and package dependencies are graphs. Our sample `routes` table stores one-way flights; Mumbai → Delhi → Mumbai is a cycle.



The problem it solves

A naive recursive walk over a graph with a cycle never ends: Mumbai → Delhi → Mumbai → Delhi … and each row differs (the total time grows), so `UNION` does not stop it. You must remember where you have been.

How it works (manual cycle detection)

1. Carry an **array of visited nodes** (`path`) in every row.

2. In the recursive term, only follow an edge if its destination is not already in the path: `WHERE r.to_city <> ALL (t.path)`.
3. Every row therefore represents a *simple path* (no repeated city); the number of simple paths is finite, so recursion ends.

```sql
-- code/05_graph_cycles.sql
WITH RECURSIVE trips AS (
    SELECT to_city AS city, ARRAY['Mumbai', to_city] AS path, minutes AS total_min
    FROM routes WHERE from_city = 'Mumbai'
    UNION ALL
    SELECT r.to_city, t.path || r.to_city, t.total_min + r.minutes
    FROM trips t
    JOIN routes r ON r.from_city = t.city
    WHERE r.to_city <> ALL (t.path)              -- never revisit a city
)
SELECT city, array_to_string(path, ' -> ') AS route, total_min
FROM trips
ORDER BY city, total_min;
```

Output:

```
   city    |                        route                         | total_min
-----------+------------------------------------------------------+-----------
 Bengaluru | Mumbai -> Bengaluru                                  |       105
 Chennai   | Mumbai -> Bengaluru -> Chennai                       |       165
 Delhi     | Mumbai -> Delhi                                      |       130
 Guwahati  | Mumbai -> Delhi -> Kolkata -> Guwahati               |       345
 Guwahati  | Mumbai -> Bengaluru -> Chennai -> Kolkata -> Guwahati |       375
 Kolkata   | Mumbai -> Delhi -> Kolkata                           |       270
 Kolkata   | Mumbai -> Bengaluru -> Chennai -> Kolkata            |       300
(7 rows)
```

`ARRAY[...]` builds a PostgreSQL array (chapter 14), `||` appends to it, and `x <> ALL (array)` is true when `x` differs from every element (chapter 6 covers `ALL`). Delhi → Mumbai was never followed because Mumbai is already in every path.

Shortest (fastest) route to each city — keep the best row per city with `DISTINCT ON` (chapter 2):

```sql
-- same trips CTE as above, then:
SELECT DISTINCT ON (city) city, total_min, array_to_string(path, ' -> ') AS route
FROM trips
WHERE city <> 'Mumbai'
ORDER BY city, total_min;
```

Output:

```
   city    | total_min |                route
-----------+-----------+---------------------------------------
 Bengaluru |       105 | Mumbai -> Bengaluru
 Chennai   |       165 | Mumbai -> Bengaluru -> Chennai
 Delhi     |       130 | Mumbai -> Delhi
 Guwahati  |       345 | Mumbai -> Delhi -> Kolkata -> Guwahati
 Kolkata   |       270 | Mumbai -> Delhi -> Kolkata
(5 rows)
```

This enumerates *every* simple path, which explodes on big, dense graphs. It is fine for small graphs and interview questions; real route planners use graph algorithms (Dijkstra, A* — classic shortest-path methods that never enumerate every path) in application code or extensions such as pgRouting, or graph databases (databases that store nodes and edges directly) (chapter 16).

The CYCLE and SEARCH clauses (PostgreSQL 14+)

SQL:1999 defines two helper clauses that PostgreSQL implemented in version 14 (the version this book runs):

- `CYCLE col SET is_cycle USING path_col` — PostgreSQL tracks the visited values of `col` for you, marks a row whose `col` value was already seen on its path with `is_cycle = true`, and does not recurse further from it.
- `SEARCH DEPTH FIRST BY col SET ord` or `SEARCH BREADTH FIRST BY col SET ord` — adds a column `ord` that you `ORDER BY` to get depth-first (follow one branch all the way down before the next) or breadth-first (finish each level before going deeper) order.

```
WITH RECURSIVE trips(city, total_min) AS (
    SELECT to_city, minutes FROM routes WHERE from_city = 'Mumbai'
    UNION ALL
    SELECT r.to_city, t.total_min + r.minutes
    FROM trips t JOIN routes r ON r.from_city = t.city
) CYCLE city SET is_cycle USING visited
SELECT city, total_min, is_cycle FROM trips
WHERE city IN ('Mumbai', 'Delhi')
ORDER BY total_min;
```

Output:

```
  city  | total_min | is_cycle
--------+-----------+----------
 Delhi  |       130 | f
 Mumbai |       265 | f
 Delhi  |       395 | t
(3 rows)
```

Why is Mumbai (265) *not* a cycle? The anchor starts from the routes' *destinations*, so "Mumbai" was not yet in the tracked path (Delhi → Mumbai). The next hop back to Delhi repeats Delhi, so that row is flagged and recursion stops there. To treat the start city as visited, begin the anchor with the start city itself (as the manual version did with `ARRAY['Mumbai', to_city]`).

```
WITH RECURSIVE org(emp_id, first_name, manager_id) AS (
    SELECT emp_id, first_name, manager_id FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.emp_id, e.first_name, e.manager_id
    FROM employees e JOIN org o ON e.manager_id = o.emp_id
) SEARCH BREADTH FIRST BY emp_id SET ord
SELECT first_name, ord FROM org ORDER BY ord LIMIT 6;
```

Output:

```
 first_name |  ord
------------+--------
 Asha       | (0,1)
 Vikram     | (1,2)
 Daniel     | (1,6)
 Lucas      | (1,10)
 Kiran      | (1,12)
 Neha       | (2,3)
(6 rows)
```

For breadth-first, `ord` is (level, id): all of level 1 comes before level 2. `DEPTH FIRST` would instead produce an array path, like our manual `path` column.

Self-check

1. *Why doesn't `UNION` stop the Mumbai–Delhi loop?* → Each lap has a different `total_min`/path, so rows never repeat exactly.
2. *What does `r.to_city <> ALL (t.path)` do?* → Follows an edge only if its destination hasn't been visited on this path.
3. *Which PostgreSQL version added `CYCLE` and `SEARCH`?* → 14.

TL;DR

Graphs can loop, so carry a visited-path array and refuse to revisit, or use PostgreSQL 14's `CYCLE` clause. Path enumeration is exponential (the work can double with every extra hop) — fine for small graphs, not a route planner.

7.6 generate_series: making rows (PostgreSQL)

What it is

`generate_series(start, stop [, step])` is a PostgreSQL **set-returning function**: used in `FROM`, it returns one row per value from `start` to `stop`. It works for integers, numerics, and timestamps/dates (step given as an `interval`).

```sql
-- code/06_generate_series.sql
SELECT * FROM generate_series(1, 5) AS n;
SELECT * FROM generate_series(10, 0, -5) AS n;
SELECT d::date AS day FROM generate_series(DATE '2025-03-01', DATE '2025-03-04', INTERVAL '1 day') AS d;
```

Output (the three results, one after another):

```
 n
---
 1
 2
 ...
 5
(5 rows)

 n
----
 10
  5
  0
(3 rows)

    day
------------
 2025-03-01
 2025-03-02
 2025-03-03
 2025-03-04
(4 rows)
```

With `date` arguments PostgreSQL picks the `timestamp with time zone` version of the function, so cast the result back to `date` for display.

The problem it solves: missing rows in reports

`GROUP BY` can only produce groups for values that **exist** in the data. A month with no orders simply doesn't appear, and a chart drawn from the result silently skips it. The fix: generate the full calendar, then `LEFT JOIN` the data to it and count.

```sql
WITH months AS (
    SELECT m::date AS month
    FROM generate_series(DATE '2025-01-01', DATE '2025-07-01', INTERVAL '1 month') AS m
)
SELECT m.month, count(o.order_id) AS orders
FROM months m
LEFT JOIN orders o ON date_trunc('month', o.placed_at) = m.month
GROUP BY m.month
ORDER BY m.month;
```

Output:

```
   month    | orders
------------+--------
 2025-01-01 |      3
 2025-02-01 |      2
 ...
 2025-06-01 |      2
 2025-07-01 |      0
(7 rows)
```

July shows `0`. Note `count(o.order_id)`, not `count(*)`: for July the `LEFT JOIN` produces one row with `NULL` order columns, and `count(*)` would count it as 1 (chapter 4).

Test data in one statement:

```
CREATE TABLE fake_events AS
SELECT g AS id, DATE '2025-01-01' + (g % 90) AS day, (g * 37) % 100 AS score
FROM generate_series(1, 1000) AS g;
SELECT count(*), min(day), max(day), round(avg(score), 1) AS avg_score FROM fake_events;
```

Output:

```
 count |    min     |    max     | avg_score
-------+------------+------------+-----------
  1000 | 2025-01-01 | 2025-03-31 |      49.5
(1 row)
```

The arithmetic keeps the data deterministic (identical on every run; no `random()`), which is handy for reproducible performance tests (chapter 12 uses this idea with millions of rows).

Dialect notes

	Numbers / dates series
PostgreSQL	`generate_series(...)`
SQL Server 2022+	`GENERATE_SERIES(start, stop [, step])` — numbers only (SQL Server — not run)
SQLite	`generate_series` table-valued function from the `series` extension (built into the `sqlite3` shell)
MySQL 8	none — use a recursive CTE (7.3) or a permanent "numbers"/"calendar" table

Many data warehouses (databases built for analysing large amounts of historical data) keep a permanent **calendar table** (one row per date with flags like `is_weekend`, `fiscal_quarter`) for the same purpose.

TL;DR

`generate_series` produces rows of numbers or timestamps. Generate the full range, `LEFT JOIN` the facts, `count(col)` — and empty days/months appear as 0.

7.7 Views

What it is

A **view** is a named query stored in the database. You query it like a table (`SELECT * FROM staff_directory`), but it stores **no data**: every time you use it, the database substitutes the view's query into yours and runs it against the current contents of the underlying (**base**) tables.

Analogy: a view is a saved search in your email client. It doesn't copy emails into a new folder; it re-runs the search each time you open it, so it's always up to date.

Why it matters

Benefit	Example
Simplicity — hide a complex join once	analysts query `staff_directory` instead of re-writing the join
Security — expose only some columns/rows	grant access to a view without `salary`, not to `employees`
Stable interface — change tables, keep the view	rename/split a table and redefine the view so old queries still work
Consistency — one definition of a metric	"active customer" is defined once, not in 20 reports

How it works

1. `CREATE VIEW name AS query;` stores the query text (parsed) in the catalog.
2. When a query mentions the view, PostgreSQL's rewriter (a stage that runs before planning and expands views) replaces the name with the stored query — just like an inlined CTE — and the optimizer plans the combined query.
3. Because nothing is stored, a view is always current and costs exactly as much as its query.

```
-- code/07_views.sql
CREATE VIEW staff_directory AS
SELECT e.emp_id, e.first_name || ' ' || e.last_name AS full_name,
       e.job_title, d.name AS department, e.email
FROM employees e
LEFT JOIN departments d USING (dept_id);

SELECT full_name, job_title, department FROM staff_directory
WHERE department = 'Sales' ORDER BY full_name;
```

Output:

```
   full_name   |   job_title   | department
---------------+---------------+------------
 Arjun Singh   | Sales Rep     | Sales
 Daniel Brooks | Sales Manager | Sales
 Priya Nair    | Sales Rep     | Sales
 Sara Khan     | Sales Rep     | Sales
(4 rows)
```

Always current — change a base table and the view reflects it immediately:

```
UPDATE employees SET job_title = 'Senior Sales Rep' WHERE emp_id = 7;
SELECT full_name, job_title FROM staff_directory WHERE emp_id = 7;
```

Output:

```
 full_name  |    job_title
------------+-----------------
 Priya Nair | Senior Sales Rep
(1 row)
```

Changing and dropping views

- `CREATE OR REPLACE VIEW` swaps the definition in place (keeping permissions), but in PostgreSQL the new query must return the **same columns, same names, same types, in the same order**; you may only **add** columns at the end.
- To remove or reorder columns: `DROP VIEW` and `CREATE VIEW` again (re-granting privileges — the permissions given with `GRANT`).
- Objects that depend on each other are tracked: you can't drop a table (or a view) that a view uses unless you add `CASCADE`, which drops the dependent views too.

```sql
-- adding hire_date at the end: allowed
CREATE OR REPLACE VIEW staff_directory AS
SELECT e.emp_id, e.first_name || ' ' || e.last_name AS full_name,
       e.job_title, d.name AS department, e.email, e.hire_date
FROM employees e
LEFT JOIN departments d USING (dept_id);

-- removing columns: refused
CREATE OR REPLACE VIEW staff_directory AS
SELECT e.emp_id, e.job_title FROM employees e;

-- dropping a table the view depends on: refused
DROP TABLE departments;
```

Output:

```
ERROR:  cannot drop columns from view
ERROR:  cannot drop table departments because other objects depend on it
DETAIL:  constraint employees_dept_id_fkey on table employees depends on table departments
constraint projects_dept_id_fkey on table projects depends on table departments
view staff_directory depends on table departments
HINT:  Use DROP ... CASCADE to drop the dependent objects too.
```

(Two foreign keys also depend on `departments`; `CASCADE` would drop those constraints and the view — rarely what you want.)

Updatable views

You can `INSERT`, `UPDATE` and `DELETE` through a view if PostgreSQL can map each view row to exactly one base-table row. Since PostgreSQL 9.3 a view is **automatically updatable** when its query:

- reads from exactly **one** table or updatable view in `FROM`;
- has no `GROUP BY`, `HAVING`, `DISTINCT`, `LIMIT`/`OFFSET`, window functions, aggregates, `UNION`/`INTERSECT`/`EXCEPT`, or `WITH` at top level.

Computed columns in such a view are simply read-only; plain columns can be written. For anything more complex you write an `INSTEAD OF` trigger (a trigger is a function the database runs automatically on a change; `INSTEAD OF` runs it in place of the write; chapter 14).

```sql
CREATE VIEW furniture AS
SELECT product_id, name, unit_price, category
FROM products
WHERE category = 'Furniture';

UPDATE furniture SET unit_price = 329.99 WHERE name = 'Ergonomic Chair';
INSERT INTO furniture VALUES (10, 'Bookshelf', 120.00, 'Furniture');
-- Nothing stops a row that falls OUTSIDE the view's WHERE...
INSERT INTO furniture VALUES (11, 'Webcam', 59.00, 'Electronics');
SELECT product_id, name, category FROM products WHERE product_id >= 10;
SELECT count(*) AS visible_in_view FROM furniture WHERE product_id >= 10;
```

Output:

```
 product_id |   name    |  category
------------+-----------+-------------
         10 | Bookshelf | Furniture
         11 | Webcam    | Electronics
(2 rows)

 visible_in_view
-----------------
               1
(1 row)
```

The Webcam went into `products` through the `furniture` view but is invisible in that view — a confusing "disappearing row". The `discontinued` column, not in the view, got its default (`false`).

`WITH CHECK OPTION` makes the view reject inserts and updates whose resulting row would not satisfy the view's `WHERE`:

```
CREATE OR REPLACE VIEW furniture AS
SELECT product_id, name, unit_price, category
FROM products
WHERE category = 'Furniture'
WITH CHECK OPTION;

INSERT INTO furniture VALUES (12, 'Monitor Arm', 79.00, 'Accessories');
```

Output:

```
ERROR:  new row violates check option for view "furniture"
DETAIL:  Failing row contains (12, Monitor Arm, Accessories, 79.00, f).
```

(`LOCAL` checks only this view's condition; `CASCADED`, the default, also checks the conditions of views it is built on.)

A view with a join or `GROUP BY` is read-only:

```
CREATE VIEW dept_payroll AS
SELECT d.name AS department, count(e.emp_id) AS headcount, sum(e.salary) AS payroll
FROM departments d LEFT JOIN employees e USING (dept_id)
GROUP BY d.name;
SELECT * FROM dept_payroll ORDER BY payroll DESC NULLS LAST;
UPDATE dept_payroll SET headcount = 0 WHERE department = 'Sales';
```

Output:

```
 department  | headcount |  payroll
-------------+-----------+-----------
 Engineering |         4 | 523000.00
 Sales       |         4 | 358000.00
 ...
 Research    |         0 |    (null)
(6 rows)

ERROR:  cannot update view "dept_payroll"
DETAIL:  Views containing GROUP BY are not automatically updatable.
HINT:  To enable updating the view, provide an INSTEAD OF UPDATE trigger or an unconditional ON UPDATE DO INSTEAD rule.
```

The catalog knows which views are updatable:

```
SELECT table_name, is_updatable
FROM information_schema.views
WHERE table_schema = 'public'
ORDER BY table_name;
```

Output:

```
   table_name    | is_updatable
-----------------+--------------
 dept_payroll    | NO
 furniture       | YES
 staff_directory | NO
(3 rows)
```

Views and security

In PostgreSQL a view accesses its base tables with the privileges of the **view's owner**, so you can `GRANT SELECT ON staff_directory TO analyst` without granting anything on `employees`; the analyst sees names and titles but never salaries. Two refinements:

- `WITH (security_barrier)` stops a cleverly written user function in the caller's `WHERE` from peeking at rows the view's own `WHERE` is meant to hide (the optimizer otherwise may evaluate the cheap user function first).
- PostgreSQL 15 added `WITH (security_invoker = true)` to make a view use the *caller's* privileges instead (PostgreSQL 15 — not run here on 14).

Roles, `GRANT` and row-level security (rules that limit which rows each user may see) are in chapter 14.

Dialect notes

	PostgreSQL	MySQL	SQLite	SQL Server
`CREATE OR REPLACE VIEW`	yes (add columns only)	yes (any change)	no — drop & create	`CREATE OR ALTER VIEW` (2016 SP1+)
Auto-updatable simple views	yes	yes	no — read-only, use `INSTEAD OF` triggers	yes
`WITH CHECK OPTION`	yes	yes	no	yes
Materialized views	yes (7.8)	no	no	"indexed views" (kept up to date automatically)

When not to use it

- **Stacking views on views** many levels deep: each looks simple, the combined query is huge and hard to tune. Keep layers shallow.
- **As a performance tool**: a plain view is exactly as fast as its query. If the query is slow, the view is slow — consider a materialized view or better indexes.
- **`SELECT *` in a view definition**: PostgreSQL expands `*` when the view is *created*, so columns added to the table later do **not** appear in the view. List columns explicitly.

Common mistakes

- Expecting a view to store data (that's a materialized view).
- Being surprised by "disappearing rows" when inserting through a filtered view — add `WITH CHECK OPTION`.
- Trying to reorder or drop columns with `CREATE OR REPLACE` in PostgreSQL.
- Adding `ORDER BY` inside a view and relying on it: order is only guaranteed by the **outer** query's `ORDER BY`.

Real-world examples

- Reporting layers: BI (business-intelligence) tools (Metabase, Tableau, Power BI) are pointed at views rather than raw tables.
- Backward compatibility: after splitting `users` into `accounts` and `profiles`, a `users` view joins them so old code keeps working.
- PostgreSQL's own `information_schema` is a set of views over the system catalogs.

In the interview

30-second answer: "A view is a stored query that acts like a virtual table. It stores no data, so it's always current and costs the same as running its query. Views simplify complex joins, provide a stable interface when tables change, and enforce security by exposing only certain columns or rows. Simple single-table views are updatable; `WITH CHECK OPTION` stops writes that would fall outside the view."

- *Does a view improve performance?* → Not by itself; it is expanded into the query. A materialized view (or indexed view in SQL Server) can.
- *Can you insert into a view?* → Yes if it's a simple single-table view (auto-updatable) or has `INSTEAD OF` triggers.

- *What happens to a view if you add a column to its table?* → Nothing; the view keeps the column list captured when it was created.

Self-check

1. *Where is a view's data stored?* → Nowhere; only its query is stored.
2. *Why did the Webcam row vanish from the `furniture` view?* → It was inserted through the view but doesn't satisfy `category = 'Furniture'`.
3. *What prevents that?* → `WITH CHECK OPTION`.
4. *Can `CREATE OR REPLACE VIEW` remove a column in PostgreSQL?* → No; drop and recreate.
5. *Why is `staff_directory` not updatable?* → It joins two tables.

TL;DR

A view is a named, stored query — virtual table, no data, always current. Use for simplicity, security and stable interfaces. Simple single-table views are writable; add `WITH CHECK OPTION`. Avoid deep view stacks and `SELECT *` definitions.

7.8 Materialized views

What it is

A **materialized view** stores the *result* of its query on disk, like a table, and only recomputes it when you run `REFRESH MATERIALIZED VIEW`. It trades **freshness** for **speed**: reads are as fast as reading a small table, but the data is only as recent as the last refresh.

Analogy: a printed report on your desk. Reading it is instant, but it doesn't change when the numbers in the system change — you have to print a new one.

How it works

1. `CREATE MATERIALIZED VIEW name AS query;` runs the query once and stores the rows (add `WITH NO DATA` to create it empty; it can't be queried until refreshed).
2. Queries read the stored rows. They can use indexes you create on the materialized view.
3. `REFRESH MATERIALIZED VIEW name;` re-runs the query and replaces the contents. This takes a strong lock (a hold on the object that makes other sessions wait): **reads are blocked** during the refresh.
4. `REFRESH MATERIALIZED VIEW CONCURRENTLY name;` builds the new result separately and applies only the differences, so readers are not blocked. It requires a **unique index** on the materialized view (plain columns, no `WHERE`) and is slower overall.

```
-- code/08_materialized_views.sql
CREATE MATERIALIZED VIEW product_sales AS
SELECT p.product_id, p.name,
       coalesce(sum(oi.quantity), 0) AS units_sold,
       coalesce(round(sum(oi.quantity * oi.unit_price * (1 - oi.discount)), 2), 0) AS revenue
FROM products p
LEFT JOIN order_items oi USING (product_id)          -- every order line
GROUP BY p.product_id, p.name;

SELECT name, units_sold, revenue FROM product_sales ORDER BY revenue DESC LIMIT 3;
```

Output:

```
      name       | units_sold | revenue
-----------------+------------+---------
 Laptop Pro 14   |          5 | 6380.05
 Standing Desk   |          2 |  978.00
 Ergonomic Chair |          3 |  962.47
(3 rows)
```

Stale until refreshed:

```
INSERT INTO orders VALUES (115, 9, NULL, '2025-06-20 09:00', 'pending', NULL);
INSERT INTO order_items VALUES (115, 5, 3, 499.00, 0.00);

SELECT name, units_sold, revenue FROM product_sales WHERE name = 'Standing Desk';
REFRESH MATERIALIZED VIEW product_sales;
SELECT name, units_sold, revenue FROM product_sales WHERE name = 'Standing Desk';
```

Output:

```
     name      | units_sold | revenue
---------------+------------+---------
 Standing Desk |          2 |  978.00
(1 row)

     name      | units_sold | revenue
---------------+------------+---------
 Standing Desk |          5 | 2475.00
(1 row)
```

Concurrent refresh needs a unique index:

```
REFRESH MATERIALIZED VIEW CONCURRENTLY product_sales;          -- fails
CREATE UNIQUE INDEX product_sales_pk ON product_sales (product_id);
REFRESH MATERIALIZED VIEW CONCURRENTLY product_sales;          -- works
CREATE INDEX product_sales_revenue ON product_sales (revenue DESC);
SELECT matviewname, ispopulated FROM pg_matviews;
```

Output:

```
ERROR:  cannot refresh materialized view "public.product_sales" concurrently
HINT:  Create a unique index with no WHERE clause on one or more columns of the materialized view.
  matviewname  | ispopulated
---------------+-------------
 product_sales | t
(1 row)
```

Trade-offs

	View	Materialized view	Summary table maintained by triggers/jobs
Stores data	no	yes	yes
Freshness	always current	as of last refresh	as current as your code keeps it
Read speed	cost of the query	fast (indexable)	fast
Write overhead	none	none at write time; refresh cost later	on every write
Effort	trivial	trivial + schedule refreshes	most code, most bugs

PostgreSQL has **no automatic or incremental refresh**: each refresh recomputes the whole query. You schedule it (cron, the classic Unix scheduler that runs commands on a timetable; the `pg_cron` extension; your job scheduler) or trigger it after batch loads (bulk imports of new data). Oracle supports incremental "fast refresh"; SQL Server's indexed views are updated automatically on every write (with many restrictions); MySQL has no materialized views.

When to use / not use

- **Use** for dashboards and reports over large data that can be minutes or hours old, expensive aggregations read far more often than the data changes, and precomputed search/lookup data.
- **Don't use** when readers need up-to-the-second numbers, or when the base data changes constantly and refreshing costs more than it saves.

Common mistakes

- Forgetting to schedule refreshes — the dashboard shows last month's numbers.
- Using plain `REFRESH` on a busy materialized view and blocking every reader for the duration.
- Treating it as a cache that updates itself (in PostgreSQL it doesn't).

In the interview

> **30-second answer:** "A view stores a query; a materialized view stores the query's result. Materialized views make expensive aggregations fast to read and can be indexed, but they're stale until refreshed. In PostgreSQL refresh is manual and full; `CONCURRENTLY` avoids blocking readers but needs a unique index."

- *How would you keep a materialized view fresh?* → Scheduled refresh (e.g. every 5 minutes with pg_cron), or refresh after each ETL batch (Extract-Transform-Load: a job that copies and reshapes data into the database); if you need real-time, maintain a summary table incrementally instead.
- *Materialized view vs cache like Redis?* → The materialized view lives in the database, is queryable with SQL and consistent with a point in time; an external cache (a fast copy of hot data kept outside the database — Redis is an in-memory key-value store) is faster per key but needs invalidation logic (removing or updating cached copies when the real data changes; see System Design Fundamentals and chapter 15).

Self-check

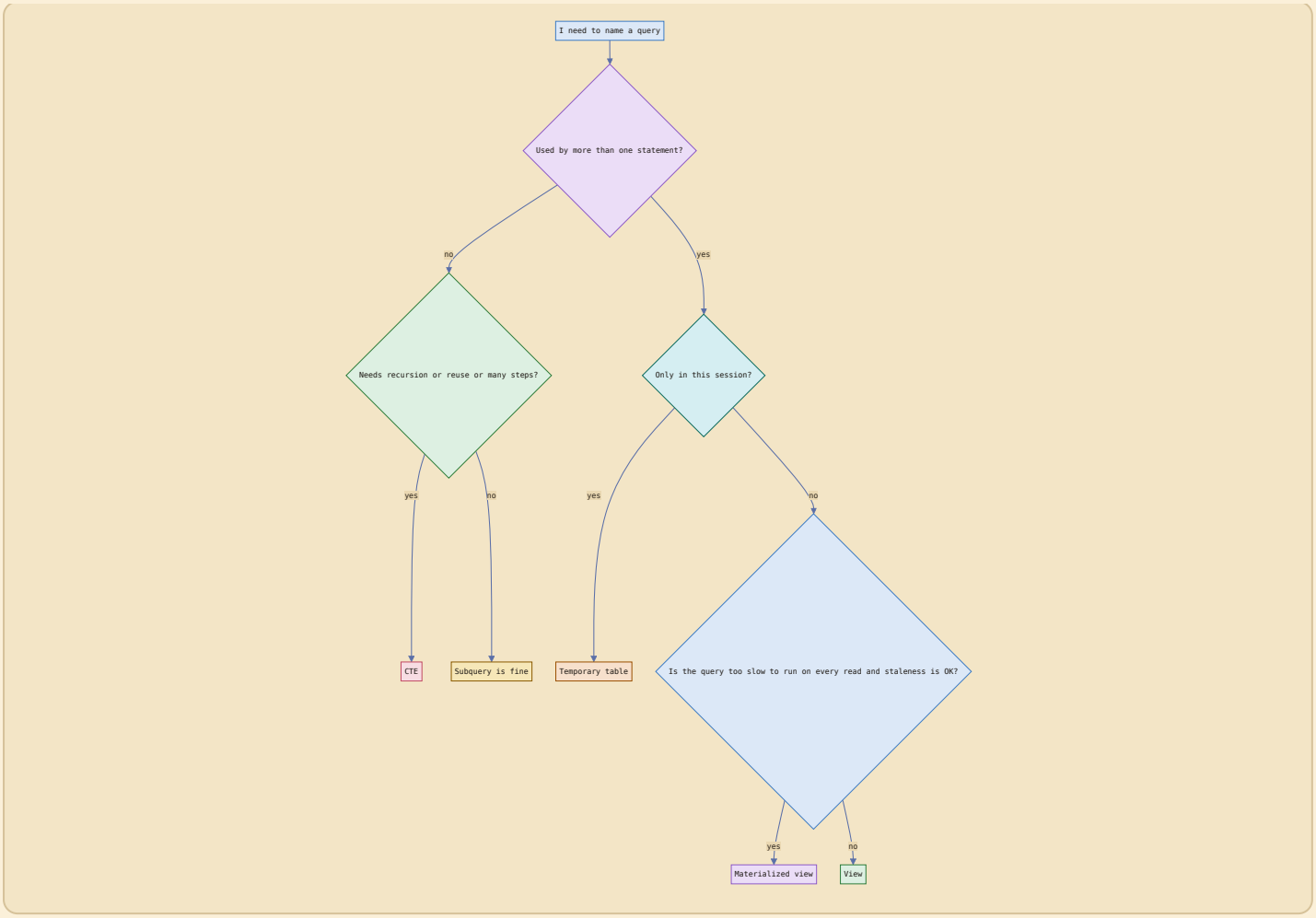
1. *Does a materialized view reflect an insert automatically in PostgreSQL?* → No; only after `REFRESH`.
2. *What does `CONCURRENTLY` need?* → A unique index on the materialized view with no `WHERE` clause.
3. *Can you index a materialized view?* → Yes, like a table.

TL;DR

Materialized view = stored result + manual refresh. Fast reads, stale data. Use `CONCURRENTLY` (with a unique index) to avoid blocking readers; schedule refreshes yourself.

7.9 Choosing the right tool

Need	Use	Lifetime	Stores rows?	Indexable?
One-off intermediate result inside one query, used once	subquery / derived table (ch. 6)	one statement	no	no
Readable multi-step query, reuse within the statement, recursion, move rows atomically	CTE	one statement	only if materialized, temporarily	no
Reusable definition for many queries and users, security, stable interface	view	permanent	no	no (index the base tables)
Fast reads of expensive results that may be slightly stale	materialized view	permanent	yes	yes
Intermediate data reused by several statements in a session, needs indexes/statistics	`CREATE TEMP TABLE` (ch. 9)	session or transaction	yes	yes



Chapter 7 TL;DR

- **CTE** (`WITH name AS (...)`) names a step for one statement; chain with commas; reuse by name. PostgreSQL 12+ inlines single-use CTEs; `MATERIALIZED` / `NOT MATERIALIZED` override.
- **Data-modifying CTEs** (PostgreSQL) run `INSERT/UPDATE/DELETE ... RETURNING` inside `WITH` — atomic multi-step changes; all parts see the pre-statement snapshot.
- **Recursive CTE** = anchor `UNION [ALL]` recursive term; the recursive term sees only the previous pass's rows; stops when a pass returns nothing. Cast types in the anchor, always have a stop condition, `ORDER BY` a path/depth.
- **Trees**: adjacency list + recursion — join children to go down, join the parent to go up, carry the top node and aggregate outside for subtree totals.
- **Graphs**: carry a visited array and refuse revisits, or use `CYCLE` (PostgreSQL 14+); `SEARCH BREADTH/DEPTH FIRST` gives an ordering column.
- `generate_series` makes number/date rows: full calendar + `LEFT JOIN` + `count(col)` fills gaps with zeros. MySQL uses a recursive CTE instead.
- **View** = stored query, no data, always current; for simplicity, security, stable interfaces. Simple single-table views are updatable; `WITH CHECK OPTION` blocks out-of-view writes; `CREATE OR REPLACE` can only append columns in PostgreSQL.
- **Materialized view** = stored result; fast, indexable, stale until `REFRESH` (`CONCURRENTLY` needs a unique index). No auto-refresh in PostgreSQL.

Next: chapter 8 — window functions, which compute running totals, rankings and comparisons with neighbouring rows without collapsing rows the way `GROUP BY` does.

Chapter 8 — Window Functions: Rankings, Running Totals and Row-to-Row Comparisons

`GROUP BY` answers "what is the total per department?" but throws away the individual rows to do it. Very often you need both at once: *each* employee **and** their department's average; *each* day's sales **and** the running total so far; *each* order **and** the previous order by the same customer. **Window functions** do exactly that. They compute a value across a set of related rows — the *window* — while keeping every row in the result.

They replace awkward self-joins (a table joined to itself) and correlated subqueries (subqueries re-run for every outer row), and they sit behind many SQL interview questions (top-N per group, running totals, second-highest salary, streaks, month-over-month growth).

What you'll learn

#	Topic	You will be able to…
8.1	`OVER()`	explain how a window function differs from `GROUP BY`
8.2	`PARTITION BY`	compute per-group values next to each row (share of total, diff from average)
8.3	`ORDER BY` in a window	build running totals and understand the default frame and its tie trap
8.4	Frames: `ROWS` / `RANGE` / `GROUPS`, `EXCLUDE`	choose exactly which rows each calculation sees
8.5	Ranking	use `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`, `PERCENT_RANK`, `CUME_DIST` and handle ties
8.6	Value functions	compare rows with `LAG` / `LEAD`, `FIRST_VALUE` / `LAST_VALUE` / `NTH_VALUE`
8.7	Running & moving calculations	running totals, moving averages, cumulative share, growth %
8.8	Top-N per group & dedup (de-duplication: removing extra copies)	the most-asked window pattern, plus "keep the latest row"
8.9	Gaps and islands	find streaks, missing ranges and sessions (bursts of activity separated by idle time)
8.10	Named windows, `FILTER`, evaluation order, performance	write tidy window queries and know how they execute

Setup. The `company` sample database from chapter 1 plus two small tables added in `code/00_schema.sql`:

Table	Rows	Why
`daily_sales(store, day, amount)`	Mumbai 1–7 March 2025; Pune the same week **except 4 March** (closed)	running totals, moving averages, frames, a gap
`logins(user_id, login_date)`	two users' login days in May 2025 with breaks	streaks (gaps and islands)

Mumbai's sales for 3 and 4 March are both 1500, and two Pune days are both 650 — deliberate ties. In `employees`, Rahul and Meera (Engineering) both earn 98 000 and Arjun and Sara (Sales) both earn 68 000.

8.1 What a window function is

What it is

A **window function** is a function that, for each row, looks at a set of rows related to it (its **window**) and returns one value — **without merging the rows**. You recognise it by the `OVER (...)` clause after the function call:

```
function_name(args) OVER (
    [PARTITION BY ...]      -- split rows into independent groups
    [ORDER BY ...]          -- order rows inside each group
    [frame clause]          -- which rows around the current one to use
)
```

Analogy: in a class photo, `GROUP BY` replaces each row of students with one sign ("Row 2: average height 150 cm"). A window function keeps every student and pins a badge on each: "your row's average is 150 cm", "you are 3rd tallest in your row", "the student to your left is 2 cm shorter".

The problem it solves

Compare `GROUP BY` with a window:

```
-- code/01_window_basics.sql
SELECT dept_id, round(avg(salary)) AS dept_avg
FROM employees WHERE dept_id IN (1, 2)
GROUP BY dept_id ORDER BY dept_id;
```

Output:

```
 dept_id | dept_avg
---------+----------
       1 |   130750
       2 |    89500
(2 rows)
```

Two rows: the individual employees are gone. With windows, every employee stays and gets the aggregates beside them:

```
SELECT first_name, dept_id, salary,
       round(avg(salary) OVER ())                    AS company_avg,
       round(avg(salary) OVER (PARTITION BY dept_id)) AS dept_avg,
       count(*)          OVER (PARTITION BY dept_id)  AS dept_size
FROM employees
WHERE dept_id IN (1, 2)
ORDER BY dept_id, salary DESC;
```

Output:

```
 first_name | dept_id |  salary   | company_avg | dept_avg | dept_size
------------+---------+-----------+-------------+----------+-----------
 Vikram     |       1 | 185000.00 |      110125 |   130750 |         4
 Neha       |       1 | 142000.00 |      110125 |   130750 |         4
 ...
 Daniel     |       2 | 150000.00 |      110125 |    89500 |         4
 Priya      |       2 |  72000.00 |      110125 |    89500 |         4
 ...
(8 rows)
```

- `OVER ()` — an empty window means "all rows of the result" (here the 8 rows left after `WHERE`), so `company_avg` is the same on every row.
- `OVER (PARTITION BY dept_id)` — the window is "all rows with my `dept_id`".

Without window functions you'd need a join to a grouped subquery (or a correlated subquery per row) for each extra column.

How it works

1. The query's `FROM`, `WHERE`, `GROUP BY` and `HAVING` run first and produce a set of rows.
2. For each window function, rows are split into **partitions** (`PARTITION BY`) and sorted inside each partition (`ORDER BY`).
3. For each row, the function looks at the rows of its window (the whole partition, or a **frame** within it — 8.3/8.4) and computes one value.
4. That value becomes a new column. Row count is unchanged.
5. Then `DISTINCT`, the final `ORDER BY` and `LIMIT` run.

Two families of functions can be used with `OVER`:

Family	Functions	Notes
Aggregate functions used as windows	`sum`, `avg`, `count`, `min`, `max`, `string_agg`, `array_agg`, `bool_and`, …	any normal aggregate becomes a window function when you add `OVER`
Dedicated window functions	`row_number`, `rank`, `dense_rank`, `percent_rank`, `cume_dist`, `ntile`, `lag`, `lead`, `first_value`, `last_value`, `nth_value`	only valid with `OVER`; the ranking ones take no arguments

Dialect notes

	Window functions available since	Frame support
PostgreSQL	8.4 (2009)	`ROWS`, `RANGE` with offsets, `GROUPS`, `EXCLUDE` (offsets/`GROUPS`/`EXCLUDE` since 11)
MySQL	8.0 (2018)	`ROWS`, `RANGE`; no `GROUPS`, no `EXCLUDE`
SQLite	3.25 (2018)	`ROWS`, `RANGE`; `GROUPS` and `EXCLUDE` since 3.28
SQL Server	ranking 2005; aggregate `ORDER BY` + frames 2012	`ROWS`; `RANGE` only with `UNBOUNDED`/`CURRENT ROW`; no `GROUPS`/`EXCLUDE`
Oracle	8i (1999)	full, plus extensions

TL;DR

`f(...) OVER (...)` computes across related rows but keeps every row. `OVER ()` = all rows; add `PARTITION BY`, `ORDER BY` and a frame to narrow it.

8.2 PARTITION BY: per-group values next to each row

What it is

`PARTITION BY expr` splits the rows into independent groups, like `GROUP BY` does, but without collapsing them. Each row's window is its own partition; the function "restarts" in every partition.

Typical uses

```sql
SELECT first_name, dept_id, salary,
       round(100.0 * salary / sum(salary) OVER (PARTITION BY dept_id), 1) AS pct_of_dept_payroll,
       salary - round(avg(salary) OVER (PARTITION BY dept_id))            AS vs_dept_avg
FROM employees
WHERE dept_id IN (1, 2)
ORDER BY dept_id, salary DESC;
```

Output:

```
 first_name | dept_id |  salary   | pct_of_dept_payroll | vs_dept_avg
------------+---------+-----------+---------------------+-------------
 Vikram     |       1 | 185000.00 |                35.4 |    54250.00
 Neha       |       1 | 142000.00 |                27.2 |    11250.00
 Rahul      |       1 |  98000.00 |                18.7 |   -32750.00
 Meera      |       1 |  98000.00 |                18.7 |   -32750.00
 ...
(8 rows)
```

"Percent of group total" and "difference from group average" are the two most common `PARTITION BY` calculations. `100.0 *` forces decimal division (chapter 3).

Partitioning rules

- You can partition by several expressions: `PARTITION BY store, date_trunc('month', day)`.
- `NULL` values form **one** partition together (just like `GROUP BY`):

```sql
SELECT first_name, dept_id, count(*) OVER (PARTITION BY dept_id) AS same_dept
FROM employees
WHERE dept_id IS NULL OR dept_id = 4
ORDER BY dept_id NULLS LAST;
```

Output:

```
 first_name | dept_id | same_dept
------------+---------+-----------
 Kiran      |       4 |         2
 Omar       |       4 |         2
 Asha       |  (null) |         2
 Rohan      |  (null) |         2
(4 rows)
```

- Different window functions in one `SELECT` can use different partitions (the first example mixed `OVER ()` with `OVER (PARTITION BY dept_id)`).

Common mistakes

- **Expecting fewer rows.** A window never removes rows; if you want one row per group, you want `GROUP BY` (or filter afterwards, 8.8).
- **Mixing a window and a non-grouped column in a `GROUP BY` query**: in a grouped query, window functions see the *grouped* rows, so their arguments must be grouped columns or aggregates (8.7 shows `sum(sum(x)) OVER ()`).

Self-check

1. *How many rows does `SELECT name, avg(salary) OVER (PARTITION BY dept_id) FROM employees` return?* → One per employee (15).
2. *Where do rows with `dept_id` NULL go?* → Together in one partition.

TL;DR

`PARTITION BY` = `GROUP BY` without collapsing. Perfect for "share of group total" and "difference from group average".

8.3 ORDER BY inside OVER: running calculations and the default frame

What it is

Adding `ORDER BY` inside `OVER` does two things:

1. It defines the order that ranking and offset functions (`row_number`, `lag`, …) use.
2. For **aggregate** window functions it changes the default window from "the whole partition" to "from the start of the partition **up to the current row**" — so `sum` becomes a **running total**, `count` a

running count, `max` a running maximum.

```sql
-- code/02_order_and_default_frame.sql
SELECT store, day, amount,
       sum(amount) OVER (PARTITION BY store ORDER BY day) AS running_total,
       count(*)    OVER (PARTITION BY store ORDER BY day) AS days_so_far
FROM daily_sales
WHERE store = 'Mumbai'
ORDER BY day;
```

Output:

```
 store  |    day     | amount  | running_total | days_so_far
--------+------------+---------+---------------+-------------
 Mumbai | 2025-03-01 | 1200.00 |       1200.00 |           1
 Mumbai | 2025-03-02 |  900.00 |       2100.00 |           2
 Mumbai | 2025-03-03 | 1500.00 |       3600.00 |           3
 Mumbai | 2025-03-04 | 1500.00 |       5100.00 |           4
 Mumbai | 2025-03-05 |  700.00 |       5800.00 |           5
 Mumbai | 2025-03-06 | 1100.00 |       6900.00 |           6
 Mumbai | 2025-03-07 | 1800.00 |       8700.00 |           7
(7 rows)
```

The default frame and the tie trap

The exact default, when `ORDER BY` is present and no frame is written, is:

```sql
RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

`RANGE` works on *values*, not physical rows: "current row" means the current row **and all its peers** — rows with the same `ORDER BY` value. So when the ordering column has ties, tied rows are added together and all get the same "running" total:

```sql
SELECT first_name, salary,
       sum(salary) OVER (ORDER BY salary)                    AS default_range,
       sum(salary) OVER (ORDER BY salary
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS rows_frame,
       sum(salary) OVER (ORDER BY salary, emp_id)            AS unique_order
FROM employees
WHERE dept_id = 2
ORDER BY salary, emp_id;
```

Output:

```
 first_name |  salary   | default_range | rows_frame | unique_order
------------+-----------+---------------+------------+--------------
 Arjun      |  68000.00 |     136000.00 |   68000.00 |     68000.00
 Sara       |  68000.00 |     136000.00 |  136000.00 |    136000.00
 Priya      |  72000.00 |     208000.00 |  208000.00 |    208000.00
 Daniel     | 150000.00 |     358000.00 |  358000.00 |    358000.00
(4 rows)
```

- `default_range`: Arjun and Sara are peers, so both see 68 000 + 68 000.
- `rows_frame`: `ROWS` counts physical rows, so the total grows one row at a time — but which tied row comes "first" is **not defined**; next time Sara could get 68 000.
- `unique_order`: adding a tie-breaker (`emp_id`) makes the order unique, so there are no peers and every frame type gives the same, **deterministic** (identical on every run) result. This is the best fix.

`OVER (...)` contains	Default window for aggregates
nothing / only `PARTITION BY`	the whole partition
`ORDER BY`	`RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` (start → current row + its peers)

Common mistakes

- **Running totals that "jump"** on duplicate dates or amounts — the peer effect. Order by a unique key, or write `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.
- **Confusing the window's `ORDER BY` with the query's `ORDER BY`**. The one inside `OVER` only controls the calculation; the output order is set by the final `ORDER BY` (without it, output order is not guaranteed).

Self-check

1. *Why can a running total show the same value on two rows?* → The default `RANGE` frame includes peers — rows with the same `ORDER BY` value.
2. *Two ways to avoid it?* → Make the `ORDER BY` unique (add a key) or use a `ROWS` frame (plus a tie-breaker for determinism).

TL;DR

`ORDER BY` in `OVER` turns aggregates into running aggregates. The default frame is `RANGE … CURRENT ROW`, which lumps ties together — add a unique tie-breaker.

8.4 Frames: ROWS, RANGE, GROUPS and EXCLUDE

What it is

The **frame** is the subset of the partition that an aggregate (and `first_value`/`last_value`/`nth_value`) looks at for the current row. It "slides" as the current row moves. Ranking functions and `lag`/`lead` ignore the frame.

```
{ ROWS | RANGE | GROUPS } BETWEEN frame_start AND frame_end [EXCLUDE ...]

frame_start / frame_end:
    UNBOUNDED PRECEDING      -- first row of the partition
    n PRECEDING              -- n rows / n units of value / n peer groups before
    CURRENT ROW
    n FOLLOWING
    UNBOUNDED FOLLOWING      -- last row of the partition
```

Short form: `ROWS 2 PRECEDING` means `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`.

Mode	`n PRECEDING` means	Peers (ties)	Needs
`ROWS`	*n physical rows* before	treated as separate rows	any `ORDER BY`
`RANGE`	rows whose **value** is within *n* of the current value (`n` can be a number or an `interval`)	always included together	exactly one `ORDER BY` column of a numeric/date/time type when an offset is used
`GROUPS` (PostgreSQL 11+)	*n peer groups* before (a group = rows with equal `ORDER BY` values)	included together	an `ORDER BY`

Frame diagrams

In the tables below, each line is one "current row" (**X**), and `x` marks the other rows inside its frame. The SQL uses `string_agg(...) OVER (frame)` to list the days in each frame — a handy trick to *see* a frame.

`ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` — a sliding window of up to 3 rows:

```sql
-- code/03_frames.sql
SELECT day, amount,
       string_agg(to_char(day, 'DD'), ',') OVER w AS frame_days,
       sum(amount) OVER w                         AS frame_sum
FROM daily_sales
WHERE store = 'Mumbai'
WINDOW w AS (ORDER BY day ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
ORDER BY day;
```

Output:

```
    day     | amount  | frame_days | frame_sum
------------+---------+------------+-----------
 2025-03-01 | 1200.00 | 01         |   1200.00
 2025-03-02 |  900.00 | 01,02      |   2100.00
 2025-03-03 | 1500.00 | 01,02,03   |   3600.00
 2025-03-04 | 1500.00 | 02,03,04   |   3900.00
 2025-03-05 |  700.00 | 03,04,05   |   3700.00
 2025-03-06 | 1100.00 | 04,05,06   |   3300.00
 2025-03-07 | 1800.00 | 05,06,07   |   3600.00
(7 rows)
```

current row	01	02	03	04	05	06	07
01	**X**	·	·	·	·	·	·
02	x	**X**	·	·	·	·	·
03	x	x	**X**	·	·	·	·
04	·	x	x	**X**	·	·	·
05	·	·	x	x	**X**	·	·
06	·	·	·	x	x	**X**	·
07	·	·	·	·	x	x	**X**

The first two frames are *shorter* — there are no earlier rows. (The `WINDOW w AS (...)` clause names a window so it can be reused; see 8.10.)

Centered: `ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING` — smoothing that looks both ways:

```sql
SELECT day, amount,
       string_agg(to_char(day, 'DD'), ',') OVER w AS frame_days,
       round(avg(amount) OVER w, 1)               AS centered_avg
FROM daily_sales
WHERE store = 'Mumbai'
WINDOW w AS (ORDER BY day ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)
ORDER BY day;
```

Output:

```
    day     | amount  | frame_days | centered_avg
------------+---------+------------+--------------
 2025-03-01 | 1200.00 | 01,02      |       1050.0
 2025-03-02 |  900.00 | 01,02,03   |       1200.0
 ...
 2025-03-07 | 1800.00 | 06,07      |       1450.0
(7 rows)
```

`ROWS` vs `RANGE` when a day is missing. Pune has no row for 4 March. "The previous 2 rows" and "the previous 2 days" now differ:

```sql
SELECT day, amount,
       string_agg(to_char(day, 'DD'), ',') OVER (ORDER BY day
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)                    AS rows_2,
       string_agg(to_char(day, 'DD'), ',') OVER (ORDER BY day
            RANGE BETWEEN INTERVAL '2 days' PRECEDING AND CURRENT ROW)  AS range_2_days
FROM daily_sales
WHERE store = 'Pune'
ORDER BY day;
```

Output:

```
    day     | amount |  rows_2  | range_2_days
------------+--------+----------+--------------
 2025-03-01 | 400.00 | 01       | 01
 2025-03-02 | 650.00 | 01,02    | 01,02
 2025-03-03 | 500.00 | 01,02,03 | 01,02,03
 2025-03-05 | 800.00 | 02,03,05 | 03,05
 2025-03-06 | 650.00 | 03,05,06 | 05,06
 2025-03-07 | 300.00 | 05,06,07 | 05,06,07
(6 rows)
```

`ROWS 2 PRECEDING` (three rows, whatever their dates):

current row	01	02	03	05	06	07
01	**X**	·	·	·	·	·
02	x	**X**	·	·	·	·
03	x	x	**X**	·	·	·
05	·	x	x	**X**	·	·
06	·	·	x	x	**X**	·
07	·	·	·	x	x	**X**

`RANGE INTERVAL '2 days' PRECEDING` (dates from current − 2 days to current):

current row	01	02	03	05	06	07
01	**X**	·	·	·	·	·
02	x	**X**	·	·	·	·
03	x	x	**X**	·	·	·
05	·	·	x	**X**	·	·
06	·	·	·	x	**X**	·
07	·	·	·	x	x	**X**

For "average of the last 3 *calendar* days", `RANGE` with an interval is correct and `ROWS` silently reaches back to 2 March. (Alternatively, fill the missing days with `generate_series` first — chapter 7.)

`ROWS` vs `RANGE` vs `GROUPS` with ties. Ordering Pune by `amount` (two days have 650):

```sql
SELECT day, amount,
       string_agg(amount::int::text, ',') OVER (ORDER BY amount
            ROWS   BETWEEN 1 PRECEDING AND CURRENT ROW) AS rows_1,
       string_agg(amount::int::text, ',') OVER (ORDER BY amount
            RANGE  BETWEEN 150 PRECEDING AND CURRENT ROW) AS range_150,
       string_agg(amount::int::text, ',') OVER (ORDER BY amount
            GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW) AS groups_1
FROM daily_sales
WHERE store = 'Pune'
ORDER BY amount, day;
```

Output:

```
    day     | amount | rows_1  |  range_150  |  groups_1
------------+--------+---------+-------------+-------------
 2025-03-07 | 300.00 | 300     | 300         | 300
 2025-03-01 | 400.00 | 300,400 | 300,400     | 300,400
 2025-03-03 | 500.00 | 400,500 | 400,500     | 400,500
 2025-03-02 | 650.00 | 500,650 | 500,650,650 | 500,650,650
 2025-03-06 | 650.00 | 650,650 | 500,650,650 | 500,650,650
 2025-03-05 | 800.00 | 650,800 | 650,650,800 | 650,650,800
(6 rows)
```

- `ROWS 1 PRECEDING`: the previous physical row, so the two 650 rows get *different* frames.
- `RANGE 150 PRECEDING`: every row whose amount is between (current − 150) and current, **including all peers**. For 650: amounts 500…650 → 500, 650, 650.
- `GROUPS 1 PRECEDING`: the current peer group plus the one group before it. For 650: groups {500} and {650, 650}. For 800: groups {650, 650} and {800}.

`EXCLUDE` (PostgreSQL 11+) removes rows from the frame: `EXCLUDE CURRENT ROW`, `EXCLUDE GROUP` (current row and its peers), `EXCLUDE TIES` (peers but not the current row), `EXCLUDE NO OTHERS` (default). Example — each day compared with the average of all *other* days:

```sql
SELECT day, amount,
       round(avg(amount) OVER (ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
                               EXCLUDE CURRENT ROW), 1) AS avg_of_other_days
FROM daily_sales
WHERE store = 'Mumbai'
ORDER BY day;
```

Output:

```
    day     | amount  | avg_of_other_days
------------+---------+-------------------
 2025-03-01 | 1200.00 |            1250.0
 2025-03-02 |  900.00 |            1300.0
 ...
 2025-03-07 | 1800.00 |            1150.0
(7 rows)
```

(Week total 8700; for 1 March: (8700 − 1200) / 6 = 1250.)

Choosing a frame

You want	Frame
Running total (deterministic)	`ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` + unique `ORDER BY`
Last *N* rows (e.g. last 7 records)	`ROWS BETWEEN N-1 PRECEDING AND CURRENT ROW`
Last *N* days of time, with gaps in data	`RANGE BETWEEN INTERVAL 'N-1 days' PRECEDING AND CURRENT ROW`
Whole partition (e.g. for `last_value`)	`ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` (or no `ORDER BY`)
Current value and the next distinct value(s)	`GROUPS BETWEEN CURRENT ROW AND 1 FOLLOWING`

Common mistakes

- **Writing `ROWS 6 PRECEDING` for "last 7 days" when dates can be missing** — use `RANGE` with an interval.
- **Forgetting the frame applies only to some functions.** `row_number`, `rank`, `lag`, `lead`, `ntile` ignore it.
- **Frame start after frame end** (e.g. `BETWEEN CURRENT ROW AND 1 PRECEDING`) — an error.
- **Porting `GROUPS` / `EXCLUDE` / `RANGE n PRECEDING` to SQL Server or MySQL** — not supported there (see the dialect table in 8.1).

In the interview

> **30-second answer:** "The frame says which rows around the current one an aggregate sees. `ROWS` counts physical rows, `RANGE` uses the ordering value — so ties are always included and you can say 'within 7 days' — and `GROUPS` counts groups of tied rows. The default with `ORDER BY` is `RANGE UNBOUNDED PRECEDING TO CURRENT ROW`, which is why running totals can repeat on ties."

- *Moving 7-day average with missing days?* → `avg(x) OVER (ORDER BY day RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW)`, or fill the calendar and use `ROWS 6 PRECEDING`.

Self-check

1. *Pune, current row 5 March: which days are in `ROWS 2 PRECEDING`? In `RANGE INTERVAL '2 days' PRECEDING`?* → 2, 3, 5 March; 3, 5 March.
2. *Which frame mode treats tied rows as separate?* → `ROWS`.
3. *What does `EXCLUDE CURRENT ROW` do?* → Removes the current row from its own frame.
4. *What is `ROWS 3 PRECEDING` short for?* → `ROWS BETWEEN 3 PRECEDING AND CURRENT ROW`.

TL;DR

Frame = which rows an aggregate sees. `ROWS` = physical rows, `RANGE` = value distance (ties included, intervals allowed), `GROUPS` = peer groups. `EXCLUDE` removes rows. Write the frame explicitly whenever it matters.

8.5 Ranking functions

What it is

Ranking functions give each row a number based on its position in the window's `ORDER BY`. They differ in **how they treat ties**.

Function	Returns	Ties
`row_number()`	1, 2, 3, … unique per row	tied rows get different numbers — **in no guaranteed order**
`rank()`	position with gaps ("Olympic" ranking)	same rank; next rank skips (1, 2, 2, 4)
`dense_rank()`	position without gaps	same rank; next rank is +1 (1, 2, 2, 3)
`percent_rank()`	(rank − 1) / (rows in partition − 1), from 0 to 1	same value
`cume_dist()`	(rows ordered before the current row + its peers) / rows in partition, from >0 to 1	same value
`ntile(n)`	bucket number 1…n, buckets as equal as possible	ties can be split across buckets

```sql
-- code/04_ranking.sql
SELECT first_name, salary,
       row_number()  OVER w AS row_number,
       rank()        OVER w AS rank,
       dense_rank()  OVER w AS dense_rank,
       round(percent_rank() OVER w :: numeric, 2) AS pct_rank,
       round(cume_dist()    OVER w :: numeric, 2) AS cume_dist,
       ntile(3)      OVER w AS ntile_3
FROM employees
WHERE dept_id IN (1, 2)
WINDOW w AS (ORDER BY salary DESC)
ORDER BY salary DESC, row_number;
```

Output:

```
 first_name |  salary   | row_number | rank | dense_rank | pct_rank | cume_dist | ntile_3
------------+-----------+------------+------+------------+----------+-----------+---------
 Vikram     | 185000.00 |          1 |    1 |          1 |     0.00 |      0.13 |       1
 Daniel     | 150000.00 |          2 |    2 |          2 |     0.14 |      0.25 |       1
 Neha       | 142000.00 |          3 |    3 |          3 |     0.29 |      0.38 |       1
 Rahul      |  98000.00 |          4 |    4 |          4 |     0.43 |      0.63 |       2
 Meera      |  98000.00 |          5 |    4 |          4 |     0.43 |      0.63 |       2
 Priya      |  72000.00 |          6 |    6 |          5 |     0.71 |      0.75 |       2
 Arjun      |  68000.00 |          7 |    7 |          6 |     0.86 |      1.00 |       3
 Sara       |  68000.00 |          8 |    7 |          6 |     0.86 |      1.00 |       3
(8 rows)
```

Reading the ties:

- Rahul and Meera tie at 98 000: `rank` 4 and 4, then Priya gets **6** (position 5 is "used up"); `dense_rank` gives 4, 4, then **5**.
- `row_number` gave Rahul 4 and Meera 5 — but nothing in `ORDER BY salary DESC` says so; it could flip. Add a tie-breaker (`ORDER BY salary DESC, emp_id`) whenever you use `row_number`.
- `percent_rank` for Priya: (6 − 1) / (8 − 1) = 0.71. `cume_dist` for Rahul/Meera: 5 of 8 rows have salary ≥ 98 000 in this descending order → 0.63.
- `ntile(3)` over 8 rows: 8 / 3 = 2 remainder 2, so the first two buckets get 3 rows and the last gets 2.

Ranking within groups — add `PARTITION BY`:

```
SELECT dept_id, first_name, salary,
       dense_rank() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rank_in_dept
FROM employees
WHERE dept_id IS NOT NULL
ORDER BY dept_id, rank_in_dept, first_name;
```

Output:

```
 dept_id | first_name |  salary   | rank_in_dept
---------+------------+-----------+--------------
       1 | Vikram     | 185000.00 |            1
       1 | Neha       | 142000.00 |            2
       1 | Meera      |  98000.00 |            3
       1 | Rahul      |  98000.00 |            3
 ...
       5 | Li         |  64000.00 |            1
(13 rows)
```

`ntile` for quartiles. Bucket everyone into four salary quartiles (the four equal-sized quarters of the sorted values), then summarise each bucket (window in a subquery, `GROUP BY` outside):

```
SELECT quartile, count(*) AS people, min(salary), max(salary)
FROM (SELECT salary, ntile(4) OVER (ORDER BY salary) AS quartile FROM employees) t
GROUP BY quartile
ORDER BY quartile;
```

Output:

```
 quartile | people |    min    |    max
----------+--------+-----------+-----------
        1 |      4 |  30000.00 |  68000.00
        2 |      4 |  68000.00 |  98000.00
        3 |      4 |  98000.00 | 142000.00
        4 |      3 | 150000.00 | 250000.00
(4 rows)
```

Note that 68 000 and 98 000 appear in two buckets each: `ntile` fills buckets by row count and **splits ties**. If tied values must stay together, compute buckets from `percent_rank` or `cume_dist` instead (or

use `percentile_cont`, an ordered-set aggregate — an aggregate that reads its input in sorted order, like a median — for exact quartile boundaries — chapter 4).

Which ranking function? (the question interviewers love)

Question	Use
"Exactly one row per …" / pagination (splitting results into pages) / dedup	`row_number()` with a unique tie-breaker
"Top 3 places, ties share a place, next place skips" (sports)	`rank()`
"The 2nd highest *distinct* salary" / "top 3 salary levels"	`dense_rank()`
"Top 10 %"	`percent_rank() <= 0.1` or `cume_dist() <= 0.1` (depending on how ties should count)
"Split into 4 equal groups"	`ntile(4)`

Nth highest value — the classic: `dense_rank` makes ties count once.

```
-- 2nd highest distinct salary (NULL if there isn't one)
SELECT max(salary) FROM (
    SELECT salary, dense_rank() OVER (ORDER BY salary DESC) AS dr FROM employees
) t WHERE dr = 2;
```

Output:

```
    max
-----------
 185000.00
(1 row)
```

Asha, the CEO, earns 250 000, so Vikram's 185 000 is the second-highest distinct value. Wrapping in `max()` returns one row even when several people share that salary, and `NULL` rather than no rows when there is no 2nd value.

Self-check

1. *Salaries 100, 90, 90, 80: rank of 80? dense_rank?* → 4; 3.
2. *Why add `emp_id` to `ORDER BY` for `row_number`?* → To make the numbering deterministic among ties.
3. *`ntile(4)` over 10 rows — bucket sizes?* → 3, 3, 2, 2.
4. *Which function answers "second highest distinct salary"?* → `dense_rank() = 2`.

TL;DR

`row_number` (unique, arbitrary among ties), `rank` (ties share, gaps after), `dense_rank` (ties share, no gaps), `ntile(n)` (n near-equal buckets), `percent_rank`/`cume_dist` (relative position 0–1). Always think about ties.

8.6 Value (offset) functions: LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE

What it is

Value functions return a value **from another row** of the window.

Function	Returns	Uses the frame?
`lag(expr [, offset [, default]])`	`expr` from `offset` rows **before** (default 1)	no — relative to the partition's order
`lead(expr [, offset [, default]])`	`expr` from `offset` rows **after**	no
`first_value(expr)`	`expr` from the frame's first row	yes
`last_value(expr)`	`expr` from the frame's last row	yes
`nth_value(expr, n)`	`expr` from the frame's *n*-th row (1-based), NULL if the frame is shorter	yes

When there is no such row (the first row has no "previous"), `lag`/`lead` return `NULL`, or the `default` you pass.

Day-over-day change:

```sql
-- code/05_value_functions.sql
SELECT store, day, amount,
       lag(amount)  OVER w          AS prev_day,
       amount - lag(amount) OVER w  AS change,
       lead(amount) OVER w          AS next_day,
       lag(amount, 2, 0) OVER w     AS two_days_ago_or_0
FROM daily_sales
WHERE day <= '2025-03-04'
WINDOW w AS (PARTITION BY store ORDER BY day)
ORDER BY store, day;
```

Output:

```
 store  |    day     | amount  | prev_day | change  | next_day | two_days_ago_or_0
--------+------------+---------+----------+---------+----------+-------------------
 Mumbai | 2025-03-01 | 1200.00 |   (null) |  (null) |   900.00 |                 0
 Mumbai | 2025-03-02 |  900.00 |  1200.00 | -300.00 |  1500.00 |                 0
 ...
 Mumbai | 2025-03-04 | 1500.00 |  1500.00 |    0.00 |   (null) |            900.00
 Pune   | 2025-03-01 |  400.00 |   (null) |  (null) |   650.00 |                 0
 Pune   | 2025-03-02 |  650.00 |   400.00 |  250.00 |   500.00 |                 0
 Pune   | 2025-03-03 |  500.00 |   650.00 | -150.00 |   (null) |            400.00
(7 rows)
```

`PARTITION BY store` stops `lag` from reaching into the other store's rows: Pune's first day has no previous day even though Mumbai's last row came before it. The `WHERE` runs before the window, so `lead` on 4 March sees nothing (5 March was filtered out).

Note that `lag` means "previous **row**", not "previous **day**": in Pune, the row before 5 March is 3 March. If you need the previous *calendar* day, fill gaps first or check the date difference (8.9).

The `last_value` trap. With `ORDER BY` in the window, the default frame ends at the current row, so `last_value` just returns the current row:

```sql
SELECT day, amount,
       first_value(amount) OVER (ORDER BY day)    AS first_amt,
       last_value(amount)  OVER (ORDER BY day)    AS last_default_frame,
       last_value(amount)  OVER (ORDER BY day
            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS last_whole_partition,
       nth_value(amount, 2) OVER (ORDER BY day
            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS second_amt
FROM daily_sales
WHERE store = 'Pune'
ORDER BY day;
```

Output:

```
    day     | amount | first_amt | last_default_frame | last_whole_partition | second_amt
------------+--------+-----------+--------------------+----------------------+------------
 2025-03-01 | 400.00 |    400.00 |             400.00 |               300.00 |     650.00
 2025-03-02 | 650.00 |    400.00 |             650.00 |               300.00 |     650.00
 2025-03-03 | 500.00 |    400.00 |             500.00 |               300.00 |     650.00
 ...
 2025-03-07 | 300.00 |    400.00 |             300.00 |               300.00 |     650.00
(6 rows)
```

Fix: extend the frame to `UNBOUNDED FOLLOWING`, or use `first_value` with the order reversed (`first_value(amount) OVER (ORDER BY day DESC)`), which needs no frame.

`first_value(first_name) OVER (PARTITION BY dept_id ORDER BY salary DESC)` puts each department's top earner beside every employee — handy for "gap to the leader" columns (the file shows it).

Time between events — days between a customer's consecutive orders (a standard retention metric, i.e. a measure of how often customers come back):

```sql
SELECT customer_id, order_id, placed_at::date AS placed,
       placed_at::date - lag(placed_at::date) OVER (PARTITION BY customer_id ORDER BY placed_at) AS
days_since_prev
FROM orders
WHERE customer_id IN (1, 2, 5)
ORDER BY customer_id, placed_at;
```

Output:

```
 customer_id | order_id |   placed   | days_since_prev
-------------+----------+------------+-----------------
           1 |      101 | 2025-01-03 |          (null)
           1 |      104 | 2025-02-11 |              39
           1 |      113 | 2025-06-02 |             111
           2 |      102 | 2025-01-17 |          (null)
           2 |      108 | 2025-04-01 |              74
 ...
(7 rows)
```

Subtracting two `date`s gives an integer number of days in PostgreSQL.

Dialect notes

- `IGNORE NULLS` (e.g. `lag(x) IGNORE NULLS OVER (...)`, "the last non-null value") is in the SQL standard and supported by Oracle, SQL Server 2022, BigQuery and Snowflake (two cloud data-warehouse products). PostgreSQL gains it in **version 19** (in beta as of September 2026); PostgreSQL 14–18 and MySQL lack it. There, emulate it with a running `count(x)` as a group key and `first_value` within each group, or a correlated subquery.
- `nth_value(...) FROM LAST` (counting from the end of the frame) exists in Oracle and a few warehouses, not in PostgreSQL, MySQL or SQL Server.

Common mistakes

- `last_value` without a full frame; `lag` without `PARTITION BY` (crosses groups); `lag` treated as "previous period" when periods can be missing.
- The first row's `amount - lag(amount)` is `NULL`; pass a `default` or use `coalesce` if you want 0.

In the interview

30-second answer: "`LAG` and `LEAD` read a column from a previous or next row in the window's order, per partition, which replaces a self-join on 'row n−1'. `FIRST_VALUE`, `LAST_VALUE` and `NTH_VALUE` read from positions in the frame — and because the default frame ends at the current row, `LAST_VALUE` needs `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`."

- *Year-over-year growth per product?* → `lag(revenue) OVER (PARTITION BY product ORDER BY year)`, then `(revenue - prev) / prev`.

- *How did people do this before window functions?* → A self-join on `t2.day = t1.day - 1` (breaks with gaps) or a correlated subquery `(SELECT ... WHERE t2.day < t1.day ORDER BY day DESC LIMIT 1)`.

Self-check

1. *What does `lag(x, 1, 0)` return on the first row?* → 0.
2. *Why does `last_value(x) OVER (ORDER BY d)` equal `x`?* → The default frame ends at the current row (plus peers).
3. *Which of these ignore the frame: `lag`, `first_value`, `lead`, `nth_value`?* → `lag` and `lead`.

TL;DR

`lag` / `lead` = previous/next row per partition (with optional offset and default).
`first_value` / `last_value` / `nth_value` read the frame — give `last_value` a full frame.

8.7 Running totals, moving averages and growth

What it is

The everyday analytics calculations are just aggregate window functions with the right frame:

Metric	Window
Running total	`sum(x) OVER (PARTITION BY g ORDER BY t ROWS UNBOUNDED PRECEDING)`
Moving average of last 3 rows	`avg(x) OVER (PARTITION BY g ORDER BY t ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`
Cumulative share	running total / `sum(x) OVER (PARTITION BY g)`
Share of total	`x / sum(x) OVER (PARTITION BY g)`
Growth vs previous period	`x / lag(x) OVER (ORDER BY t) - 1`

```sql
-- code/06_running_and_moving.sql
SELECT store, day, amount,
       sum(amount) OVER (PARTITION BY store ORDER BY day
                         ROWS UNBOUNDED PRECEDING)                        AS running_total,
       round(avg(amount) OVER (PARTITION BY store ORDER BY day
                         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 1)     AS moving_avg_3,
       count(*) OVER (PARTITION BY store ORDER BY day
                         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)         AS rows_in_avg,
       round(100.0 * sum(amount) OVER (PARTITION BY store ORDER BY day ROWS UNBOUNDED PRECEDING)
                   / sum(amount) OVER (PARTITION BY store), 1)             AS pct_of_week_so_far
FROM daily_sales
ORDER BY store, day;
```

Output:

```
 store  |    day     | amount  | running_total | moving_avg_3 | rows_in_avg | pct_of_week_so_far
--------+------------+---------+---------------+--------------+-------------+--------------------
 Mumbai | 2025-03-01 | 1200.00 |       1200.00 |       1200.0 |           1 |               13.8
 Mumbai | 2025-03-02 |  900.00 |       2100.00 |       1050.0 |           2 |               24.1
 Mumbai | 2025-03-03 | 1500.00 |       3600.00 |       1200.0 |           3 |               41.4
 ...
 Mumbai | 2025-03-07 | 1800.00 |       8700.00 |       1200.0 |           3 |              100.0
 ...
 Pune   | 2025-03-03 |  500.00 |       1550.00 |        516.7 |           3 |               47.0
 Pune   | 2025-03-05 |  800.00 |       2350.00 |        650.0 |           3 |               71.2
 ...
 Pune   | 2025-03-07 |  300.00 |       3300.00 |        583.3 |           3 |              100.0
(13 rows)
```

Two things to notice:

1. The first two moving averages use only 1 and 2 rows (`rows_in_avg`) — a "warm-up" period. Averages over fewer points than intended can mislead on a chart.
2. Pune's 5 March average (650.0) uses 2, 3 and 5 March because the frame is `ROWS` — see 8.4 for the `RANGE` alternative.

To hide warm-up rows, show the average only when the frame is full: `CASE WHEN count(*) OVER w = 3 THEN avg(amount) OVER w END` (in `code/06_running_and_moving.sql`).

Windows on top of GROUP BY

Window functions run **after** `GROUP BY`, so they can work on the grouped rows. That's how you get "monthly revenue and the running total of monthly revenue" in one query. The inner `sum` is the normal aggregate (per month); the outer `sum(...) OVER` adds up those monthly sums:

```
SELECT date_trunc('month', o.placed_at)::date AS month,
       sum(oi.quantity * oi.unit_price * (1 - oi.discount))::numeric(10,2) AS revenue,
       sum(sum(oi.quantity * oi.unit_price * (1 - oi.discount)))
           OVER (ORDER BY date_trunc('month', o.placed_at)::date)::numeric(10,2) AS running_revenue
FROM orders o JOIN order_items oi USING (order_id)
WHERE o.status <> 'cancelled'
GROUP BY 1
ORDER BY 1;
```

Output:

```
   month    | revenue | running_revenue
------------+---------+-----------------
 2025-01-01 | 1871.90 |         1871.90
 2025-02-01 |  811.49 |         2683.39
 ...
 2025-06-01 | 1516.60 |         7403.62
(6 rows)
```

`sum(sum(...))` looks odd but is correct. It's often clearer to aggregate in a CTE (Common Table Expression — a named subquery written with `WITH`, chapter 7) first and apply windows in the outer query — as here for **month-over-month growth**:

```
WITH monthly AS (
    SELECT date_trunc('month', o.placed_at)::date AS month,
           sum(oi.quantity * oi.unit_price * (1 - oi.discount)) AS revenue
    FROM orders o JOIN order_items oi USING (order_id)
    WHERE o.status <> 'cancelled'
    GROUP BY 1
)
SELECT month, round(revenue, 2) AS revenue,
       round(lag(revenue) OVER (ORDER BY month), 2) AS prev_month,
       round(100.0 * (revenue / lag(revenue) OVER (ORDER BY month) - 1), 1) AS mom_growth_pct
FROM monthly
ORDER BY month;
```

Output:

```
   month    | revenue | prev_month | mom_growth_pct
------------+---------+------------+----------------
 2025-01-01 | 1871.90 |     (null) |         (null)
 2025-02-01 |  811.49 |    1871.90 |          -56.6
 2025-03-01 |  804.95 |     811.49 |           -0.8
 ...
 2025-06-01 | 1516.60 |     249.93 |          506.8
(6 rows)
```

If a month could have *no* orders, generate the months first (chapter 7) — otherwise `lag` would compare with two months back. Guard divisions with `NULLIF(prev, 0)` if the previous value can be zero.

Real-world examples

Account balance after each transaction (running `sum` ordered by time and id), 7-day moving average of daily active users, stock level as a running sum of `+received -shipped`.

TL;DR

Running total = `sum` + `ROWS UNBOUNDED PRECEDING`; moving average = `avg` + `ROWS n PRECEDING` (watch warm-up rows and gaps); growth = value / `lag(value)` − 1. Windows run after `GROUP BY`, so `sum(sum(x)) OVER (...)` works — a CTE is often clearer.

8.8 Top-N per group and de-duplication

The problem

"Top 2 earners in each department", "latest order per customer", "best-selling product per category" — these **top-N-per-group** questions are the most common window-function interview problem. `LIMIT` can't do it (it limits the whole result, not each group), and `GROUP BY` returns only the aggregated value, not the rest of the row.

Why you need a subquery or CTE

Window functions are computed *after* `WHERE`, so you cannot filter on them in the same query level:

```
-- code/07_top_n_per_group.sql
SELECT first_name, salary FROM employees
WHERE row_number() OVER (ORDER BY salary DESC) <= 3;
```

Output:

```
ERROR:  window functions are not allowed in WHERE
LINE 2: WHERE row_number() OVER (ORDER BY salary DESC) <= 3;
              ^
```

(They're not allowed in `GROUP BY` or `HAVING` either.) The pattern is always: **compute the rank in a CTE or subquery, filter in the outer query.**

How it works

1. In a CTE, add `row_number()` / `rank()` / `dense_rank()` `OVER (PARTITION BY group ORDER BY measure DESC)`.
2. In the outer query, `WHERE rank_col <= N`.
3. Pick the ranking function according to how ties should be treated.

```
WITH ranked AS (
    SELECT d.name AS dept, e.first_name, e.salary,
           row_number() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC, e.emp_id) AS rn,
           rank()       OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC)           AS rnk,
           dense_rank() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC)           AS drnk
    FROM employees e JOIN departments d USING (dept_id)
)
SELECT dept, first_name, salary, rn, rnk, drnk
FROM ranked
WHERE dept IN ('Engineering', 'Sales') AND rnk <= 3
ORDER BY dept, rn;
```

Output:

```
    dept     | first_name |  salary   | rn | rnk | drnk
-------------+------------+-----------+----+-----+------
 Engineering | Vikram     | 185000.00 |  1 |   1 |    1
 ...
 Engineering | Rahul      |  98000.00 |  3 |   3 |    3
 Engineering | Meera      |  98000.00 |  4 |   3 |    3
 Sales       | Daniel     | 150000.00 |  1 |   1 |    1
 Sales       | Priya      |  72000.00 |  2 |   2 |    2
 Sales       | Arjun      |  68000.00 |  3 |   3 |    3
 Sales       | Sara       |  68000.00 |  4 |   3 |    3
(8 rows)
```

The three filters give different answers when there are ties — here for Sales:

```sql
-- ranked = Sales only; then:
SELECT 'row_number <= 3' AS rule, string_agg(first_name, ', ' ORDER BY rn) AS kept FROM ranked WHERE rn <= 3
UNION ALL
SELECT 'rank <= 3',       string_agg(first_name, ', ' ORDER BY rn) FROM ranked WHERE rnk <= 3
UNION ALL
SELECT 'dense_rank <= 2', string_agg(first_name, ', ' ORDER BY rn) FROM ranked WHERE drnk <= 2;
```

Output:

```
      rule       |            kept
-----------------+----------------------------
 row_number <= 3 | Daniel, Priya, Arjun
 rank <= 3       | Daniel, Priya, Arjun, Sara
 dense_rank <= 2 | Daniel, Priya
(3 rows)
```

Filter	Meaning	Rows per group
`row_number() <= N`	exactly N rows; ties broken by your tie-breaker	exactly N (or fewer)
`rank() <= N`	everyone whose position is within the top N; ties included	can exceed N
`dense_rank() <= N`	everyone having one of the top N *distinct* values	can far exceed N

Ask in an interview: "If two people tie for 3rd, should both be returned?"

Latest row per key (de-duplication). Same pattern with `N = 1`, ordered newest first:

```sql
SELECT customer_id, order_id, placed_at, status
FROM (
    SELECT o.*, row_number() OVER (PARTITION BY customer_id ORDER BY placed_at DESC) AS rn
    FROM orders o
) t
WHERE rn = 1
ORDER BY customer_id;
```

Output:

```
 customer_id | order_id |      placed_at      |  status
-------------+----------+---------------------+-----------
           1 |      113 | 2025-06-02 10:30:00 | pending
           2 |      108 | 2025-04-01 12:00:00 | delivered
           3 |      111 | 2025-05-06 07:55:00 | cancelled
 ...
           8 |      109 | 2025-04-09 16:45:00 | delivered
(8 rows)
```

Find duplicates to delete — number the rows within each duplicate group, and everything with `rn > 1` is an extra copy. The duplicate Anita Shah (chapter 1's messy email) is found after normalising the email:

```sql
WITH dups AS (
    SELECT customer_id,
           row_number() OVER (PARTITION BY lower(trim(email)) ORDER BY customer_id) AS rn
    FROM customers
    WHERE email IS NOT NULL
)
SELECT customer_id AS would_delete FROM dups WHERE rn > 1;
```

Output:

```
 would_delete
--------------
            8
(1 row)
```

To actually delete: `DELETE FROM customers WHERE customer_id IN (SELECT customer_id FROM dups WHERE rn > 1)` (after re-pointing that customer's orders — chapter 9 covers `DELETE` and foreign keys — columns whose values must match a key in another table).

Alternatives

Approach	Top-1	Top-N	Notes
Window + filter (above)	yes	yes	portable, flexible about ties
`DISTINCT ON (group) ... ORDER BY group, measure DESC`	yes	no	PostgreSQL only, concise
`LATERAL (... ORDER BY ... LIMIT N)` per group (ch. 6)	yes	yes	fastest when an index (sorted lookup structure) on (group, measure) exists and groups are few
Correlated subquery `WHERE salary = (SELECT max(...) ...)`	yes (with ties)	awkward	classic pre-window approach
`QUALIFY rn <= N`	yes	yes	filter on windows without a subquery — Snowflake, BigQuery, DuckDB, Teradata; **not** PostgreSQL / MySQL / SQL Server

For top-1, `SELECT DISTINCT ON (dept_id) dept_id, first_name, salary FROM employees WHERE dept_id IS NOT NULL ORDER BY dept_id, salary DESC, emp_id` returns each department's best-paid person in one short query (the file runs it).

In the interview

30-second answer: "Rank rows inside each group with a window function in a CTE — `PARTITION BY` the group, `ORDER BY` the measure descending — then filter `rank <= N` in the outer query, because window functions can't appear in `WHERE`. I'd choose `row_number`, `rank` or `dense_rank` depending on whether ties should be included, and add a tie-breaker for determinism."

- *Top 3 salaries per department with ties?* → `dense_rank() <= 3` (three distinct salary levels) or `rank() <= 3` (three places).
- *How to make it fast on a big table?* → Index `(dept_id, salary DESC)`; PostgreSQL can then read each partition pre-sorted; for few groups a `LATERAL ... LIMIT N` uses the index directly.

Self-check

1. *Why can't you write `WHERE rank() OVER (...) = 1`?* → Window functions are evaluated after `WHERE`.
2. *Top 2 per group, ties allowed to exceed 2 — which function?* → `rank()` (or `dense_rank()` for distinct values).
3. *How do you keep only the newest row per `user_id`?* → `row_number() OVER (PARTITION BY user_id ORDER BY created_at DESC, id DESC) = 1` in a subquery.

TL;DR

Rank in a CTE, filter outside. `row_number` = exactly N, `rank` = N places with ties, `dense_rank` = N distinct values. `rn = 1` newest-first is the standard dedup. PostgreSQL shortcut for top-1: `DISTINCT ON`.

8.9 Gaps and islands

What it is

An **island** is a run of consecutive values (consecutive days a user logged in, consecutive seat numbers that are free, consecutive readings above a threshold). A **gap** is a missing stretch between islands. "Longest login streak", "find missing invoice numbers" and "group events into sessions" are all gaps-and-islands problems.

How it works — the row-number trick

For a run of consecutive dates, the date goes up by 1 each row and so does `row_number()`. Their **difference is therefore constant within a run** and changes when a day is skipped. That constant identifies the island.

1. Number the rows per user in date order: `rn = row_number() OVER (PARTITION BY user_id ORDER BY login_date)`.
2. Compute `grp = login_date - rn` (a date minus an integer is a date).
3. `GROUP BY user_id, grp`: each group is one island; `min` / `max` / `count` describe it.

```sql
-- code/08_gaps_and_islands.sql
SELECT user_id, login_date,
       row_number() OVER (PARTITION BY user_id ORDER BY login_date) AS rn,
       login_date - row_number() OVER (PARTITION BY user_id ORDER BY login_date)::int AS grp
FROM logins
WHERE user_id = 1
ORDER BY login_date;
```

Output:

```
 user_id | login_date | rn |    grp
---------+------------+----+------------
       1 | 2025-05-01 |  1 | 2025-04-30
       1 | 2025-05-02 |  2 | 2025-04-30
       1 | 2025-05-03 |  3 | 2025-04-30
       1 | 2025-05-06 |  4 | 2025-05-02
       1 | 2025-05-07 |  5 | 2025-05-02
       1 | 2025-05-10 |  6 | 2025-05-04
(6 rows)
```

The value of `grp` has no meaning of its own — it's just a label shared by the rows of one run. (`row_number()` returns `bigint`; the `::int` cast is needed because PostgreSQL defines `date - integer`, not `date - bigint`.)

Each island (streak):

```sql
WITH marked AS (
    SELECT user_id, login_date,
           login_date - row_number() OVER (PARTITION BY user_id ORDER BY login_date)::int AS grp
    FROM logins
)
SELECT user_id, min(login_date) AS streak_start, max(login_date) AS streak_end,
       count(*) AS days
FROM marked
GROUP BY user_id, grp
ORDER BY user_id, streak_start;
```

Output:

```
 user_id | streak_start | streak_end | days
---------+--------------+------------+------
       1 | 2025-05-01   | 2025-05-03 |    3
       1 | 2025-05-06   | 2025-05-07 |    2
       1 | 2025-05-10   | 2025-05-10 |    1
       2 | 2025-05-02   | 2025-05-05 |    4
       2 | 2025-05-09   | 2025-05-10 |    2
(5 rows)
```

Longest streak per user — one more step (`DISTINCT ON` keeps the top row per user):

```sql
WITH marked AS (...same as above...),
streaks AS (
    SELECT user_id, grp, count(*) AS days, min(login_date) AS from_day
    FROM marked GROUP BY user_id, grp
)
SELECT DISTINCT ON (user_id) user_id, days AS longest_streak, from_day
FROM streaks
ORDER BY user_id, days DESC, from_day;
```

Output:

```
 user_id | longest_streak |  from_day
---------+----------------+------------
       1 |              3 | 2025-05-01
       2 |              4 | 2025-05-02
(2 rows)
```

If the data can contain the same date twice for a user, de-duplicate first (or use `dense_rank` instead of `row_number`), otherwise the trick breaks.

Gaps with `lead`. Compare each row with the next one; a difference greater than 1 is a gap:

```sql
SELECT user_id,
       login_date + 1              AS gap_start,
       next_login - 1              AS gap_end,
       next_login - login_date - 1 AS missing_days
FROM (
    SELECT user_id, login_date,
           lead(login_date) OVER (PARTITION BY user_id ORDER BY login_date) AS next_login
    FROM logins
) t
WHERE next_login - login_date > 1
ORDER BY user_id, gap_start;
```

Output:

```
 user_id | gap_start  |  gap_end   | missing_days
---------+------------+------------+--------------
       1 | 2025-05-04 | 2025-05-05 |            2
       1 | 2025-05-08 | 2025-05-09 |            2
       2 | 2025-05-06 | 2025-05-08 |            3
(3 rows)
```

The same query with integer ids finds missing invoice or ticket numbers.

The general method: flag and running sum

The row-number trick needs "consecutive" to mean "+1". For any other rule — events less than 30 minutes apart, temperature above a threshold, same status as the previous row — use two steps:

1. **Flag** the first row of each island: `CASE WHEN <this row does NOT continue the previous one> THEN 1 ELSE 0 END`, using `lag`.
2. **Running sum** of the flags: `sum(flag) OVER (ORDER BY ...)` numbers the islands 1, 2, 3, …

Here: **sessionization** — a new session starts when more than 30 minutes have passed since the previous event:

```sql
WITH ev(user_id, at) AS (VALUES
    (7, TIMESTAMP '2025-05-01 09:00'), (7, '2025-05-01 09:10'), (7, '2025-05-01 09:25'),
    (7, '2025-05-01 11:00'), (7, '2025-05-01 11:05'),
    (7, '2025-05-01 15:30')
), flagged AS (
    SELECT user_id, at,
           CASE WHEN at - lag(at) OVER (PARTITION BY user_id ORDER BY at) <= INTERVAL '30 minutes'
                THEN 0 ELSE 1 END AS new_session
    FROM ev
)
SELECT user_id, at, new_session,
       sum(new_session) OVER (PARTITION BY user_id ORDER BY at) AS session_no
FROM flagged
ORDER BY at;
```

Output:

```
 user_id |         at          | new_session | session_no
---------+---------------------+-------------+------------
       7 | 2025-05-01 09:00:00 |           1 |          1
       7 | 2025-05-01 09:10:00 |           0 |          1
       7 | 2025-05-01 09:25:00 |           0 |          1
       7 | 2025-05-01 11:00:00 |           1 |          2
       7 | 2025-05-01 11:05:00 |           0 |          2
       7 | 2025-05-01 15:30:00 |           1 |          3
(6 rows)
```

The first event has no previous one: `lag` is `NULL`, the comparison is `NULL` (not true), so the `CASE` falls to `ELSE 1` — a new session, as intended. Group by `(user_id, session_no)` to get session start, end and length. (`VALUES` as a CTE creates inline sample rows; the first row's `TIMESTAMP` literal sets the column type.)

Real-world examples

App streaks and contribution graphs, web-analytics sessions (30 minutes of inactivity is a common cut-off), periods when CPU (the processor) load stayed above 90 %, missing cheque or invoice numbers.

In the interview

30-second answer: "For consecutive dates or integers, subtract `row_number()` from the value; the difference is constant within each run, so grouping by it gives the islands with `min`, `max`, `count`. For general rules, flag each row that starts a new island using `lag`, then a running `sum` of the flags gives an island id. Gaps come from `lead(value) - value > 1`."

- *Users who logged in 3+ consecutive days?* → Islands query, then `HAVING count(*) >= 3`, select distinct users.
- *What if a user logs in twice on the same day?* → De-duplicate to one row per day first, or use `dense_rank`.

Self-check

1. *Why is `login_date - row_number()` constant inside a streak?* → Both increase by exactly 1 per row while the days are consecutive.
2. *What does the running sum of "new island" flags give?* → A sequential island number per row.
3. *How do you list missing numbers between consecutive ids?* → `lead(id) - id > 1` → gap from `id + 1` to `lead(id) - 1`.

TL;DR

Islands: `value - row_number()` is a run label → `GROUP BY` it. General rule: flag starts with `lag`, running `sum` of flags = island id. Gaps: `lead(v) - v > 1`.

8.10 Named windows, FILTER, evaluation order and performance

The WINDOW clause

When several functions share a window, define it once with `WINDOW name AS (...)` (after `HAVING`, before `ORDER BY`) and write `OVER name`. A window can also be based on another: `OVER (w ROWS 2 PRECEDING)` adds a frame to named window `w` (the base window must not have its own frame).

```
-- code/09_named_windows_filter_plan.sql
SELECT store, day, amount,
       sum(amount) OVER w_run    AS running_total,
       max(amount) OVER w_run    AS best_so_far,
       rank()      OVER w_amount AS amount_rank
FROM daily_sales
WHERE store = 'Pune'
WINDOW w_run    AS (PARTITION BY store ORDER BY day),
       w_amount AS (PARTITION BY store ORDER BY amount DESC)
ORDER BY day;
```

Output:

```
 store |    day     | amount | running_total | best_so_far | amount_rank
-------+------------+--------+---------------+-------------+-------------
 Pune  | 2025-03-01 | 400.00 |        400.00 |      400.00 |           5
 Pune  | 2025-03-02 | 650.00 |       1050.00 |      650.00 |           2
 Pune  | 2025-03-03 | 500.00 |       1550.00 |      650.00 |           4
 Pune  | 2025-03-05 | 800.00 |       2350.00 |      800.00 |           1
 Pune  | 2025-03-06 | 650.00 |       3000.00 |      800.00 |           2
 Pune  | 2025-03-07 | 300.00 |       3300.00 |      800.00 |           6
(6 rows)
```

`max(...) OVER w_run` is a running maximum ("best day so far"). The `WINDOW` clause is supported by PostgreSQL, MySQL 8, SQLite and SQL Server 2022 (not earlier SQL Server).

FILTER on window aggregates

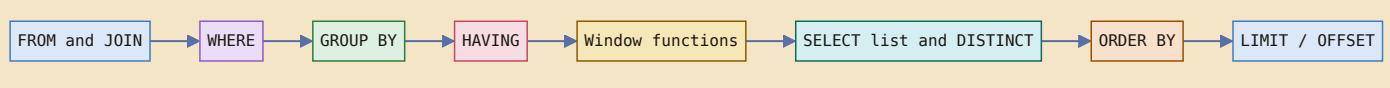
The `FILTER (WHERE ...)` clause from chapter 4 also works on aggregate window functions (PostgreSQL and SQLite; elsewhere use `count(CASE WHEN cond THEN 1 END) OVER (...)`):

```
SELECT day, amount,
       count(*) FILTER (WHERE amount >= 1500) OVER (ORDER BY day) AS big_days_so_far
FROM daily_sales
WHERE store = 'Mumbai'
ORDER BY day;
```

Output:

```
    day     | amount  | big_days_so_far
------------+---------+-----------------
 2025-03-01 | 1200.00 |               0
 2025-03-02 |  900.00 |               0
 2025-03-03 | 1500.00 |               1
 2025-03-04 | 1500.00 |               2
 2025-03-05 |  700.00 |               2
 2025-03-06 | 1100.00 |               2
 2025-03-07 | 1800.00 |               3
(7 rows)
```

Where windows fit in the logical order



Consequences:

- **`WHERE` changes the window.** `rank() OVER (ORDER BY salary DESC)` with `WHERE dept_id = 1` ranks only Engineering (Vikram 1, Neha 2, Rahul and Meera 3). For company-wide ranks shown only for Engineering, rank in a subquery **without** the filter and filter outside.
- Windows **can't** be used in `WHERE`, `GROUP BY` or `HAVING` (they don't exist yet) → wrap in a subquery/CTE (8.8).
- Windows **can** be used in `ORDER BY` (`ORDER BY rank() OVER (...)`) and can take aggregates as arguments in grouped queries (8.7).
- `DISTINCT` runs after windows, so `SELECT DISTINCT dept_id, count(*) OVER (PARTITION BY dept_id)` works (one row per department).
- `LIMIT` runs last, so `row_number() ... LIMIT 5` numbers *all* rows first.

Performance

PostgreSQL evaluates windows with a **WindowAgg** node (one step in the execution plan that `EXPLAIN` prints) that reads rows sorted by the `PARTITION BY` + `ORDER BY` keys:

```
EXPLAIN (COSTS OFF)
SELECT store, day, sum(amount) OVER (PARTITION BY store ORDER BY day)
FROM daily_sales;

EXPLAIN (COSTS OFF)
SELECT first_name, rank() OVER (PARTITION BY dept_id ORDER BY salary DESC)
FROM employees;
```

Output:

```
                       QUERY PLAN
---------------------------------------------------------
 WindowAgg
   ->  Index Scan using daily_sales_pkey on daily_sales
(2 rows)

               QUERY PLAN
----------------------------------------
 WindowAgg
   ->  Sort
         Sort Key: dept_id, salary DESC
         ->  Seq Scan on employees
(4 rows)
```

- In the first plan the primary key `(store, day)` (the columns that uniquely identify each row, which PostgreSQL backs with an index) already delivers rows in the needed order, so there's **no sort**. In the second, PostgreSQL must sort by `dept_id, salary DESC`.
- **Index tip:** an index on `(partition columns, order columns)` can remove the sort — e.g. `CREATE INDEX ON employees (dept_id, salary DESC)` (chapter 12).
- Windows with **different** `PARTITION BY`/`ORDER BY` need separate sorts; windows sharing the same definition are computed in one pass — another reason to reuse a named window.
- Sliding frames with `sum`/`count`/`avg` are computed incrementally (by adjusting the previous row's result rather than starting over), but `min`/`max` over a moving frame (and frames with `EXCLUDE`) may need re-scanning the frame, which gets expensive for large frames.
- A window over millions of rows sorts all of them; filter in `WHERE` first when the logic allows.

Chapter 8 TL;DR

- **Window function** = `f(...) OVER (PARTITION BY … ORDER BY … frame)`: computed per row over related rows; **row count never changes**.
- `PARTITION BY` = groups without collapsing → share of total, difference from group average.
- `ORDER BY` **in** `OVER` makes aggregates running; the default frame `RANGE UNBOUNDED PRECEDING … CURRENT ROW` includes ties — use a unique order or `ROWS`.

- **Frames**: `ROWS` (physical rows), `RANGE` (value distance, intervals, ties included), `GROUPS` (peer groups), `EXCLUDE` — PostgreSQL 11+ for offsets/`GROUPS`/`EXCLUDE`.
- **Ranking**: `row_number` (unique), `rank` (gaps), `dense_rank` (no gaps), `ntile`, `percent_rank`, `cume_dist`. Always decide how ties behave.
- **Value functions**: `lag`/`lead` (ignore frame), `first_value`/`last_value`/`nth_value` (use frame — give `last_value` a full frame). `IGNORE NULLS` only from PostgreSQL 19.
- **Patterns**: running total, moving average, growth vs `lag`, top-N per group (rank in CTE, filter outside), dedup (`row_number() = 1`), gaps & islands (`value − row_number()`, or flag + running sum).
- **Order of evaluation**: after `WHERE`/`GROUP BY`/`HAVING`, before `DISTINCT`/`ORDER BY`/`LIMIT`. No `QUALIFY` in PostgreSQL.
- **Performance**: one sort per distinct window; an index on (partition, order) columns can remove it.

Next: chapter 9 — DDL, DML and constraints: creating and changing tables, keys and constraints, and inserting, updating and deleting data safely.

Chapter 9 — DDL, DML and Constraints: Building Tables and Changing Data Safely

So far the book has only *read* data. This chapter is about creating the structures that hold data and changing what is in them:

- **DDL — Data Definition Language**: statements that define structure: `CREATE`, `ALTER`, `DROP`, `TRUNCATE` (and `CREATE INDEX`).
- **Keys and constraints**: rules attached to tables — primary and foreign keys, `UNIQUE`, `CHECK`, `NOT NULL`, `DEFAULT` — enforced on every change, whichever application writes.
- **DML — Data Manipulation Language**: statements that change rows: `INSERT`, `UPDATE`, `DELETE`, plus PostgreSQL's "insert or update" (`ON CONFLICT`) and `RETURNING`.
- **Backup and restore**: copying a database and getting data back after a mistake — run for real with `pg_dump` and `pg_restore` (PostgreSQL's command-line programs that write a database out to a file and load it back in).

Designing tables (entities, relationships, normal forms — sets of rules for keeping each fact in only one place) is chapter 10 and chapter 11; here we focus on the statements.

What you'll learn

#	Topic	You will be able to…
9.1	Data types	pick the right type for numbers, text, dates, booleans, and know what each rejects
9.2	`CREATE`	create databases, schemas, tables (also from queries and as copies), temporary tables
9.3	Keys and constraints	explain super/candidate/primary/alternate/foreign/composite/unique keys and use every constraint, including `ON DELETE` actions
9.4	Generated keys	use identity columns, sequences, `serial`, UUIDs and generated columns
9.5	`ALTER`	add/rename/drop columns, change types, add constraints safely on live tables
9.6	`DROP` and `TRUNCATE`	remove tables or empty them, and compare with `DELETE`
9.7	Indexes (brief)	create/drop indexes; clustered vs non-clustered
9.8	`INSERT`	insert single/multiple rows, defaults, `INSERT … SELECT`, `RETURNING`
9.9	`UPDATE`	update with conditions, expressions, `CASE`, other tables
9.10	`DELETE`	delete with conditions, other tables, soft deletes
9.11	Upsert and `MERGE`	`INSERT … ON CONFLICT`, and what `MERGE` (PostgreSQL 15+) looks like
9.12	Backup and restore	logical vs physical backups; `pg_dump` / `pg_restore`; other databases' commands

Setup. Examples run on the `company` sample database (chapter 1, section 1.11); each file in `code/` starts from a fresh copy and creates any extra tables it needs. Errors are shown on purpose — each `ERROR:` in an output block is the database refusing something.

9.1 Data types

What it is

Every column has a **data type** saying what kind of value it holds (whole number, decimal, text, date…). The type decides storage, which operations work, how values sort and — just as important — which values

are **rejected**. A type is the first and cheapest constraint.

The common types

Kind	PostgreSQL type	Holds	Use for	MySQL / SQL Server notes
Integer	`smallint` (2 bytes)	−32 768 … 32 767	small ranges: ranks, ratings	same names
	`integer` / `int` (4 bytes)	about ±2.1 billion	counts, most ids	same
	`bigint` (8 bytes)	about ±9.2 × 10^{18}	large ids, populations, byte counts	same
Exact decimal	`numeric(p, s)` / `decimal(p, s)`	*p* digits in total, *s* after the point	**money**, anything that must add up exactly	same
Approximate	`real`, `double precision`	binary floating point (fast but approximate)	measurements, science, ML (machine-learning) features	`FLOAT` / `DOUBLE`; SQL Server `float`
Fixed text	`char(n)`	exactly *n* characters, padded with spaces	fixed codes (rarely worth it)	same
Variable text	`varchar(n)`	up to *n* characters	text with a real maximum length	SQL Server: `nvarchar(n)` for Unicode (text in any alphabet)
	`text`	any length	most text in PostgreSQL	MySQL `TEXT` has limits and index restrictions
Date / time	`date`	a calendar day	birthdays, due dates	same
	`time`	time of day	opening hours	same
	`timestamp`	date + time, **no** time zone	local "wall clock" times	MySQL `DATETIME`
	`timestamptz`	an exact instant (stored in UTC — the world's reference clock — shown in the session's time zone; a *session* is your current connection)	event times, created_at	MySQL `TIMESTAMP` (converted to UTC); SQL Server `datetimeoffset`
Year only	(none) — use `smallint` + `CHECK`, or `date`			MySQL has `YEAR`
Boolean	`boolean`	true / false / NULL	flags	MySQL `BOOLEAN` = `TINYINT(1)`; SQL Server `bit`
Other useful	`uuid`, `jsonb`, arrays, `inet`, ranges		see chapter 14	

More on type conversion, casts and functions per type is in chapter 3.

Seeing the rules in action:

```sql
-- code/01_data_types.sql
SET TimeZone = 'UTC';   -- make timestamptz output the same on every machine
CREATE TABLE type_demo (
    qty        integer,
    views      bigint,
    rank_no    smallint,
    price      numeric(10,2),
    ratio      double precision,
    code       char(5),
    name       varchar(10),
    notes      text,
    born_on    date,
    starts_at  time,
    created    timestamp,
    created_tz timestamptz,
    active     boolean,
    birth_year smallint CHECK (birth_year BETWEEN 1900 AND 2100)  -- no YEAR type in PostgreSQL
);
INSERT INTO type_demo VALUES
 (3, 9000000000, 7, 19.999, 0.1::float8 + 0.2::float8, 'AB', 'Tara', 'long text ...',
  '1994-09-22', '09:30', '2025-06-01 10:00', '2025-06-01 10:00+05:30', true, 1994);

SELECT price, ratio, '[' || code || ']' AS code_padded, length(code) AS len_code,
       octet_length(code) AS bytes_code, created, created_tz
FROM type_demo;
```

Output:

```
 price |        ratio        | code_padded | len_code | bytes_code |       created       |       created_tz
-------+---------------------+-------------+----------+------------+---------------------+------------------------
 20.00 | 0.30000000000000004 | [AB]        |        2 |          5 | 2025-06-01 10:00:00 | 2025-06-01 04:30:00+00
(1 row)
```

What happened:

- `price` 19.999 was **rounded** to 2 decimal places → 20.00.
- `ratio`: 0.1 + 0.2 in binary floating point is not exactly 0.3 (`0.1::float8` is PostgreSQL's shorthand for a **cast** — converting a value to another type, here an 8-byte float) (the file also shows `0.1::numeric + 0.2::numeric = 0.3` is true, the `float8` version false). That's why money is never stored in `float`.
- `code` is stored padded to 5 characters (5 bytes), but PostgreSQL ignores a `char`'s trailing spaces, so `length` says 2 — confusing; prefer `text` / `varchar`.
- `created_tz`: 10:00 in India (+05:30) was stored as an instant and shown in the session time zone (UTC) as 04:30. **What the types reject:**

```sql
INSERT INTO type_demo (name) VALUES ('Christopher Columbus');
INSERT INTO type_demo (rank_no) VALUES (40000);
INSERT INTO type_demo (price) VALUES (123456789.00);
INSERT INTO type_demo (born_on) VALUES ('2025-02-30');
INSERT INTO type_demo (birth_year) VALUES (1850);
```

Output:

```
ERROR:  value too long for type character varying(10)
ERROR:  smallint out of range
ERROR:  numeric field overflow
DETAIL:  A field with precision 10, scale 2 must round to an absolute value less than 10^8.
ERROR:  date/time field value out of range: "2025-02-30"
ERROR:  new row for relation "type_demo" violates check constraint "type_demo_birth_year_check"
...
```

`numeric(10,2)` allows 10 digits of which 2 are after the point, so at most 8 before it ($< 10^8$). MySQL in non-strict mode (a lenient setting that turns many errors into warnings) would silently truncate some of these values instead of failing; PostgreSQL always errors.

How to choose

1. **Money** → `numeric(p, s)` (or integer cents), never `float`. **Ids/counts** → `integer`, or `bigint` if it could ever pass 2 billion.
2. **Text** → `text` in PostgreSQL; `varchar(n)` only when the maximum is a real business rule (no speed difference). SQL Server: `nvarchar` for Unicode.
3. **Instants** → `timestamptz`; `timestamp` only for wall-clock times that must not shift (a shop's opening time). **Days** → `date`.
4. Pick the smallest type that safely fits — changing it later on a big table is expensive (9.5).

Common mistakes

Money in `float` (rounding errors); dates as `text` (`'03/04/2025'` — March or April? can't sort); `varchar(255)` everywhere by habit (a meaningless limit); `timestamp` for events across time zones (ambiguous instants).

9.2 CREATE: databases, schemas and tables

What it is

`CREATE` makes new database objects. The ones you'll use most:

Statement	Creates
`CREATE DATABASE name;`	a new, separate database on the server (the running database program)
`CREATE SCHEMA name;`	a namespace **inside** a database (like a folder); tables are then `schema.table`
`CREATE TABLE name (...);`	a table: columns, types, constraints
`CREATE TABLE name AS SELECT ...;`	a table filled with a query's result
`CREATE TABLE name (LIKE other INCLUDING ALL);`	an empty copy of another table's structure
`CREATE TEMP TABLE name (...);`	a table visible only to your session, dropped automatically at the end
`CREATE VIEW`, `CREATE INDEX`, `CREATE SEQUENCE`, `CREATE FUNCTION`…	covered in chapters 7, 9.7, 9.4, 14

How CREATE TABLE works

```
CREATE TABLE [IF NOT EXISTS] [schema.]table_name (
    column_name  data_type  [column constraints],   -- one line per column
    ...,
    [table constraints]                             -- e.g. PRIMARY KEY (a, b)
);
```

Square brackets in a syntax outline like this mark optional parts — you don't type the brackets.

1. The name must be unique within its schema (default `public`); each column gets a name, a type and optionally constraints (9.3) and a `DEFAULT`.
2. Multi-column constraints go after the columns.
3. `IF NOT EXISTS` turns "already exists" into a notice (an informational message, not an error) — handy in setup scripts, but it does **not** check that the existing table has the same columns.

```sql
-- code/02_create.sql
CREATE DATABASE school_demo;          -- a new database (then dropped again in the script)
CREATE SCHEMA school;

CREATE TABLE school.students (
    student_id   integer      PRIMARY KEY,
    first_name   varchar(50)  NOT NULL,
    last_name    varchar(50)  NOT NULL,
    roll_number  integer      UNIQUE,
    email        text,
    enrolled_on  date         NOT NULL DEFAULT current_date
);

CREATE TABLE IF NOT EXISTS school.students (student_id integer);

CREATE TABLE school.bad (
    id   integer PRIMARY KEY,
    name text,
);
```

Output:

```
NOTICE:  relation "students" already exists, skipping
ERROR:  syntax error at or near ")"
LINE 4: );
```

A comma after the last column is a syntax error (PostgreSQL, MySQL, SQL Server) — a common typo after deleting the last line.

Tables from queries and copies:

```sql
CREATE TABLE school.sales_staff AS
SELECT emp_id, first_name, salary FROM employees WHERE dept_id = 2;
SELECT * FROM school.sales_staff ORDER BY emp_id;
```

Output:

```
 emp_id | first_name |  salary
--------+------------+-----------
      6 | Daniel     | 150000.00
      7 | Priya      |  72000.00
 ...
(4 rows)
```

`CREATE TABLE … AS` copies column names, types and data, but **no constraints, defaults or indexes**. (`SELECT … INTO new_table FROM …` does the same in PostgreSQL and SQL Server.) To copy only the structure, with constraints and defaults, use `LIKE`:

```sql
CREATE TABLE school.products_copy (LIKE products INCLUDING ALL);   -- 0 rows, same columns/constraints
```

Temporary tables (`CREATE TEMP TABLE scratch (id int, note text);`) live in a private per-session schema (the file's catalog query — the **catalog** is PostgreSQL's set of internal tables describing every object — shows a `pg_temp_N` schema and `relpersistence = 't'` instead of `'p'`) and vanish when the session ends — or at transaction end with `ON COMMIT DROP`. Use them for multi-step processing inside one session (7.9).

> **Key term — transaction:** a group of statements the database treats as one unit: either all of them take effect or none do. You start one with `BEGIN`, make it permanent with `COMMIT`, or undo everything since `BEGIN` with `ROLLBACK`. Without `BEGIN`, PostgreSQL wraps each statement in its own tiny transaction. Transactions are covered in depth in chapter 13.

Dialect notes

	PostgreSQL	MySQL	SQL Server	SQLite
Database vs schema	database contains schemas	"schema" = "database" (synonyms)	database contains schemas (`dbo` default)	one file = one database
Switch database	reconnect (`\c db` in psql, PostgreSQL's command-line client)	`USE db;`	`USE db;`	open another file / `ATTACH`
`CREATE TABLE IF NOT EXISTS`	yes	yes	no (check `OBJECT_ID` first)	yes
Temporary table	`CREATE TEMP TABLE`	`CREATE TEMPORARY TABLE`	`CREATE TABLE #name`	`CREATE TEMP TABLE`
`CREATE DATABASE` inside a transaction	not allowed	implicit commit (silently ends the open transaction first)	not allowed	n/a

---

9.3 Keys and constraints

What they are

A **key** is a column (or set of columns) whose values identify rows. A **constraint** is a rule the database checks on every write; a statement that would break it fails completely. Applications have bugs and there is usually more than one of them, but every write passes through the database — so that is where the rules belong.

The kinds of keys

Think of a `members` table with `member_id`, `email`, `phone`, `full_name`:

Key	Definition	In `members`
Super key	any column set that is unique for every row (may include unnecessary columns)	`{member_id}`, `{member_id, full_name}`, `{email, phone}`, …
Candidate key	a *minimal* super key — drop any column and it stops being unique; a table can have several	`{member_id}`, `{email}`
Primary key (PK)	the candidate key chosen as main identifier: unique **and** `NOT NULL`, one per table	`member_id`
Alternate key	a candidate key that wasn't chosen as primary; enforced with `UNIQUE`	`email`
Unique key	columns declared `UNIQUE`: no duplicates, NULLs allowed (below); many per table	`email`, `phone`
Composite key	two or more columns; only the *combination* must be unique	`(club_id, member_id)` in `club_memberships`
Foreign key (FK)	column(s) whose values must match a key (usually the PK) of another (or the same) table	`club_memberships.member_id → members.member_id`
Natural vs surrogate	natural = a real-world value (email, ISBN); surrogate = a generated number/UUID (9.4)	`email` is natural, `member_id` is surrogate

Keys and their theory (functional dependencies) are developed further in chapter 10 and chapter 11.

The constraints

Constraint	Rule	Written as
`NOT NULL`	a value is required	`col type NOT NULL`
`DEFAULT`	value used when an `INSERT` omits the column (not a check, but declared alongside)	`col type DEFAULT expr`
`UNIQUE`	no two rows share the value(s); NULLs don't count as equal	`col type UNIQUE` / `UNIQUE (a, b)`
`PRIMARY KEY`	`UNIQUE` + `NOT NULL`; one per table; creates a unique index (a lookup structure, 9.7)	`col type PRIMARY KEY` / `PRIMARY KEY (a, b)`
`CHECK`	a boolean expression must not be false	`CHECK (price >= 0)` / `CHECK (end_date >= start_date)`
`FOREIGN KEY`	value must exist in the referenced key (or be NULL)	`col type REFERENCES other(col)` / `FOREIGN KEY (a) REFERENCES other(x)`
`EXCLUDE` (PostgreSQL)	no two rows may "conflict" by an operator (e.g. overlapping bookings of one room)	see chapter 14

A **column constraint** is written on the column; a **table constraint** after the columns (required for multi-column rules). Name important ones with `CONSTRAINT name …` — the name appears in errors and is needed to drop it; otherwise PostgreSQL generates names like `members_email_key`.

```
-- code/03_keys_and_constraints.sql
CREATE TABLE members (
    member_id  integer PRIMARY KEY,                       -- primary key
    email      text    NOT NULL UNIQUE,                   -- candidate key -> alternate key
    phone      text    UNIQUE,                            -- unique key, NULL allowed
    full_name  text    NOT NULL,
    plan       text    NOT NULL DEFAULT 'free'
                       CONSTRAINT plan_is_known CHECK (plan IN ('free', 'pro', 'team')),
    joined_on  date    NOT NULL DEFAULT current_date,
    left_on    date,
    CONSTRAINT left_after_joined CHECK (left_on IS NULL OR left_on >= joined_on)   -- table-level check
);

INSERT INTO members (member_id, email, phone, full_name, joined_on)
VALUES (1, 'ravi@mail.example', NULL, 'Ravi Kumar', '2025-01-10'),
       (2, 'lena@mail.example', NULL, 'Lena Ortiz', '2025-02-01');     -- two NULL phones: fine
SELECT member_id, email, phone, plan FROM members ORDER BY member_id;
```

Output:

```
 member_id |       email       | phone  | plan
-----------+-------------------+--------+------
         1 | ravi@mail.example | (null) | free
         2 | lena@mail.example | (null) | free
(2 rows)
```

`plan` was filled from its `DEFAULT`. Now each rule refusing bad data:

```
INSERT INTO members (member_id, email, full_name) VALUES (1, 'x@mail.example', 'Dup Id');
INSERT INTO members (member_id, email, full_name, joined_on) VALUES (NULL, 'y@mail.example', 'No Id', '2025-03-01');
INSERT INTO members (member_id, email, full_name) VALUES (3, 'ravi@mail.example', 'Dup');
INSERT INTO members (member_id, email, full_name, plan, joined_on) VALUES (4, 'z@mail.example', 'Z', 'gold', '2025-03-01');
INSERT INTO members (member_id, email, full_name, joined_on, left_on)
VALUES (5, 'w@mail.example', 'W', '2025-03-01', '2025-01-01');
```

Output:

```
ERROR:  duplicate key value violates unique constraint "members_pkey"
DETAIL:  Key (member_id)=(1) already exists.
ERROR:  null value in column "member_id" of relation "members" violates not-null constraint
...
ERROR:  duplicate key value violates unique constraint "members_email_key"
...
ERROR:  new row for relation "members" violates check constraint "plan_is_known"
...
ERROR:  new row for relation "members" violates check constraint "left_after_joined"
...
```

Two subtle rules:

- **`CHECK` passes when its expression is `NULL`.** A check fails only when the expression is *false*; `NULL` ("unknown") is accepted. That's why `left_after_joined` explicitly allows `left_on IS NULL`, and why a column that must be present also needs `NOT NULL`. (Three-valued logic — every condition is true, false or unknown: chapter 2.)
- **`UNIQUE` allows several `NULL`s** in PostgreSQL, MySQL, SQLite and Oracle, because NULL is not equal to NULL. SQL Server allows only **one** NULL in a unique constraint (use a filtered unique index — an index over only the rows matching a `WHERE` — for "unique when present"). PostgreSQL 15 added `UNIQUE NULLS NOT DISTINCT` to treat NULLs as equal (PostgreSQL 15 — not run).

Foreign keys and referential actions

A foreign key keeps **referential integrity**: a child row can't point to a parent that doesn't exist. The interesting question is what happens when the *parent* is deleted or its key changes. You choose with `ON DELETE` / `ON UPDATE`:

Action	When the referenced parent row is deleted (or its key updated)
`NO ACTION` (default)	error if children still reference it (checked at the end of the statement; can be deferred)
`RESTRICT`	error, checked immediately (can't be deferred)
`CASCADE`	delete (or update) the child rows too
`SET NULL`	set the child's FK column(s) to NULL
`SET DEFAULT`	set the child's FK column(s) to their default

```
CREATE TABLE clubs (
    club_id  integer PRIMARY KEY,
    name     text NOT NULL UNIQUE
);
CREATE TABLE club_memberships (
    club_id    integer REFERENCES clubs(club_id)     ON DELETE CASCADE,
    member_id  integer REFERENCES members(member_id) ON DELETE RESTRICT,
    role       text NOT NULL DEFAULT 'member',
    PRIMARY KEY (club_id, member_id)                                  -- composite key
);
CREATE TABLE club_events (
    event_id   integer PRIMARY KEY,
    club_id    integer REFERENCES clubs ON DELETE SET NULL,           -- "REFERENCES clubs" = its PK
    title      text NOT NULL
);
INSERT INTO clubs VALUES (10, 'Chess'), (20, 'Hiking');
INSERT INTO club_memberships (club_id, member_id) VALUES (10, 1), (10, 2), (20, 1);
INSERT INTO club_events VALUES (100, 10, 'Blitz night'), (101, 20, 'Sunrise trek');

INSERT INTO club_memberships (club_id, member_id) VALUES (10, 1);       -- composite PK duplicate
INSERT INTO club_memberships (club_id, member_id) VALUES (30, 1);       -- no club 30
DELETE FROM members WHERE member_id = 2;                                  -- RESTRICT
```

Output:

```
ERROR:  duplicate key value violates unique constraint "club_memberships_pkey"
...
ERROR:  insert or update on table "club_memberships" violates foreign key constraint
"club_memberships_club_id_fkey"
DETAIL:  Key (club_id)=(30) is not present in table "clubs".
ERROR:  update or delete on table "members" violates foreign key constraint "club_memberships_member_id_fkey" on
table "club_memberships"
DETAIL:  Key (member_id)=(2) is still referenced from table "club_memberships".
```

Deleting the Chess club triggers both `CASCADE` (its memberships go) and `SET NULL` (its event stays, orphaned on purpose):

```
DELETE FROM clubs WHERE club_id = 10;
SELECT * FROM club_memberships ORDER BY club_id, member_id;
SELECT * FROM club_events ORDER BY event_id;
```

Output:

```
 club_id | member_id |  role
---------+-----------+--------
      20 |         1 | member
(1 row)

 event_id | club_id |    title
----------+---------+--------------
      100 |  (null) | Blitz night
      101 |      20 | Sunrise trek
(2 rows)
```

Choosing an action: `CASCADE` for rows that are *part of* the parent (order lines, memberships); `RESTRICT` / `NO ACTION` for things that must not vanish silently (a customer with orders); `SET NULL` when the child should survive without the link.

Every constraint is recorded in the `pg_constraint` catalog (`contype` `p` = primary key, `u` = unique, `f` = foreign key, `c` = check); the file lists the eight created here.

Deferrable constraints. Constraints are normally checked after each statement; `DEFERRABLE INITIALLY DEFERRED` postpones the check to `COMMIT`, allowing temporarily inconsistent states — e.g. two rows that reference each other:

```
CREATE TABLE people (
    person_id  integer PRIMARY KEY,
    partner_id integer REFERENCES people DEFERRABLE INITIALLY DEFERRED
);
BEGIN;
INSERT INTO people VALUES (1, 2);   -- person 2 doesn't exist yet: allowed for now
INSERT INTO people VALUES (2, 1);
COMMIT;                             -- checked here: both exist
```

Both rows are committed. (`UNIQUE`, `PRIMARY KEY`, `REFERENCES` and `EXCLUDE` can be deferrable in PostgreSQL; `CHECK` and `NOT NULL` cannot.)

Dialect notes

InnoDB is MySQL's default **storage engine** (the component that actually stores and reads rows); SQLite's `PRAGMA` is its command for changing settings.

	PostgreSQL	MySQL (InnoDB)	SQL Server	SQLite
`CHECK` enforced	yes	since 8.0.16 (parsed but ignored before)	yes	yes
FK enforced	yes	yes	yes	only after `PRAGMA foreign_keys = ON`
Index auto-created on FK column	**no** — create one yourself	yes	no	no
Multiple NULLs in `UNIQUE`	yes	yes	no (one)	yes
Deferrable constraints	yes	no	no	FKs only

When not to rely on constraints alone

Constraints check single rows (and FK existence). Multi-row rules ("at most 5 members per club") need triggers (functions the database runs automatically when rows are inserted, updated or deleted), `EXCLUDE` constraints or careful transactions (chapter 13, chapter 14).

Common mistakes

- **No index on the FK column**: in PostgreSQL each parent delete then scans the whole child table.
- `ON DELETE CASCADE` **everywhere**: one delete can wipe out large parts of the database.
- **Forgetting** `NOT NULL` next to `CHECK` or `UNIQUE` when a value is required.
- **A changeable natural key** (email, phone) as the PK — every FK must change with it. Use a surrogate PK plus `UNIQUE` on the natural key.

In the interview

30-second answer: "A candidate key is a minimal set of columns that identifies a row; the primary key is the chosen one — unique, not null, one per table; the rest are alternate keys enforced with `UNIQUE` (NULLs allowed). A composite key combines columns. A foreign key references another table's key, and `ON DELETE CASCADE / SET NULL / RESTRICT` decides what happens to children. `CHECK`, `NOT NULL` and `DEFAULT` complete the set."

- *Primary key vs unique key?* → PK: one per table, never NULL, the row's identity (often the clustered index — the one that stores the rows themselves in key order, 9.7 — in SQL Server/MySQL). UNIQUE: many per table, NULLs allowed.
- *Natural or surrogate PK?* → Usually a surrogate (`bigint` identity or UUID) for stability, plus `UNIQUE` on the natural key to keep the business rule.

Self-check

1. *Can a table have two primary keys?* → No; one primary key (possibly composite), any number of unique keys.
2. *Does* `CHECK (qty > 0)` *reject NULL?* → No; add `NOT NULL`.
3. *Does PostgreSQL index a foreign-key column automatically?* → No.
4. *Is* `{member_id, email}` *a candidate key?* → No — it's a super key but not minimal (`member_id` alone is unique).

9.4 Generated keys: identity, sequences, serial, UUID, generated columns

What it is

Most tables use a generated **surrogate key**. PostgreSQL generates numbers with a **sequence** — an object that hands out increasing values (`nextval`); an **identity column** (SQL standard, PostgreSQL 10+) is a column tied to its own sequence.

Syntax	Meaning
`GENERATED ALWAYS AS IDENTITY`	database always generates the value; supplying one is an error (unless `OVERRIDING SYSTEM VALUE`)
`GENERATED BY DEFAULT AS IDENTITY`	generated when you don't supply one; you may supply your own
`serial` / `bigserial`	older PostgreSQL shorthand: integer column + sequence + `DEFAULT nextval(...)`
`uuid DEFAULT gen_random_uuid()`	random 128-bit id — a UUID, *universally unique identifier* (built in since PostgreSQL 13)
`GENERATED ALWAYS AS (expr) STORED`	a **generated column**: computed from other columns on every write (PostgreSQL 12+)

```
-- code/04_identity_and_sequences.sql
CREATE TABLE tickets (
    ticket_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    subject    text NOT NULL,
    price      numeric(8,2) NOT NULL,
    qty        integer NOT NULL DEFAULT 1,
    total      numeric(10,2) GENERATED ALWAYS AS (price * qty) STORED
);
INSERT INTO tickets (subject, price) VALUES ('Printer jam', 10);
INSERT INTO tickets (subject, price, qty) VALUES ('VPN access', 5, 3), ('New laptop', 1200, 1)
RETURNING ticket_id, total;
```

Output:

```
 ticket_id |  total
-----------+---------
         2 |   15.00
         3 | 1200.00
(2 rows)
```

```
INSERT INTO tickets (ticket_id, subject, price) VALUES (99, 'Manual id', 1);
INSERT INTO tickets (subject, price, total) VALUES ('Fake total', 1, 500);
```

Output:

```
ERROR:  cannot insert a non-DEFAULT value into column "ticket_id"
DETAIL:  Column "ticket_id" is an identity column defined as GENERATED ALWAYS.
HINT:  Use OVERRIDING SYSTEM VALUE to override.
ERROR:  cannot insert a non-DEFAULT value into column "total"
DETAIL:  Column "total" is a generated column.
```

Sequences never go backwards — gaps are normal:

```
INSERT INTO tickets (ticket_id, subject, price) OVERRIDING SYSTEM VALUE VALUES (99, 'Manual id', 1);
BEGIN;
INSERT INTO tickets (subject, price) VALUES ('Will be rolled back', 1);
ROLLBACK;
INSERT INTO tickets (subject, price) VALUES ('After rollback', 2);
SELECT ticket_id, subject, total FROM tickets ORDER BY ticket_id;
```

Output:

```
 ticket_id |    subject     |  total
-----------+----------------+---------
         1 | Printer jam    |   10.00
 ...
         3 | New laptop     | 1200.00
         5 | After rollback |    2.00
        99 | Manual id      |    1.00
(5 rows)
```

Number 4 went to the rolled-back insert and is never reused: sequences are deliberately **not** transactional (a rollback doesn't undo them), so concurrent inserts (inserts running at the same moment in different sessions) never wait for each other. Never assume ids are gap-free (for legal invoice numbers, generate them separately under a lock — a **lock** is a claim on a row or table that makes other sessions wait until you finish).

Why identity beats `serial`: `serial` is just a default, so a manual id doesn't advance the sequence and a later generated one collides:

```
CREATE TABLE legacy (id serial PRIMARY KEY, label text);
INSERT INTO legacy (label) VALUES ('a'), ('b');       -- ids 1, 2
INSERT INTO legacy (id, label) VALUES (3, 'manual');  -- allowed
INSERT INTO legacy (label) VALUES ('c');              -- sequence gives 3 -> collision
```

Output:

```
ERROR:  duplicate key value violates unique constraint "legacy_pkey"
DETAIL:  Key (id)=(3) already exists.
```

`GENERATED ALWAYS` refuses the manual insert up front. (After loading your own ids, resync: `SELECT setval('legacy_id_seq', (SELECT max(id) FROM legacy));`.)

A standalone sequence, e.g. for invoice numbers shared by several tables:

```
CREATE SEQUENCE invoice_no START 1000 INCREMENT 10;
SELECT nextval('invoice_no') AS first, nextval('invoice_no') AS second, currval('invoice_no') AS current;
-- 1000 | 1010 | 1010
```

`currval` returns the last value *this session* got from `nextval`.

UUIDs: `key_id uuid PRIMARY KEY DEFAULT gen_random_uuid()` gives 128-bit ids (36 characters as text). They can be generated anywhere (even in the app) without coordination and don't reveal row counts, but take 16 bytes (vs 8 for `bigint`) and random values scatter index inserts. Time-ordered UUIDv7 (a UUID whose first bits are a timestamp, so new ids sort roughly by creation time) reduces that; PostgreSQL 18 adds a built-in `uuidv7()` (not run here).

Dialect notes

	Auto-increment syntax
PostgreSQL	`GENERATED {ALWAYS | BY DEFAULT} AS IDENTITY` (10+), `serial` (legacy)
MySQL	`AUTO_INCREMENT`
SQL Server	`IDENTITY(1,1)`; `SEQUENCE` objects since 2012
SQLite	`INTEGER PRIMARY KEY` (alias of the rowid, SQLite's hidden row number); `AUTOINCREMENT` prevents reuse of deleted ids
Oracle	identity columns since 12c; sequences

Generated columns: PostgreSQL 12–17 `STORED` only; PostgreSQL 18 adds `VIRTUAL` (computed on read) and makes it the default when neither keyword is given (not run); MySQL 5.7+ supports both; SQL Server has "computed columns" (optionally `PERSISTED`).

9.5 ALTER: changing existing tables

What it is

`ALTER TABLE` changes an existing table's structure — usually with data in it — without recreating it.

Task	PostgreSQL
Add a column	`ALTER TABLE t ADD COLUMN c type [DEFAULT …] [NOT NULL];`
Drop a column	`ALTER TABLE t DROP COLUMN c;`
Rename a column / table	`ALTER TABLE t RENAME COLUMN a TO b;` / `ALTER TABLE t RENAME TO u;`
Change a type	`ALTER TABLE t ALTER COLUMN c TYPE newtype [USING expr];`
Set / drop a default	`ALTER TABLE t ALTER COLUMN c SET DEFAULT …;` / `DROP DEFAULT`
Set / drop NOT NULL	`ALTER TABLE t ALTER COLUMN c SET NOT NULL;` / `DROP NOT NULL`
Add / drop a constraint	`ALTER TABLE t ADD CONSTRAINT name …;` / `DROP CONSTRAINT name;`

How it works (with real data)

```sql
-- code/05_alter.sql
CREATE TABLE staff (id integer PRIMARY KEY, first_name text, last_name text);
INSERT INTO staff VALUES (1, 'Tara', 'Mendes'), (2, 'Omar', 'Mendes'), (3, 'Yuki', 'Tanaka');

ALTER TABLE staff ADD COLUMN salary integer;
ALTER TABLE staff ADD COLUMN country text NOT NULL DEFAULT 'US';
SELECT * FROM staff ORDER BY id;
```

Output:

```
 id | first_name | last_name | salary | country
----+------------+-----------+--------+---------
  1 | Tara       | Mendes    | (null) | US
  2 | Omar       | Mendes    | (null) | US
  3 | Yuki       | Tanaka    | (null) | US
(3 rows)
```

Existing rows get NULL, or the default if there is one. Since PostgreSQL 11, adding a column with a constant default is instant (no table rewrite — copying every row into a new file).

```sql
ALTER TABLE staff RENAME COLUMN first_name TO given_name;
ALTER TABLE staff RENAME TO team;

UPDATE team SET salary = 50000 + id * 1000;
ALTER TABLE team ALTER COLUMN salary TYPE numeric(10,2) USING salary::numeric(10,2);
ALTER TABLE team ALTER COLUMN last_name TYPE integer;                -- text -> integer?
```

Output:

```
ERROR:  column "last_name" cannot be cast automatically to type integer
HINT:  You might need to specify "USING last_name::integer".
```

`USING` tells PostgreSQL how to compute the new value from the old one; without it only automatic (implicit/assignment) casts — the conversions PostgreSQL considers safe to apply on its own — are allowed.

Adding `NOT NULL` to a column that has NULLs fails — `ALTER TABLE team ADD COLUMN email text NOT NULL;` gives *ERROR: column "email" of relation "team" contains null values*. Add, backfill (fill in values for the existing rows), then constrain:

```
ALTER TABLE team ADD COLUMN email text;
UPDATE team SET email = lower(given_name) || '@corp.example';
ALTER TABLE team ALTER COLUMN email SET NOT NULL;
```

Constraints on live tables: `NOT VALID` then `VALIDATE`. Adding a `CHECK` or FK normally scans the whole table under a strong lock. On a big table, add it `NOT VALID` (enforced for new and changed rows only), fix old data, then `VALIDATE CONSTRAINT` (scans under a lighter lock):

```
ALTER TABLE team ADD CONSTRAINT team_email_uq UNIQUE (email);
INSERT INTO team (id, given_name, last_name, salary, email) VALUES (4, 'Leo', 'Brandt', -5, 'leo@corp.example');
ALTER TABLE team ADD CONSTRAINT salary_positive CHECK (salary > 0);           -- fails: Leo's row
ALTER TABLE team ADD CONSTRAINT salary_positive CHECK (salary > 0) NOT VALID;  -- accepted
ALTER TABLE team VALIDATE CONSTRAINT salary_positive;                         -- fails until fixed
UPDATE team SET salary = 48000 WHERE id = 4;
ALTER TABLE team VALIDATE CONSTRAINT salary_positive;                         -- succeeds
```

Output:

```
ERROR:  check constraint "salary_positive" of relation "team" is violated by some row
ERROR:  check constraint "salary_positive" of relation "team" is violated by some row
```

(`UNIQUE`/`PRIMARY KEY` can't be `NOT VALID`; build a unique index `CONCURRENTLY` (without blocking writes, 9.7) and then `ADD CONSTRAINT … UNIQUE USING INDEX` instead.)

Several changes in one statement (one lock, one pass):

```
ALTER TABLE team
    DROP COLUMN country,
    ADD COLUMN hired_on date,
    ALTER COLUMN last_name SET NOT NULL;
SELECT conname, pg_get_constraintdef(oid) AS definition
FROM pg_constraint WHERE conrelid = 'team'::regclass ORDER BY conname;
```

Output:

```
     conname     |           definition
-----------------+--------------------------------
 salary_positive | CHECK ((salary > (0)::numeric))
 staff_pkey      | PRIMARY KEY (id)
 team_email_uq   | UNIQUE (email)
(3 rows)
```

Note the primary key is still called `staff_pkey`: renaming a table doesn't rename its constraints or indexes.

Locks and production safety

Most `ALTER TABLE` forms take an **ACCESS EXCLUSIVE** lock — no reads or writes until it finishes. Metadata-only changes (edits to the table's description in the catalog that don't touch its rows: nullable column, constant default, rename, drop column — which only hides it) hold it briefly; **type changes that rewrite the table** can hold it for minutes. Production (the live system real users depend on) practice: set `lock_timeout` (so a statement gives up instead of waiting indefinitely for a lock), use `NOT VALID` + `VALIDATE` and `CREATE INDEX CONCURRENTLY`, and "expand → migrate → contract" (add the new column, backfill in batches — chunks of a few thousand rows at a time — switch the app, drop the old one). Locks: chapter 13.

Dialect notes

Task	PostgreSQL	MySQL	SQL Server
Change type	`ALTER COLUMN c TYPE t`	`MODIFY COLUMN c t`	`ALTER COLUMN c t`
Rename column	`RENAME COLUMN a TO b`	`RENAME COLUMN a TO b` (8.0) / `CHANGE a b t`	`EXEC sp_rename 't.a', 'b', 'COLUMN'`
Rename table	`ALTER TABLE t RENAME TO u`	`RENAME TABLE t TO u`	`EXEC sp_rename 't', 'u'`
DDL inside a transaction (rollback-able)	yes	no — implicit commit	yes (mostly)

Common mistakes

`NOT NULL` without a default or backfill (fails); a type change on a huge table in business hours (long exclusive lock); renaming columns that application SQL still uses (views follow renames automatically in PostgreSQL, application code does not).

9.6 DROP and TRUNCATE (vs DELETE)

What they are

- **`DROP`** removes an object entirely: the table's definition, data, indexes, constraints, triggers and permissions. `DROP TABLE t;`, `DROP DATABASE d;`, `DROP SCHEMA s;`, `DROP VIEW v;`, `DROP INDEX i;`.
- **`TRUNCATE`** removes **all rows** from a table but keeps the table itself (structure, indexes, grants). `TRUNCATE TABLE t;` (the word `TABLE` is optional in PostgreSQL and MySQL, required in SQL Server and Oracle).
- **`DELETE`** (DML, 9.10) removes rows one by one, optionally only those matching a `WHERE`.

Analogy: `DELETE` erases lines from a notebook one by one; `TRUNCATE` swaps in a fresh empty notebook; `DROP` throws the notebook away.

How TRUNCATE works

```sql
-- code/06_drop_and_truncate.sql
CREATE TABLE audit_log (log_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY, msg text NOT NULL);
INSERT INTO audit_log (msg) SELECT 'event ' || g FROM generate_series(1, 5) g;

TRUNCATE audit_log;
SELECT count(*) AS rows_after_truncate FROM audit_log;                    -- 0
INSERT INTO audit_log (msg) VALUES ('after truncate') RETURNING log_id;   -- 6: identity continues
TRUNCATE audit_log RESTART IDENTITY;
INSERT INTO audit_log (msg) VALUES ('after restart') RETURNING log_id;    -- 1
```

Output (the three results): `0`, then `log_id = 6`, then `log_id = 1`.

In PostgreSQL, `TRUNCATE` is **transactional** — you can roll it back:

```sql
BEGIN;
TRUNCATE audit_log;
SELECT count(*) AS inside_tx FROM audit_log;       -- 0
ROLLBACK;
SELECT count(*) AS after_rollback FROM audit_log;  -- 1
```

A table referenced by a foreign key can't be truncated alone, because that would orphan the child rows; `CASCADE` truncates the referencing tables too:

```
TRUNCATE customers;
BEGIN;
TRUNCATE customers CASCADE;
SELECT (SELECT count(*) FROM customers) AS customers,
       (SELECT count(*) FROM orders) AS orders,
       (SELECT count(*) FROM order_items) AS items;
ROLLBACK;
```

Output:

```
ERROR:  cannot truncate a table referenced in a foreign key constraint
DETAIL:  Table "orders" references "customers".
HINT:  Truncate table "orders" at the same time, or use TRUNCATE ... CASCADE.
NOTICE:  truncate cascades to table "orders"
NOTICE:  truncate cascades to table "order_items"
 customers | orders | items
-----------+--------+-------
         0 |      0 |     0
(1 row)
```

How DROP works

`DROP TABLE audit_log;` removes the table; querying it afterwards gives *relation "audit_log" does not exist*, a second `DROP` errors, and `DROP TABLE IF EXISTS` only prints a notice — the form to use in scripts. `DROP` refuses while other objects depend on the table (default `RESTRICT`); `CASCADE` drops the dependents too — views are dropped, and FKs *pointing at* the table are removed (the child table and its rows stay):

```
CREATE VIEW big_spenders AS SELECT customer_id FROM orders GROUP BY customer_id HAVING count(*) > 2;
DROP TABLE orders;
BEGIN;
DROP TABLE orders CASCADE;
SELECT count(*) AS order_items_still_there FROM order_items;
ROLLBACK;
```

Output:

```
ERROR:  cannot drop table orders because other objects depend on it
DETAIL:  constraint order_items_order_id_fkey on table order_items depends on table orders
view big_spenders depends on table orders
HINT:  Use DROP ... CASCADE to drop the dependent objects too.
NOTICE:  drop cascades to 2 other objects
...
 order_items_still_there
-------------------------
                      22
(1 row)
```

`DROP DATABASE name;` removes a whole database (you can't be connected to it, and in PostgreSQL it can't run inside a transaction). `DROP SCHEMA s CASCADE;` removes a schema and everything in it.

DELETE vs TRUNCATE vs DROP

	`DELETE FROM t [WHERE …]`	`TRUNCATE t`	`DROP TABLE t`
Kind	DML	DDL	DDL
Removes	chosen rows (or all)	all rows	rows **and** the table
`WHERE` possible	yes	no	no
Speed on big tables	slow: row by row, all logged (each change written to the transaction log)	fast: releases the data files	fast
Space returned to the OS	usually not: dead rows (old, deleted versions) are reused after `VACUUM` (PostgreSQL's cleanup process), the file rarely shrinks	yes	yes
Row-level `ON DELETE` triggers	fire	don't fire (PostgreSQL has `ON TRUNCATE` triggers)	n/a
FK checks	per row (or cascades)	refused if referenced, unless `CASCADE`	refused if referenced, unless `CASCADE`
Identity / auto-increment	continues	PostgreSQL: continues unless `RESTART IDENTITY`; MySQL & SQL Server: reset	gone
Rollback	yes	inside a transaction in PostgreSQL & SQL Server; **not** in MySQL & Oracle (implicit commit)	same as `TRUNCATE`
Lock	row locks	exclusive table lock	exclusive table lock

"Can `TRUNCATE`/`DROP` be undone?" therefore depends on the database — in MySQL the only way back is a backup (9.12); Oracle can undo a `DROP TABLE` from its recycle bin (`FLASHBACK TABLE t TO BEFORE DROP`, if enabled) but not a `TRUNCATE`. SQLite has no `TRUNCATE`; `DELETE FROM t` without `WHERE` is optimized to do the same job.

Common mistakes

- Running `DROP`/`TRUNCATE` on the wrong connection (production instead of dev) — make production visually distinct and use reviewed migrations (versioned, checked-in scripts that change the schema step by step).
- `… CASCADE` without reading what it cascades to — try it inside `BEGIN … ROLLBACK` first, as above.
- `DELETE` to empty a 100-million-row table (hours, huge log) when `TRUNCATE` takes a second.
- Expecting `TRUNCATE` to be invisible to concurrent transactions: it isn't MVCC-safe (MVCC, *multi-version concurrency control*, lets each transaction read a **snapshot** — a frozen, consistent view of the data as of one moment; chapter 13) — a transaction with an older snapshot that reads the table afterwards sees it empty.

In the interview

> **30-second answer:** "`DELETE` is DML: selected rows, one at a time, fires triggers, keeps the table. `TRUNCATE` is DDL: all rows at once by deallocating storage — much faster, no `WHERE`, no delete triggers, often resets identity; rollback-able in PostgreSQL and SQL Server, not in MySQL/Oracle. `DROP` removes the table itself."

Self-check

1. *Empty a staging table (a scratch table where incoming data lands before processing) quickly and reset its ids in PostgreSQL?* → `TRUNCATE staging RESTART IDENTITY;`
2. *Can you `TRUNCATE` a table that other tables reference by FK?* → Only with `CASCADE` (or truncating them in the same statement).
3. *What does `DROP TABLE orders CASCADE` do to `order_items`?* → Drops the FK constraint on it; the table and rows remain.

9.7 Indexes (in brief)

An **index** is a separate data structure (by default a B-tree — a balanced, always-sorted tree of keys) that finds rows by column values without reading the whole table — like a book's index sending you straight to the page. This section shows only the DDL; how indexes work, their kinds, `EXPLAIN` (the command that shows how the database plans to run a query), and when they help or hurt are in chapter 12.

```sql
-- code/07_indexes_brief.sql
CREATE INDEX idx_employees_last_name ON employees (last_name);
CREATE INDEX idx_orders_customer_placed ON orders (customer_id, placed_at DESC);   -- composite

-- A UNIQUE index can enforce rules a UNIQUE constraint can't, e.g. case-insensitive uniqueness:
CREATE UNIQUE INDEX idx_customers_email_ci ON customers (lower(trim(email)));
```

Output:

```
ERROR:  could not create unique index "idx_customers_email_ci"
DETAIL:  Key (lower(TRIM(BOTH FROM email)))=(anita.shah@mail.example) is duplicated.
```

The sample data really has a duplicate customer (chapter 1's Anita Shah, entered twice with different capitalisation) — the index found it. After cleaning (`UPDATE customers SET email = NULL WHERE customer_id = 8`) the same statement succeeds.

Primary keys and unique constraints create their indexes automatically — `pg_indexes` (a catalog view listing every index) lists `employees_pkey` and `employees_email_key` next to the ones created above. Other index DDL: `DROP INDEX [IF EXISTS] name;`, and `CREATE INDEX CONCURRENTLY …` / `DROP INDEX CONCURRENTLY …`, which avoid blocking writes on a live table (slower, and not allowed inside a transaction block).

Clustered vs non-clustered

	Clustered index	**Non-clustered (secondary) index**
What it is	the table's rows are **stored in the index's order** — the index *is* the table	a separate structure of (key → pointer to the row)
How many	at most one per table	many
SQL Server	`CREATE CLUSTERED INDEX ix ON t (col);` (the PK is clustered by default)	`CREATE [NONCLUSTERED] INDEX ix ON t (col);`
MySQL InnoDB	always the primary key (no PK → the first `UNIQUE NOT NULL` index, else a hidden row id)	every other index; it stores the PK value as the pointer
PostgreSQL	**none** — tables are unordered "heaps"; `CLUSTER t USING ix;` re-sorts the table once, but new rows aren't kept in order	every index is secondary

```sql
CLUSTER orders USING idx_orders_customer_placed;   -- one-time physical re-ordering in PostgreSQL
```

A clustered index makes range scans (reading every key between two values) on its key fast (neighbouring keys are neighbouring rows) but makes the choice of key important: a random key (like a

random UUID) causes page splits on insert (a full storage page — the fixed-size block, 8 KB in PostgreSQL, that data is kept in — must be split in two to make room).

Cost of indexes: each index must be maintained on every insert and on updates of its columns (and cleaned up after deletes), and takes disk space. Index what queries filter, join and sort on — not every column.

9.8 INSERT

What it is

`INSERT` adds new rows to a table.

```
INSERT INTO table_name [(column1, column2, ...)]
VALUES (value1, value2, ...) [, (...), ...]
[ON CONFLICT ...]            -- 9.11
[RETURNING ...];             -- PostgreSQL: give back the inserted rows
-- or
INSERT INTO table_name [(columns)] SELECT ...;
```

How it works

1. The **column list** names the columns you supply, in order. Omitted columns get their `DEFAULT`, else NULL (an error if `NOT NULL` without a default).
2. Without a column list you must supply **every** column in table order — fragile: adding a column later breaks the statement.
3. Every row is checked against types and constraints; if any row fails, the **whole statement** fails.

```
-- code/08_insert.sql
CREATE TABLE contacts (
    contact_id  integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    first_name  text NOT NULL,
    last_name   text NOT NULL,
    age         integer CHECK (age >= 0),
    city        text DEFAULT 'Pune',
    created_at  timestamp NOT NULL DEFAULT '2025-06-01 09:00'
);

INSERT INTO contacts (first_name, last_name, age) VALUES ('Tara', 'Mendes', 28);

INSERT INTO contacts (first_name, last_name, age, city) VALUES      -- several rows at once
    ('Omar', 'Mendes', 31, 'Mumbai'),
    ('Yuki', 'Tanaka', NULL, DEFAULT);                              -- DEFAULT keyword

INSERT INTO contacts VALUES (DEFAULT, 'Leo', 'Brandt', 40, 'Delhi', '2025-06-02 10:00');  -- all columns

INSERT INTO contacts (first_name, last_name) VALUES ('Ines', 'Silva')
RETURNING contact_id, city, created_at;
```

Output (of the `RETURNING`):

```
 contact_id | city |     created_at
------------+------+---------------------
          5 | Pune | 2025-06-01 09:00:00
(1 row)
```

`RETURNING` hands back generated values (ids, defaults) in the same round trip (one request to the server and its reply) — no second query, no race (a **race condition** is a bug whose outcome depends on the timing of concurrent sessions).

`INSERT … SELECT` copies rows produced by any query:

```
INSERT INTO contacts (first_name, last_name, city)
SELECT first_name, last_name, city FROM customers WHERE country = 'UK'
RETURNING contact_id, first_name, city;
```

Output:

```
 contact_id | first_name |  city
------------+------------+--------
          6 | Ben        | London
          7 | George     | London
(2 rows)
```

Errors worth recognising:

```
INSERT INTO contacts (first_name, last_name) VALUES ("Ana", "Lima");
INSERT INTO contacts (first_name, last_name) VALUES ('Ana', 'Lima', 22);
INSERT INTO contacts (first_name, last_name, age) VALUES ('Ok', 'Row', 20), ('Bad', 'Row', -1);
SELECT count(*) AS still_7 FROM contacts;
```

Output:

```
ERROR:  column "Ana" does not exist
ERROR:  INSERT has more expressions than target columns
ERROR:  new row for relation "contacts" violates check constraint "contacts_age_check"
DETAIL:  Failing row contains (9, Bad, Row, -1, Pune, 2025-06-01 09:00:00).
 still_7
---------
       7
(1 row)
```

- In standard SQL (PostgreSQL, SQLite, SQL Server by default, Oracle) **double quotes name identifiers** (names of tables, columns and other objects); strings use single quotes. MySQL accepts double-quoted strings by default, so MySQL-style code often breaks elsewhere.
- The multi-row insert failed as a whole — the valid 'Ok' row was not kept (but it consumed identity values 8 and 9: more gaps).

Bulk loading: multi-row `VALUES` or `INSERT … SELECT` beats one statement per row; for millions of rows use `COPY table FROM file` (psql: `\copy` for a client file), MySQL `LOAD DATA INFILE`, SQL Server `BULK INSERT` / `bcp`.

Dialect notes

	PostgreSQL	MySQL	SQL Server	SQLite
Multi-row `VALUES`	yes	yes	yes (max 1000 rows per statement)	yes
Get generated values back	`RETURNING`	`LAST_INSERT_ID()`	`OUTPUT inserted.*` / `SCOPE_IDENTITY()`	`RETURNING` (3.35+) / `last_insert_rowid()`
`INSERT … DEFAULT VALUES`	yes	`INSERT INTO t () VALUES ()`	yes	yes

9.9 UPDATE

What it is

`UPDATE` changes column values in existing rows.

```
UPDATE table_name
SET column1 = expr1, column2 = expr2, ...
[FROM other_tables]          -- PostgreSQL: use other tables to compute / filter
[WHERE condition]            -- which rows; WITHOUT it, EVERY row changes
[RETURNING ...];
```

How it works

1. `WHERE` picks the rows, as in `SELECT`. No `WHERE` = all rows.
2. Every `SET` expression is evaluated from the row's **old** values, then all assignments apply together.
3. New rows must satisfy all constraints, or the statement fails and nothing changes.
4. PostgreSQL writes a new version of each updated row (MVCC — chapter 13); `VACUUM` later removes the old one.

Raise Sales salaries by 10 %:

```
-- code/09_update.sql
UPDATE employees
SET salary = salary * 1.10
WHERE dept_id = 2
RETURNING emp_id, first_name, salary;
```

Output:

```
 emp_id | first_name |  salary
--------+------------+-----------
      6 | Daniel     | 165000.00
      7 | Priya      |  79200.00
 ...
(4 rows)
```

Percentages are multiplications: +10 % is `* 1.10`, +50 % is `* 1.5`, doubling is `* 2` — check the factor against the requirement.

All `SET` expressions see the old row:

```
UPDATE products
SET unit_price   = unit_price * 0.9,
    discontinued = unit_price * 0.9 < 40          -- uses the OLD unit_price
WHERE category = 'Furniture'
RETURNING name, unit_price, discontinued;
```

Output:

```
      name       | unit_price | discontinued
-----------------+------------+--------------
 Standing Desk   |     449.10 | f
 Ergonomic Chair |     314.99 | f
 Desk Lamp       |      31.50 | t
(3 rows)
```

(In MySQL, assignments are applied left to right, so a later expression sees the *new* value of an earlier column — a real portability trap. PostgreSQL and SQL Server follow the standard.)

Different changes for different rows with `CASE`:

```
UPDATE orders
SET status = CASE WHEN shipped_at IS NULL THEN 'cancelled' ELSE 'delivered' END
WHERE status IN ('pending', 'shipped') AND placed_at < '2025-06-10'
RETURNING order_id, status;
```

Output:

```
 order_id |  status
----------+-----------
      107 | delivered
 ...
      113 | cancelled
(4 rows)
```

Using another table — PostgreSQL's `UPDATE … FROM` joins the target to other tables:

```
ALTER TABLE customers ADD COLUMN lifetime_orders integer NOT NULL DEFAULT 0;
UPDATE customers c
SET lifetime_orders = t.n
FROM (SELECT customer_id, count(*) AS n FROM orders GROUP BY customer_id) t
WHERE t.customer_id = c.customer_id;
SELECT customer_id, first_name, lifetime_orders FROM customers ORDER BY customer_id LIMIT 4;
```

Output:

```
 customer_id | first_name | lifetime_orders
-------------+------------+-----------------
           1 | Anita      |               3
           2 | Ben        |               2
 ...
(4 rows)
```

Make sure each target row matches **at most one** row of the `FROM` source; if it matches several, PostgreSQL uses one of them unpredictably. That's why the example aggregates first. The portable alternative is a correlated scalar subquery (a subquery that returns one value and is re-run for each outer row) in `SET`:

```
UPDATE departments d
SET budget = (SELECT coalesce(sum(salary), 0) * 1.5 FROM employees e WHERE e.dept_id = d.dept_id)
WHERE d.dept_id = 5
RETURNING name, budget;                          -- Support | 96000.00
```

The classic accident — and the habit that prevents it. `UPDATE` without `WHERE` updates every row. Practise risky statements inside a transaction, check the effect, then `COMMIT` or `ROLLBACK`:

```
BEGIN;
UPDATE employees SET salary = 1;
SELECT count(*) AS rows_now_paid_1 FROM employees WHERE salary = 1;
ROLLBACK;
SELECT count(*) AS rows_paid_1_after_rollback FROM employees WHERE salary = 1;
```

Output:

```
 rows_now_paid_1
-----------------
              15
(1 row)

 rows_paid_1_after_rollback
----------------------------
                          0
(1 row)
```

Other safety habits: run the `WHERE` as a `SELECT count(*)` first; check psql's `UPDATE n` row count; use `RETURNING`; in MySQL clients, `--safe-updates` refuses key-less updates.

Constraints still apply: setting `dept_id = 99` fails the foreign key, `unit_price = -1` fails the `CHECK` (both shown in the file).

9.10 DELETE

What it is

`DELETE` removes rows from a table.

```
DELETE FROM table_name
[USING other_tables]         -- PostgreSQL: join to other tables
[WHERE condition]            -- WITHOUT it, EVERY row is deleted
[RETURNING ...];
```

How it works

1. `WHERE` picks the rows; without it all rows go (the table remains).
2. Deleting a parent that still has children fails or cascades, as its foreign keys declare (9.3).
3. In PostgreSQL the rows are only marked dead; `VACUUM` reclaims the space later.

```
-- code/10_delete.sql
DELETE FROM order_items
WHERE order_id = 105
RETURNING order_id, product_id, quantity;
```

Output:

```
 order_id | product_id | quantity
----------+------------+----------
      105 |          1 |        2
(1 row)
```

```
DELETE FROM customers WHERE customer_id = 1;
```

Output:

```
ERROR:  update or delete on table "customers" violates foreign key constraint "orders_customer_id_fkey" on table
"orders"
DETAIL:  Key (customer_id)=(1) is still referenced from table "orders".
```

Deleting based on another table — PostgreSQL's `DELETE … USING`, or the portable subquery form:

```
DELETE FROM order_items oi
USING orders o
WHERE o.order_id = oi.order_id
  AND o.status = 'cancelled'
RETURNING oi.order_id, oi.product_id;          -- (111, 2)

DELETE FROM orders
WHERE status = 'cancelled'
  AND order_id NOT IN (SELECT order_id FROM order_items)
RETURNING order_id;                            -- 105, 111

DELETE FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
RETURNING customer_id, first_name;
```

Output of the last statement:

```
 customer_id | first_name
-------------+------------
           4 | Divya
           9 | Hana
          10 | Ivan
(3 rows)
```

(Divya's only order was a cancelled one, removed just before.) Children first, then parents — the order foreign keys demand. (`NOT IN` with a subquery is safe here only because `order_items.order_id` can't be NULL; prefer `NOT EXISTS` — see chapter 6.)

Delete everything — safely practised:

```
BEGIN;
DELETE FROM order_items;
SELECT count(*) AS items_inside_tx FROM order_items;       -- 0
ROLLBACK;
SELECT count(*) AS items_after_rollback FROM order_items;  -- 20
```

Soft delete. Many applications never physically delete business data; they mark it:

```
ALTER TABLE products ADD COLUMN deleted_at timestamp;
UPDATE products SET deleted_at = '2025-06-30 12:00' WHERE name = 'Desk Lamp';
CREATE VIEW active_products AS SELECT * FROM products WHERE deleted_at IS NULL;
SELECT count(*) AS all_products, (SELECT count(*) FROM active_products) AS active FROM products;
```

Output:

```
 all_products | active
--------------+--------
            9 |      8
(1 row)
```

	Hard delete	Soft delete
Undo / audit	only from backups or audit tables	trivial: clear `deleted_at`
Queries	simple	every query must filter (use a view, or row-level security — per-user rules that hide rows)
Unique constraints	simple	a deleted row still blocks its email — use a partial unique index (PostgreSQL's filtered index) `WHERE deleted_at IS NULL`
Privacy law ("right to be forgotten")	satisfied	must still purge eventually

Deleting millions of rows: do it in batches (e.g. `DELETE … WHERE id IN (SELECT id … LIMIT 10000)` repeated) to keep transactions and locks short; for time-based data, partitioned tables (one logical table stored as several physical pieces, e.g. one per month) let you `DROP` a whole old partition instantly (chapter 15).

Dialect notes: UPDATE / DELETE using another table

	UPDATE	DELETE
PostgreSQL	`UPDATE t SET … FROM s WHERE t.id = s.id`	`DELETE FROM t USING s WHERE t.id = s.id`
SQL Server	`UPDATE t SET … FROM t JOIN s ON …`	`DELETE t FROM t JOIN s ON …`
MySQL	`UPDATE t JOIN s ON … SET t.col = …`	`DELETE t FROM t JOIN s ON …`
SQLite	`UPDATE t SET … FROM s WHERE …` (3.33+)	subquery only (`WHERE id IN (SELECT …)`)

9.11 Upsert: INSERT … ON CONFLICT, and MERGE

What it is

An **upsert** means "insert this row, or if its key already exists, update (or skip) it". `SELECT`-then-`INSERT`-or-`UPDATE` in application code has a race: two sessions both see "not there" and both insert.

PostgreSQL (9.5+) does it atomically (as one indivisible step that no other session can slip in between) with `INSERT … ON CONFLICT`.

How it works

```
INSERT INTO t (cols) VALUES (...)
ON CONFLICT (conflict_target)          -- the unique/PK column(s), or ON CONSTRAINT name
DO NOTHING                             -- skip rows that would conflict
-- or
DO UPDATE SET col = EXCLUDED.col, ...  -- EXCLUDED = the row you tried to insert
          [WHERE condition];           -- update only when this is true
```

1. PostgreSQL tries to insert each row; if it would violate the unique index named by the conflict target, it runs the `DO` action instead of raising an error.
2. In `DO UPDATE`, `t.col` (or an alias) is the **existing** row, `EXCLUDED.col` the **proposed** one.
3. Each row ends up inserted, updated or skipped — atomically, even under concurrency.

```
-- code/11_upsert.sql
CREATE TABLE inventory (
    sku         text PRIMARY KEY,
    qty         integer NOT NULL CHECK (qty >= 0),
    updated_at  timestamp NOT NULL
);
INSERT INTO inventory VALUES ('MOUSE-1', 10, '2025-06-01 08:00'), ('HUB-2', 4, '2025-06-01 08:00');

INSERT INTO inventory VALUES ('MOUSE-1', 99, '2025-06-02 08:00'), ('LAMP-3', 7, '2025-06-02 08:00')
ON CONFLICT (sku) DO NOTHING
RETURNING sku, qty;
```

Output:

```
  sku   | qty
--------+-----
 LAMP-3 |   7
(1 row)
```

MOUSE-1 already existed, so it was skipped silently (and isn't in `RETURNING`).

Insert or add to the existing stock:

```
INSERT INTO inventory AS i VALUES
    ('MOUSE-1', 5, '2025-06-03 08:00'),
    ('KEYB-4',  2, '2025-06-03 08:00')
ON CONFLICT (sku) DO UPDATE
SET qty        = i.qty + EXCLUDED.qty,
    updated_at = EXCLUDED.updated_at
RETURNING sku, qty, updated_at, (xmax <> 0) AS was_update;
```

Output:

```
   sku   | qty |     updated_at      | was_update
---------+-----+---------------------+------------
 MOUSE-1 |  15 | 2025-06-03 08:00:00 | t
 KEYB-4  |   2 | 2025-06-03 08:00:00 | f
(2 rows)
```

(`xmax` is a hidden system column on every PostgreSQL row; `xmax <> 0` is a common trick to tell updated from inserted rows; it relies on an internal detail, so don't build business logic on it.)

Conditional update — only if the incoming data is newer (a common rule when syncing from several sources):

```
INSERT INTO inventory AS i VALUES ('HUB-2', 1, '2025-05-01 00:00')    -- older than what we have
ON CONFLICT (sku) DO UPDATE
SET qty = EXCLUDED.qty, updated_at = EXCLUDED.updated_at
WHERE EXCLUDED.updated_at > i.updated_at
RETURNING sku, qty;
```

Output:

```
 sku | qty
-----+-----
(0 rows)
```

Nothing changed: HUB-2 keeps qty 4 from 1 June (the file lists the table afterwards). **Rules and errors:**

- The conflict target must match a unique index or constraint (`ON CONFLICT (sku)` or `ON CONFLICT ON CONSTRAINT inventory_pkey`). `DO NOTHING` may omit the target (then any unique violation is skipped).
- One statement can't affect the same row twice:

```
INSERT INTO inventory VALUES ('PEN-5', 1, '2025-06-04'), ('PEN-5', 2, '2025-06-04')
ON CONFLICT (sku) DO UPDATE SET qty = EXCLUDED.qty;
```

Output:

```
ERROR:  ON CONFLICT DO UPDATE command cannot affect row a second time
HINT:  Ensure that no rows proposed for insertion within the same command have duplicate constrained values.
```

De-duplicate the input first (e.g. `SELECT DISTINCT ON (sku) …`).

- Other constraint violations (a `CHECK`, a different unique index) still raise errors.

MERGE (PostgreSQL 15+, SQL Server, Oracle)

`MERGE` is the SQL-standard statement for "compare a source with a target and insert, update or delete accordingly". PostgreSQL added it in **version 15** (SQL Server has had it since 2008, Oracle since 9i); this book's server is 14, where `MERGE` is a syntax error. The general shape (PostgreSQL 15+ / SQL Server — not run here):

```
MERGE INTO inventory AS t
USING incoming AS s ON t.sku = s.sku
WHEN MATCHED AND s.qty = 0 THEN DELETE
WHEN MATCHED THEN UPDATE SET qty = s.qty, updated_at = s.updated_at
WHEN NOT MATCHED THEN INSERT (sku, qty, updated_at) VALUES (s.sku, s.qty, s.updated_at);
```

	`INSERT … ON CONFLICT`	`MERGE`
Can delete	no	yes (`WHEN MATCHED … THEN DELETE`)
Needs a unique index	yes	no — any join condition
Safe against concurrent inserts of the same key	yes (that's its purpose)	not by itself: two sessions can both take the `NOT MATCHED` branch → unique violation; retry or lock
`RETURNING`	yes	PostgreSQL 17+ (which also added `WHEN NOT MATCHED BY SOURCE` for deleting target rows missing from the source)

Rule of thumb in PostgreSQL: **upserts of single rows/batches → `ON CONFLICT`; synchronising a table from a staging table with deletes → `MERGE`** (15+).

Dialect notes

	Upsert syntax
PostgreSQL	`INSERT … ON CONFLICT … DO NOTHING / DO UPDATE SET … EXCLUDED.col` (9.5+); `MERGE` (15+)
MySQL	`INSERT … ON DUPLICATE KEY UPDATE col = VALUES(col)` (or row alias `AS new … new.col`, 8.0.19+); `INSERT IGNORE` ≈ do nothing
SQLite	`INSERT … ON CONFLICT(col) DO UPDATE SET col = excluded.col` (3.24+); `INSERT OR IGNORE` / `INSERT OR REPLACE`
SQL Server	`MERGE`, or `UPDATE` + `INSERT … WHERE NOT EXISTS` with locking hints
Oracle	`MERGE`

In the interview

30-second answer: "In PostgreSQL, `INSERT … ON CONFLICT (key) DO UPDATE SET col = EXCLUDED.col` — atomic and concurrency-safe because it relies on the unique index; `DO NOTHING` skips duplicates. `MERGE` (PostgreSQL 15+, SQL Server, Oracle) is the standard multi-branch statement that can also delete, but doesn't by itself protect against concurrent inserts."

Self-check

1. *What does `EXCLUDED` refer to?* → The row that was proposed for insertion.
2. *What does `ON CONFLICT` need?* → A unique index/constraint matching the conflict target.

9.12 Backup and restore

What it is

A **backup** is a copy you can rebuild the database from; a **restore** does the rebuilding. Backups protect against what replication (keeping live copies — **replicas** — of the database on other servers; see System Design Fundamentals) can't: a mistaken `DELETE`/`DROP` (replicas copy it instantly), data-corrupting bugs, ransomware (malware that encrypts your files and demands payment), losing a data centre.

Two terms decide a backup strategy:

- **RPO — recovery point objective**: how much recent data you can afford to lose (e.g. 5 minutes).
- **RTO — recovery time objective**: how long you can afford to be down while restoring (e.g. 1 hour).

Kinds of backups

Kind	What is copied	Tools (PostgreSQL)	Good for	Limits
Logical	SQL / an archive of schema + data	`pg_dump`, `pg_dumpall`	small/medium DBs, single tables, moving across versions	slow for huge DBs; restore rebuilds indexes
Physical	the database files themselves	`pg_basebackup`, file-system snapshots, pgBackRest, Barman	large DBs, fast full restore	whole cluster (in PostgreSQL: one server instance with all its databases) only; same major version (e.g. 16.x)
Continuous (PITR)	a base backup **plus** the write-ahead log (WAL) stream	WAL archiving + base backup	restore to *any moment* (just before the bad `DELETE`)	more setup and storage

> **Key term — WAL and PITR:** the **write-ahead log (WAL)** is a file of every change, written *before* the change is applied to the tables, so the database can replay it after a crash. **Point-in-time recovery (PITR)** restores a **base backup** (a full physical copy) and then replays archived WAL up to any chosen moment — for example one second before a bad `DELETE`.

Other terms: **full** (everything), **differential** (changes since the last full), **incremental** (changes since the last backup of any kind; PostgreSQL 17 added incremental `pg_basebackup`), **hot/online** (taken while the database is in use — all of the above are).

pg_dump and pg_restore, for real

`pg_dump` takes a **consistent** snapshot of one database while others keep reading and writing (it runs inside a single repeatable-read transaction — an isolation level where the transaction sees the data exactly as it was when it started; chapter 13). Formats:

Format flag	Output	Restore with	Notes
`-F p` (plain, default)	a `.sql` script	`psql -f`	readable, editable; replayed statement by statement (`--single-transaction` for all-or-nothing)
`-F c` (custom)	one compressed archive	`pg_restore`	pick objects to restore, reorder, parallel restore (`-j`)
`-F d` (directory)	a folder, one file per table	`pg_restore`	the only format that can **dump** in parallel (`-j`)
`-F t` (tar)	a tar archive (a standard Unix file bundle)	`pg_restore`	like custom, but no compression or parallel restore

The script does a full round trip on the book's private server, in a temporary directory deleted afterwards:

```
# code/12_backup_restore.sh (key lines; CONN="-h /tmp -p 5544 -U postgres")
pg_dump $CONN -d shop -F p -f "$WORK/shop.sql"          # plain SQL script
pg_dump $CONN -d shop -F c -f "$WORK/shop.dump"         # custom archive
pg_dump $CONN -d shop -F d -j 2 -f "$WORK/shop_dir"     # directory format, 2 parallel jobs
pg_restore -l "$WORK/shop.dump"                         # list the archive's contents
psql $CONN -d shop -c "DROP TABLE order_items"          # the "disaster"
pg_restore $CONN -d shop -t order_items "$WORK/shop.dump"      # bring back one table
pg_restore $CONN -d shop_copy -j 2 "$WORK/shop.dump"    # full restore into a new, empty DB
psql $CONN -d shop_plain -f "$WORK/shop.sql"            # plain dumps are replayed by psql
pg_dump $CONN -d shop -t products --schema-only         # structure only (--data-only: rows only)
```

Output (run with `bash chapters/09-ddl-dml-and-constraints/code/12_backup_restore.sh` from the book root):

```
== 1. a database to protect
shop: 10 customers, 14 orders, 22 order_items
== 2. logical backups in three formats
...
-- first CREATE TABLE in the plain dump:
CREATE TABLE public.customers (
    customer_id integer NOT NULL,
    first_name text NOT NULL,
...
-- plain dump contains 8 COPY blocks (one per table with data)
== 3. what is inside the custom-format archive (pg_restore -l = table of contents)
TABLE public customers postgres
TABLE public orders postgres
TABLE DATA public customers postgres
...
== 4. disaster: someone drops a table
ERROR:  relation "order_items" does not exist
== 5. restore just that table (definition + data) from the archive
shop: 10 customers, 14 orders, 22 order_items
-- note: -t restores the table and its rows, not its PK/FK constraints:
0 constraints on order_items
== 6. full restore into a NEW database, 2 parallel jobs
shop_copy: 10 customers, 14 orders, 22 order_items
3 constraints on order_items
== 7. a plain dump is restored with psql
shop_plain: 10 customers, 14 orders, 22 order_items
== 8. schema-only and data-only dumps of one table
CREATE TABLE statements in schema-only dump: 1, in data-only dump: 0
...
```

What it shows:

1. A plain dump is just SQL: `CREATE TABLE`s, a `COPY … FROM stdin` block of rows per table, then constraints and indexes (added *after* the data, which is faster). `stdin` means *standard input*: the rows follow inline in the same script.
2. A custom archive has a **table of contents** (`pg_restore -l`); `-t`, `-n` (schema) and `-L` (edited list) choose what to restore.
3. `-t` restores a table's definition and rows **but not its constraints or indexes** (separate archive entries) — re-add them with `-L` or by re-running the DDL.

Useful options: `--clean --if-exists` (drop objects before recreating), `--create` (create the database itself), `--no-owner` (when restoring as a different user), `-j N` (parallelism). `pg_dump` dumps one database; **roles and tablespaces** (database users/groups, and named disk locations for data) are cluster-wide, so also run `pg_dumpall --globals-only`.

Backup and restore in other databases

	Full backup	Restore	Notes
PostgreSQL	`pg_dump` (logical), `pg_basebackup` + WAL (physical, PITR)	`pg_restore` / `psql`; restore base backup + replay WAL to a target time	command-line tools, not SQL statements
MySQL	`mysqldump`, MySQL Shell dump utilities (logical); Percona XtraBackup / MySQL Enterprise Backup (physical)	`mysql < dump.sql`; binary logs (MySQL's log of every change) for point-in-time	
SQL Server	`BACKUP DATABASE db TO DISK = '…\db.bak'` (also `WITH DIFFERENTIAL`, `BACKUP LOG`, `TO URL` for cloud storage)	`RESTORE DATABASE db FROM DISK = '…' [WITH NORECOVERY / RECOVERY, STOPAT = …]`	backup and restore are T-SQL (SQL Server's SQL dialect) statements; the others mostly use separate tools
SQLite	`.backup file` in the shell, `VACUUM INTO 'file'`	copy the file back / `.restore`	the database is one file

Two corrections to things some tutorials show (SQL Server — not run):

- `BACKUP DATABASE` has no `WITH SCHEDULE = …` option. Backups are scheduled outside the statement — SQL Server Agent jobs or maintenance plans (elsewhere: cron or systemd timers — the standard Linux job schedulers — or a backup tool's scheduler).
- There is no `RESTORE TABLE`. To recover one table, restore the backup under **another database name** (`RESTORE DATABASE db_old FROM DISK = … WITH MOVE …`) and copy the table across — like our `pg_restore -t` or `shop_copy`.

Best practices

- **3-2-1 rule**: 3 copies, on 2 kinds of storage, 1 off-site.
- **Test restores regularly** — an untested backup is a hope. Automate a restore into a scratch server and run sanity queries (row counts, as the script does).
- **Encrypt** backups and restrict who can read them.
- **Match RPO/RTO**: nightly `pg_dump` alone risks 24 h of loss; PITR gets it to seconds.
- **Replication is not a backup** — replicas faithfully copy `DROP TABLE` (chapter 15).
- Managed databases (run for you by a cloud provider: Amazon RDS, Cloud SQL, Azure) provide automated snapshots + PITR within a retention window (how long backups are kept).

In the interview

> **30-second answer:** "Logical backups (`pg_dump`) export schema and data — flexible, single tables, portable across versions, slow for huge databases. Physical backups copy the data files; with archived WAL they allow point-in-time recovery. Choose by RPO/RTO, keep 3-2-1 copies, encrypt, test restores — and replication doesn't protect against a bad `DELETE`."

- *Someone ran `DELETE` without `WHERE` at 14:03 — how do you recover?* → PITR: restore the last base backup to a new server and replay WAL up to 14:02:59, then copy the lost rows back (or fail over to it — switch the application to use it).
- *pg_dump on a busy database — does it block?* → No; it reads a consistent snapshot. It does take light locks that block DDL like `DROP` or `ALTER` on dumped tables while it runs.

Self-check

1. *Which `pg_dump` formats can `pg_restore` read?* → custom, directory and tar (plain is replayed with `psql`).
2. *Which format can dump in parallel?* → Directory (`-F d -j N`).
3. *What does `pg_dump` not include that `pg_dumpall --globals-only` does?* → Roles and tablespaces.

Chapter 9 TL;DR

- **Types** validate as well as store: `numeric` for money, `integer`/`bigint` for ids, `text`, `timestamptz`, `date`, `boolean`.
- `CREATE` databases, schemas, tables (`IF NOT EXISTS`, `AS SELECT` = data without constraints, `LIKE … INCLUDING ALL` = structure, `TEMP`).
- **Keys**: super ⊇ candidate → one primary (unique + not null) + alternate keys (`UNIQUE`); composite keys; foreign keys with `ON DELETE CASCADE | SET NULL | RESTRICT | NO ACTION`. `CHECK` rejects only false; `UNIQUE` allows several NULLs (one in SQL Server). Name constraints; index FK columns.
- **Generated keys**: `GENERATED ALWAYS AS IDENTITY` (not `serial`), sequences have gaps, UUIDs for distributed ids, `GENERATED … STORED` for derived columns.
- `ALTER`: add/drop/rename columns, `TYPE … USING`, `SET NOT NULL` after backfilling, `NOT VALID` + `VALIDATE` on big tables; watch exclusive locks.
- `DROP` removes the object, `TRUNCATE` empties it fast (transactional in PostgreSQL, `RESTART IDENTITY`, `CASCADE`), `DELETE` removes chosen rows.
- **Indexes**: `CREATE [UNIQUE] INDEX [CONCURRENTLY]`; clustered vs non-clustered; details in chapter 12.

- **DML**: `INSERT` (list columns, multi-row, `INSERT … SELECT`), `UPDATE` (`SET` sees old values, `FROM`), `DELETE` (`USING`, children first), all with `RETURNING`. Practise destructive statements in `BEGIN … ROLLBACK`.
- **Upsert**: `INSERT … ON CONFLICT … DO NOTHING / DO UPDATE SET … = EXCLUDED.…`; `MERGE` needs PostgreSQL 15+.
- **Backups**: `pg_dump` (plain/custom/directory) + `pg_restore` (selective, parallel); physical + WAL for PITR; RPO/RTO, 3-2-1, test restores.

Next: chapter 10 — data modeling and ER diagrams, which decides *which* tables and keys to create in the first place.

Chapter 10 — Data Modeling and ER Diagrams

Up to now the tables in this book were handed to you. In real work nobody hands you tables: somebody describes a business in a few sentences ("we rent bikes to people"), and you must decide what to store, how the pieces connect, and which rules the database should enforce. That decision is **data modeling**, and it matters more than any single query you will write, because any fact left out of the model is a question no SQL, however clever, can ever answer.

This chapter walks the whole path: from a plain-language brief, to a drawing that business people can check (an **ER diagram**), to precise tables with keys and constraints (the **relational model** — the theory of storing all data as tables of rows and columns, §10.11), to `CREATE TABLE` statements running on PostgreSQL. Chapter 11 then tests those tables for repeated data, and Chapter 17 applies everything to nine full case studies.

What you'll learn

1. What a data model is, who reads it, and the three levels it is drawn at (conceptual, logical, physical).
2. A repeatable seven-step modeling process, and the best practices that separate easy-to-change models from painful ones.
3. The ER model: entities, the five kinds of attribute, relationships, degree, cardinality, participation, relationship attributes and weak entities — in **Chen notation** and in **crow's-foot notation** (the one `mermaid erDiagram` draws — Mermaid is a tool that turns a short text description into a diagram).
4. A complete worked example: an employee-management ER diagram built step by step.
5. The relational model: relations, tuples, attributes, domains, `NULL`, the seven kinds of key and the integrity constraints.
6. The mechanical procedure that turns any ER diagram into tables, and the judgement calls it leaves to you.

All runnable code is in `code/`: `00_schema.sql` builds a small bike-rental database; `01`–`04` run on a fresh database each (see Chapter 1 for the private PostgreSQL 14 server). Every "Output:" block below is real psql output (psql is PostgreSQL's command-line client).

10.1 What data modeling is

What it is

Data modeling is deciding three things before you write any SQL:

Part	Plain meaning	Usually becomes
Entity	a *thing* you keep data about (Customer, Bike, Rental) — almost always a noun	a table
Attribute	one *fact* about an entity (a customer's email, a bike's frame number)	a column
Relationship	a *link* between entities (a customer *rents* a bike)	a foreign key (a column holding another table's key), or a table of its own

The result — a **data model** — is a picture of those three things and nothing else. It says nothing about indexes (lookup structures that speed up searches), `VARCHAR` lengths or which database product you will use. Analogy: it is the architect's floor plan, drawn before anyone picks brick suppliers.

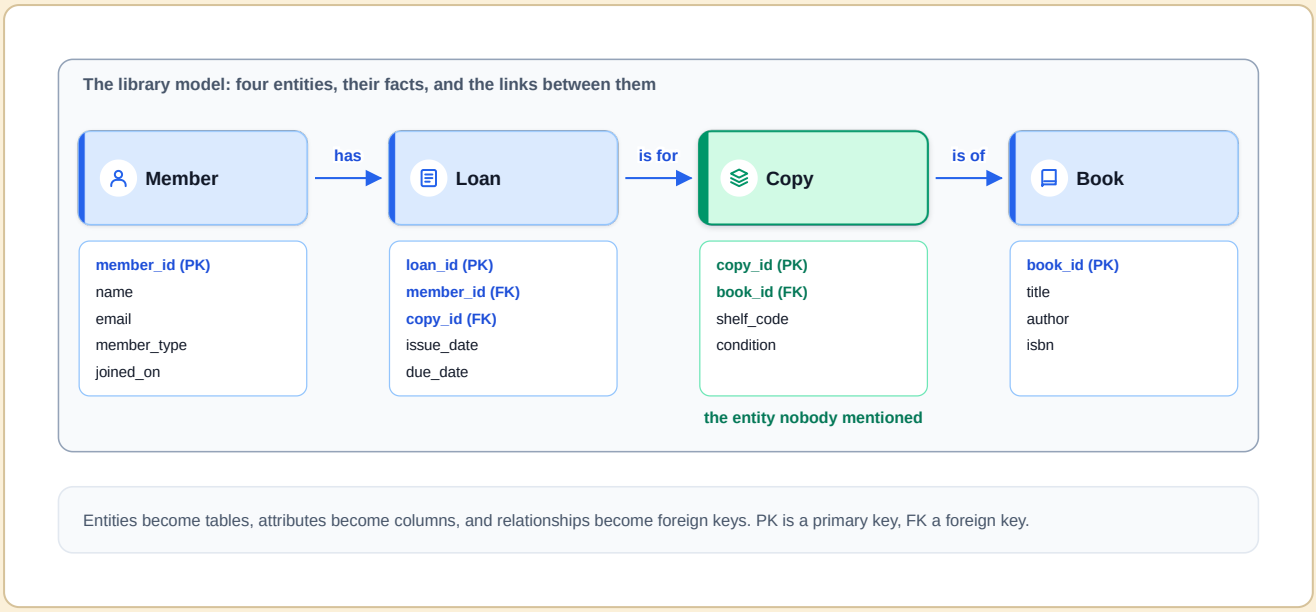
The problem it solves — the question you cannot answer

A small bike-rental shop says: *"We rent bikes to customers."* That sounds like two tables, `customer` and `bike`. Then the owner asks: *"We own three 'Urban 3' bikes. One came back with a bent wheel — which one,*

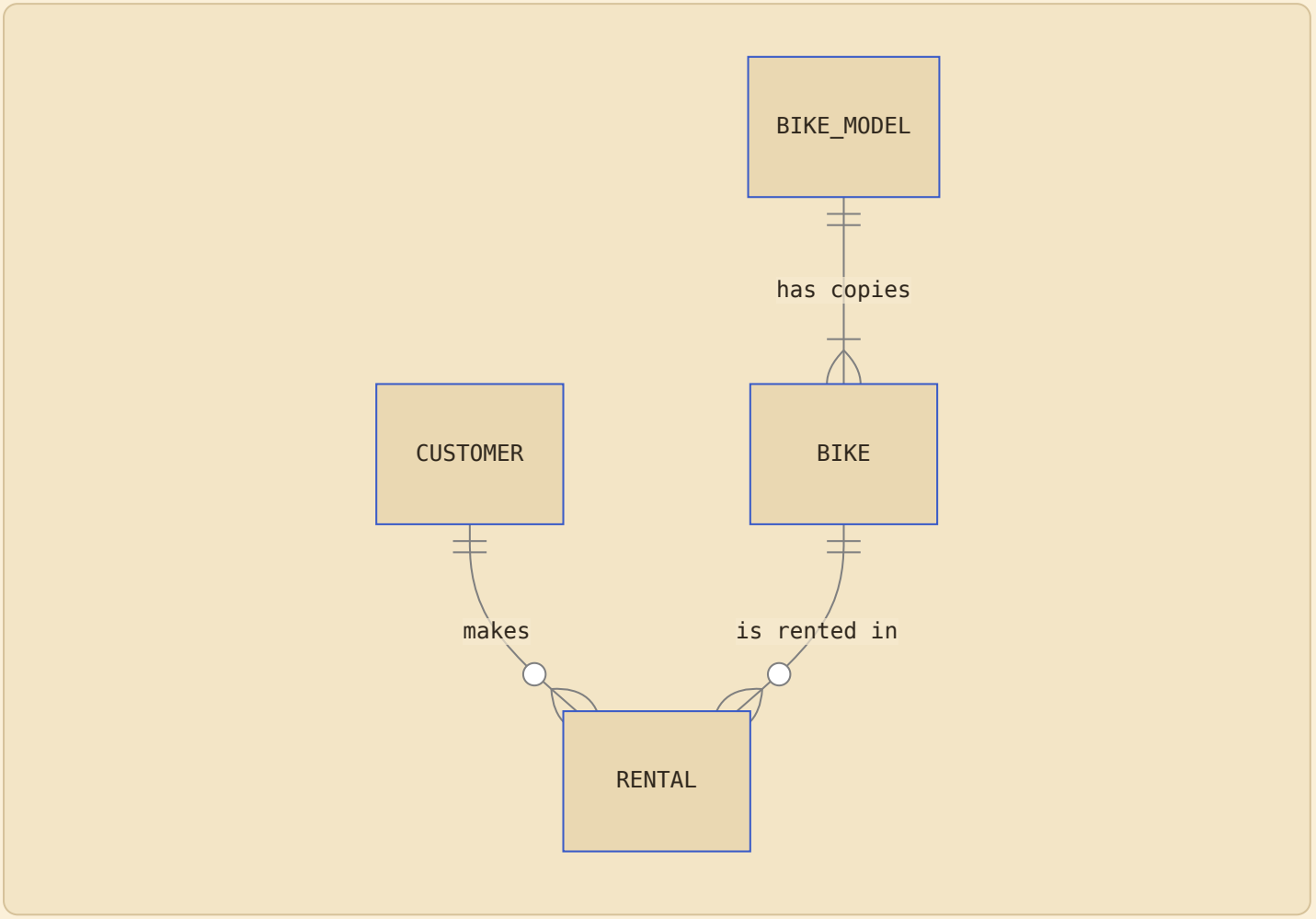
and who had it?"

If your `bike` table actually describes a **model** of bike (name, type, hourly rate), it cannot answer. The physical object with a frame number is a different thing from the product it is an example of. Add a separate entity for the physical bike and the question becomes a short join. The code got no smarter; the database simply began recording one more fact.

Figure — the same idea in the course's library example: a Book (a title) is not a Copy (the object on the shelf) (course diagram)



The bike shop's model therefore has four entities: `customer`, `bike_model` (the "title"), `bike` (the physical object) and `rental` (who had which bike, when).



Who reads a data model

Reader	What they check
Business people (shop owner, registrar, librarian)	"Is anything missing? Are the rules right?" — they spot a forgotten entity in seconds
Developers	which tables exist and what each one means
Database administrators	storage, indexes, backups (mostly at the physical level)
Analysts	where each fact lives, so reports are correct

Since non-technical readers must follow it, you describe the model in everyday language first and add technical detail afterwards — which is why §10.2 has three levels.

What data modeling is *not*

- **Not writing SQL.** SQL comes last and is nearly mechanical once the model is right (§10.15).
- **Not performance tuning.** Indexes and query plans (the step-by-step strategy the database picks for running a query) come later (Chapter 12). Quickly answering the wrong question helps nobody.
- **Not application code.** Two very different apps can share one model.

Why it matters

- **Fewer wrong answers** — rules in the model become constraints that every program must obey.
- **Less repeated data** — each fact stored once cannot contradict itself.
- **Cheaper change** — a model that matches reality absorbs new requirements; one that doesn't fights each of them.
- **Shared vocabulary** — when the team says "rental", everyone means the same thing.

TL;DR

A data model = entities (things) + attributes (facts) + relationships (links). Entities → tables, attributes → columns, relationships → foreign keys. If the model does not hold a fact, no query can recover it.

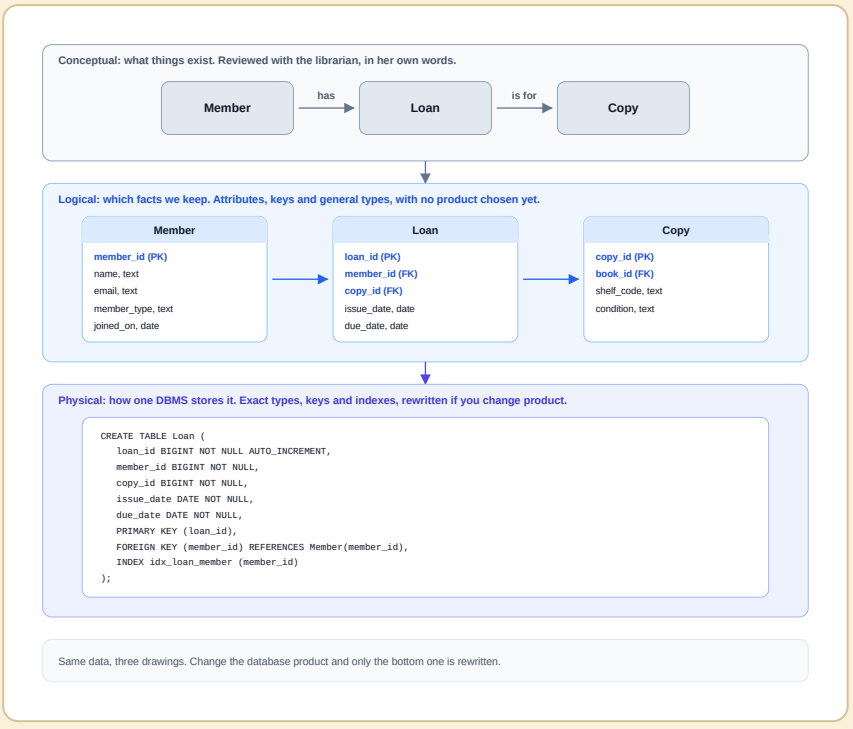
10.2 The three levels of a data model

What it is

The same model is drawn three times, each time with more detail, for a different audience:

	Conceptual	Logical	Physical
Shows	entities and relationships only	+ attributes, keys, general types (text/number/date)	+ exact types, indexes, partitioning (splitting a big table into pieces), storage options
Detail	low	medium	high
Main audience	business people	designers, developers	DBAs (database administrators), developers
Tied to a DBMS (database management system, e.g. PostgreSQL) product?	no	no	**yes**
Question it answers	"Is every thing present, and is each link correct?"	"Which facts are recorded, and which keys join the tables?"	"How do we store this well on *this* database?"
Bike-shop example	Customer *rents* Bike; Bike *is of* BikeModel	`rental(rental_id, customer_id → customer, bike_id → bike, out_at, due_at, back_at)`	`CREATE TABLE rental (... timestamp ..., INDEX on customer_id)`

Figure — the same data at the three levels (course diagram)



How it works

1. **Conceptual** — boxes and lines, in the users' own words. Technology-independent: it would be equally valid for a relational database, a document store (a database that keeps JSON-like documents instead of tables) or a paper ledger.
2. **Logical** — add attributes, primary and foreign keys, and general types. This is where **normalization** happens (removing repeated data — Chapter 11). Still product-neutral.
3. **Physical** — rewrite for one DBMS: `bigint` vs `int`, `numeric(6,2)`, `timestamp`, indexes for the known queries, identity columns (keys the database numbers automatically), partitioning.

The physical version of the rental table from `code/00_schema.sql`:

```sql
CREATE TABLE rental (
  rental_id   bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id bigint      NOT NULL REFERENCES customer(customer_id),
  bike_id     bigint      NOT NULL REFERENCES bike(bike_id),
  out_at      timestamp   NOT NULL,
  due_at      timestamp   NOT NULL,
  back_at     timestamp,                          -- NULL = bike still out (meaningful NULL)
  CHECK (due_at > out_at),
  CHECK (back_at IS NULL OR back_at >= out_at)
);
CREATE INDEX rental_customer_idx ON rental(customer_id);   -- "rentals of this customer"
CREATE INDEX rental_bike_idx     ON rental(bike_id);       -- "who has this bike"
```

The entity came from the conceptual model, the columns and keys from the logical model; only the exact types, the identity column and the two indexes are physical decisions. Identity syntax differs by product, which is exactly why this level is product-specific:

Auto-numbered key	Syntax
PostgreSQL 10+	`bigint GENERATED ALWAYS AS IDENTITY` (older: `bigserial`)
MySQL	`BIGINT AUTO_INCREMENT`
SQLite	`INTEGER PRIMARY KEY` (becomes the rowid, SQLite's built-in row number)
SQL Server	`BIGINT IDENTITY(1,1)`

Why bother with three levels

- **Every reader gets a version pitched at them.** A shop owner reviewing three boxes finds a missing entity; the same person shown `CREATE TABLE` finds nothing.
- **Mistakes are caught while cheap.** A missing entity costs a conversation at the conceptual level and a migration (a script that changes an existing schema and its data) once two million rows exist.
- **Technology is isolated.** Switch from MySQL to PostgreSQL and only the physical model is rewritten. This is **data independence** (changing how data is stored without rewriting the programs that use it; defined in Chapter 1) applied at design time.

The levels are not a one-way street: working out the logical model often shows the conceptual one was wrong, and you go back. That is cheap precisely because the conceptual drawing is small.

Common mistakes

- Showing column types to business reviewers (they stop reading).
- Calling a diagram with attributes and keys "conceptual" — once keys appear it is logical.
- Treating logical and physical as the same: *logical = what the business needs; physical = how one product stores it.*

Self-check

1. Which level first shows primary keys? → **Logical.**
2. You migrate from SQL Server to PostgreSQL. Which levels change? → **Only the physical.**
3. Where does normalization happen? → **Logical level.**

TL;DR

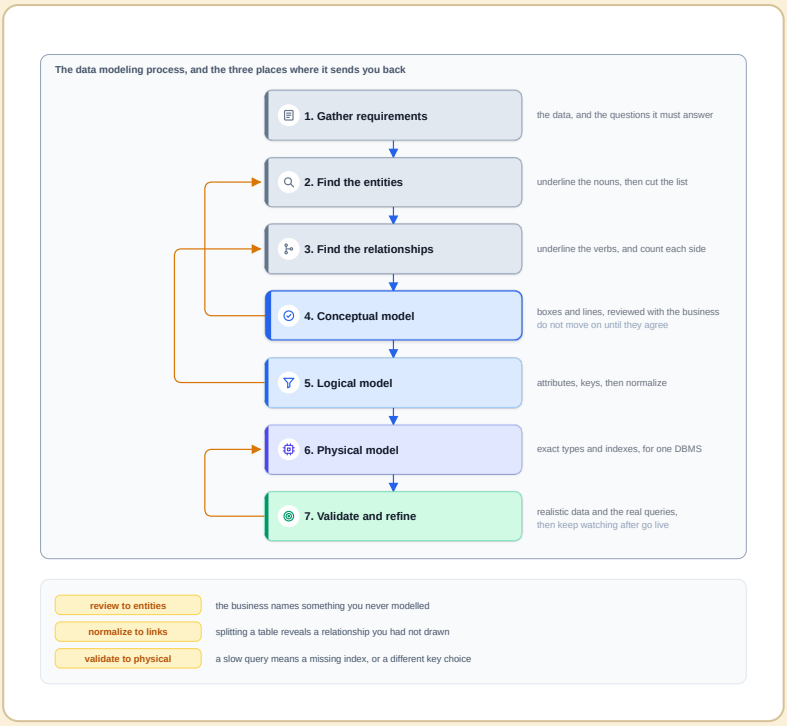
Conceptual (things + links, for the business) → logical (attributes, keys, general types, normalized) → physical (exact types, indexes, one DBMS). Only the physical level depends on the product.

10.3 The data modeling process

What it is

A fixed order of work where every step produces something you can show someone. Starting with tables is the classic mistake — it fixes the answer before you understand the question.

Figure — seven steps and the three places the work loops back (course diagram)



How it works — the bike shop, step by step

1. **Gather requirements — including the questions.** Talk to the people doing the work. Write down what must be recorded *and* what the system must answer:
   - Customers are casual or members; members may keep a bike longer.
   - The shop owns several physical bikes of each model.
   - A customer takes a bike at a time and must return it by a due time.
   - Late returns cost 2.00 per started hour.
   - Staff must see which bikes are out and who has them.

   An unanswerable question is a bug in the design, and right now it costs nothing to fix.
2. **Find the entities: underline the nouns.** customer, member, model, bike, time, fee, staff. Keep a noun as an entity only if you store *several facts* about it and there are *many instances* of it:
   - customer, bike_model, bike, rental → entities.
   - time → one value → an attribute.
   - casual/member → two kinds of customer → start with a `customer_type` attribute.
   - late fee → calculable from times → a **derived** value (§10.6), not stored.
   - staff → users of the system, not necessarily data in it.
3. **Find the relationships: underline the verbs between nouns.** customer *rents* bike; bike *is of* bike_model. For each, ask "how many on each side?" (cardinality, §10.7) and write it in words: one customer has many rentals; each rental is for one bike.
4. **Draw and review the conceptual model** with the owner, reading it back as sentences. She will correct you ("customers can also *reserve* a bike for tomorrow") — a new entity for the price of one box. Do not continue until the business agrees.
5. **Design the logical model.** Attributes, primary keys, foreign keys, general types; then normalize.

6. **Build the physical model.** Exact types, and an index for every question from step 1 (`rental.customer_id`, `rental.bike_id`).
7. **Validate, implement, refine.** Load realistic volumes (not ten rows — enough rows for slowness to show up), run the step-1 questions, check correctness and speed. After go-live (the launch to real users) keep watching: a model right for 10 thousand rentals may need a new index at 10 million.

The loops back are part of the process

Trigger	Sends you back to
Stakeholder (anyone with a say in the system) review reveals something new (reservations)	step 2 (entities)
Normalization splits a table and exposes a hidden link	step 3 (relationships)
Validation finds a slow query	step 6 (index or key choice)

Looping back is the method working, not failing: every loop costs less than releasing the error.

Trade-offs to keep in mind throughout

- **Normalize vs denormalize** — normalize first (split tables so each fact is stored once); add redundancy (deliberate duplicate copies) back only for a measured slow query (Chapter 11).
- **Growth** — rows in three years, not today; it changes key and index choices.
- **Access control** — mark sensitive data early (emails, payments).
- **Integrity** — each business rule you collected in step 1 belongs in a constraint, not in an `if` inside the application.

The model answering the step-1 questions (`code/01_bike_questions.sql`)

```sql
-- Q1: which bikes are out right now, and who has them?
SELECT b.frame_no, m.model_name, c.full_name, r.due_at
FROM rental r
JOIN bike b       ON b.bike_id = r.bike_id
JOIN bike_model m ON m.model_id = b.model_id
JOIN customer c   ON c.customer_id = r.customer_id
WHERE r.back_at IS NULL
ORDER BY b.frame_no;
-- Q2: bikes never rented (NOT EXISTS anti-join)   Q4: late fee, derived at read time
SELECT r.rental_id, c.full_name,
       ceil(extract(epoch FROM (r.back_at - r.due_at)) / 3600) * 2.00 AS late_fee
FROM rental r JOIN customer c USING (customer_id)
WHERE r.back_at > r.due_at;
```

Output:

```
--- Q1: which bikes are out right now, and who has them?
 frame_no | model_name |  full_name   |       due_at
----------+------------+--------------+---------------------
 FR-2001  | Ridge 29   | Meera Nair   | 2026-09-27 18:00:00
 FR-3001  | Volt City  | Lena Fischer | 2026-09-27 10:30:00
(2 rows)

--- Q2: which physical bikes have never been rented?
 frame_no | model_name
----------+------------
 FR-1002  | Urban 3
 FR-1003  | Urban 3
(2 rows)

--- Q3: how many bikes of each model exist, and how many are out?
 model_name | bikes | out_now
------------+-------+---------
 Ridge 29   |     1 |       1
 Urban 3    |     3 |       0
 Volt City  |     1 |       1
(3 rows)

--- Q4: late returns, with a late fee of 2.00 per started hour (fee is derived, not stored)
 rental_id | full_name  | late_fee
-----------+------------+----------
         2 | Tom Okafor |     2.00
(1 row)
```

Q2 and Q3 are only possible because `bike` (the physical object) is separate from `bike_model`.

In the interview

30-second answer: "I start from requirements *and the questions the system must answer*. Nouns give candidate entities, verbs give relationships. I draw a conceptual model and review it with the business, then add attributes and keys and normalize (logical), then choose types and indexes for the target database (physical), and validate with realistic data and the real queries. I expect to loop back."

- *How do you decide whether a noun is an entity or an attribute?* → Entity if it carries several facts and has many instances; a single value (a date, a colour) is an attribute.
- *When do you add indexes?* → At the physical level, for the queries written down in requirements — not "on every column".

TL;DR

Requirements (+ questions) → nouns = entities → verbs = relationships → conceptual review → logical (keys, normalize) → physical (types, indexes) → validate with real data; loop back when reality disagrees.

10.4 Best practices in data modeling

Two models can both work on day one and differ enormously a year later. The difference comes from small early decisions. Here are the rules, each with the reason.

Practice	Rule	Why
Questions first	Write the list of questions before drawing.	A model is only "correct" relative to the questions it must answer.
Rules in the database	Every stated rule → a `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `UNIQUE` or `CHECK`.	A constraint protects every program that ever connects, including tomorrow's ad-hoc (one-off, written for a single task) script; an `if` protects only the program it is in.
One naming convention	Decide once (`snake_case` = lower case with underscores, singular *or* plural tables) and never mix. Same thing = same name everywhere (`customer_id`, never `cust_no` elsewhere). Name FKs after their target.	Joins read naturally; nobody has to guess.
Types on purpose	Dates as `date`/`timestamp`, never text; money as `numeric`, never `float`/`double`; text sized for real data. Prefer the narrowest type that fits.	Text dates accept "31 February" and sort wrongly; floating point cannot represent most decimals exactly, so totals drift; narrower rows = more rows per page (the fixed-size block the database reads from disk) = fewer disk reads.
Decide what NULL means	`NOT NULL` whenever the fact must exist; allow `NULL` only when "missing" is meaningful (`back_at IS NULL` = bike still out).	`NULL` means unknown/not applicable, is not 0 or '' and is not equal to anything (§10.12).
Keys deliberately	Surrogate primary key (a generated id with no business meaning) + `UNIQUE` on the natural key (a real-world identifier: `email`, `frame_no`). Keep keys narrow.	Natural values change; a PK is copied into every FK and index.
Normalize, then denormalize with evidence	Default to 3NF (third normal form, the usual target of normalization); denormalize only after a measured slow query that an index could not fix — and document why.	Redundant copies can disagree (Chapter 11).
Index for real queries	Index what the requirement questions filter/join on; nothing else.	Every index slows every write and costs disk.
Design for predictable growth	Estimate rows in 3 years; don't build tables for imagined requirements.	Speculative tables become the parts nobody understands.
Document meaning	For each entity/column: what it means, allowed values, units.	Anyone can see `status` is text; only docs say which 4 values it holds.

The CHECK constraints in the bike schema are this practice in action: a rental due before it starts, or returned before it left, is refused for every client.

```
CHECK (due_at > out_at),
CHECK (back_at IS NULL OR back_at >= out_at)
```

Common mistakes

- **A list in one column** — `phones = '555-1, 555-2'` cannot be searched, counted or constrained → separate table.
- **Numbered columns** — `phone1, phone2, phone3` stores a list across columns and breaks the day someone owns a fourth phone.
- **One huge table** — 60 columns, mostly empty per row, is several entities that were never separated.
- **Over-normalizing** — seven joins for an everyday question is a real cost; 3NF is the usual stopping point.
- **Rules only in code** — some other program will break them.

Real-world examples

- **Stripe's API** (the programming interface of the payments company Stripe) represents amounts as integers in the smallest currency unit (cents) — the same idea as "never float for money".
- **ISO country codes** (the international standard two-letter codes such as `IN`, `GB`) are `char(2)` — "size text for the data" in the wild.

TL;DR

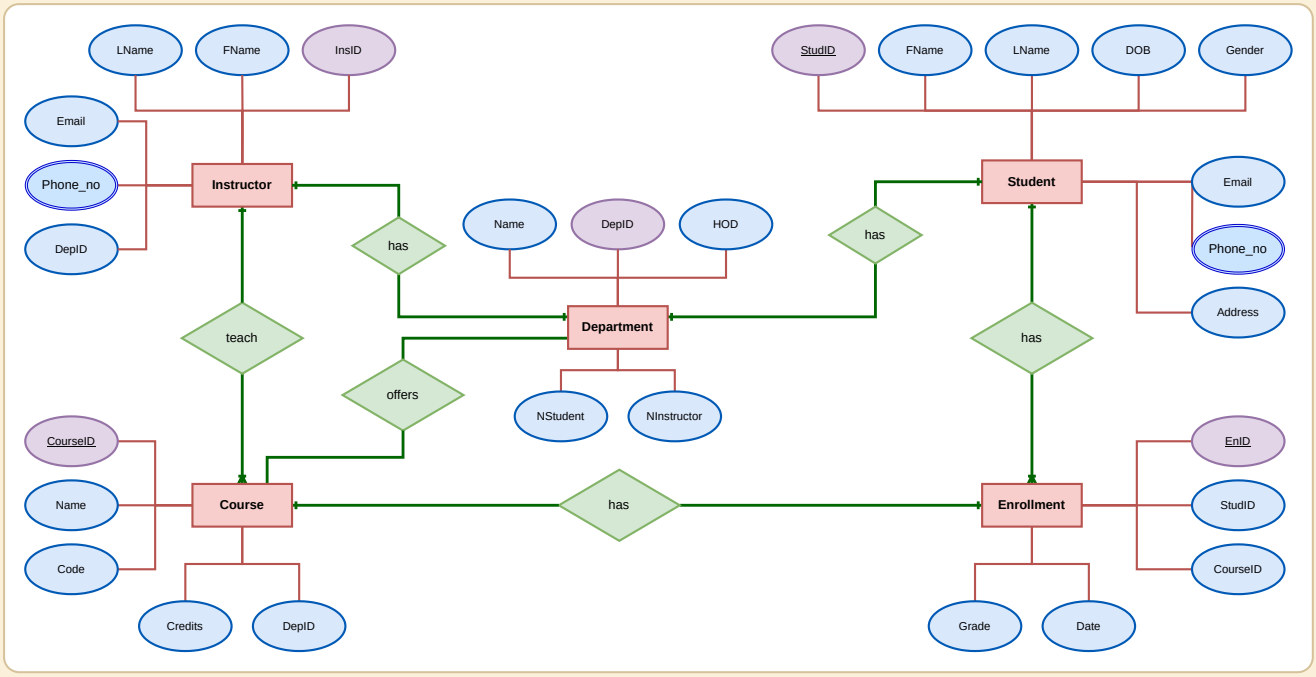
Questions first; constraints in the schema; one naming style; correct types (dates as dates, money as `numeric`); explicit `NOT NULL`; surrogate PK + unique natural key; normalize to 3NF, denormalize with measurements; index only real queries; document meaning.

10.5 The Entity-Relationship (ER) model and its notations

What it is

The **Entity-Relationship model** (ER model), proposed by Peter Chen in 1976, lets you sketch entities, attributes and relationships on paper long before you create a table. An **ER diagram (ERD)** is one such drawing. It is how you draw the conceptual model (entities + relationships only) and then the logical model (the same diagram with attributes, keys and cardinalities added). It never names a DBMS.

Figure — a complete Chen-notation ER diagram for a student management system; by §10.10 you can read every symbol (course diagram)



Two notations for the same model

	Chen notation	Crow's-foot notation (IE / Martin)
Entity	rectangle	rectangle with a title bar
Attribute	ellipse attached by a line	a row listed *inside* the rectangle
Relationship	diamond labelled with a verb	the line itself, labelled with a verb
Cardinality	`1`, `N`, `M` written near the diamond (or `(min,max)` pairs)	the *shape* of each line end (see table below)
Participation	double line = total, single = partial	inner symbol: bar = at least one, circle = may be zero
Used by	textbooks, exams, teaching	nearly every design tool (dbdiagram, MySQL Workbench, ERwin, Lucidchart), and `mermaid erDiagram`
Strength	shows the concepts explicitly	compact; fits 40 tables on a screen

Neither is "more correct". Learn Chen to understand the concepts; you will read crow's foot immediately afterwards. In this book the course's Chen figures show the concepts, and every schema is also drawn in crow's foot with mermaid.

Reading crow's-foot line ends — each end has an *outer* symbol (maximum) and an *inner* symbol (minimum):

Mermaid syntax (left end / right end)	Picture	Meaning (min, max)
`|o` / `o|`	circle + bar	zero or one `(0,1)`
`||` / `||`	bar + bar	exactly one `(1,1)`
`}o` / `o{`	circle + crow's foot	zero or more `(0,N)`
`}|` / `|{`	bar + crow's foot	one or more `(1,N)`

So `CUSTOMER ||--o{ RENTAL : makes` reads: *each rental belongs to exactly one customer; a customer makes zero or more rentals*. Strictly, mermaid draws a solid line `--` for an **identifying** relationship (the child's key includes the parent's — a weak entity, §10.9) and a dashed line `..` for a non-identifying one. This book draws every line solid for readability; a weak entity is recognised by a PK that contains the owner's key.

Strengths and limits of an ERD

- **Good at:** making the things, their links and the counts on each link visible, so even a non-technical reviewer notices what is missing.
- **Not a table diagram:** a single M:N diamond turns into *three* tables (§10.7).
- **Not a performance document:** no indexes or storage.
- **Not a process diagram:** it says a student enrols in a course, not which screens or approvals happen.

> **TL;DR**

The ER model (Chen, 1976) draws entities, attributes and relationships. Chen = rectangles/ellipses/diamonds (teaching); crow's foot = attributes inside boxes and cardinality in line ends (industry, mermaid). Same model, two pictures.

10.6 Entities and the five kinds of attribute

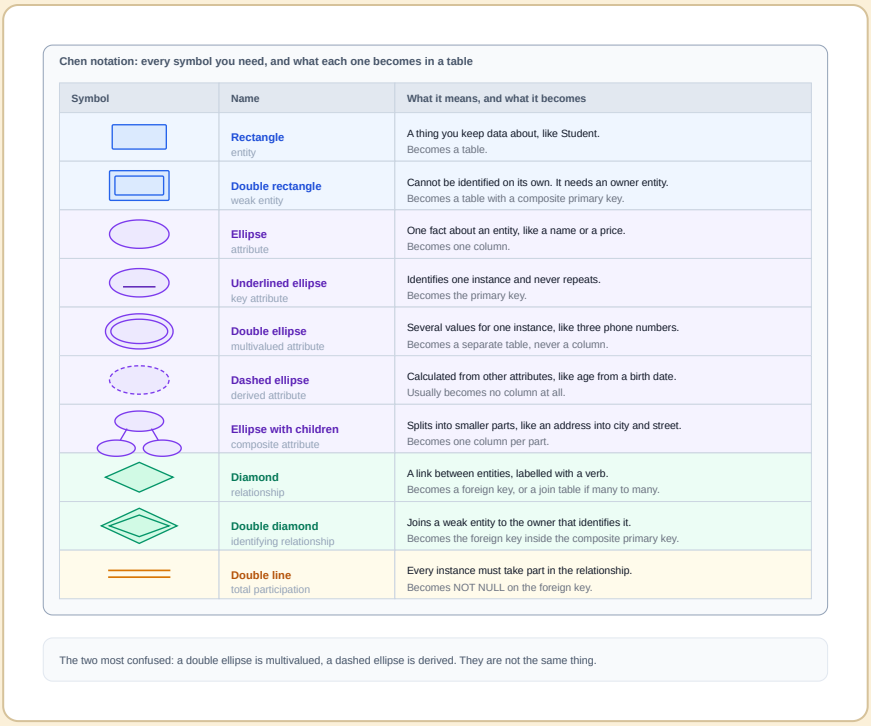
Entities: type, instance, set

An **entity** is any object or concept in the business that you record data about and can distinguish from others like it. Three words are used loosely but mean different things:

Term	Meaning	Example
Entity type	the general kind; what the rectangle is labelled	`Employee`
Entity instance	one particular member	employee #2, Ravi Menon
Entity set	all instances that exist right now	every row in the `employee` table today

You *draw* types; the database *stores* sets.

Figure — every Chen symbol and its meaning (course diagram)

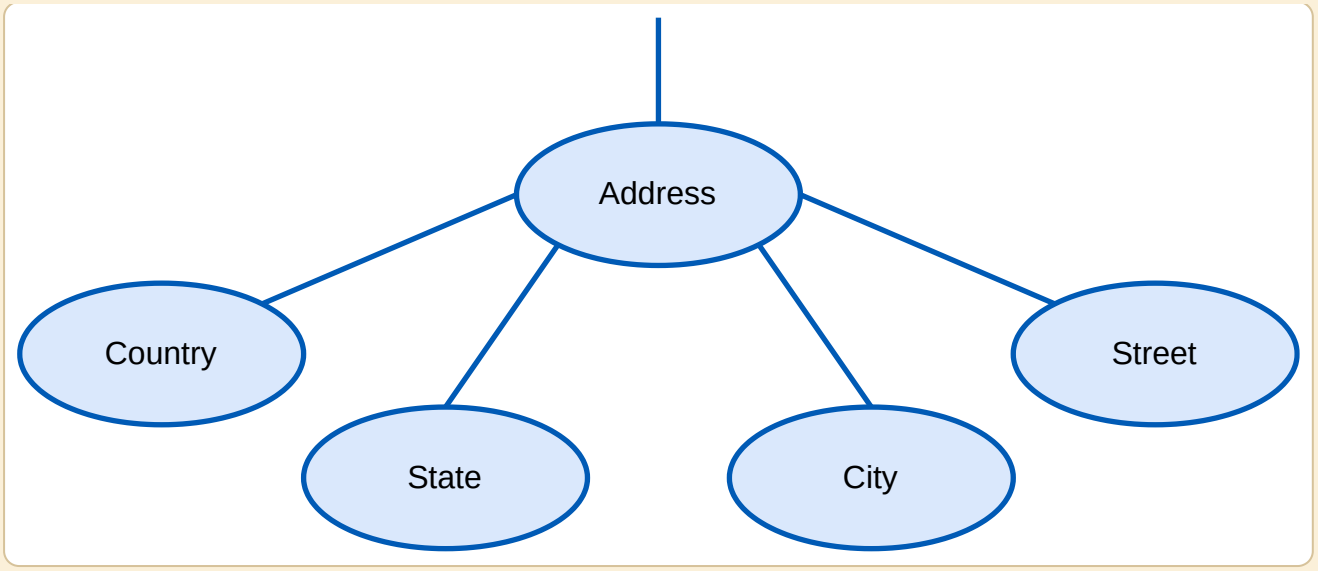


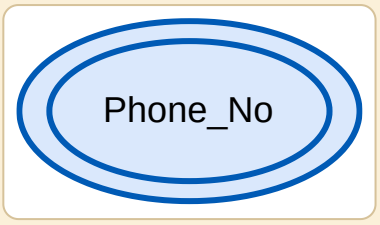
The five kinds of attribute

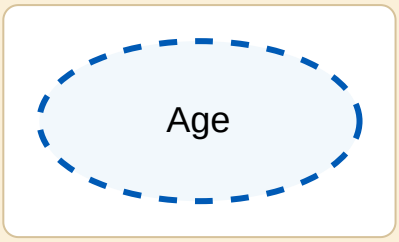
Kind	Chen symbol	Example	Becomes in tables
Key	ellipse, name **underlined**	`emp_id`	the primary key
Simple (atomic)	plain ellipse	`birth_date`	one column
Composite	ellipse with child ellipses	`full_name` = first + middle initial + last	one column **per part**
Multivalued	**double** ellipse	`phone` (an employee may have several)	a **separate table** (owner key + value)
Derived	**dashed** ellipse	`age` (from `birth_date`)	usually **no column** — computed on read

The two most confused: **double ellipse = multivalued**, **dashed ellipse = derived**.

Figure — composite, multivalued and derived attributes in Chen notation (course diagrams)



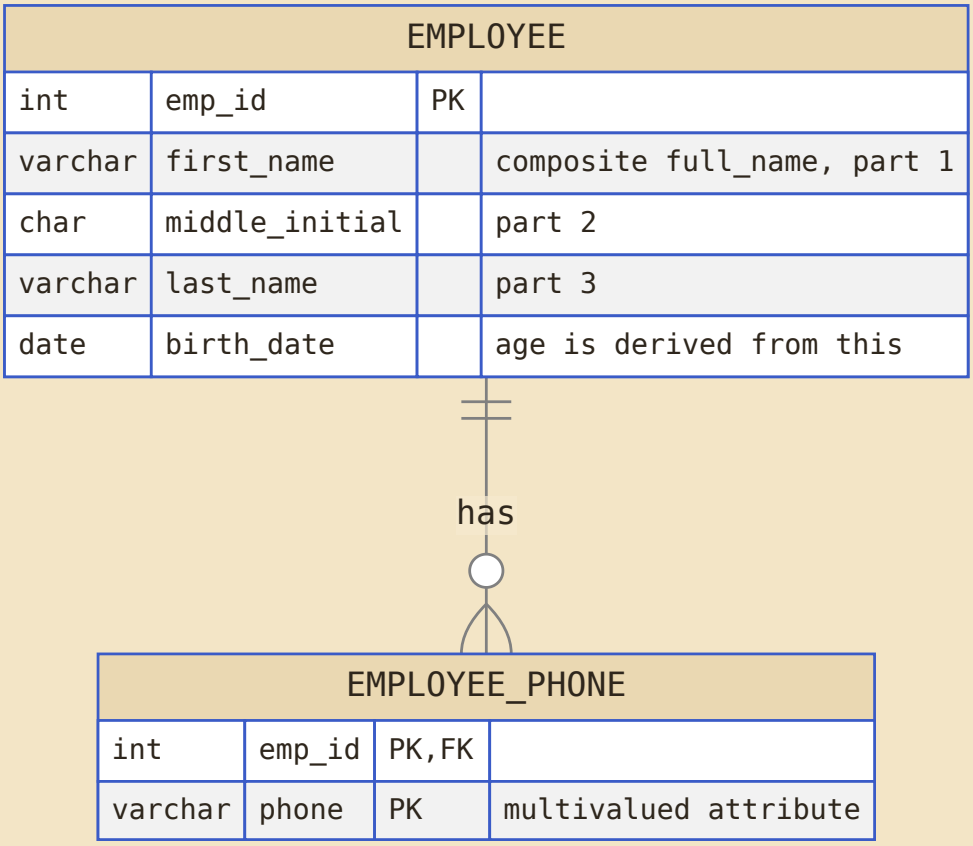




Why each "becomes" rule holds:

- **Composite → split.** If the whole address sits in one column you cannot search by city. (If you *never* query the parts, one column is fine — split the day you need to.)
- **Multivalued → own table.** A relational cell holds exactly one value (§10.12), so a list cannot be a column. `phone1, phone2, phone3` is the mistake the double ellipse exists to prevent.
- **Derived → compute.** A stored `age` is wrong a year later. Store a derived value only if it is expensive to compute *and* you accept keeping it fresh — or if it must be **frozen in time** (an order total at the prices of the day of purchase is not really derivable from *today's* data).

In crow's foot there are no special shapes: the composite is already split into columns, the multivalued attribute is already a separate box, and a derived attribute is usually omitted or marked with a comment:



Self-check

1. A double ellipse means…? → **Multivalued** (e.g. phone numbers).
2. `age` next to `birth_date` — which kind, and what column? → **Derived**; no column, compute it.
3. `address` split into street/city/postcode is…? → **Composite**; one column per part.

TL;DR

Key (underlined) → PK; simple → column; composite → one column per part; multivalued (double ellipse) → separate table; derived (dashed) → computed, not stored.

10.7 Relationships: degree, cardinality and participation

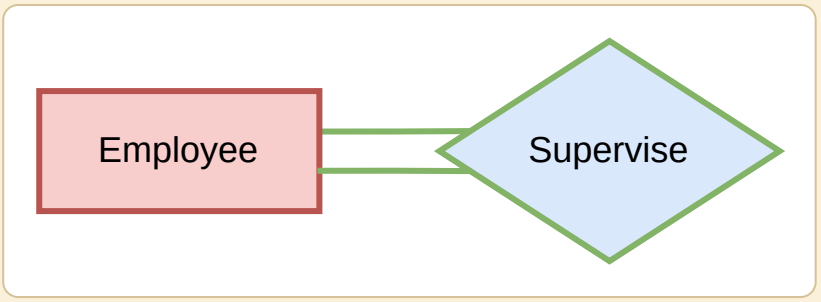
A **relationship** is an association between entities, drawn in Chen as a diamond labelled with a verb that reads well aloud ("Employee *works in* Department", not "Employee *has* Department"). As with entities, the **relationship type** is the general association you draw, and the **relationship set** is the actual pairs stored (employee 2 works in department 10, …).

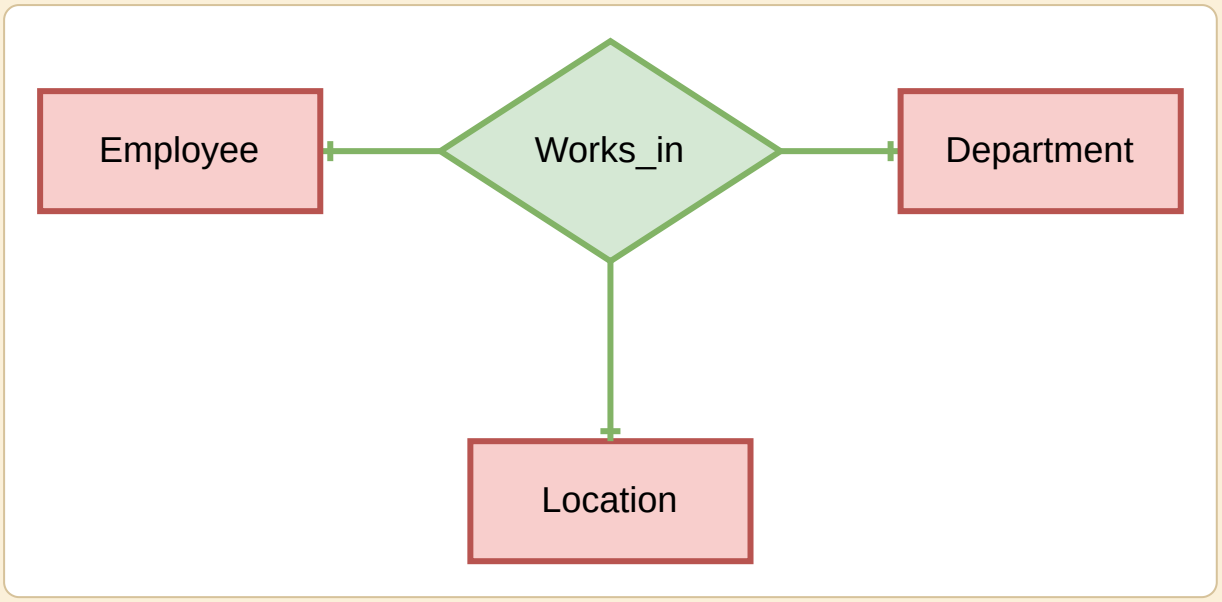
Three separate questions describe every relationship: **degree** (how many entity *types*), **cardinality** (how many *instances* on each side) and **participation** (is taking part *required*).

Degree

Degree	Name	Example	Tables
1	**Unary** (recursive)	an employee *supervises* employees	self-referencing FK: `employee.supervisor_id → employee.emp_id`
2	**Binary**	employee *works in* department	the usual case
3	**Ternary**	supplier *supplies* part *to* site	one table with three FKs, usually all three in the PK

Figure — unary and ternary relationships (course diagrams)





A ternary relationship records a fact that three binary relationships cannot: *which* combination is true. The demo in `code/04_ternary_and_weak_entity.sql` stores three supplier-part-site facts, then keeps only the binary pairs and joins them back:

```
WITH sp AS (SELECT DISTINCT supplier_id, part_id FROM supplies),
     ss AS (SELECT DISTINCT supplier_id, site_id FROM supplies)
SELECT ... FROM sp JOIN ss USING (supplier_id) ...   -- rebuild from pairs
```

Output:

```
--- the ternary fact: who supplies what, where
 supplier | part  | city
----------+-------+-------
 Acme     | bolt  | Leeds
 Acme     | panel | Pune
 Bolt Co  | bolt  | Pune
(3 rows)

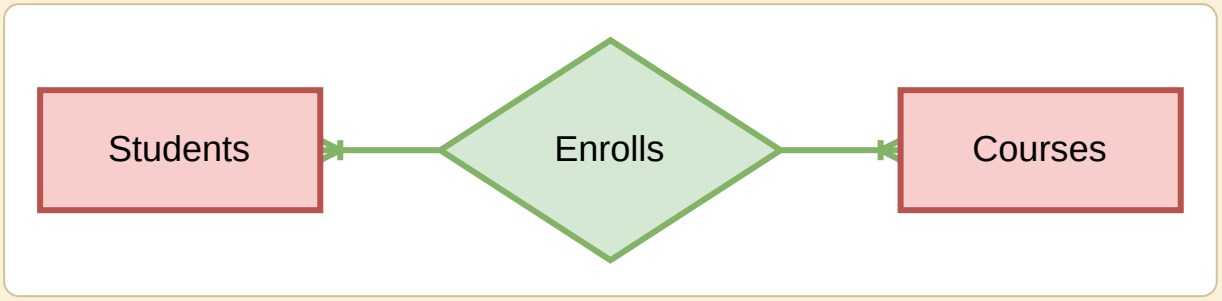
--- keep only binary pairs (supplier-part, supplier-site) and join them back:
 supplier | part  | city  | really_true
----------+-------+-------+-------------
 Acme     | bolt  | Leeds | t
 Acme     | bolt  | Pune  | f
 Acme     | panel | Leeds | f
 Acme     | panel | Pune  | t
 Bolt Co  | bolt  | Pune  | t
(5 rows)
```

From the pairs alone you cannot tell whether Acme sends bolts to Pune. Ternary relationships are rare, though — before drawing one, check the fact truly needs all three at once (Chapter 11's 5NF, §11.13, is the formal version of this question).

Cardinality (mapping cardinality)

Cardinality	Meaning	Example
1:1	each side links to at most one of the other	employee ↔ payroll record
1:N	one on the left, many on the right	a department employs many employees
N:1	1:N viewed from the opposite side	many employees work in one department
M:N	many on both sides	employees work on many projects; a project has many employees

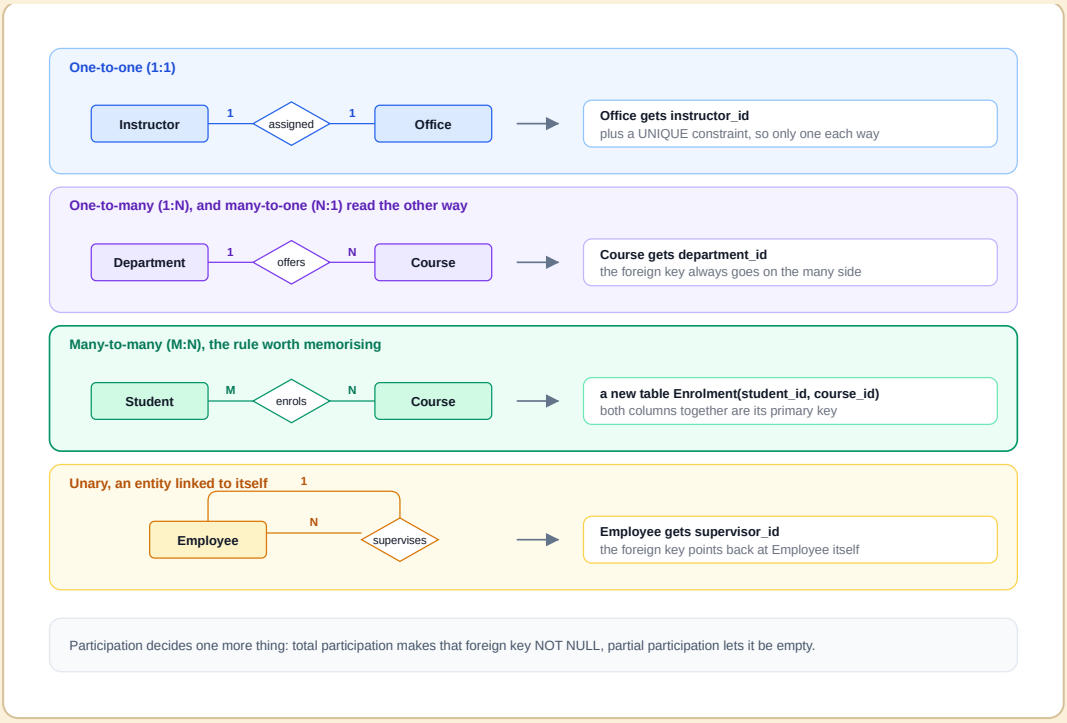
Figure — many-to-many in Chen notation (course diagram)



1:N and N:1 are one relationship described from opposite ends; both names are correct.

How each cardinality becomes tables

Figure — cardinality to foreign keys (course diagram)



Cardinality	Rule	Why
1:N / N:1	FK on the **many** side (`employee.dept_id`)	the "one" side would need a list in a cell
M:N	a new **junction table** (also *join*, *bridge* or *associative* table) holding both FKs, which together form its PK (`works_on(emp_id, project_id)`)	there is no single side to put it on
1:1	FK on **either** side + `UNIQUE`; prefer the side with total participation so it can be `NOT NULL`	without `UNIQUE` it silently becomes 1:N
Unary	FK to the same table	

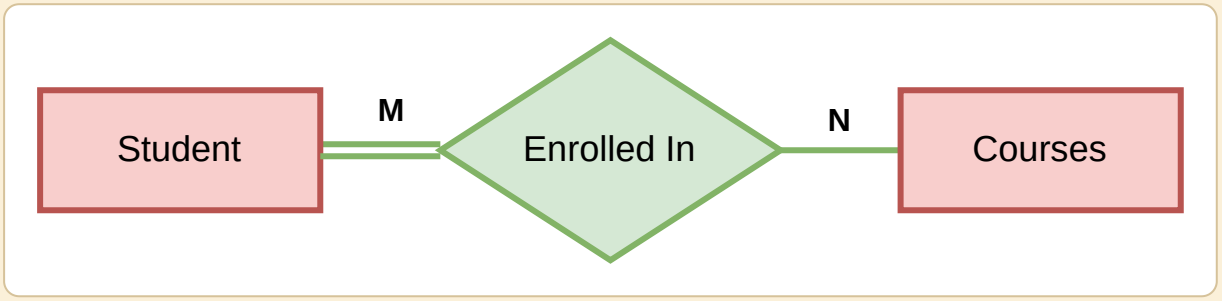
Memorise this one: **every M:N link in a diagram costs three tables in SQL.**

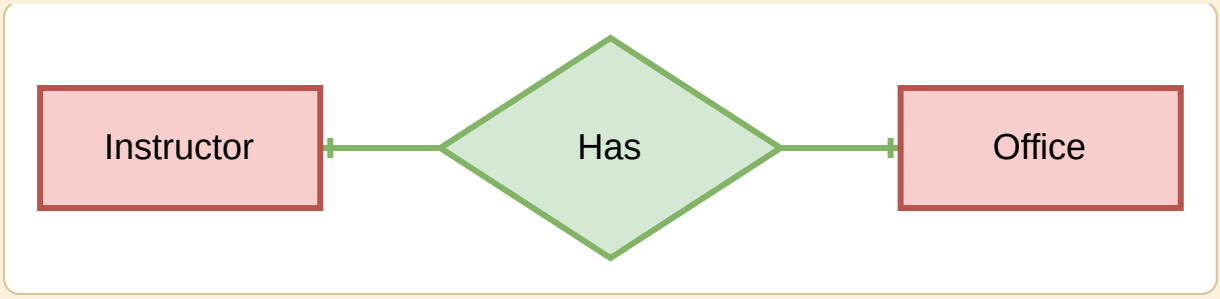
Participation

Participation says whether every instance *must* take part.

	Chen drawing	Meaning	Becomes
Total (mandatory)	double line	every employee must work in a department	FK `NOT NULL`
Partial (optional)	single line	an employee may have no supervisor	FK allows `NULL`

Figure — total (double line) and partial (single line) participation (course diagrams)





Skipping participation is easy and expensive: it decides whether a column can be empty, and that decision affects every query (outer vs inner joins, `IS NULL` checks).

The (min, max) notation

Design tools often label each end with a pair **(min, max)** — the fewest and most times one instance can take part:

Pair	Meaning	Participation	Crow's foot
`(0,1)`	at most once, maybe never	partial	`o|`
`(1,1)`	exactly once	total	`||`
`(0,N)`	any number, including none	partial	`o{`
`(1,N)`	at least once	total	`|{`

The minimum is the participation; the maximum is the cardinality. That is why tools prefer the pair: one label says both.

Careful: in Chen's `(min,max)` style the pair is usually written next to the entity it describes ("each employee works in (1,1) departments"), which is the *opposite* end from where crow's foot draws its symbol. Always check a diagram's legend.

In the interview

30-second answer: "Degree counts the entity types involved — one (unary), two (binary) or three (ternary). Cardinality is how many instances link: 1:1, 1:N or M:N. Participation is whether taking part is mandatory. 1:N puts the FK on the many side; M:N needs a junction table; 1:1 is an FK plus UNIQUE; total participation makes the FK NOT NULL."

- *Where does the FK go in a 1:N?* → Many side. *Why not the one side?* → It would need a list in one cell, which violates 1NF (first normal form: one value per cell).
- *How do you enforce 1:1 in SQL?* → FK + `UNIQUE` on it (or share the same PK in both tables).
- *Can you enforce "every department has at least one employee" with a plain FK?* → No; an FK constrains the child side. You need a deferred constraint trigger (a trigger — code the database runs automatically on a change — whose check waits until commit) or application logic.

TL;DR

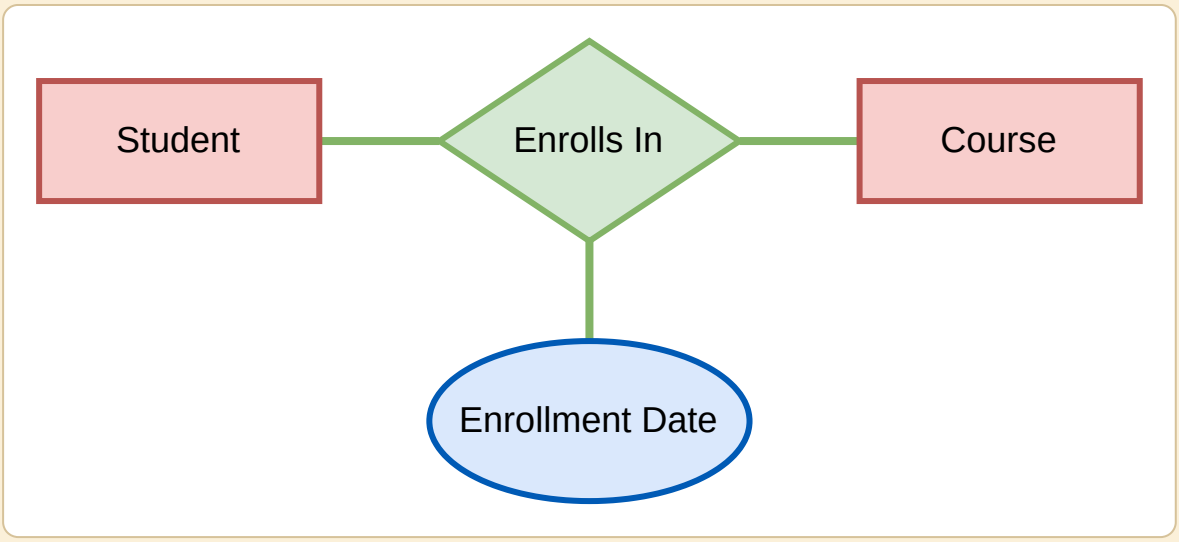
Degree = number of entity types; cardinality = max instances per side; participation = min (mandatory or not). 1:N → FK on many side; M:N → junction table; 1:1 → FK + UNIQUE; unary → self-FK; total → NOT NULL.

10.8 Attributes of relationships

What it is

Some facts describe the **link**, not either entity. The hours an employee spends on a project depend on *both* the employee and the project. In Chen notation such an attribute hangs off the diamond.

Figure — an attribute on a relationship (course diagram)



The test: *would you need to know both entities to know the value?*

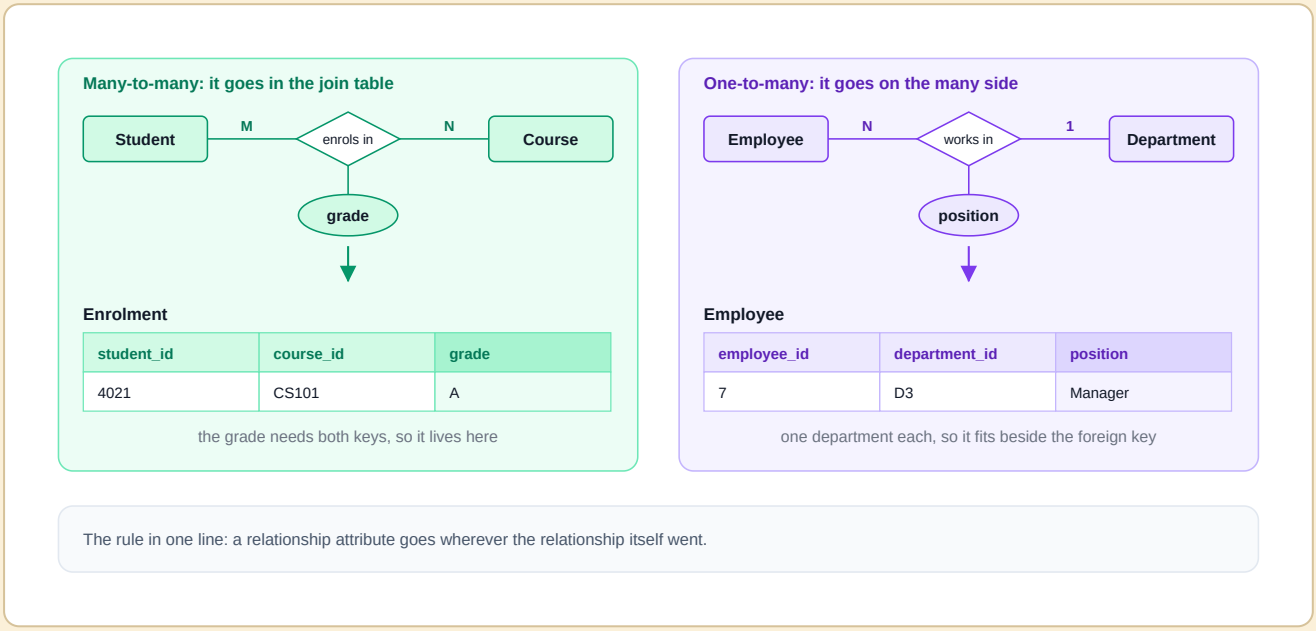
Attribute	Needs	Belongs on
`weekly_hours`	employee **and** project	the relationship *works on*
`start_date` of an assignment	employee **and** project	the relationship
`title`	project only	Project
`last_name`	employee only	Employee

Where it lands in tables

Put the attribute wherever the relationship itself ended up:

- **M:N** → a column in the junction table: `works_on(emp_id, project_id, weekly_hours)`.
- **1:N** → the many side, beside the FK (if an employee works in exactly one department, `role_in_dept` sits on `employee` next to `dept_id`).
- **1:1** → wherever the FK went.

Figure — placement depends on cardinality (course diagram)



Putting it on an entity forces a false rule: `weekly_hours` on `employee` allows only one number per employee, however many projects they work on.

When a relationship should become an entity

When a link collects many facts (a rental has out time, due time, return time, fee), promote it to an entity. The tables come out identical; the diagram just gains a box for a noun people already use. **Test:** if the business has a noun for the link ("rental", "loan", "enrolment", "booking"), make it an entity. That is why `rental` in the bike shop is an entity, not a diamond between Customer and Bike.

> **TL;DR**
>
> If a fact needs both ends of a link, it is a relationship attribute; it lands in the junction table (M:N) or the many side (1:N). A link with its own name and many facts becomes an entity.

10.9 Weak entities

What it is

A **weak entity** has nothing of its own that singles it out. "Room 101" is ambiguous — many buildings have one. Only *(Main building, room 101)* identifies it. The entity it depends on is the **owner** (or identifying / strong entity).

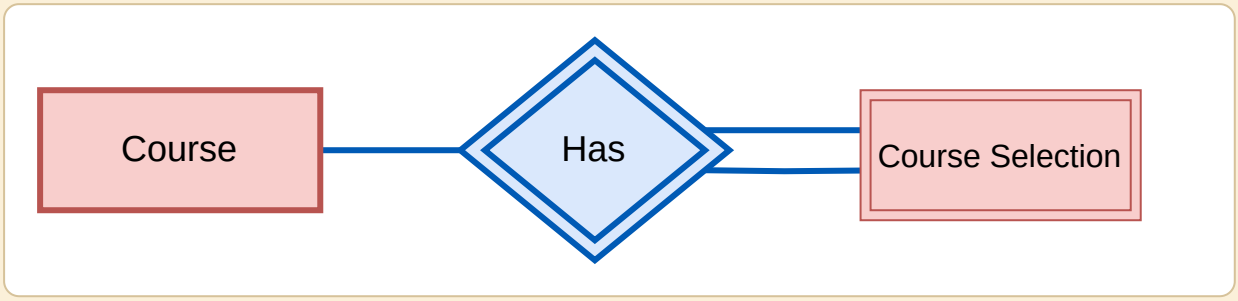
Four properties:

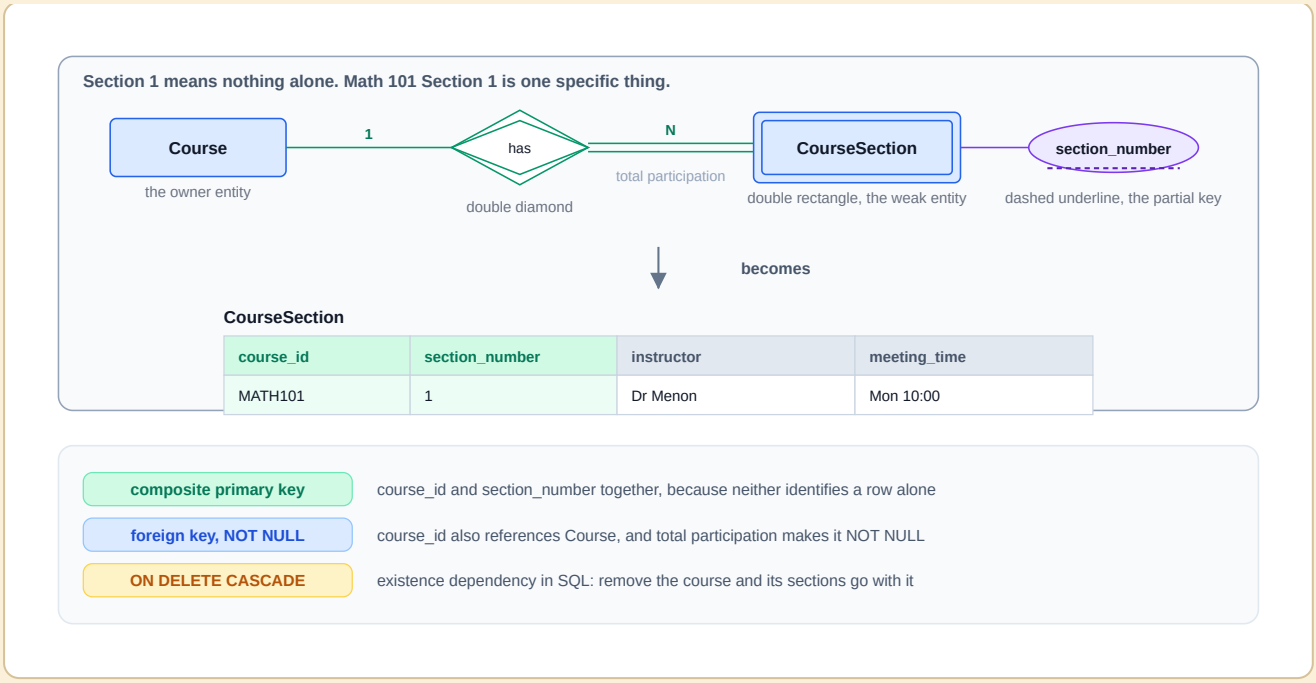
1. **No key of its own** — nothing inside it is unique across all instances.
2. **A partial key** (also called a **discriminator**) — unique *within one owner* (`room_no`).
3. **Existence dependency** — delete the building and its rooms mean nothing.
4. **Total participation in the identifying relationship** — no room belongs to no building.

How it is drawn

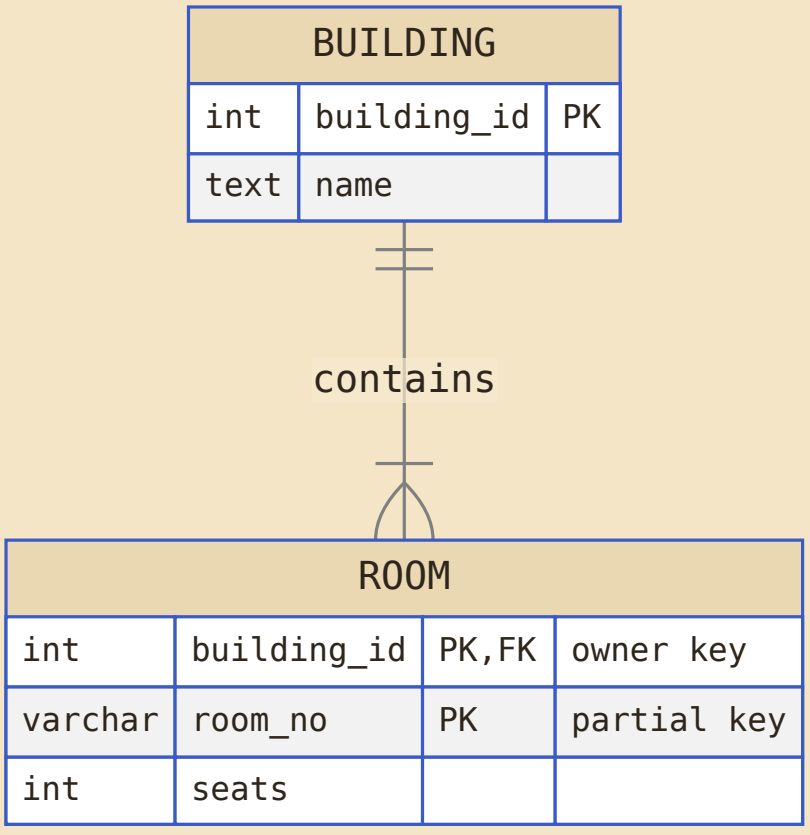
Chen symbol	Meaning
double rectangle	the weak entity
double diamond	the **identifying relationship**
dashed underline	the partial key
double line (weak side)	total participation

Figure — a weak entity in Chen notation, and its table (course diagrams)





In crow's foot (mermaid), a weak entity is shown by an **identifying (solid) line** and a PK that includes the owner's key:



What it becomes in SQL — two valid options

```
-- Option A: composite key = owner key + partial key
CREATE TABLE room_a (
  building_id int NOT NULL REFERENCES building ON DELETE CASCADE,
  room_no     varchar(6) NOT NULL,                -- partial key (discriminator)
  seats       int,
  PRIMARY KEY (building_id, room_no)
);
-- Option B: surrogate key + UNIQUE on (owner key, partial key)
CREATE TABLE room_b (
  room_id     int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  building_id int NOT NULL REFERENCES building ON DELETE CASCADE,
  room_no     varchar(6) NOT NULL,
  seats       int,
  UNIQUE (building_id, room_no)                   -- without this, duplicates slip in
);
```

Output (both options refuse a duplicate room, and cascade when the owner goes):

```
--- both options refuse a second room 101 in building 1
NOTICE:  room_a: duplicate key value violates unique constraint "room_a_pkey"
NOTICE:  room_b: duplicate key value violates unique constraint "room_b_building_id_room_no_key"
--- existence dependency: delete the Annex, its rooms go too
 room_a_rows | room_b_rows
-------------+-------------
           1 |           1
(1 row)
```

	Composite PK (A)	Surrogate PK (B)
Enforces "no duplicate partial key per owner"	automatically	only with the extra `UNIQUE` (easy to forget)
Referencing it from other tables	two columns	one column
Key width copied into child FKs/indexes	wider	narrow

Common examples: order → order line (`(order_id, line_no)`), building → room, invoice → invoice line. **Weak or not is a design decision:** bank transactions numbered per account are weak; with a global transaction id they are strong. A reasonable default is option B *with* the unique constraint.

Self-check

1. What is a partial key? → An attribute unique only within one owner (e.g. `line_no` within an order).
2. Which SQL clause expresses existence dependency? → `ON DELETE CASCADE` on the owner FK.
3. Why is the owner FK `NOT NULL`? → Participation in the identifying relationship is total.

TL;DR

Weak entity = identified only together with its owner. PK = owner key + partial key (or surrogate + `UNIQUE(owner, partial)`), owner FK `NOT NULL` and `ON DELETE CASCADE`. Chen: double rectangle, double diamond, dashed underline.

10.10 Worked example — an employee-management ER diagram

The brief

A company wants to:

1. Record each employee: name, date of birth, one or more phone numbers, and the department they belong to (every employee has exactly one department).
2. Keep one payroll record per employee (monthly salary, cash advance, tax); no payroll record is shared by two employees. Net pay is salary − advance − tax.

3. Record who supervises whom (the CEO has no supervisor).
4. Record which projects employees work on and for how many hours a week (many-to-many).
5. Record each employee's dependants (children, spouse) for insurance; a dependant is identified by name *within* an employee.

Step 1 — entities (nouns)

Employee, Department, Payroll, Project, Dependant. ("Record", "information" and "number" are words about data, not things.)

Step 2 — attributes, classified

Entity	Attribute	Kind	Reason
Employee	`emp_id`	key	issued once, never repeats
	`full_name`	composite	first name, middle initial, last name
	`birth_date`	simple	one value
	`phone`	**multivalued**	"one or more"
	`age`	**derived**	from `birth_date`
Payroll	`payroll_id`	key	
	`monthly_salary`, `cash_advance`, `tax`	simple, **stored**	typed in by a person — nothing computes them
	`net_pay`	**derived**	salary − advance − tax
Department	`dept_id` (key), `dept_name`		
Project	`project_id` (key), `title`		
Dependant	`dep_name`	**partial key**	unique only within one employee
	`relationship`	simple	

The recurring trap is salary vs net pay: **if someone types the value in, it is stored; if you can compute it from stored attributes, it is derived.**

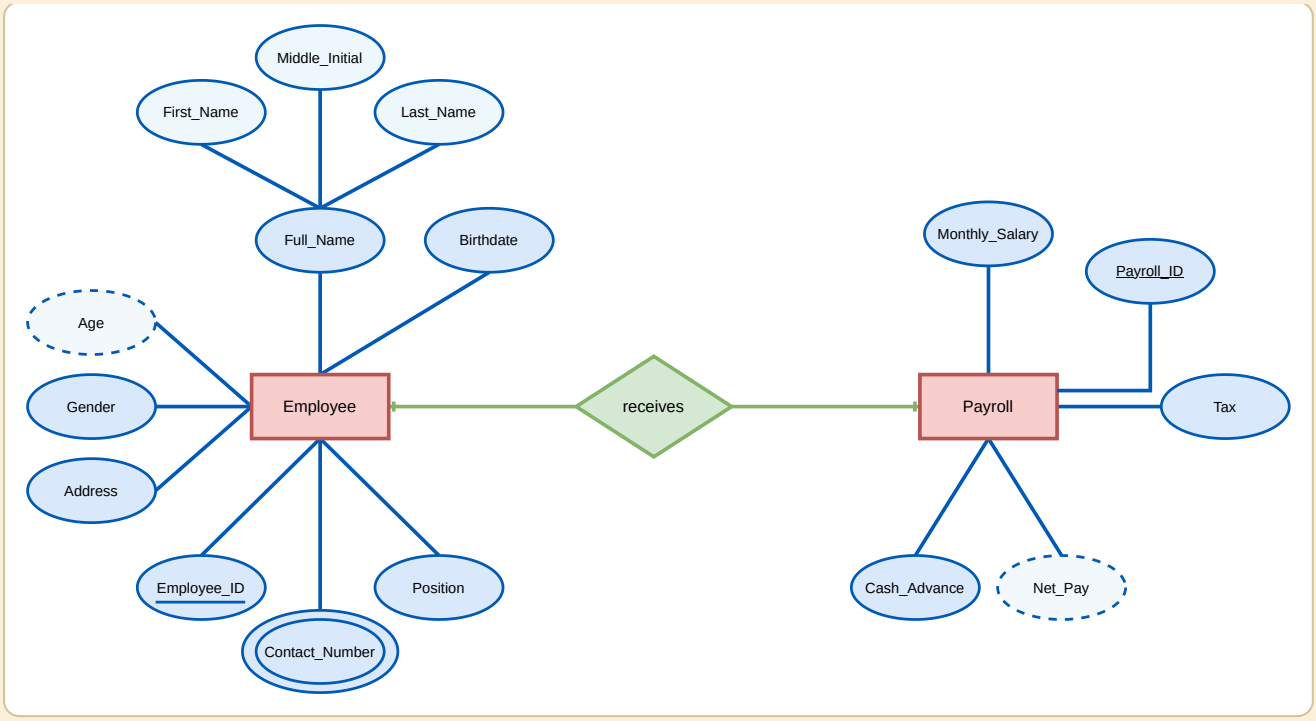
Steps 3–4 — relationships with degree, cardinality and participation

Relationship	Degree	Cardinality	Participation
Employee *works in* Department	binary	N:1	employee total (brief says "exactly one")
Employee *is paid through* Payroll	binary	1:1	both total
Employee *supervises* Employee	unary	1:N	partial (CEO has none)
Employee *works on* Project, attribute `weekly_hours`	binary	M:N	partial
Employee *has* Dependant (identifying)	binary	1:N	dependant total

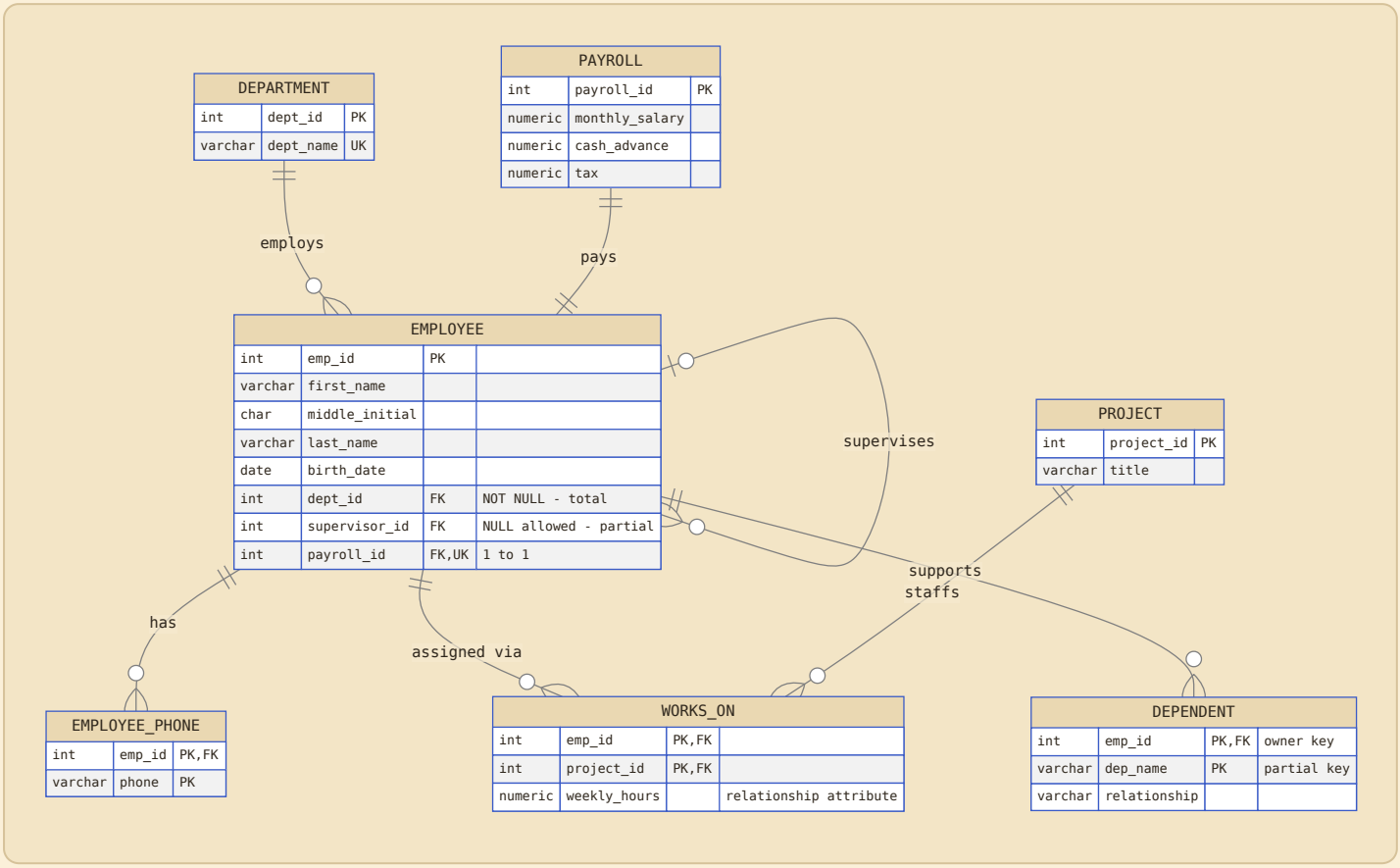
Step 5 — the diagram

The course's Chen diagram for the core of this system (Employee + Payroll, with composite, multivalued and derived attributes) shows every special symbol at once:

Figure — Employee and Payroll in Chen notation: underlined keys, composite Full_Name, double-ellipse Contact_Number, dashed Age and Net_Pay (course diagram)



The full design in crow's-foot notation:



Read it back as sentences — if one sounds wrong, the diagram is wrong: "Each employee works in exactly one department; a department employs zero or more employees (the brief does not require every department to have staff). Each employee has exactly one payroll record and vice versa. An employee may supervise several employees and has at most one supervisor…"

Design check: does Payroll deserve its own table?

A 1:1 with total participation on **both** sides usually describes one thing stored in two places. Reasons to keep two tables anyway:

- **Access control** — salaries can be granted to HR only (`GRANT` gives a user or role a permission: `GRANT SELECT ON payroll TO hr`; see Chapter 14).
- **Row width** — the frequently read `employee` row stays narrow.
- **Optional data** — if many employees had no payroll row, a separate table stores nothing for them instead of several `NULL`s.

Reasons to merge: no join, and no way to have an employee without payroll. Either answer is acceptable in an interview **if you notice the question and justify your choice.** Here we keep them separate for access control. §10.15 converts this diagram to tables and runs it.

TL;DR

Nouns → entities; classify every attribute (key/simple/composite/multivalued/derived); settle degree, cardinality and participation separately for each verb; read the diagram back as sentences; a 1:1 with total participation both ways should always prompt "one table or two?".

10.11 The relational model

What it is

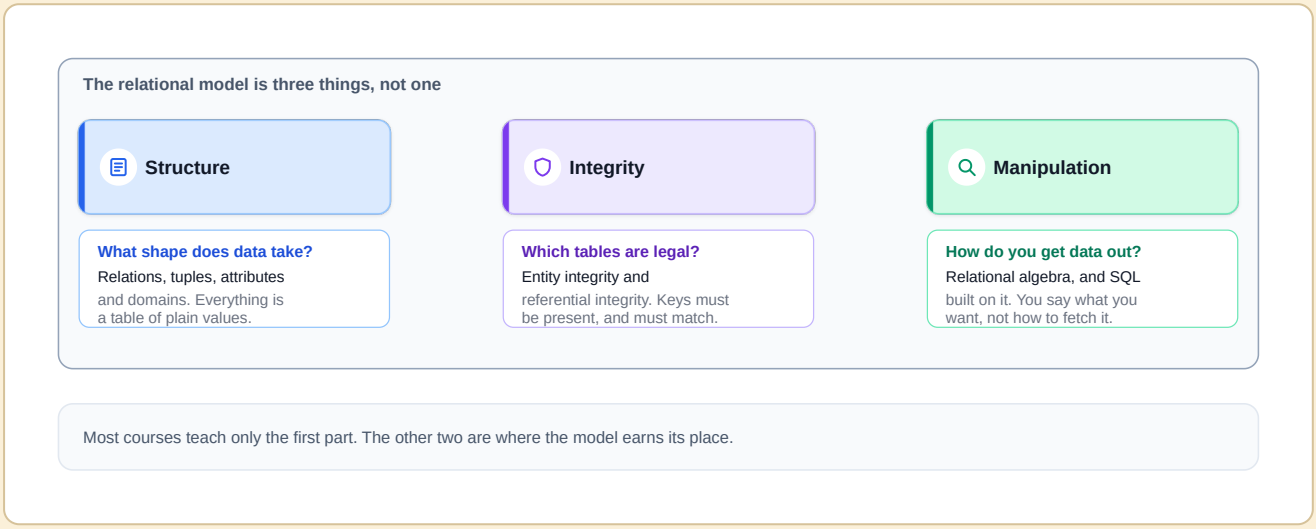
The ER diagram is a drawing; a database stores **relations** (tables). The **relational model**, published by **Edgar F. Codd in 1970**, says: store all data as tables of values, and let programs *describe* the data they want while the system works out how to fetch it.

Before it, hierarchical and network databases (older designs where records were linked in trees or webs) made programs walk stored paths of pointers (stored disk addresses of other records) (employee record → pointer → department record). Reorganize the file and every program walking that path broke.

The three parts of the model

Part	Question it answers	Content
Structure	What shape is the data?	Only relations: a name, named columns, rows of single values. No nested records, arrays or pointers.
Integrity	Which tables are legal?	Keys and constraints declared once on the table; the DBMS refuses any change that breaks them.
Manipulation	How do I get data out?	Declarative queries (SQL: you state *what* you want, not how to fetch it), built on **relational algebra**: operations (select, project, join, union, difference …) that take relations and return a relation.

Figure — structure, integrity, manipulation (course diagram)



Links are values, not pointers

`employee.dept_id = 10` links to the department whose `dept_id` is 10 — the shared *value* is the whole link. Consequences:

- The link survives copying the table to another server, reloading in a different order, or rebuilding indexes.
- The DBMS can **check** the link (a foreign key refuses `dept_id = 99` if no department 99 exists). A raw pointer cannot be checked like that.

What you get

- **One rule reaches every program** — constraints live with the data.
- **Physical data independence** — queries name *what*, not *how*, so indexes and storage can change without editing queries.
- **Some logical data independence** — adding a column or hiding a split behind a view (a saved query that behaves like a table) does not break existing queries. It is *not* total: renaming a column or shrinking its type still breaks the programs that use it.

What it does not promise

- **Nothing about storage** — files, pages and indexes belong to the storage engine (the part of the DBMS that reads and writes the files) (Chapter 12).
- **Joins are not free** — matching values costs work that grows with table size.
- **A table is not always an entity** — tables also come from M:N relationships (`works_on`) and multivalued attributes (`employee_phone`).

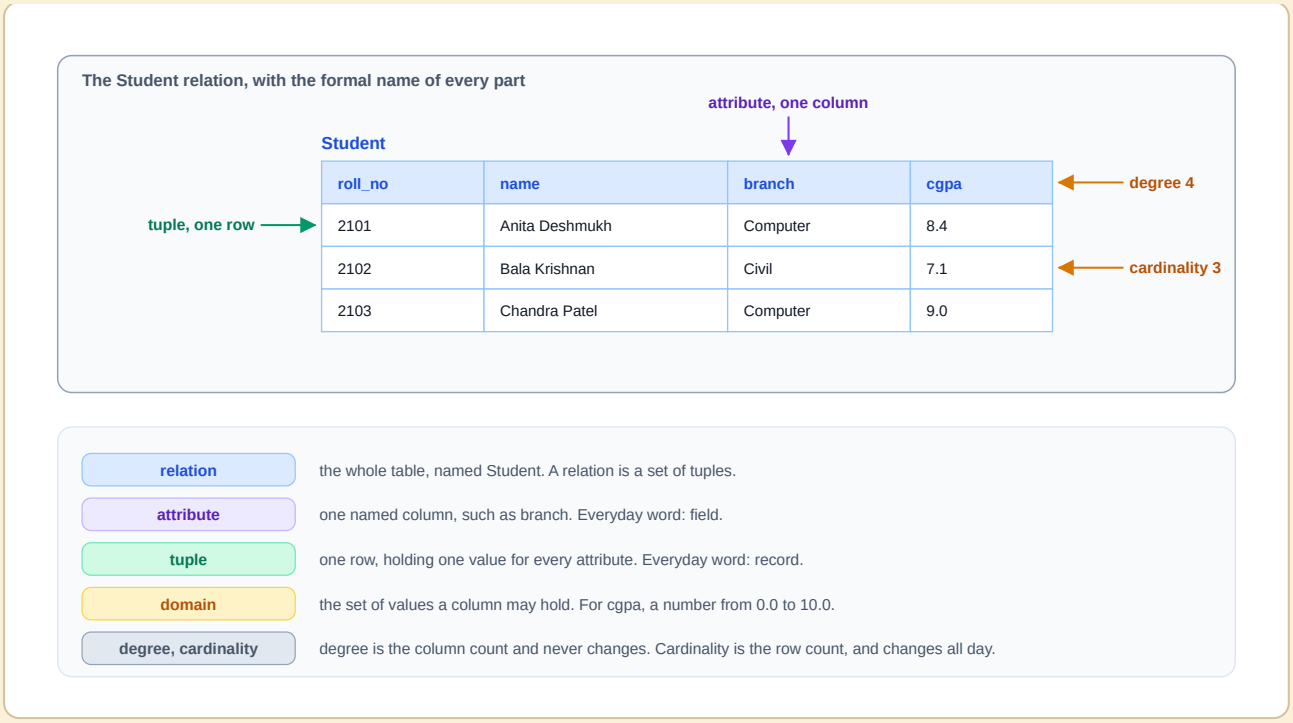
> **TL;DR**
>
> Codd (1970): everything is a relation of single values; integrity rules are declared on tables; queries are declarative. Rows link by shared values, which can be checked and survive reorganization.

10.12 Relations, tuples, attributes — the precise vocabulary

The core words

Formal term	SQL term	Older file term	Meaning
relation	table	file	named set of rows with fixed named columns
tuple	row	record	one value for each attribute
attribute	column	field	a named column
domain	data type + constraints	field type	the set of allowed values for an attribute
degree (arity)	number of columns	number of fields	a property of the schema
cardinality	number of rows	number of records	a property of the current data

Figure — anatomy of a relation (course diagram)



- **Domain** is what you *mean*, which is more than a type: `int` is a type; "integers 1–500" is the domain you meant for room capacity, enforced with `CHECK`.
- **Schema vs instance.** The **relation schema** is the design — `room(room_id, code, capacity)` — and rarely changes. The **relation instance** is the set of rows at this moment and changes with every insert. A key belongs to the *schema*, so it has to be true for any data the table could ever legally contain — the current rows prove nothing.
- **Two meanings of "cardinality".** In §10.7 it meant how many instances link (1:N); here it means a row count. Both are standard; context tells you which.

Five properties of every relation

1. **One value per cell** (atomic) — no lists, no nested records. This is why multivalued attributes need their own table, and it is the heart of 1NF (§11.7).
2. **Rows have no inherent order** — a relation is a set, so if you need sorted output you must ask for it with `ORDER BY`.
3. **Columns have no inherent order either** — you refer to them by name. (Real SQL tables do remember the order columns were declared in, so `SELECT *` always lists them the same way; still, list the columns you want explicitly.)
4. **Attribute names are unique within a relation.**
5. **No duplicate tuples** — a set has no duplicates. *Here SQL differs:* a SQL table accepts identical rows unless a `PRIMARY KEY` or `UNIQUE` forbids them. Declaring a key closes the gap.

NULL

`NULL` marks a **missing** value: either *unknown* (grade not yet computed) or *not applicable* (no middle name). It is not zero and not the empty string. SQL uses **three-valued logic**: comparing anything with `NULL` gives *unknown*, and `WHERE` keeps only rows where the condition is *true* (Chapter 2 covers this in depth). From `code/03_keys_and_integrity.sql`:

```
SELECT (SELECT count(*) FROM lecturer WHERE room_id = NULL)  AS eq_null,
       (SELECT count(*) FROM lecturer WHERE room_id IS NULL) AS is_null,
       count(room_id) AS count_col, count(*) AS count_star
FROM lecturer;
```

Output:

```
 eq_null | is_null | count_col | count_star
---------+---------+-----------+------------
       0 |       2 |         1 |          3
(1 row)
```

`= NULL` matches nothing, `IS NULL` works, `count(col)` skips nulls while `count(*)` counts rows. So whenever a column is nullable, each query touching it must decide how missing values are treated.

Self-check

1. `student(roll_no, name, cgpa, dept)` holds 8,000 rows. Degree? Cardinality? → **4** and **8,000**.
2. Does a SQL table forbid duplicate rows by default? → **No** — only a key or `UNIQUE` does.
3. Why can't you prove `{name}` is a key by looking at the rows? → Keys are schema properties; rows can only *disprove* one (two equal names).

TL;DR

Relation = table, tuple = row, attribute = column, domain = allowed values; degree = #columns (schema), cardinality = #rows (instance). Cells are atomic, order is meaningless, formal relations have no duplicates (SQL tables do unless keyed). `NULL` = missing; test with `IS NULL`.

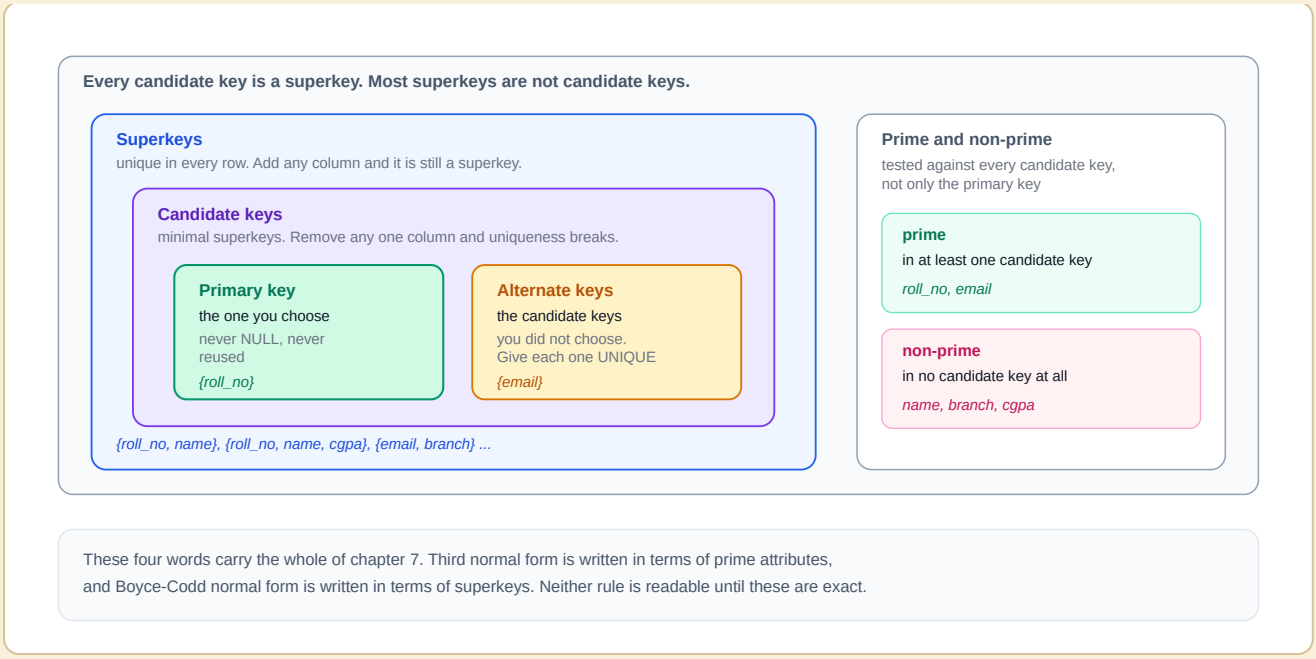
10.13 Keys in relational databases

Two identical-looking rows — same name, department and year — and a support ticket asking to change "that student". Without something that tells rows apart, the request has no answer. **Keys** make identity exist. Chapter 11 states its rules with exactly these definitions, so precision matters.

Running example: `lecturer(lecturer_id, email, name, room_id)`, where ids are issued once and emails are unique.

Key	Definition	Example
Superkey	a set of attributes that can **never** repeat in two rows (in any legal instance)	`{lecturer_id}`, `{email}`, `{lecturer_id, name}`, all four columns …
Candidate key	a **minimal** superkey — remove any attribute and it stops being unique	`{lecturer_id}`, `{email}`
Primary key	the one candidate key you *choose*; unique and never `NULL`	`lecturer_id`
Alternate key	any candidate key other than the chosen primary one — declare it `UNIQUE`	`email`
Composite key	any key with 2+ attributes (a shape, not a separate kind)	`booking(room_id, day, slot)`
Foreign key	attributes whose values must match a key in a (possibly the same) table, or be `NULL`	`lecturer.room_id → room.room_id`
Natural key	made of real-world data	email, ISBN, frame number
Surrogate key	system-generated, no business meaning	identity integer, UUID

Figure — superkeys ⊇ candidate keys ⊇ primary key, with prime and non-prime attributes (course diagram)



Key facts to be exact about:

- **A superkey stays a superkey when you add columns to it**, so there are many of them; candidate keys are the ones with nothing spare. Minimality is about *removability*, not size — a 3-column candidate key is fine if none can go.
- **All candidate keys are superkeys, but the reverse is rarely true.**
- **Prime attribute** = appears in *some* candidate key (any of them, not only the chosen primary key). **Non-prime** = belongs to none. In `lecturer`, `lecturer_id` and `email` are prime; `name` and `room_id` are non-prime. 3NF is defined with these words (§11.9).
- **Composite example.** `enrolment(student_id, course_id, term, grade)`: students may retake a course another term, so only `{student_id, course_id, term}` identifies a row, and dropping any of the three breaks it.
- **Foreign keys:** may be `NULL` (no link yet — that is not a violation); may reference any `PRIMARY KEY` or `UNIQUE` column set; may reference their own table (`employee.supervisor_id`), which is how a unary relationship becomes a column.
- **Surrogate vs natural:** surrogates are narrow and never change; natural keys are readable and sometimes save a join, but change when the business changes (a new roll-number format means updating every FK). **Default: surrogate PK + `UNIQUE` natural key** — the `customer` table in `00_schema.sql` does exactly that.

In the interview

30-second answer: "A superkey uniquely identifies rows; a candidate key is a minimal superkey; the primary key is the candidate I pick — unique and not null; the other candidates are alternate keys and get UNIQUE constraints. A foreign key references a primary or unique key and may be null. My default is a generated surrogate PK, with the natural key kept unique."

- *Is `{id, name}` a candidate key?* → A superkey, not a candidate key: `name` can be dropped.
- *UUID vs bigint surrogate?* → UUIDs can be generated anywhere without coordination but are 16 bytes vs 8 and random UUIDs scatter B-tree (the default, always-sorted index structure) inserts; time-ordered UUIDs (v7) reduce that.
- *Is an attribute of an alternate key prime?* → Yes — prime means "in any candidate key".

TL;DR

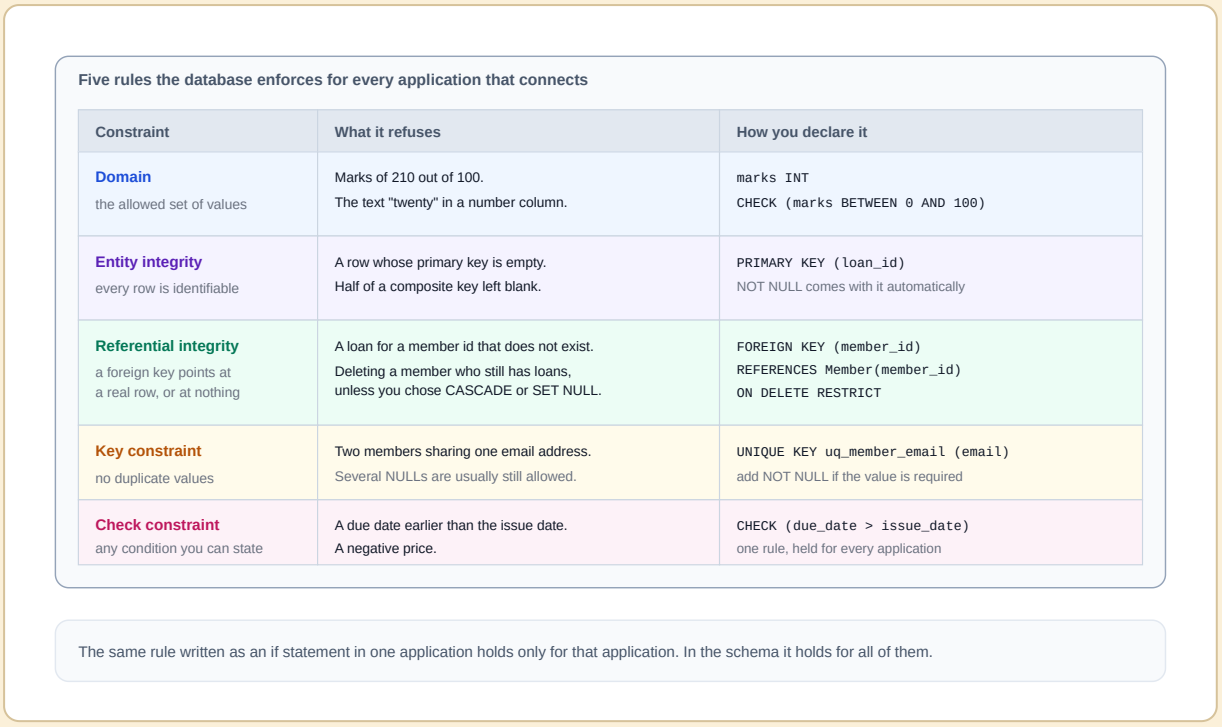
Superkey (unique) ⊇ candidate key (minimal) ⊇ primary key (chosen, NOT NULL); other candidates = alternate keys (UNIQUE). Prime = in some candidate key. FKs match a PK/UNIQUE or are NULL. Surrogate PK + unique natural key is the safe default.

10.14 Relational integrity constraints

What it is

An **integrity constraint** is a condition stored data is never allowed to break. Whatever program connects, the DBMS tests it during each `INSERT`, `UPDATE` and `DELETE` and refuses any change that would violate it.

Figure — the integrity constraints and the SQL that declares each (course diagram)



Constraint	Rule	SQL
Domain	a value must be in the attribute's domain	the data type, plus `CHECK (capacity BETWEEN 1 AND 500)`
Entity integrity	no part of a primary key may be `NULL`	`PRIMARY KEY` (implies `NOT NULL`)
Key	candidate keys are unique	`PRIMARY KEY`, `UNIQUE`
Referential integrity	an FK value matches an existing referenced row, or is `NULL` — no **orphan rows**	`FOREIGN KEY … REFERENCES …`
(also declared the same way)	must always have a value / default when omitted	`NOT NULL`, `DEFAULT` (DEFAULT never refuses anything, so it is not strictly a constraint)

Why entity integrity: a primary key identifies a row, and `NULL` means "unknown" — a row with unknown identity cannot be distinguished, so the model rejects it. For composite keys *every* column must be present.

What happens when a referenced (parent) row is deleted or its key updated

Action (`ON DELETE` / `ON UPDATE`)	Effect	Use when
`NO ACTION` (default)	error if children exist; in PostgreSQL the check runs at the end of the statement and can be `DEFERRABLE` (postponed) to the end of the transaction — the group of statements committed together	general default
`RESTRICT`	error immediately, cannot be deferred	you want an immediate refusal
`CASCADE`	delete/update the children too	weak entities, pure child data (phones, order lines) — dangerous elsewhere
`SET NULL`	children's FK becomes `NULL` (column must be nullable)	optional relationships
`SET DEFAULT`	children's FK becomes its default	rare

Dialect notes: MySQL/InnoDB (MySQL's default storage engine) treats `NO ACTION` as `RESTRICT` and rejects `SET DEFAULT`; SQL Server has no `RESTRICT` keyword (use `NO ACTION`); SQLite enforces FKs only after `PRAGMA foreign_keys = ON` (a SQLite settings command). MySQL enforces `CHECK` only from 8.0.16 (earlier versions parsed and ignored it).

Running it (`code/03_keys_and_integrity.sql`)

Each failing statement is wrapped in a `DO` block (a one-off piece of PostgreSQL procedural code) that catches the error and prints it as a `NOTICE` (an informational message), so the script keeps going.

```
CREATE TABLE room (
  room_id  int PRIMARY KEY,                                  -- entity integrity: unique + NOT NULL
  code     varchar(10) NOT NULL UNIQUE,                      -- alternate (candidate) key
  capacity int NOT NULL CHECK (capacity BETWEEN 1 AND 500)   -- domain constraint
);
CREATE TABLE lecturer (
  lecturer_id int PRIMARY KEY,
  name        varchar(60) NOT NULL,
  email       varchar(120) UNIQUE,                           -- UNIQUE but nullable
  room_id     int REFERENCES room(room_id) ON DELETE SET NULL  -- optional link
);
CREATE TABLE booking (
  room_id  int  NOT NULL REFERENCES room(room_id) ON DELETE RESTRICT,
  day      date NOT NULL,
  slot     int  NOT NULL CHECK (slot BETWEEN 1 AND 8),
  PRIMARY KEY (room_id, day, slot)                           -- composite key
);
-- each violation is attempted inside:  DO $$ BEGIN ... EXCEPTION WHEN ... THEN RAISE NOTICE ... END $$;
```

Output:

```
--- 1. domain: capacity 900 is outside the allowed values
NOTICE:  new row for relation "room" violates check constraint "room_capacity_check"
--- 2. entity integrity: a primary key may not be NULL
NOTICE:  null value in column "room_id" of relation "room" violates not-null constraint
--- 3. key constraint: the alternate key code must be unique
NOTICE:  duplicate key value violates unique constraint "room_code_key"
--- 4. referential integrity: no orphan rows
NOTICE:  insert or update on table "booking" violates foreign key constraint "booking_room_id_fkey"
--- 5. UNIQUE lets several rows have NULL (Dr Silva and Dr Wu both have no email)
   name   | email
----------+-------
 Dr Silva |
 Dr Wu    |
(2 rows)

--- 6. ON DELETE RESTRICT: room 1 still has bookings
NOTICE:  update or delete on table "room" violates foreign key constraint "booking_room_id_fkey" on table
"booking"
--- 7. ON DELETE SET NULL: deleting room 2 empties Dr Silva's room_id
   name   | room_id
----------+---------
 Dr Park  |       1
 Dr Silva |
 Dr Wu    |
(3 rows)
```

UNIQUE and NULL — a classic surprise

Because `NULL` is never equal to `NULL`, most databases let a `UNIQUE` column hold **many** nulls (output 5). If "unique when present" is not what you want, add `NOT NULL`.

DBMS	Multiple NULLs in a UNIQUE column?
PostgreSQL	yes by default; PostgreSQL 15+ can say `UNIQUE NULLS NOT DISTINCT` to allow only one (not run here — our server is 14)
MySQL (InnoDB)	yes
SQLite	yes
SQL Server	**no — only one NULL**; use a filtered unique index (one covering only rows that match its `WHERE`) `WHERE col IS NOT NULL` for "many"

Why rules belong in the schema

1. **They hold for everybody** — web app, mobile backend (the server code behind a phone app), nightly import, a hand-typed fix at midnight.
2. **They cannot drift** — one copy, not one per application.
3. **They document meaning** — the `CREATE TABLE` tells a reader what the data is.

Costs: constraints add a little work to each write, and a thoughtless `CASCADE` can wipe out far more rows than someone meant. Both are small next to the benefit of a single, central enforcement point.

Common mistakes

- Forgetting to declare alternate keys → two customers with the same email.
- `ON DELETE CASCADE` on a "real" relationship (deleting a customer silently deletes their invoices).
- Assuming a nullable `UNIQUE` column means "at most one missing".
- Leaving FKs off "for speed" — orphans appear and joins silently lose rows.

TL;DR

Domain (type + CHECK), entity integrity (PK never NULL), key (UNIQUE), referential (FK matches or NULL; choose RESTRICT/NO ACTION/CASCADE/SET NULL on delete). UNIQUE usually allows many NULLs. Put rules in the schema so they hold for every client.

10.15 Converting an ER diagram to tables

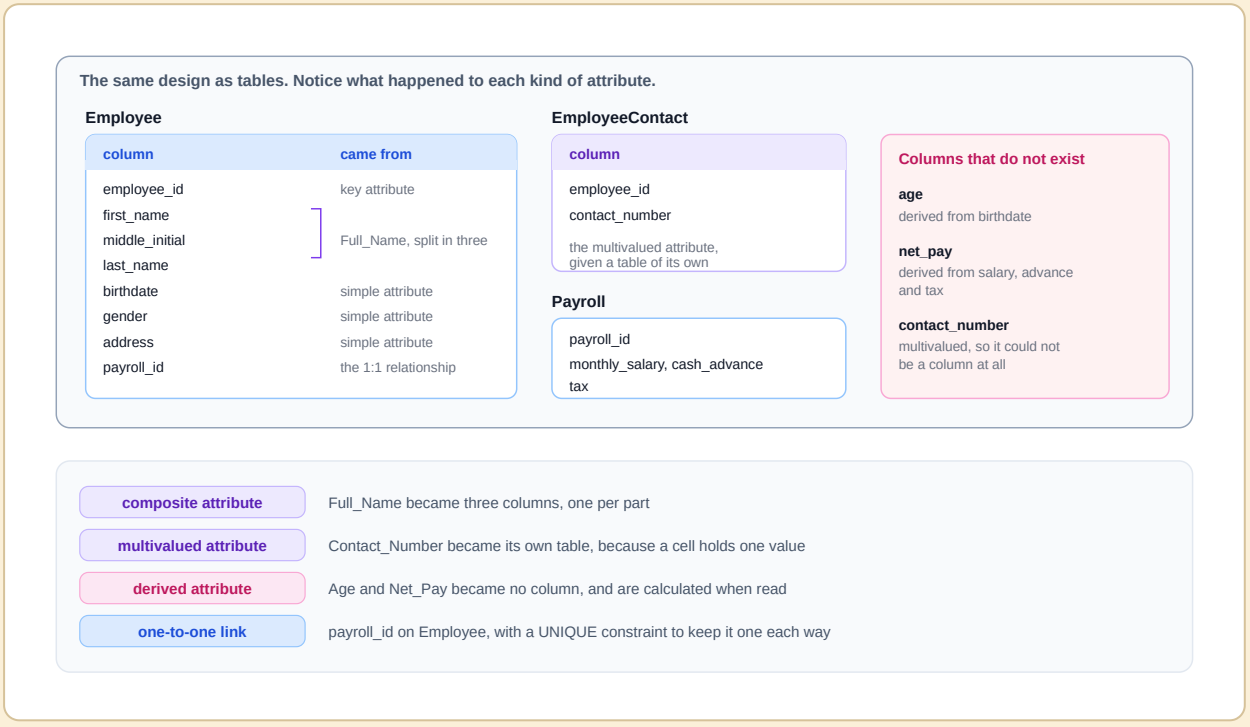
The procedure

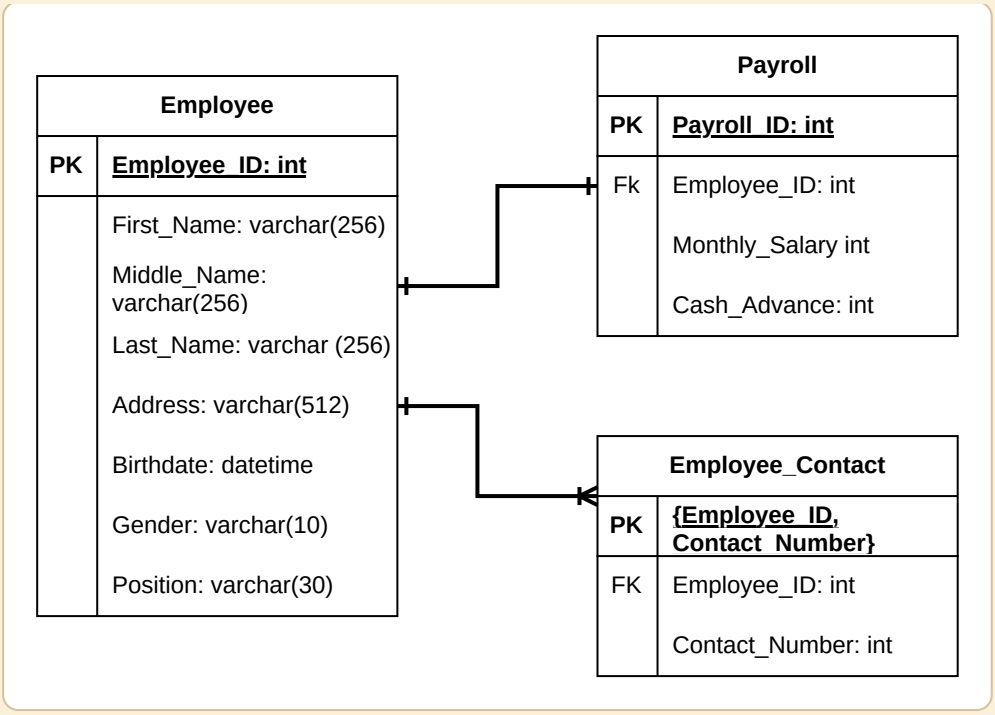
Once the diagram is right, conversion is mechanical — one decision per construct:

#	ER construct	Relational result
1	Strong entity	a table; key attribute → primary key
2	Simple attribute	a column
3	Composite attribute	one column per part (no column for the whole)
4	Derived attribute	no column (compute in the query or a view)
5	Multivalued attribute	own table: owner key + value, both in the PK, `ON DELETE CASCADE`
6a	1:N / N:1 relationship	FK on the many side
6b	M:N relationship	junction table: both FKs, together the PK
6c	1:1 relationship	FK + `UNIQUE` on either side, prefer the total-participation side
6d	Unary relationship	FK to the same table
6e	Ternary relationship	table with three FKs (PK usually all three)
7	Relationship attribute	follows the relationship: junction table (M:N) or many side (1:N)
8	Weak entity	table with PK = owner key + partial key; owner FK `NOT NULL`, usually `ON DELETE CASCADE`
9	Participation	total → FK `NOT NULL`; partial → FK nullable

Create tables in **dependency order** — parents before children, since a foreign key can only point at a table that already exists.

Figure — the course's Employee/Payroll example as tables: the composite splits, the multivalued attribute becomes a table, the derived attributes vanish (course diagrams)




Employee
PK
Employee_ID: int
First_Name: varchar(256)
Middle_Name: varchar(256)
Last_Name: varchar (256)
Address: varchar(512)
Birthdate: datetime
Gender: varchar(10)
Position: varchar(30)
Payroll
PK
Payroll_ID: int
Fk
Employee_ID: int
Monthly_Salary int
Cash_Advance: int
Employee_Contact
PK
{Employee_ID, Contact_Number}
FK
Employee_ID: int
Contact_Number: int

Running the procedure on §10.10 (`code/02_employee_er_to_tables.sql`)

```
CREATE TABLE department (                       -- strong entity -> table
  dept_id   int PRIMARY KEY,
  dept_name varchar(60) NOT NULL UNIQUE
);
CREATE TABLE payroll (                          -- strong entity -> table
  payroll_id     int PRIMARY KEY,
  monthly_salary numeric(10,2) NOT NULL CHECK (monthly_salary >= 0),  -- stored (typed in)
  cash_advance   numeric(10,2) NOT NULL DEFAULT 0,
  tax            numeric(10,2) NOT NULL DEFAULT 0
  -- net_pay is DERIVED -> no column (computed in the view below)
);
CREATE TABLE employee (
  emp_id         int PRIMARY KEY,                              -- key attribute
  first_name     varchar(40) NOT NULL,                         -- composite attribute full_name
  middle_initial char(1),                                      --   split into
  last_name      varchar(40) NOT NULL,                         --   one column per part
  birth_date     date NOT NULL,                                -- simple; age is DERIVED
  dept_id        int  NOT NULL REFERENCES department(dept_id),  -- 1:N, total participation -> NOT NULL
  supervisor_id  int  REFERENCES employee(emp_id),             -- unary 1:N, partial -> NULL allowed
  payroll_id     int  NOT NULL UNIQUE REFERENCES payroll(payroll_id)  -- 1:1 -> FK + UNIQUE
);
CREATE TABLE employee_phone (                   -- multivalued attribute -> own table
  emp_id int         NOT NULL REFERENCES employee(emp_id) ON DELETE CASCADE,
  phone  varchar(20) NOT NULL,
  PRIMARY KEY (emp_id, phone)
);
CREATE TABLE project (project_id int PRIMARY KEY, title varchar(80) NOT NULL);
CREATE TABLE works_on (                         -- M:N -> junction table
  emp_id     int NOT NULL REFERENCES employee(emp_id),
  project_id int NOT NULL REFERENCES project(project_id),
  weekly_hours numeric(4,1) NOT NULL CHECK (weekly_hours > 0),  -- relationship attribute
  PRIMARY KEY (emp_id, project_id)
);
CREATE TABLE dependent (                        -- weak entity
  emp_id       int NOT NULL REFERENCES employee(emp_id) ON DELETE CASCADE,  -- owner key
  dep_name     varchar(40) NOT NULL,                                       -- partial key
  relationship varchar(20) NOT NULL,
  PRIMARY KEY (emp_id, dep_name)                                           -- owner key + partial key
);
-- Derived attributes live in a view (fixed "as of" date so the output is repeatable):
CREATE VIEW employee_card AS
SELECT e.emp_id,
       concat_ws(' ', e.first_name, e.middle_initial, e.last_name)   AS full_name,
       date_part('year', age(date '2026-09-28', e.birth_date))::int  AS age,
       p.monthly_salary - p.cash_advance - p.tax                    AS net_pay,
       (SELECT string_agg(phone, ', ' ORDER BY phone) FROM employee_phone ph
         WHERE ph.emp_id = e.emp_id)                                 AS phones
FROM employee e JOIN payroll p USING (payroll_id);
```

Output:

```
--- employee_card (derived: full_name, age, net_pay; multivalued: phones)
 emp_id |  full_name   | age | net_pay |              phones
--------+--------------+-----+---------+----------------------------------
      1 | Aisha Khan   |  41 | 4900.00 | +44 20 7946 0001
      2 | Ravi K Menon |  33 | 4250.00 | +91 80 4000 0003, +91 98450 00002
      3 | Sofia Rossi  |  28 | 3100.00 |
(3 rows)

--- the unary relationship: who reports to whom
 employee | supervisor
----------+------------
 Aisha    |
 Ravi     | Aisha
 Sofia    | Aisha
(3 rows)

--- 1:1 is enforced by UNIQUE: a second employee on payroll 501 is refused
NOTICE:  refused: duplicate key value violates unique constraint "employee_payroll_id_key"
--- weak entity: deleting the owner removes its dependents (ON DELETE CASCADE)
 dependents_left | phones_left
-----------------+-------------
               0 |           0
(1 row)
```

(Before deleting employee 2 the script first removes their `works_on` rows and clears the `supervisor_id` pointing at them, because those FKs use the default `NO ACTION` — the database would otherwise refuse, which is exactly referential integrity at work.)

Checking a conversion

1. **Count:** 5 entities + 1 multivalued attribute + 1 M:N relationship = 7 tables. ✓
2. **Every relationship produced something** (an FK, a UNIQUE FK or a table). A relationship that produced nothing was missed.
3. **No derived attribute survived** as a column — or, if one did, a comment says why.
4. **No multivalued attribute left in its parent** — a list column is a 1NF violation (Chapter 11).

> **TL;DR**
>
> Entities → tables; simple → columns; composite → parts; derived → nothing; multivalued → own table; 1:N → FK on many side; M:N → junction table; 1:1 → FK + UNIQUE; weak → owner key + partial key with CASCADE; total participation → NOT NULL. Parents first.

10.16 Best practices for ER → relational mapping

The procedure is mechanical for small diagrams; large ones need judgement in a few places.

Situation	Recommendation	Why / exception
Composite attribute	Store the parts; do **not** also store the whole. Build the whole in a query or view.	Two copies disagree after the first partial update. Exception: never query the parts → one column is fine.
Multivalued attribute	Always a separate table with a composite PK (owner key + value). Never numbered columns.	The composite PK stops the same phone being stored twice for one person.
Derived attribute	Compute on read. Store only if expensive *or* it must be frozen in time (order total at purchase prices) — and comment the column.	Otherwise the next engineer deletes it as a "mistake".
1:1 with total participation both sides	Probably one table. Keep two only for access control, row width, or mostly-absent optional data.	Two tables always joined 1:1 are cost without benefit.
M:N	The junction table is mandatory; relationship attributes live there; give it a business name (`enrolment`, `booking`) not `student_course`.	Add a surrogate key only if other tables must reference a single row — and keep `UNIQUE` on the pair so nobody enrols twice.
Weak entity with a surrogate key	Keep `UNIQUE (owner_key, partial_key)`.	Otherwise the rule the weak entity expressed is enforced nowhere.
Naming	One style for tables and columns; FKs named after their target; **role names** when two FKs hit the same table (`origin_airport_id`, `destination_airport_id`, not `airport_id_1/2`).	Readable joins.
Normalization	Run the normal forms over the result (Chapter 11); expect one or two further splits.	A careful ERD lands *near* 3NF, rarely exactly on it.
Large diagrams	Create parent tables before children and finish one area (say orders) before starting the next (inventory); record each choice you made that the rules left open (merged 1:1s, stored derived columns, surrogate keys on weak entities).	Those are the choices someone will question later.

Common mistakes

- Converting an M:N with an FK on one side (only works if it was really 1:N).
- Keeping `full_name` *and* `first_name`/`last_name`.
- Surrogate key on a junction table without the pair `UNIQUE`.
- Creating child tables before parents in a migration script.

In the interview

30-second answer (for "turn this ERD into tables"): "Each strong entity becomes a table with its key as PK. Composite attributes split into columns, derived ones are computed, multivalued ones get their own table. 1:N puts the FK on the many side, M:N gets a junction table holding both keys plus any relationship attributes, 1:1 is an FK with UNIQUE. A weak entity's PK combines its owner's key with its partial key, with cascade delete, and total participation means NOT NULL. Then I check the result against the normal forms."

- *Where does `grade` go in student–course?* → In the enrolment junction table.
- *Would you merge a 1:1?* → Yes unless access control, row width or sparse optional data justify splitting.

Self-check

1. Two FKs from `flight` to `airport` — how do you name them? → By role: `origin_airport_id`, `destination_airport_id`.
2. A junction table has a surrogate PK. What else must it have? → `UNIQUE` on the two FK columns.
3. Is the converted schema automatically in 3NF? → No; check it with the normal forms.

TL;DR

Split composites without keeping the whole; multivalued → table; derived → computed (unless frozen); question every total 1:1; junction tables hold relationship attributes and a unique pair; name by role; convert in dependency order; then normalize.

Key terms

Term	Meaning
Data model	picture of entities, attributes and relationships
Conceptual / logical / physical model	things + links / + attributes, keys, general types / + exact types, indexes for one DBMS
Data independence	changing storage (physical) or some schema details (logical) without changing applications
Entity type / instance / set	the kind / one member / all current members
Key, simple, composite, multivalued, derived attribute	identifies / one value / has parts / many values / computed
Relationship type / set	the association / the actual pairs
Degree	number of entity types in a relationship (unary, binary, ternary)
Cardinality (ER)	max instances per side: 1:1, 1:N, M:N
Participation	total (mandatory, double line) or partial (optional)
(min, max)	min = participation, max = cardinality
Junction table	table implementing an M:N relationship
Weak entity, owner, partial key (discriminator), identifying relationship	entity identified only with its owner, via owner key + partial key
Chen vs crow's-foot	ellipse/diamond notation vs attributes-in-box + line-end notation
Relation, tuple, attribute, domain	table, row, column, allowed values
Degree / cardinality (relation)	number of columns / number of rows
Schema vs instance	design vs current data
Superkey, candidate, primary, alternate, composite, foreign, natural, surrogate key	see §10.13
Prime / non-prime attribute	in some candidate key / in none
Domain, entity, key, referential integrity	allowed values / PK not NULL / uniqueness / FK matches or NULL
Orphan row	child row whose FK points at nothing
RESTRICT / NO ACTION / CASCADE / SET NULL / SET DEFAULT	what happens to children when a parent is deleted/updated

Chapter 10 TL;DR

- **Model before tables.** Entities (nouns) → tables, attributes → columns, relationships (verbs) → FKs or junction tables. A fact you do not model is a question you cannot answer.
- **Three levels:** conceptual (for the business), logical (attributes, keys, normalization), physical (types, indexes, one DBMS).
- **Process:** requirements + questions → entities → relationships → conceptual review → logical → physical → validate; loop back freely.
- **Best practices:** constraints in the schema, one naming style, dates as dates, money as `numeric`, explicit `NOT NULL`, surrogate PK + unique natural key, 3NF by default, index real queries.
- **ER notation:** Chen (rectangle, ellipse, diamond; double ellipse = multivalued, dashed = derived, double rectangle/diamond = weak) vs crow's foot (`||`, `o|`, `o{`, `|{`), which mermaid draws.
- **Relationships:** degree, cardinality, participation are three separate questions; (min,max) = (participation, cardinality).
- **Relational model (Codd 1970):** relations of atomic values, links by value, declarative queries; SQL tables allow duplicates unless keyed; `NULL` needs `IS NULL`.
- **Keys:** superkey ⊇ candidate (minimal) ⊇ primary; alternates get `UNIQUE`; prime = in any candidate key.
- **Integrity:** domain, entity, key, referential (+ ON DELETE choices); UNIQUE usually allows many NULLs.
- **Mapping:** composite → parts, multivalued → table, derived → view, 1:N → FK on many, M:N → junction, 1:1 → FK + UNIQUE, weak → owner key + partial key + CASCADE, total → NOT NULL. Then normalize — Chapter 11.

Next: Chapter 11 — Normalization, which checks a design for redundancy and splits tables safely.

Chapter 11 — Normalization

Chapter 10 turned a business description into tables. This chapter asks the next question: **does each fact live in exactly one place?** When it doesn't, the table can contradict itself — a customer with two cities, a doctor with two specialties — and some facts cannot be stored or are lost by accident. **Normalization** is the precise, rule-based way to find such tables and split them safely.

Normalization is often taught as vague advice ("avoid redundancy"). Here it is built on one exact tool, the **functional dependency**, so that "is this table in 3NF?" (third normal form, §11.9) has a checkable yes/no answer. A small Python program (`code/fd_tools.py`) computes closures (everything a set of columns determines, §11.4), candidate keys (minimal column sets that identify a row) and normal-form violations, and every worked answer in the chapter was verified with it. The SQL files show the anomalies (the insert/update/delete failures of §11.6) happening on PostgreSQL and disappearing after decomposition (splitting one table into several).

What you'll learn

1. Functional dependencies: the definition, where they come from, and their types (trivial, full, partial, transitive, multivalued).
2. Armstrong's inference rules, attribute closure, finding every candidate key, and minimal covers.
3. The three anomalies (insert, update, delete) — demonstrated live.
4. 1NF, 2NF, 3NF, BCNF, 4NF and 5NF: the rule, a violating table, the fix, and the proof that nothing was lost.
5. Lossless decomposition and dependency preservation — and why 3NF, not BCNF, is the usual stopping point.
6. Four exercises with full solutions, and when to deliberately **denormalize**.

Files in `code/`: `00_schema.sql` (one wide order table), `01`–`07` (SQL demos, each on a fresh database) and `fd_tools.py` (run with Python 3: `python fd_tools.py`). All "Output:" blocks are real.

11.1 Functional dependencies

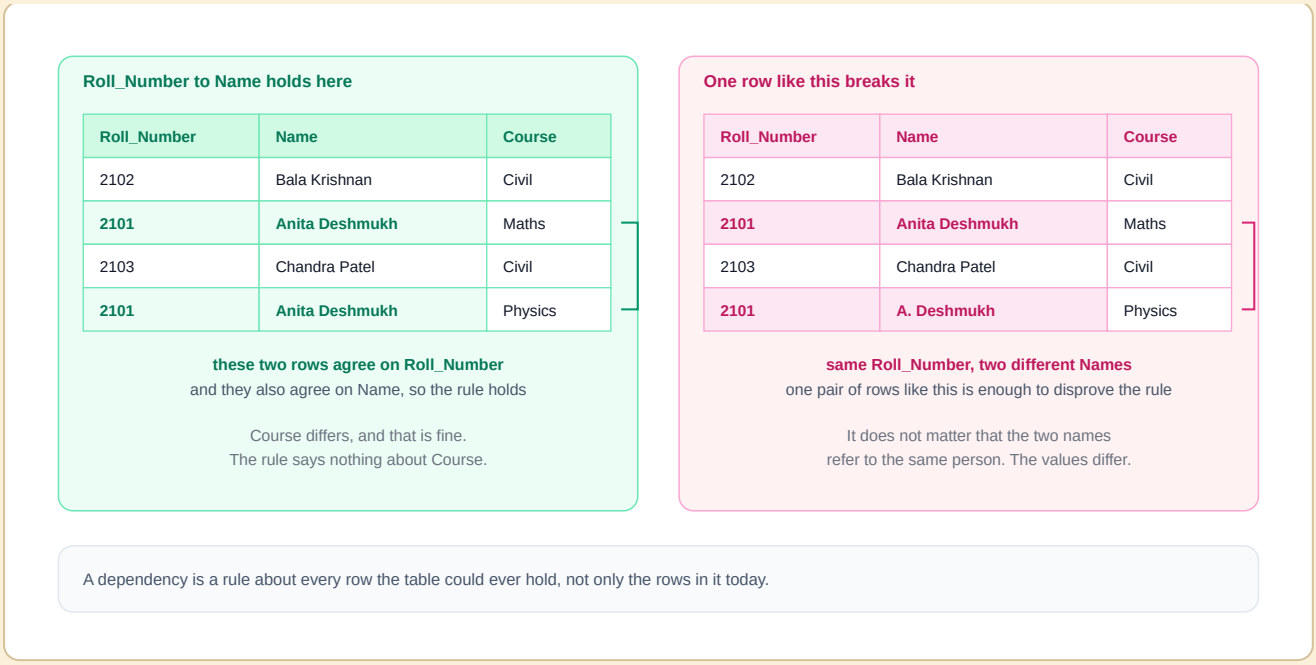
What it is

A **functional dependency (FD)** says that the values of one set of columns *fix* the values of another. Written `X -> Y`, read "X determines Y" (or "Y is functionally dependent on X").

The test, in terms of rows: take any two rows. If they agree on every column of `X`, they must agree on every column of `Y`. A single pair of rows that agrees on `X` but differs on `Y` proves `X -> Y` false.

Analogy: a national ID number determines a person's date of birth — two records with the same ID and different birthdays mean one record is wrong. But a birthday does not determine an ID: millions of people share one.

Figure — the FD rule in rows: two rows that agree on the left side must agree on the right side (course diagram)



The running example

This chapter uses one deliberately bad table from a small online shop (`code/00_schema.sql`):

order_id	product_id	order_date	customer_id	customer_name	customer_city	product_name	list_price	qty
O1	P1	2026-09-01	C1	Nadia Lopez	Lisbon	Kettle	29.00	1
O1	P2	2026-09-01	C1	Nadia Lopez	Lisbon	Mug	6.50	4
O2	P1	2026-09-03	C2	Jonas Berg	Oslo	Kettle	29.00	1
O3	P3	2026-09-05	C1	Nadia Lopez	Lisbon	Teapot	18.00	2
O4	P4	2026-09-06	C3	Priya Das	Kochi	Tea tin	9.00	3

The shop owner confirms four rules — these are the FDs:

#	FD	In words
F1	`order_id -> order_date, customer_id`	an order is placed on one date by one customer
F2	`customer_id -> customer_name, customer_city`	a customer has one name and one city
F3	`product_id -> product_name, list_price`	a product has one name and one current list price
F4	`{order_id, product_id} -> qty`	a product appears once per order, with one quantity

Vocabulary

- **Determinant** — the left side (`customer_id` in F2). Several normal forms are phrased in terms of determinants.
- **Dependent** — the right side.
- A side with several columns is **composite** (`{order_id, product_id}` in F4).

An FD is a rule, not a summary of today's rows

Look at the table: every `customer_name` is different, so `customer_name -> customer_city` *happens* to hold in these five rows. It is **not** a real dependency — two customers called Nadia Lopez can live in different cities. Rows can **disprove** an FD (by showing a counter-pair) but can **never prove** one. FDs are constraints on every legal state of the table, like `NOT NULL`.

Where FDs come from: ask the people who own the data, one question per candidate rule — *"If I tell you the X, do you always know the Y?"* Customer id → city? "Yes, always." City → customer? "No." Write every

answer down before designing tables; that list is the input to everything in this chapter.

Direction matters: `X -> Y` says nothing about `Y -> X`. Both can hold (an id and a unique email determine each other), but each needs its own check.

FDs and keys are the same idea

From §10.13: a **superkey** is a set that never repeats. In FD terms: **`X` is a superkey precisely when `X` functionally determines all columns** (two rows equal on `X` would be equal everywhere, i.e. the same row). A **candidate key** is then a superkey from which nothing can be removed. So the tool that tests FDs (closure, §11.4) is also the tool that finds keys.

Why the rest of the chapter needs this

Each normal form is a statement about which FDs may exist in a table: 2NF bans one shape, 3NF another, BCNF a third. Once the FDs are written down, "is this table normalized?" stops being opinion.

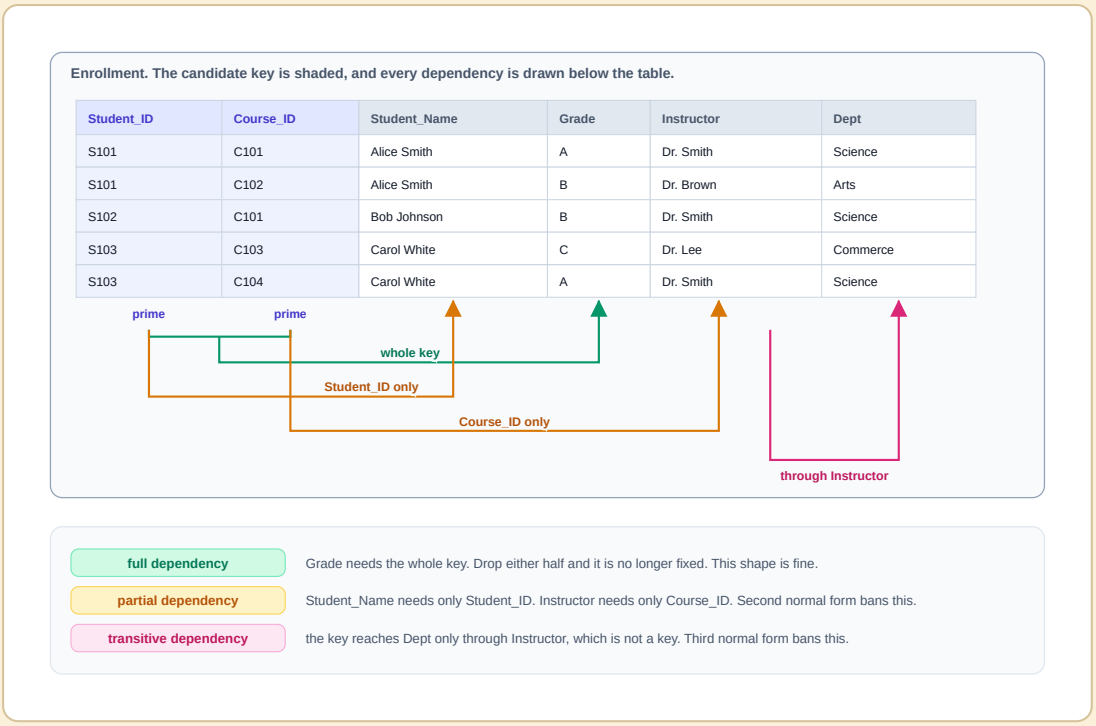
TL;DR

`X -> Y`: rows equal on X are equal on Y. One counter-pair disproves it; data never proves it — FDs come from business rules. X is a superkey iff X determines all columns.

11.2 Types of functional dependencies

In the running example the only candidate key is `{order_id, product_id}`, so **prime attributes** (members of some candidate key — §10.13) are `order_id` and `product_id`; all others are **non-prime**.

Figure — full, partial and transitive dependencies on one table (course diagram, using the course's enrolment example)



Type	Definition	Example (running table)	Banned by
Trivial	the right side is contained in the left: `{order_id, product_id} -> product_id`	always true; says nothing	never a problem — every rule says "non-trivial"
Non-trivial	the right side adds a column the left side lacks	`customer_id -> customer_city`	—
Completely non-trivial	left and right share no column	`product_id -> list_price`	—
Full	needs its *whole* (composite) determinant: remove any column and it fails	`{order_id, product_id} -> qty` (O1 has quantities 1 and 4; P1 appears in O1 and O2)	—
Partial	a *proper part* of a candidate key already determines a non-prime column	`order_id -> order_date`, `product_id -> product_name`	**2NF**
Transitive	from `X -> Y` plus `Y -> Z` you get `X -> Z`, where `Y` is not a superkey and `Z` is non-prime	`order_id -> customer_id -> customer_city`	**3NF**
Determinant not a superkey	any non-trivial `X -> Y` with X not a superkey	all of F1–F3 here	**BCNF**
Multivalued (`X ->> Y`)	for one X, the *set* of Y values is independent of the other columns	a restaurant's dishes vs its delivery zones (§11.12)	**4NF**

Notes worth being precise about:

- **Full vs partial only makes sense for composite determinants.** With a one-column key there is no "part", so a partial dependency is impossible.
- **The two conditions on "transitive" matter.** Without "Y is not a superkey" every table would be full of transitive dependencies (the key determines itself determines everything).
- **Partial and transitive can overlap:** `order_id -> customer_city` is partial (order_id is part of the key) *and* transitive (via customer_id).
- **Multivalued dependencies** are a different kind of constraint (sets of values, not single values); every FD is also an MVD (multivalued dependency), not vice versa. §11.12 treats them.

Self-check

1. `{order_id, product_id} -> order_id` — which type? → Trivial.
2. Why is `product_id -> product_name` partial? → `product_id` is a proper part of the only candidate key and `product_name` is non-prime.
3. In a table with a single-column key, which of partial/transitive can exist? → Only transitive.

TL;DR

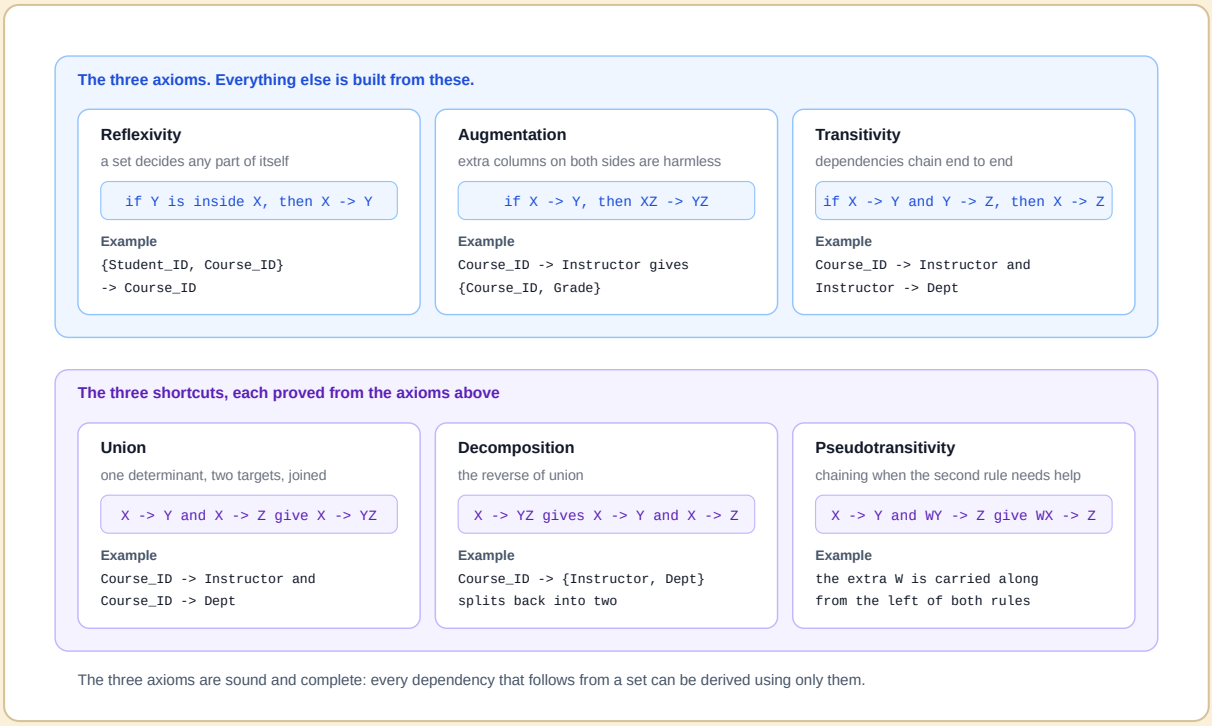
Trivial (always true) · full (needs whole key) · partial (part of a key → non-prime; kills 2NF) · transitive (key → non-key → non-prime; kills 3NF) · non-superkey determinant (kills BCNF) · multivalued (kills 4NF).

11.3 Armstrong's inference rules

What it is

A set of FDs implies more FDs than it lists. Nobody wrote `order_id -> customer_city`, yet F1 (`order_id -> customer_id`) and F2 (`customer_id -> customer_city`) force it. The given FDs `F` plus everything they imply is the **closure of F**, written **F^+**. The rules for deriving implied FDs are **Armstrong's axioms** (W. W. Armstrong, 1974).

Figure — the three axioms and the three derived rules (course diagram)



The three axioms

Axiom	Rule	Example	Why it is true
Reflexivity	if `Y ⊆ X` then `X -> Y`	`{order_id, product_id} -> product_id`	rows equal on X are equal on any part of X (these are the trivial FDs)
Augmentation	if `X -> Y` then `XZ -> YZ`	from F2: `{customer_id, product_id} -> {customer_city, product_id}`	rows equal on X and Z are equal on Y (by the FD) and on Z (by assumption)
Transitivity	if `X -> Y` and `Y -> Z` then `X -> Z`	F1 + F2: `order_id -> customer_city`	chain the two guarantees

The axioms are:

- **Sound** — they never derive an FD that is false in some table satisfying F.
- **Complete** — every FD that holds in *all* tables satisfying F can be derived with them.

So these three are enough; the next three add convenience, not power.

Three derived rules

Rule	Pattern	Example
Union	`X -> Y` and `X -> Z` ⟹ `X -> YZ`	`order_id -> order_date` and `order_id -> customer_city` ⟹ `order_id -> {order_date, customer_city}`
Decomposition (split)	`X -> YZ` ⟹ `X -> Y` and `X -> Z`	F3 ⟹ `product_id -> list_price`
Pseudotransitivity	`X -> Y` and `WY -> Z` ⟹ `WX -> Z`	suppose a customer leaves one review per product: `{customer_id, product_id} -> review_stars`. With F1 (`order_id -> customer_id`), taking W = product_id: `{order_id, product_id} -> review_stars`

Decomposition is why worked examples first rewrite every FD with a single column on the right — the list becomes easier to scan.

Deriving union from the axioms (a common exam question):

1. Augment `X -> Y` with X: `X -> XY` (since XX = X).
2. Augment `X -> Z` with Y: `XY -> YZ`.
3. Transitivity on 1 and 2: `X -> YZ`. ∎

Decomposition follows from reflexivity (`YZ -> Y`) plus transitivity; pseudotransitivity from augmenting `X -> Y` with W and then transitivity.

What the rules are for

Finding candidate keys (most "X determines everything" facts are derived, not given) and checking claims like "this table is in 3NF", which depend on implied FDs too. Applying the rules by hand is slow for 6+ columns, so in practice you use attribute closure (next), which the axioms justify.

`fd_tools.py` confirms both derivations above:

```
order_id -> customer_city follows? True
customer_id -> order_id follows?   False
pseudotransitivity: order_id -> customer_id and {customer_id, product_id} -> review_stars
  so {order_id, product_id} -> review_stars? True
```

TL;DR

F⁺ = all FDs implied by F. Axioms: reflexivity, augmentation, transitivity — sound and complete. Shortcuts: union, decomposition, pseudotransitivity.

11.4 Attribute closure and finding candidate keys

What it is

The **closure of an attribute set X**, written **X⁺**, is every attribute X determines, directly or through chains. It always contains X itself.

The algorithm

1. Start with `result = X`.
2. Scan the FDs. For any FD whose **entire** left side is inside `result`, add its right side.
3. Repeat full passes until a pass adds nothing. `result` is X⁺.

The only classic mistake is firing an FD whose left side is only *partly* present. Repeated passes matter because an FD can become usable halfway through (once an earlier FD adds the column it needs).

The core of `fd_tools.py`:

```
def closure(attrs, fds, trace=False):
    """X+ : keep adding the right side of any FD whose WHOLE left side is already in the set."""
    result = set(attrs)
    changed = True
    while changed:
        changed = False
        for lhs, rhs in fds:
            if lhs <= result and not rhs <= result:   # lhs fully present, something new to add
                result |= rhs
                changed = True
    return frozenset(result)
```

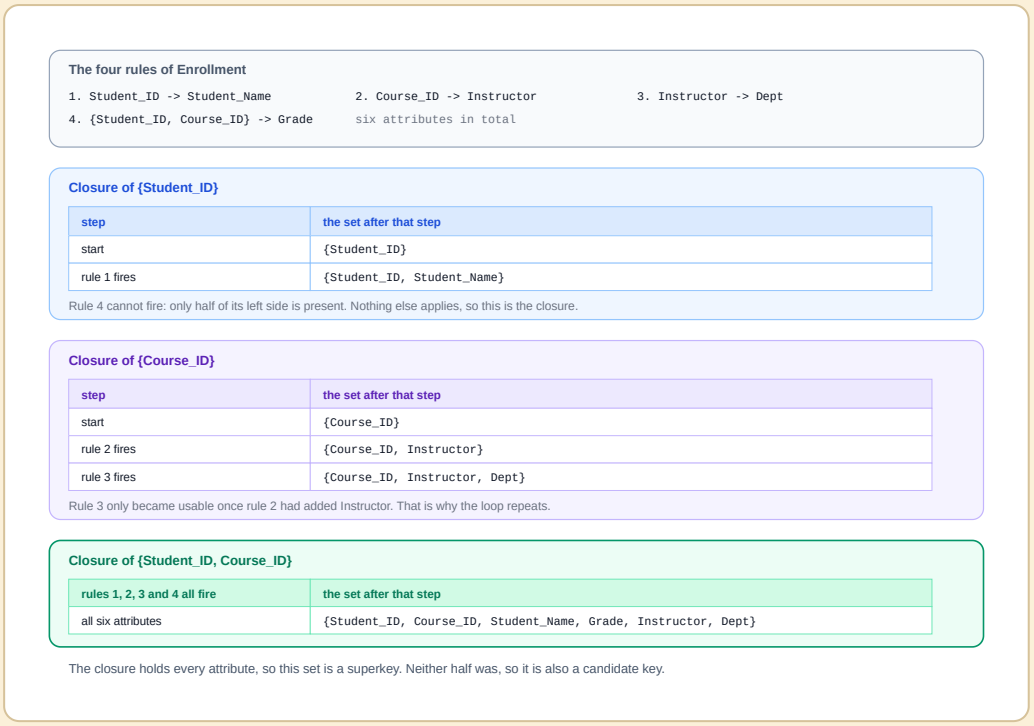
Worked closures on the running example

{order_id}⁺ — start `{order_id}`. F1 fires (`order_id` present) → add `order_date, customer_id`. Now F2's left side `customer_id` is present → add `customer_name, customer_city`. F3 needs `product_id` — absent. F4 needs *both* `order_id` and `product_id` — only one present, **skip**. Nothing more fires.

Output of `fd_tools.py`:

```
closure trace for {order_id}:
    step 1: {order_id} -> {customer_id, order_date} fires, set = {customer_id, order_date, order_id}
    step 2: {customer_id} -> {customer_city, customer_name} fires, set = {customer_city, customer_id,
customer_name, order_date, order_id}
{order_id}+ = {customer_city, customer_id, customer_name, order_date, order_id}
{product_id}+ = {list_price, product_id, product_name}
{order_id, product_id}+ = {customer_city, customer_id, customer_name, list_price, order_date, order_id,
product_id, product_name, qty}
```

Figure — the closure loop step by step on the course's enrolment example (course diagram)



Two uses of closure

Question	Test
Is X a **superkey**?	X^+ contains **all** attributes of the table
Is X a **candidate key**?	X is a superkey **and** no proper subset is (it is enough to test the subsets with one attribute removed)
Does **X -> Y** follow from F?	$Y \subseteq X^+$

On the running example: `{order_id, product_id}+` is all nine columns → superkey. `{order_id}+` (5 columns) and `{product_id}+` (3 columns) are not → neither half is a superkey → **`{order_id, product_id}` is a candidate key**. And `customer_id -> order_id` does **not** follow (`order_id ∉ {customer_id}+`) — matching reality: a customer has many orders.

Finding *all* candidate keys systematically

1. **Classify attributes** by where they appear in the FDs:
   - never on any right side → **must be in every key** (nothing can derive them);
   - only on right sides (never on a left) → **in no key** (they add nothing);
   - on both sides → maybe.
2. Compute the closure of the "must" set. If it is everything, it is the *only* candidate key.
3. Otherwise add "maybe" attributes one at a time, then two at a time, …, skipping any set that contains a key already found (it would not be minimal).

Running example: `order_id` and `product_id` never appear on a right side → both in every key, and their closure is everything → the unique candidate key. That shortcut is why `fd_tools.candidate_keys` starts from that "core".

A harder example (Exercise 3 below): R(A, B, C, D, E) with `A -> B`, `BC -> D`, `D -> A`, `C -> E`. `C` is never on a right side → in every key; `C⁺ = {C, E}` is not everything. Try adding one attribute:

```
{C}+ = {C, E}
{A, C}+ = {A, B, C, D, E}
{B, C}+ = {A, B, C, D, E}
{C, D}+ = {A, B, C, D, E}
{C, E}+ = {C, E}
```

So there are **three** candidate keys: `{A,C}`, `{B,C}`, `{C,D}`. Prime attributes: A, B, C, D; non-prime: E. Tables with several overlapping keys like this are exactly where BCNF differs from 3NF (§11.11).

In the interview

30-second answer: "To test whether X is a key, compute X⁺: start with X and repeatedly add the right side of any FD whose left side is fully contained, until nothing changes. If X⁺ is the whole relation, X is a superkey; if no smaller subset is, it's a candidate key. Attributes that never appear on a right-hand side must be in every key, which is where I start."

- *How do you check whether an FD is implied?* → Compute the closure of its left side and see if the right side is inside.
- *Complexity?* → In big-O notation (how the work grows as the input grows), a naive loop is O(|F|² · |attrs|) per closure; finding all keys can be exponential (doubling with every extra attribute) in the worst case (a relation can have exponentially many keys), but class-sized problems are tiny.

TL;DR

X⁺ = everything X determines (fire only FDs whose whole LHS is present; loop until stable). Superkey ⟺ X⁺ = all attributes; candidate key = minimal superkey; X -> Y implied ⟺ Y ⊆ X⁺. Never-on-the-right attributes are in every key.

11.5 Minimal (canonical) cover

What it is

Different FD lists can say the same thing. A **minimal cover** (also called a **canonical cover**, F_c) is an equivalent list with no waste:

1. every right side is a **single** attribute (use decomposition);
2. no left side has an **extraneous** attribute (one you can drop without changing F⁺);
3. no FD is **redundant** (derivable from the others).

It is the input to the 3NF synthesis algorithm (§11.14) and a common interview/exam exercise.

Worked example

F = { `A -> BC`, `B -> C`, `AB -> D` }

1. Split: `A -> B`, `A -> C`, `B -> C`, `AB -> D`.
2. Extraneous LHS: in `AB -> D`, is B extraneous? `A⁺` (using the current list) = {A, B, C, D} contains D → yes, replace with `A -> D`.
3. Redundant FDs: `A -> C` follows from `A -> B` and `B -> C` → remove it.

Result `{A -> B, B -> C, A -> D}` — the same as `fd_tools.py` prints:

```
=== minimal cover of {A->BC, B->C, AB->D}: {A}->{B}, {B}->{C}, {A}->{D}
```

A minimal cover is not always unique (the order you remove things can matter), but every minimal cover implies exactly F^+.

TL;DR

Minimal cover = singleton right sides, no extraneous left attributes, no redundant FDs; equivalent to F. Needed for 3NF synthesis.

11.6 Anomalies — why normalization exists

The problem it solves

The wide order table stores "Nadia lives in Lisbon" three times and "P1 is a Kettle at 29.00" twice. That repetition causes three named failures:

Anomaly	What goes wrong	In the running table
Insertion	you cannot record one fact without inventing an unrelated one	a new product nobody has ordered yet has no row to live in — the key needs an `order_id`
Update	one real-world change must be repeated on many rows; miss one and the table contradicts itself	Nadia moves: 3 rows must change
Deletion	deleting one fact silently deletes another	cancel Priya's only order and the shop forgets Priya *and* the tea tin

Figure — the three anomalies on the course's enrolment table (course diagram)



Seeing them happen (`code/01_anomalies.sql`)

```sql
-- INSERT: a new product with no order
INSERT INTO order_line_wide (order_id, product_id, ..., product_name, list_price, qty)
VALUES (NULL, 'P5', ..., 'Milk jug', 12.00, 1);              -- inside a DO block that catches the error
-- UPDATE: Nadia moves, but a careless WHERE edits only one of her rows
UPDATE order_line_wide SET customer_city = 'Porto' WHERE customer_id = 'C1' AND order_id = 'O3';
-- DELETE: cancel O4
DELETE FROM order_line_wide WHERE order_id = 'O4';
```

Output:

```
--- the same facts are stored many times
 customer_id | customer_name | customer_city | rows_holding_it
-------------+---------------+---------------+-----------------
 C1          | Nadia Lopez   | Lisbon        |               3
 C2          | Jonas Berg    | Oslo          |               1
 C3          | Priya Das     | Kochi         |               1
(3 rows)

--- INSERT anomaly: a new product nobody has ordered yet has nowhere to go
NOTICE:  refused: null value in column "order_id" of relation "order_line_wide" violates not-null constraint
--- UPDATE anomaly: Nadia moves to Porto, but the update misses one of her rows
 customer_id | cities_on_file
-------------+----------------
 C1          | {Lisbon,Porto}
(1 row)

--- DELETE anomaly: cancel order O4 and customer C3 and product P4 vanish with it
 rows_about_priya | rows_about_tea_tin
------------------+--------------------
                0 |                  0
(1 row)
```

What normalization does

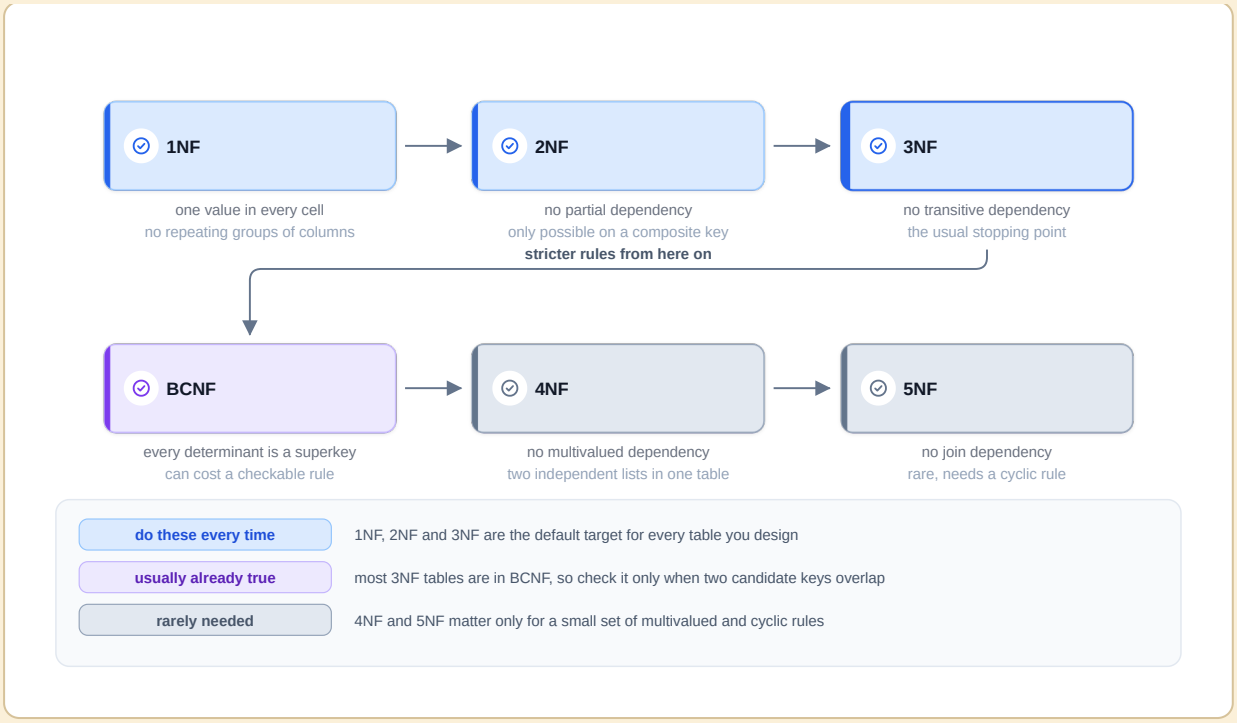
Normalization splits a table along its functional dependencies so that **every fact has exactly one home** — "Nadia lives in Lisbon" in one row of a `customer` table. One row to update, nothing unrelated to delete, and every fact can be inserted on its own. It does this with **decomposition** (replacing a table by projections of it — a *projection* is a table made of some of its columns, duplicates removed) that must be **lossless** (§11.10).

The ladder of normal forms

Each **normal form** is a named test a table passes or fails. Each assumes the previous one and is stricter:

Form	Removes	One-line rule	Origin
1NF	lists and repeating groups in cells	every cell holds one atomic value	Codd 1970
2NF	partial dependencies	no non-prime attribute depends on part of a candidate key	Codd 1971
3NF	transitive dependencies	no non-prime attribute depends on a non-superkey	Codd 1971
BCNF	remaining FD redundancy	every non-trivial FD's determinant is a superkey	Boyce & Codd 1974
4NF	independent multivalued facts	every non-trivial MVD's determinant is a superkey	Fagin 1977
5NF	join dependencies not implied by keys	the table cannot be split losslessly except via its keys	Fagin 1979

Figure — the ladder of normal forms (course diagram)



(There are also domain-key NF and 6NF — used in temporal databases (which record how values change over time) and some data-warehouse (a database built for analysis and reporting rather than day-to-day transactions) styles — which are beyond everyday design.)

You do not always climb to the top. Each step removes redundancy and usually adds a join. **3NF is where most practical designs stop**: it removes the redundancy behind the worst anomalies and always has a safe decomposition (§11.14). Climbing higher is something you choose on purpose when a particular problem demands it. Putting redundancy back on purpose is **denormalization** (§11.16).

TL;DR

Redundancy → insertion, update and deletion anomalies. Normalization decomposes tables along FDs so each fact is stored once. 1NF → 2NF → 3NF → BCNF → 4NF → 5NF, each stricter; most designs stop at 3NF.

11.7 First normal form (1NF)

What it is

A table is in **1NF** when:

1. every cell holds a **single, indivisible (atomic)** value — no lists, sets or nested records;
2. there are **no repeating groups** — no `phone1, phone2, phone3` columns;
3. rows are distinguishable — the table has a key (which, in the relational model, every relation has).

It is purely about the **shape of a cell**.

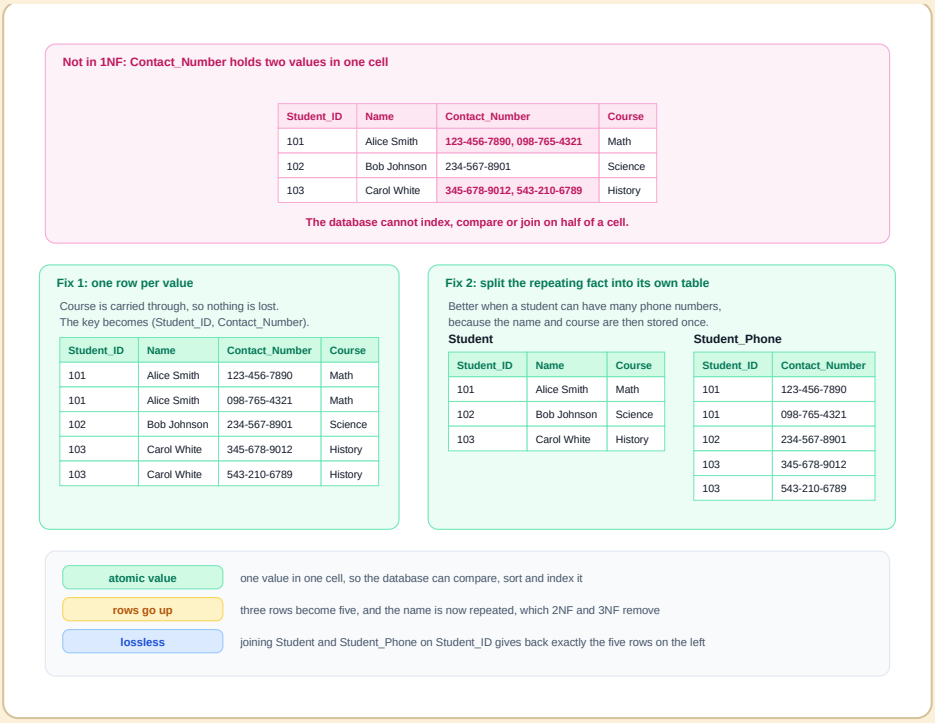
The problem it solves

```
vol_id | name  | languages
     1 | Hana  | Japanese, English
     3 | Ama   | Twi, English, French
```

"Who speaks English?" needs a text search (`LIKE '%English%'`), which also matches "Old English" or a typo; you cannot count languages, index them usefully, or constrain their spelling.

How to fix it — two routes

Figure — a list in a cell, fixed by one-row-per-value or by a separate table (course diagram)



1. **One row per value** in the same table (the key becomes `(vol_id, language)`). This reaches 1NF — but copies the other columns onto every new row.
2. **A separate table** for the repeating fact — `volunteer(vol_id, name)` + `volunteer_language(vol_id, language)`. This is what 2NF would produce from route 1 anyway (the name depends only on `vol_id`), and it is what §10.6 predicted for a multivalued attribute.

`code/03_first_normal_form.sql` repairs both kinds of violation in SQL (*unpivoting* turns several columns into rows; `CROSS JOIN LATERAL` lets the `VALUES` list read the current row's columns):

```
-- a list in one cell -> one row per value (string_to_table: PostgreSQL 14+)
CREATE TABLE volunteer_language AS
SELECT vol_id, trim(lang) AS language
FROM volunteer_packed, string_to_table(languages, ',') AS lang;
ALTER TABLE volunteer_language ADD PRIMARY KEY (vol_id, language);

-- a repeating group phone1..phone3 -> rows (unpivot with LATERAL VALUES)
SELECT g.guest_id, g.name, p.phone
FROM guest_sideways g
CROSS JOIN LATERAL (VALUES (g.phone1), (g.phone2), (g.phone3)) AS p(phone)
WHERE p.phone IS NOT NULL
ORDER BY g.guest_id, p.phone;
```

Output:

```
--- repair: one row per value (string_to_table needs PostgreSQL 14+)
 vol_id | language
--------+----------
      1 | English
      1 | Japanese
      2 | Spanish
      3 | English
      3 | French
      3 | Twi
(6 rows)

--- now counting and exact matching are ordinary SQL
 language | speakers
----------+----------
 English  |        2
 French   |        1
 Japanese |        1
 Spanish  |        1
 Twi      |        1
(5 rows)

--- unpivot the phone columns into rows; guests without a phone produce no rows
 guest_id | name  |  phone
----------+-------+----------
       10 | Ivo   | 555-0101
       10 | Ivo   | 555-0102
       12 | Malik | 555-0301
       12 | Malik | 555-0303
(4 rows)
```

Dialect note: on PostgreSQL < 14 use `unnest(string_to_array(languages, ','))`; MySQL 8 has `JSON_TABLE` or a recursive CTE; SQL Server has `STRING_SPLIT` and `CROSS APPLY (VALUES …)` for the unpivot. (Not run here.)

What 1NF does *not* say — the two classic misconceptions

- **1NF does not require column values to be unique.** Only keys are unique; `name` may repeat freely.
- **1NF does not reduce redundancy — route 1 increases it.** Splitting one row into three copies every other value three times. Removing that repetition is 2NF's and 3NF's job.

Also: a normalization step splits tables or rows but **never drops a column**. If a column disappears, information was lost and the design is wrong.

Borderline cases

- **Structured text** (a whole address in one column) — a 1NF problem only if your application queries the parts.
- **JSON / arrays** — JSON is a text format for nested data (`{"key": [1, 2]}`); both are strictly non-atomic. PostgreSQL has first-class `jsonb` (JSON stored in a parsed binary form) and arrays with indexes (Chapter 14); storing a document *on purpose* (sparse attributes — ones most rows don't have — or payloads you never filter on) is a legitimate design. The question is: do you search, join or constrain on what is inside? If yes, it belongs in rows.

Common mistakes

- Comma-separated ids in a column ("tag_ids = '3,7,9'") — the *SQL Antipatterns* book calls this **Jaywalking**; you cannot join on it or enforce FKs.
- Numbered columns (`item1…item5`).
- Believing 1NF means "no duplicate values".

TL;DR

1NF = atomic cells, no repeating groups, a key. Fix lists with one row per value or (better) a child table. 1NF does not remove redundancy and does not require unique values.

11.8 Second normal form (2NF)

What it is

A table is in **2NF** when it is in 1NF **and no non-prime attribute is partially dependent on any candidate key** — every non-key column depends on the *whole* key.

A table whose candidate keys all have **one column** is automatically in 2NF (there is no "part" of a single column).

Finding the violation in the running table

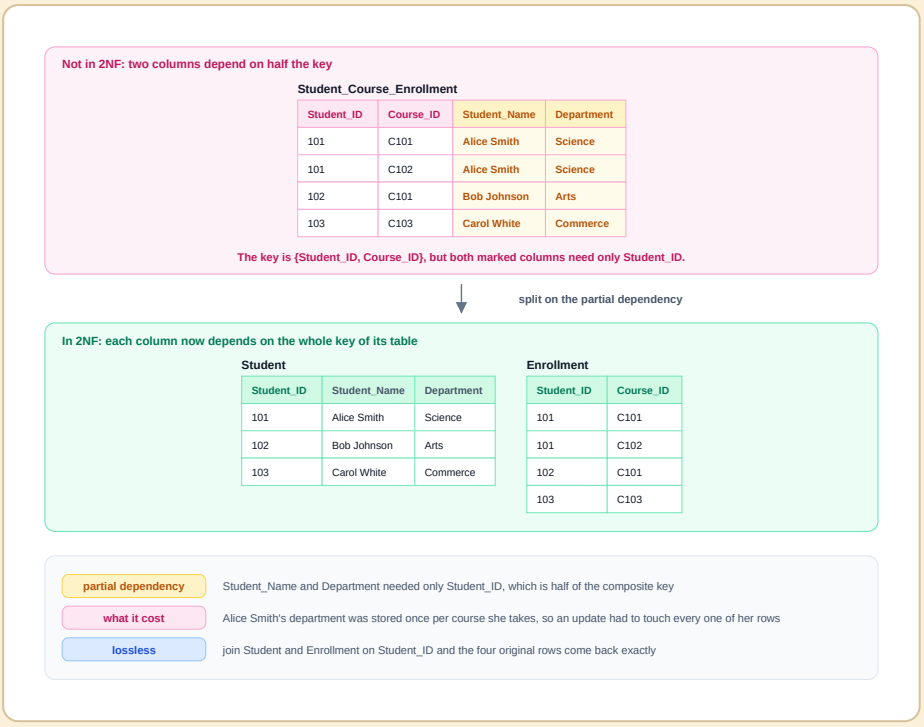
Candidate key `{order_id, product_id}`. Test each non-prime column: does it need the whole key?

Column	Needs	Verdict
`qty`	order **and** product	full ✓
`order_date`, `customer_id`	`order_id` only	partial ✗
`customer_name`, `customer_city`	`order_id` only (via customer_id)	partial ✗
`product_name`, `list_price`	`product_id` only	partial ✗

`fd_tools.py` agrees:

```
candidate keys: {order_id, product_id}
2NF violations: ['{order_id} -> {customer_city, customer_id, customer_name, order_date}', '{product_id} ->
{list_price, product_name}']
```

Figure — a composite-key table, its partial dependencies and the 2NF split (course diagram)



How to fix it

1. For each part of the key that has dependents, create a table keyed by that part and move its dependents there.
2. Leave the full-key columns in a table keyed by the whole key.
3. Group the moved columns **by the key part that determines them** — one new table per part.

Result:

- `order_header(order_id, order_date, customer_id, customer_name, customer_city)`

- `product(product_id, product_name, list_price)`
- `order_item(order_id, product_id, qty)`

`product` and `order_item` are now clean. `order_header` still repeats Nadia's city once per order — that is a *transitive* dependency, which 2NF cannot see. Next section.

Common mistakes

- Moving only one part's dependents (e.g. splitting off `product` but leaving order facts in the composite-key table) — still not 2NF.
- Checking only the primary key: 2NF is about **every candidate key** and **non-prime** attributes (Exercise 1 has a second candidate key).

TL;DR

2NF = 1NF + no non-prime column determined by part of a candidate key. Only composite keys can violate it. Fix: one table per key part with its dependents, plus the full-key table.

11.9 Third normal form (3NF)

What it is

3NF requires 2NF **plus: no non-prime attribute is transitively dependent on a candidate key** — non-key columns depend on the key, the whole key, and nothing but the key. (A popular mnemonic, usually credited to Bill Kent.)

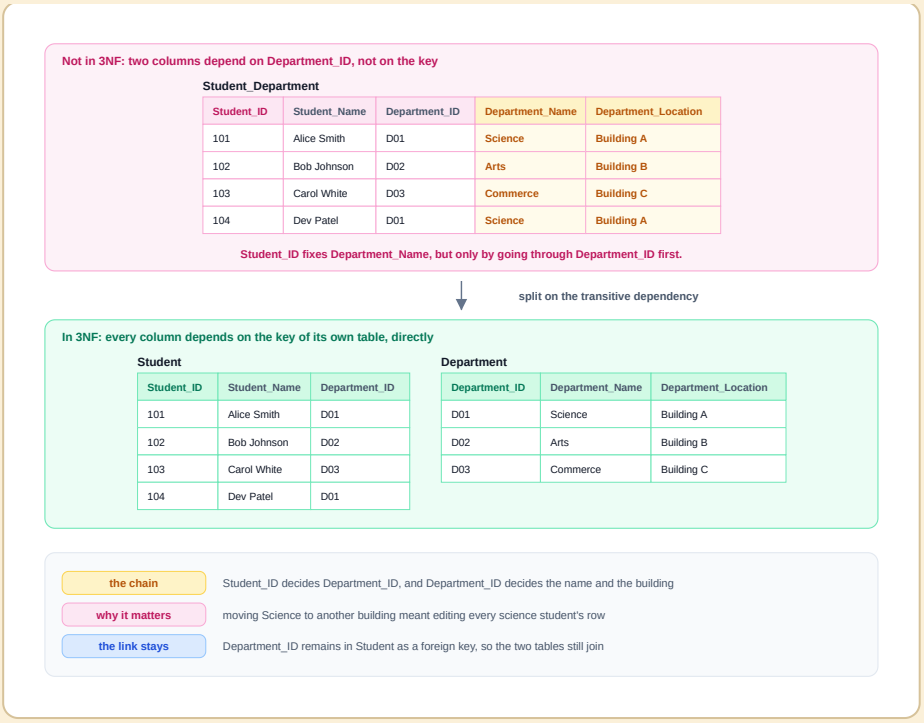
The precise textbook form (Zaniolo's), which you should know for interviews: **for every non-trivial FD `X -> A`, either X is a superkey, or A is a prime attribute.** It also covers 2NF, so a table meeting it is automatically in 2NF.

Finding the violation

`order_header(order_id, order_date, customer_id, customer_name, customer_city)` has a single-column key, so it is in 2NF. But:

`order_id -> customer_id` and `customer_id -> customer_city`, and `customer_id` is **not a superkey** here (Nadia has three orders), and `customer_city` is **non-prime** → transitive dependency → **not 3NF**. Check with the precise form: FD `customer_id -> customer_city` — is `customer_id` a superkey? No. Is `customer_city` prime? No. Both escapes fail.

Figure — a transitive dependency through a non-key column and the 3NF split (course diagram)



How to fix it

Move the columns at the **far end of the chain** into a new table whose key is the **middle** column:

- `customer(customer_id, customer_name, customer_city)`
- `orders(order_id, order_date, customer_id → customer)`

The middle column stays in `orders` as an FK — without it there would be nothing to join on.

2NF vs 3NF side by side

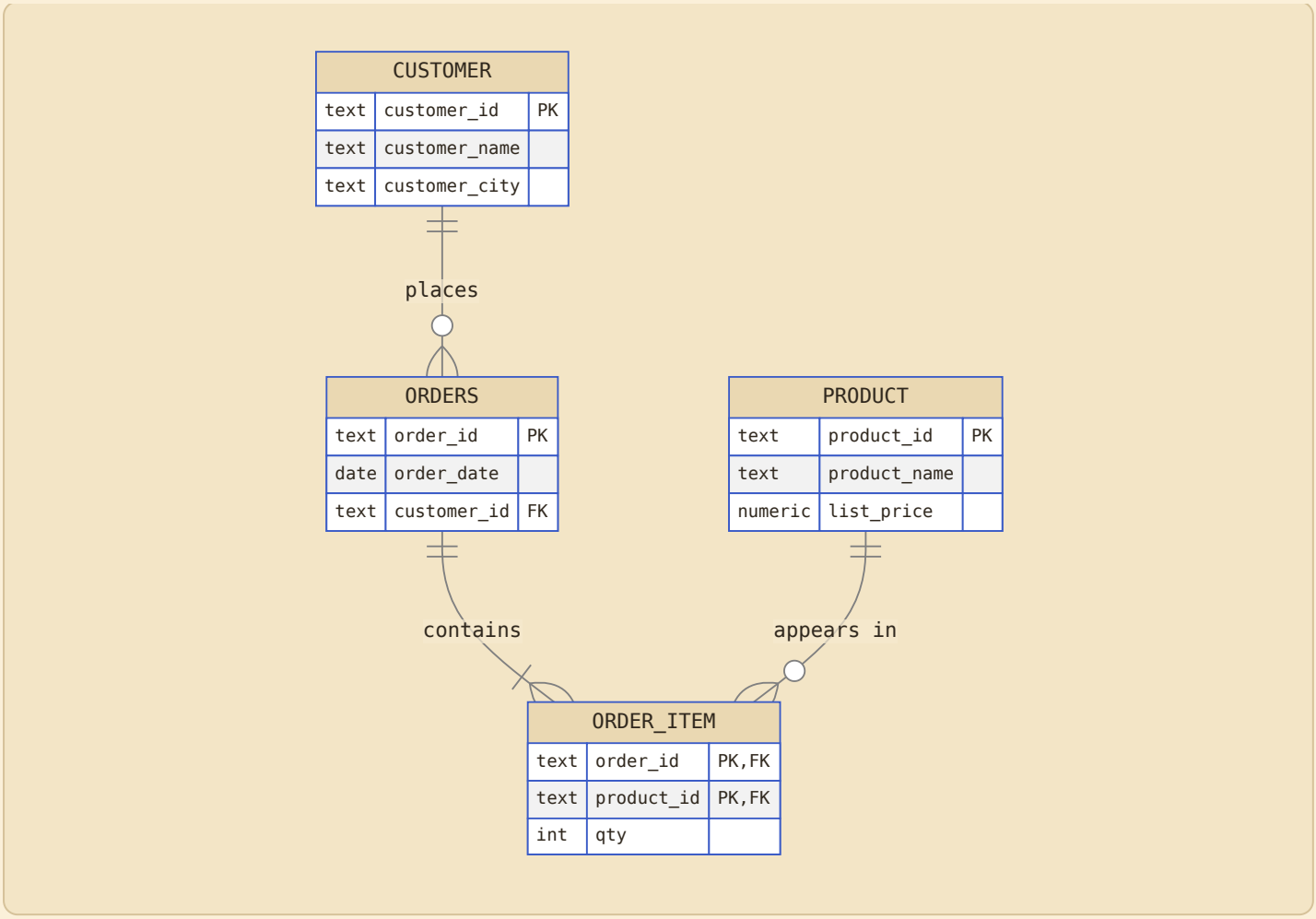
	Dependency runs through	Possible when
2NF violation	**part** of a candidate key	the key is composite
3NF violation	a **non-key** column	any key shape, including one column

So 3NF must be checked on every table; 2NF only on tables with composite keys.

The complete running example in 3NF

The 3NF synthesis algorithm (§11.14) in `fd_tools.py` produces exactly our four tables:

```
3NF synthesis: ['{customer_id, order_date, order_id}', '{customer_city, customer_id, customer_name}',
'{list_price, product_id, product_name}', '{order_id, product_id, qty}']
```



§11.10 builds these tables in PostgreSQL and re-runs the anomalies.

In the interview

30-second answer: "1NF: atomic values. 2NF: no non-key attribute depends on part of a composite key. 3NF: no non-key attribute depends on another non-key attribute — formally, for every non-trivial FD X → A, X is a superkey or A is prime. I normalize to 3NF by default because it removes the update anomalies and there's always a lossless, dependency-preserving 3NF decomposition."

- *Give a 3NF violation with a single-column key.* → `employee(emp_id, dept_id, dept_name)`: `dept_id -> dept_name`.
- *Is a table with only key columns in 3NF?* → Yes (no non-prime attributes at all); check BCNF separately.

TL;DR

3NF = 2NF + no non-key → non-key dependency (formally: every FD's LHS is a superkey or RHS is prime). Fix: split the chain's tail into a table keyed by the middle column, which stays behind as an FK.

11.10 Decomposition done right: lossless joins and dependency preservation

Every split must satisfy two properties — one mandatory, one highly desirable.

Lossless (non-additive) join — mandatory

Joining the pieces must return **exactly** the original rows — nothing lost, **nothing invented**. (A lossy split usually *adds* rows, "spurious tuples"; the information lost is which combinations were true.)

Test for a two-way split of R into R1 and R2: the split is lossless iff the shared columns `R1 ∩ R2` are a **superkey of R1 or of R2** (i.e. `R1 ∩ R2 -> R1` or `R1 ∩ R2 -> R2` is in F^+). Every split in this chapter shares a column that is the key of one piece.

`code/04_lossy_join.sql` shows what happens otherwise. `staff(emp, dept, city)` with `emp -> dept, city`, split two ways:

```
-- bad: (emp, city) + (dept, city); city is a key of neither piece
WITH ec AS (SELECT DISTINCT emp, city FROM staff),
     dc AS (SELECT DISTINCT dept, city FROM staff)
SELECT ec.emp, dc.dept, ec.city,
       (ec.emp, dc.dept, ec.city) IN (SELECT emp, dept, city FROM staff) AS real_row
FROM ec JOIN dc USING (city) ORDER BY 1, 2;
```

Output:

```
--- bad split: (emp, city) and (dept, city). The shared column city is a key of neither piece.
 emp |  dept   | city  | real_row
-----+---------+-------+----------
 Ana | Sales   | Leeds | t
 Ana | Support | Leeds | f
 Ben | Sales   | Leeds | f
 Ben | Support | Leeds | t
 Cai | Sales   | York  | t
(5 rows)

--- good split: (emp, dept) and (emp, city). The shared column emp is a key of both.
 invented_rows
---------------
             0
(1 row)
```

Dependency preservation — strongly desirable

Every original FD should still be checkable **inside a single table** (so a `PRIMARY KEY`/`UNIQUE` constraint enforces it), or be implied by FDs that are. If not, enforcing the rule needs a join — a trigger (code the database runs automatically on every change) or application code. §11.11 shows a split that is lossless but not dependency-preserving.

The running example decomposed in PostgreSQL (`code/02_decompose_and_fix.sql`)

```
CREATE TABLE customer AS SELECT DISTINCT customer_id, customer_name, customer_city FROM order_line_wide;
CREATE TABLE product  AS SELECT DISTINCT product_id, product_name, list_price     FROM order_line_wide;
CREATE TABLE orders   AS SELECT DISTINCT order_id, order_date, customer_id        FROM order_line_wide;
CREATE TABLE order_item AS SELECT order_id, product_id, qty                       FROM order_line_wide;
ALTER TABLE customer   ADD PRIMARY KEY (customer_id);
ALTER TABLE product    ADD PRIMARY KEY (product_id);
ALTER TABLE orders     ADD PRIMARY KEY (order_id),
                       ADD FOREIGN KEY (customer_id) REFERENCES customer;
ALTER TABLE order_item ADD PRIMARY KEY (order_id, product_id),
                       ADD FOREIGN KEY (order_id)   REFERENCES orders ON DELETE CASCADE,
                       ADD FOREIGN KEY (product_id) REFERENCES product;
-- lossless check: (original EXCEPT rejoined) and (rejoined EXCEPT original) must both be empty
```

Output:

```
--- row counts after the split
 customers | products | orders | order_items
-----------+----------+--------+-------------
         3 |        4 |      4 |           5
(1 row)

--- lossless? join the pieces back and compare with the original, both directions
 missing_rows | invented_rows
--------------+---------------
            0 |             0
(1 row)

--- INSERT: a product with no orders is now fine
--- UPDATE: Nadia moves once, in one row
 customer_city
---------------
 Porto
(1 row)

--- DELETE: cancel O4; Priya and the tea tin survive
 priya_rows | tea_tin_rows | o4_items
------------+--------------+----------
          1 |            1 |        0
(1 row)
```

All three anomalies are gone, and `SELECT DISTINCT` + `ADD PRIMARY KEY` is the standard way to perform such a split on live data (if `ADD PRIMARY KEY` fails, the data already contradicts the FD you assumed — a useful check in itself). The pieces also preserve every FD: each of F1–F4 lives inside one table.

TL;DR

Lossless: shared columns must be a key of one side (else the join invents rows). Dependency-preserving: each FD checkable in one table. Verify a split with `EXCEPT` both ways.

11.11 Boyce–Codd normal form (BCNF)

What it is

BCNF holds when the left side of **every non-trivial FD `X -> Y` is a superkey** — "every determinant is a (super)key". Compared with 3NF's precise form, BCNF simply removes the second escape ("…or Y is prime"). Hence every BCNF table is in 3NF; not vice versa. It is sometimes called "3.5NF".

When does 3NF pass but BCNF fail? Only if the table has **several composite candidate keys sharing at least one attribute**. With a single candidate key, 3NF and BCNF coincide — which is why reaching 3NF usually means reaching BCNF too.

A 3NF table that is not in BCNF

A clinic's rules: each doctor has exactly one specialty (`doctor -> specialty`); a patient sees exactly one doctor per specialty (`{patient, specialty} -> doctor`).

patient	specialty	doctor
Ali	Cardiology	Dr Osei
Ali	Dermatology	Dr Lind
Beth	Cardiology	Dr Osei
Beth	Neurology	Dr Tan
Chen	Cardiology	Dr Moreau

`fd_tools.py`:

```
=== visit (3NF, not BCNF)
candidate keys: {doctor, patient}, {patient, specialty}
prime: {doctor, patient, specialty}  non-prime: {}
2NF violations: none
3NF violations: none
BCNF violations: ['{doctor} -> {specialty}']
highest normal form: 3NF
BCNF decomposition: ['{doctor, specialty}', '{doctor, patient}']
FDs no longer checkable in one piece: ['{patient, specialty} -> {doctor}']
```

Why: both candidate keys are composite and overlap on `patient`, so every attribute is prime. `doctor -> specialty` passes 3NF because `specialty` is prime, but fails BCNF because `doctor` is not a superkey. The redundancy is real: "Dr Osei is a cardiologist" is stored once per patient, with the usual anomalies — from `code/05_bcnf.sql`:

```
--- the fact "Dr Osei is a cardiologist" is stored once per patient
  doctor   |  specialty  | copies
-----------+-------------+--------
 Dr Osei   | Cardiology  |      2
 ...
--- nothing stops the copies disagreeing (update anomaly)
 doctor  |      specialties
---------+-----------------------
 Dr Osei | {Cardiology,Oncology}
(1 row)
```

Decomposing

Split on the offending FD: `doctor_specialty(doctor PK, specialty)` and `patient_doctor(patient, doctor)` (PK both columns). The shared column `doctor` is the key of `doctor_specialty`, so the join is lossless (the script prints `invented_rows 0, missing_rows 0`).

The honest cost: a lost dependency

`{patient, specialty} -> doctor` now spans two tables, so no single `PRIMARY KEY`/`UNIQUE` can enforce it:

```
INSERT INTO patient_doctor VALUES ('Ali', 'Dr Moreau');     -- accepted!
```

Output:

```
--- the cost: {patient, specialty} -> doctor is no longer checked by any one table
 patient | specialty  |         doctors
---------+------------+-------------------------
 Ali     | Cardiology | {"Dr Moreau","Dr Osei"}
(1 row)
```

Ali now has two cardiologists, which the business forbids. **General result:** you can always split a table into **3NF** pieces that are both lossless and dependency-preserving; for **BCNF** that is not guaranteed. This is why practitioners usually stop at 3NF and move to BCNF only when the redundancy actually hurts.

A practical middle way (also in `05_bcnf.sql`): keep the 3NF table (so `{patient, specialty}` stays its PK) and add a master (single authoritative) `doctor_specialty` table plus a **composite foreign key** `(doctor, specialty) → doctor_specialty(doctor, specialty) ON UPDATE CASCADE`. The copies can no longer disagree, and one update propagates:

```
NOTICE:  refused: insert or update on table "visit_3nf" violates foreign key constraint
"visit_3nf_doctor_specialty_fkey"
 patient |    specialty    | doctor
---------+-----------------+---------
 Ali     | Cardiac surgery | Dr Osei
 Beth    | Cardiac surgery | Dr Osei
(2 rows)
```

The trap: "looks like BCNF, is really 2NF"

`enrol(student, course, instructor)` with `course -> instructor`, key `{student, course}`. It is tempting to call this a BCNF example, but `instructor` is **non-prime** and hangs off **half** of the key → a **2NF** violation; the table is not even in 2NF. `fd_tools.py`: `2NF violations: ['{course} -> {instructor}'] … highest normal form: 1NF`. **Rule of thumb:** a non-prime dependent whose determinant is a proper subset of a key signals a 2NF problem; only when every attribute involved is prime and a determinant still is not a superkey is it a genuine BCNF problem.

In the interview

30-second answer: "BCNF says every determinant must be a superkey. It's stricter than 3NF, which also allows X → A when A is prime. The two only differ for tables with several composite candidate keys that overlap. BCNF decompositions are always lossless but not always dependency-preserving, so sometimes I stay at 3NF and enforce the rule with a composite foreign key."

- *Example of 3NF-not-BCNF?* → Patient/specialty/doctor with doctor → specialty.
- *What's lost in the BCNF split?* → The FD {patient, specialty} → doctor can't be enforced by one table's key.

TL;DR

BCNF: every non-trivial FD's LHS is a superkey. Differs from 3NF only with overlapping composite candidate keys. Split on the violating FD (lossless), but you may lose dependency preservation.

11.12 Fourth normal form (4NF) — independent multivalued facts

What it is

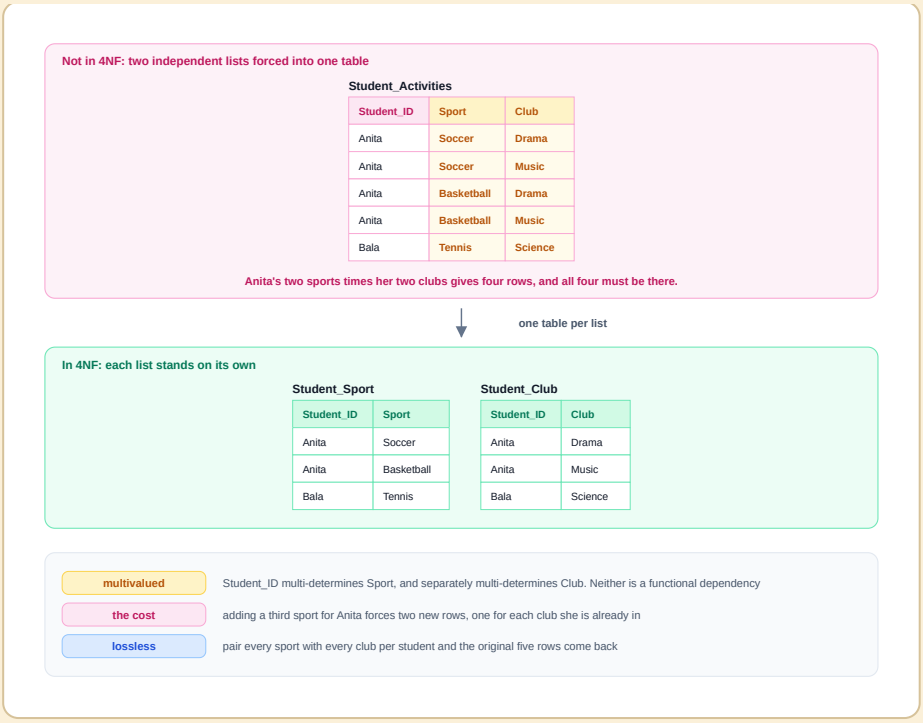
3NF and BCNF are about FDs (one value determines one value). **4NF** handles a different redundancy: two **independent lists** about the same thing stored in one table.

A restaurant has a set of dishes and a separately chosen set of delivery zones. Stored together, every dish must be paired with every zone:

restaurant	dish	zone
Saffron	Biryani	North
Saffron	Biryani	Centre
Saffron	Dal	North
Saffron	Dal	Centre
Nori	Ramen	East

Here `restaurant` does not functionally determine `dish` (Saffron has two). What holds is a **multivalued dependency (MVD)**, written `restaurant ->> dish`: for a given restaurant, the set of dishes is the same whatever zone you look at (and symmetrically `restaurant ->> zone` — MVDs come in complementary pairs). The table's only key is all three columns, so it is in BCNF — and still redundant.

Figure — two independent lists stored as a full cross product, and the 4NF split (course diagram, using sports and clubs)



The rule

A table is in **4NF** when it is in BCNF and **for every non-trivial MVD `X ->> Y`, X is a superkey**. In practice: **one table per independent list**.

The signature and the damage (`code/06_4nf_5nf.sql`)

The tell-tale sign is a **full cross product** per key value: 2 dishes × 2 zones = 4 rows. Adding one dish needs one new row *per zone*; forget one and the table claims that dish is not delivered somewhere:

```
--- restaurant ->> dish | zone: every dish is paired with every zone (a full cross product)
 restaurant | dishes | zones | rows
------------+--------+-------+------
 Nori       |      1 |     1 |    1
 Saffron    |      2 |     2 |    4
(2 rows)

--- adding one dish to Saffron needs one row PER ZONE; forget one and the table lies
 restaurant |     dish     | zones_listed
------------+--------------+--------------
 Saffron    | Biryani      |            2
 Saffron    | Dal          |            2
 Saffron    | Paneer tikka |            1
(3 rows)

--- 4NF split: one table per independent list; the join rebuilds the original exactly
 dish_rows | zone_rows | invented_rows
-----------+-----------+---------------
         3 |         3 |             0
(1 row)
```

Five rows became 3 + 3, and every new fact is now a single row. The split is lossless precisely because the lists were independent (an MVD `X ->> Y` is exactly the condition for splitting R into XY and X(R−Y) losslessly).

Caution: if the lists are *not* independent (a dish is only delivered to some zones), then the three-column table is the correct design and there is no MVD. As with FDs, the business rule decides, not the rows.

TL;DR

MVD `X ->> Y`: for each X, the set of Y values is independent of the other columns (visible as a full cross product). 4NF: every non-trivial MVD's LHS is a superkey → store each independent list in its own

table.

11.13 Fifth normal form (5NF) — facts that need three tables

What it is

5NF (also **project-join normal form, PJ/NF**) handles a rarer case: a table with no FD or MVD problems that nonetheless can be split into **three (or more)** projections that join back exactly — while **no two** of them can. The constraint behind it is a **join dependency (JD)**.

Classic shape — a **cyclic business rule**: *if a supplier supplies a part, and the part is used in a project, and the supplier supplies that project, then the supplier supplies that part to that project.*

supplier	part	project
Acme	bolt	Bridge
Acme	bolt	Tower
Acme	nut	Tower
Brix	bolt	Tower

Why two tables are not enough

Take the three pairwise projections — supplier-part (`sp`), part-project (`pj`), supplier-project (`sj`). **Every** two-way join invents a false row; the three-way join does not (`code/06_4nf_5nf.sql`):

```
--- rows INVENTED by each two-way join (none of these is true)
 pieces | supplier | part | project
--------+----------+------+---------
 pj+sj  | Brix     | nut  | Tower
 sp+pj  | Brix     | bolt | Bridge
 sp+sj  | Acme     | nut  | Bridge
(3 rows)

--- the three-way join invents nothing: 5NF stores sp, pj and sj instead of spj
 invented_rows
---------------
             0
(1 row)
```

For example `sp+pj` claims Brix supplies bolts to Bridge; the third table `sj` rules it out, because Brix does not supply the Bridge project at all.

Figure — why two pairwise tables invent a row that the third rules out (course diagram)



The rule

A table is in **5NF** when every non-trivial join dependency on it is implied by its candidate keys. Practically: when a real cyclic rule links three things, keep the **three two-column tables** (here `sp`, `pj`, `sj`) and derive the combined view by a three-way join. Then inserting one pairwise fact automatically implies the right combinations.

How often 4NF and 5NF come up

Rarely. Independent lists usually get separate tables from the start (they are separate multivalued attributes in the ER diagram), and three-way rules are often simpler to enforce with a constraint or in code. Know them to **name** the problem precisely and to recognise the cross-product and invented-row signatures. Proof of correctness is the same as always: join the pieces and get exactly the original rows.

> **TL;DR**
>
> 5NF (PJ/NF): no join dependency except those implied by keys. Signature: every two-way join of the projections invents rows but the three-way join doesn't → store the three pairwise tables.

11.14 Putting it together — a normalization procedure

Step by step for any table

1. **Write down the FDs** from business rules (not from sample rows). Split right sides to single attributes.
2. **Find all candidate keys** with closure (§11.4); mark prime / non-prime.
3. **1NF:** atomic cells, no repeating groups.
4. **2NF:** any non-prime attribute determined by part of a candidate key? → split per key part.
5. **3NF:** any FD `X -> A` with X not a superkey and A non-prime? → split the chain.
6. **BCNF (only if overlapping composite keys exist):** any FD with a non-superkey LHS? → split, and check what you lose.
7. **4NF/5NF (only if you see cross products / three-way rules).**
8. **Verify every split:** lossless (shared columns are a key of one side; `EXCEPT` both ways is empty) and ideally dependency-preserving; no column dropped.

Two textbook algorithms (both in `fd_tools.py`)

	3NF synthesis (Bernstein)	BCNF decomposition
Steps	1. compute a minimal cover · 2. one table per group of FDs with the same LHS · 3. if no table contains a candidate key, add one · 4. drop tables contained in others	while some table has an FD X → Y with X not a superkey *of that table*: replace it by `X⁺` (within the table) and `(table − X⁺) ∪ X`
Lossless?	always	always
Dependency-preserving?	**always**	not always
Redundancy left?	possibly (only the 3NF-not-BCNF kind)	none from FDs

Run on Exercise 3's relation (R(A,B,C,D,E), `A -> B, BC -> D, D -> A, C -> E`):

```
BCNF decomposition: ['{A, B}', '{C, E}', '{A, D}', '{C, D}']
FDs lost by it: ['{B, C} -> {D}']
3NF synthesis: ['{A, B}', '{B, C, D}', '{A, D}', '{C, E}']
```

The BCNF result loses `BC -> D`; the 3NF result keeps it (in `{B, C, D}`) at the cost of some redundancy. This trade-off is the whole BCNF-vs-3NF story in one line.

TL;DR

FDs → keys → check 1NF, 2NF, 3NF (BCNF/4NF/5NF when their signatures appear) → split → verify lossless + dependency-preserving. 3NF synthesis always preserves dependencies; BCNF decomposition may not.

11.15 Exercises (with solutions)

Try each one on paper before looking at the solution — the real skill is noticing the dependency in a table you have never seen. Every answer below was checked with `fd_tools.py` (its output is quoted).

Exercise 1 — workshop sign-ups (target: 2NF, and beyond)

A conference records sign-ups in one table:

attendee_id	workshop_id	attendee_name	attendee_email	workshop_title	room	seat_no
A1	W1	Kofi	kofi@mail.example	Intro to SQL	R12	4
A1	W2	Kofi	kofi@mail.example	Indexing	R07	9
A2	W1	Mei	mei@mail.example	Intro to SQL	R12	5
A3	W3	Lars	lars@mail.example	Data modeling	R12	1

Rules: an attendee has one name and one email; **emails are unique per attendee**; a workshop has one title and one room; each attendee has one seat in each workshop they attend. Is the table in 2NF? Normalize it to 3NF.

Solution

FDs: `attendee_id -> attendee_name, attendee_email`; `attendee_email -> attendee_id`; `workshop_id -> workshop_title, room`; `{attendee_id, workshop_id} -> seat_no`.

Keys — the twist: because email identifies an attendee, there are **two** candidate keys:

```
candidate keys: {attendee_email, workshop_id}, {attendee_id, workshop_id}
prime: {attendee_email, attendee_id, workshop_id}  non-prime: {attendee_name, room, seat_no, workshop_title}
2NF violations: ['{attendee_email} -> {attendee_name}', '{attendee_id} -> {attendee_name}', '{workshop_id} ->
{room, workshop_title}']
highest normal form: 1NF
```

`attendee_email` is **prime** (it is in a candidate key), so `attendee_id -> attendee_email` is not itself a 2NF problem — but `attendee_name`, `workshop_title` and `room` are non-prime and depend on halves of keys. **Not in 2NF.**

Decomposition (the 3NF synthesis agrees):

```
3NF synthesis: ['{attendee_email, attendee_id, attendee_name}', '{room, workshop_id, workshop_title}',
'{attendee_id, seat_no, workshop_id}']
```

- `attendee(attendee_id PK, attendee_name, attendee_email UNIQUE)` — both keys enforced.
- `workshop(workshop_id PK, workshop_title, room)`
- `signup(attendee_id FK, workshop_id FK, seat_no, PK (attendee_id, workshop_id))`

All three are in BCNF: every determinant in each is a key of that table. Lossless: `signup` shares `attendee_id` (key of `attendee`) and `workshop_id` (key of `workshop`). **Common wrong answer:** splitting off only `attendee` and leaving the workshop columns in the composite-key table — still a partial dependency on `workshop_id`. **Beyond the question:** is `(workshop_id, seat_no)` unique too (one person per seat)? If so, declare `UNIQUE (workshop_id, seat_no)` on `signup` — a rule no normal form would have told you to add.

Exercise 2 — flights (target: 3NF, and question the assumption)

flight_id	flight_date	tail_number	aircraft_type	seat_capacity
9001	2026-10-01	G-EZAA	A320	186
9002	2026-10-01	G-EZBB	A320	180
9003	2026-10-02	G-EZAA	A320	186
9004	2026-10-02	EI-FJK	B737-800	189

`flight_id` is the key. A flight is operated by one aircraft (identified by its tail number); each aircraft has one type and one seat capacity. Is the table in 2NF? In 3NF? Normalize it.

Solution

2NF: yes, automatically — the only candidate key `{flight_id}` has one column, so nothing can depend on "part" of it (and cells are atomic).

FDs: `flight_id -> flight_date, tail_number`; `tail_number -> aircraft_type, seat_capacity`. Chain: `flight_id -> tail_number -> aircraft_type`, and `tail_number` is not a key here (G-EZAA flies 9001 and 9003) → **transitive dependency → not 3NF**.

```
2NF violations: none
3NF violations: ['{tail_number} -> {aircraft_type}', '{tail_number} -> {seat_capacity}']
highest normal form: 2NF
BCNF decomposition: ['{aircraft_type, seat_capacity, tail_number}', '{flight_date, flight_id, tail_number}']
```

Tables: `aircraft(tail_number PK, aircraft_type, seat_capacity)` and `flight(flight_id PK, flight_date, tail_number FK)`. The middle column `tail_number` stays in `flight` as the FK. Now a cabin refit on G-EZAA is one update, and a newly delivered aircraft can be recorded before its first flight.

Question the assumption. A tempting "further" step is `aircraft_type -> seat_capacity` and a third table `aircraft_type(type, capacity)`. The data refutes it: G-EZAA and G-EZBB are both A320s with 186 and 180 seats — capacity depends on each airframe's cabin layout, not on the type. One counter-pair kills the FD. Had the business told you "every A320 has 186 seats", the third table would be right; **a dependency is a claim about the world**, and when you state your assumptions in an interview, say which ones the answer depends on.

Exercise 3 — abstract relation (keys, closure, BCNF vs 3NF)

R(A, B, C, D, E) with F = { `A -> B`, `BC -> D`, `D -> A`, `C -> E` }. (a) Find all candidate keys. (b) Highest normal form? (c) Give a BCNF decomposition and say what it loses. (d) Give a 3NF decomposition.

Solution

(a) `C` never appears on a right side → in every key; `C⁺ = {C, E}`. Adding one attribute: `{A,C}⁺`, `{B,C}⁺`, `{C,D}⁺` are all of R; `{C,E}⁺ = {C,E}`. **Keys: AC, BC, CD** (any other superkey contains one of these).

(b) Prime = {A, B, C, D}; non-prime = {E}. `C -> E`: C is part of a key, E non-prime → partial dependency → **only 1NF**.

```
candidate keys: {A, C}, {B, C}, {C, D}
2NF violations: ['{C} -> {E}']
3NF violations: ['{C} -> {E}']
BCNF violations: ['{A} -> {B}', '{D} -> {A}', '{C} -> {E}']
highest normal form: 1NF
```

Note `A -> B` and `D -> A` are fine for 3NF (B and A are prime) but violate BCNF.

(c) BCNF: `{A,B}`, `{C,E}`, `{A,D}`, `{C,D}` — lossless, but **`BC -> D` is lost** (B and C, D are never together in one table).

(d) 3NF synthesis: `{A,B}`, `{B,C,D}`, `{A,D}`, `{C,E}` — `{B,C,D}` contains the key BC, so it is lossless, and every FD lives in one table.

Exercise 4 — quick-fire: keys and highest normal form

#	Relation and FDs
Q1	R(A,B,C): `A -> B`, `B -> C`
Q2	R(A,B,C): `AB -> C`, `C -> B`
Q3	R(A,B,C,D): `AB -> C`, `B -> D`
Q4	R(A,B,C): `A -> B`, `B -> A`, `A -> C`
Q5	Tutor(tutor, subject, language): a tutor's subjects and the languages they teach in are independent

Solution

```
Q1 R(A,B,C)  A->B, B->C          keys: {A}               highest NF: 2NF
Q2 R(A,B,C)  AB->C, C->B         keys: {A, B}, {A, C}    highest NF: 3NF
Q3 R(A,B,C,D)  AB->C, B->D       keys: {A, B}            highest NF: 1NF
Q4 R(A,B,C)  A->B, B->A, A->C    keys: {A}, {B}          highest NF: BCNF
```

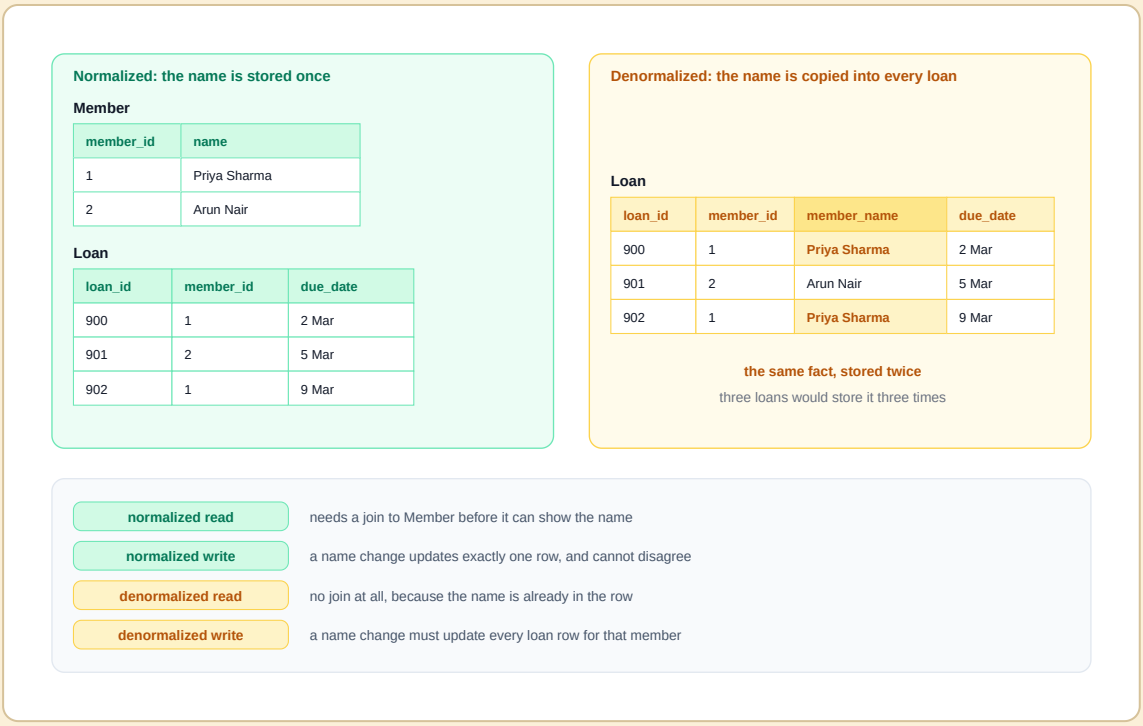
- Q1: transitive A → B → C (single key, so 2NF holds).
- Q2: the textbook 3NF-not-BCNF shape — `C -> B` with B prime, C not a superkey.
- Q3: `B -> D` is partial (B is part of key AB, D non-prime).
- Q4: A and B are both keys; every determinant is a key.
- Q5: no non-trivial FDs, key = all three columns → BCNF; but `tutor ->> subject` (and `->> language`) is a non-trivial MVD with a non-superkey LHS → **BCNF, not 4NF**. Split into `tutor_subject` and `tutor_language`.

11.16 Denormalization — when and how to break the rules

What it is

Denormalization is deliberately adding redundancy back into a normalized design to make important reads faster or simpler — accepting extra write work and a consistency risk. It is a conscious, measured trade, not "skipping normalization": you must know exactly which redundancy you add and how it stays correct.

Figure — the same data normalized and denormalized, with the cost of each (course diagram)



Why it matters — the trade

	Normalized	Denormalized
Each fact stored	once	more than once
Writes	touch one row; cannot disagree	touch several places; can drift if not maintained
Reads	may need joins / aggregation	fewer joins; precomputed answers
Storage	smaller	larger
Best for	OLTP (online transaction processing: many small writes)	read-heavy, reporting, analytics (OLAP, online analytical processing)

Common techniques

Technique	Example	Keeping it correct
Stored summary / counter column	`orders.item_count`, a post's `like_count`	trigger in the same transaction, or batch recompute
Materialized view (a query result saved as a table)	nightly sales-by-product report	`REFRESH MATERIALIZED VIEW` (stale — out of date — in between; `CONCURRENTLY`, which refreshes without blocking readers, needs a unique index)
Duplicated column to avoid a join	copying `customer_name` onto `orders` for a list page	trigger or application code on every change — the riskiest option
Pre-joined / wide tables	star-schema dimensions in a data warehouse (Kimball-style dimensional modeling)	rebuilt by ETL, not updated in place
Caches outside the DB	Redis copy of a product page	expiry / invalidation (see the system-design notes)

Key terms — star schema, ETL, cache: a **star schema** is the classic data-warehouse layout: one long, narrow *fact* table of events (sales) surrounded by wide *dimension* tables describing them

(product, store, date) — Ralph Kimball popularised this "dimensional modeling". **ETL** (extract, transform, load) is the batch job that copies data out of the operational database, reshapes it and loads it into the warehouse. A **cache** is a fast copy of data kept closer to the reader (Redis is a popular in-memory one); *expiry* and *invalidation* are how stale copies get thrown away. More in System Design Fundamentals.

Running it (`code/07_denormalization.sql`)

```
CREATE TABLE orders (order_id text PRIMARY KEY, customer text NOT NULL,
                     item_count int NOT NULL DEFAULT 0);   -- DENORMALIZED: derivable from order_item
CREATE TABLE order_item (
  order_id   text REFERENCES orders ON DELETE CASCADE,
  product_id text REFERENCES product,
  qty        int  NOT NULL CHECK (qty > 0),
  price_paid numeric(8,2) NOT NULL,   -- NOT redundancy: the price at order time is its own fact
  PRIMARY KEY (order_id, product_id)
);
-- AFTER INSERT/UPDATE OF qty/DELETE trigger adjusts orders.item_count in the same transaction
CREATE MATERIALIZED VIEW sales_by_product AS
SELECT p.name, sum(oi.qty) AS units, sum(oi.qty * oi.price_paid) AS revenue
FROM order_item oi JOIN product p USING (product_id) GROUP BY p.name;
```

Output:

```
--- stored item_count vs the value recomputed from order_item
 order_id | stored | recomputed
----------+--------+------------
 O1       |      4 |          4
 O2       |      2 |          2
(2 rows)

--- the list price changes later; price_paid on old orders must not
 order_id |  name  | price_paid | todays_price
----------+--------+------------+--------------
 O1       | Kettle |      29.00 |        31.00
 O2       | Kettle |      27.00 |        31.00
(2 rows)

--- after a new sale, before REFRESH (stale copy)
  name  | units | revenue
--------+-------+---------
 Kettle |     3 |   83.00
 Mug    |     3 |   19.50
(2 rows)

--- after REFRESH
  name  | units | revenue
--------+-------+---------
 Kettle |     3 |   83.00
 Mug    |    13 |   84.50
(2 rows)
```

Two lessons: the trigger keeps the counter exact (but adds a write to every item change and a row lock on the order — other transactions changing that order must wait); the materialized view is cheap to read but **stale until refreshed**.

Not everything that looks redundant is. `price_paid` looks like a copy of `list_price`, but the FD is `{order_id, product_id} -> price_paid` — the price *on that day* — a different fact from today's list price. In the running example we assumed `product_id -> list_price`; in a real shop you would store both. Always ask: *is this the same fact, or a snapshot of it?*

When to denormalize — the rule

1. Normalize first (3NF).
2. Find a specific, important query that is too slow **on realistic data**, and check that indexes, query rewrites and caching did not fix it.

3. Add the smallest redundancy that fixes it, **with an enforced maintenance path** (trigger, materialized view refresh, ETL), and document which query it serves.

When not to denormalize

- Write-heavy tables (every write pays the maintenance).
- When the data must be exactly consistent at all times and you have no reliable maintenance mechanism.
- "Because joins are slow" in general — joins on indexed keys are what relational databases are built for (Chapter 12).

Real-world examples

- **Data warehouses** (Snowflake, BigQuery, Redshift — cloud warehouse services) routinely use denormalized star schemas: facts in a narrow table, wide dimension tables — because data is loaded in batches and read by many analytical queries.
- **Social feeds** keep counters (likes, followers) as stored columns rather than `COUNT(*)` on every page view.
- Wide-column and document stores (non-relational "NoSQL" databases; Chapter 16) are designed around query-shaped, denormalized data.

In the interview

> **30-second answer:** "I normalize to 3NF by default so every fact has one home. I denormalize only for a measured read bottleneck that indexing didn't solve — typically a counter column maintained by a trigger, a materialized view for reports, or a star schema for analytics — and I make sure the redundant copy has a single, enforced maintenance path."

- *Is storing the order total denormalization?* → If it is recomputable from current data, yes; if it freezes historical prices, it is its own fact.
- *Materialized view vs trigger-maintained table?* → MV: simple, periodically stale; trigger: always current, adds write cost and locking.

TL;DR

Denormalize = add controlled redundancy for reads, after normalizing and measuring. Use counters with triggers, materialized views or warehouse star schemas; every copy needs an enforced maintenance path. Snapshots (price paid) are facts, not redundancy.

Key terms

Term	Meaning
Functional dependency `X -> Y`	rows equal on X are equal on Y
Determinant / dependent	left / right side of an FD
Trivial / non-trivial FD	RHS ⊆ LHS / RHS adds a new attribute
Full / partial dependency	needs the whole composite determinant / a proper part of a key suffices
Transitive dependency	key → non-superkey → non-prime attribute
Multivalued dependency `X ->> Y`	for each X, the set of Y values is independent of the rest
Join dependency	the table equals the join of certain projections
F^+ / X^+	closure of an FD set / of an attribute set
Armstrong's axioms	reflexivity, augmentation, transitivity (sound and complete)
Union, decomposition, pseudotransitivity	derived inference rules
Minimal (canonical) cover	equivalent FD set with single RHS, no extraneous attributes, no redundant FDs
Prime / non-prime attribute	in some candidate key / in none
Insertion / update / deletion anomaly	can't add a fact alone / must change many rows / deleting loses an unrelated fact
1NF, 2NF, 3NF, BCNF, 4NF, 5NF	see §11.6 table
Lossless (non-additive) decomposition	joining pieces returns exactly the original rows
Spurious tuple	row invented by a lossy join
Dependency preservation	every FD enforceable within one piece
3NF synthesis / BCNF decomposition	the two standard normalization algorithms
Denormalization	deliberate, maintained redundancy for read performance
Materialized view	stored query result refreshed on demand

Chapter 11 TL;DR

- **FDs are the foundation.** `X -> Y` is a business rule; data can disprove it, never prove it. X is a superkey iff X^+ is everything.
- **Tools:** Armstrong's axioms (sound, complete), attribute closure (fire only fully-present LHS; loop), candidate keys (start with attributes never on a right side), minimal covers.
- **Anomalies** (insert/update/delete) come from storing a fact more than once; normalization decomposes tables so each fact lives in one place.
- **1NF** atomic cells · **2NF** no non-prime on part of a key · **3NF** every FD's LHS is a superkey or RHS is prime · **BCNF** every determinant is a superkey · **4NF** no independent lists (MVDs) · **5NF** no join dependencies beyond keys.
- **Every split must be lossless** (shared columns = key of one side; check with `EXCEPT` both ways) and ideally dependency-preserving. 3NF can always have both; BCNF sometimes loses an FD — which is why 3NF is the default stopping point.
- **Denormalize** only for a measured read problem, with an enforced way to keep copies correct; snapshots such as `price_paid` are separate facts, not redundancy.

- Next: Chapter 12 makes the joins that normalization creates fast; Chapter 17 applies Chapters 10–11 to full designs.